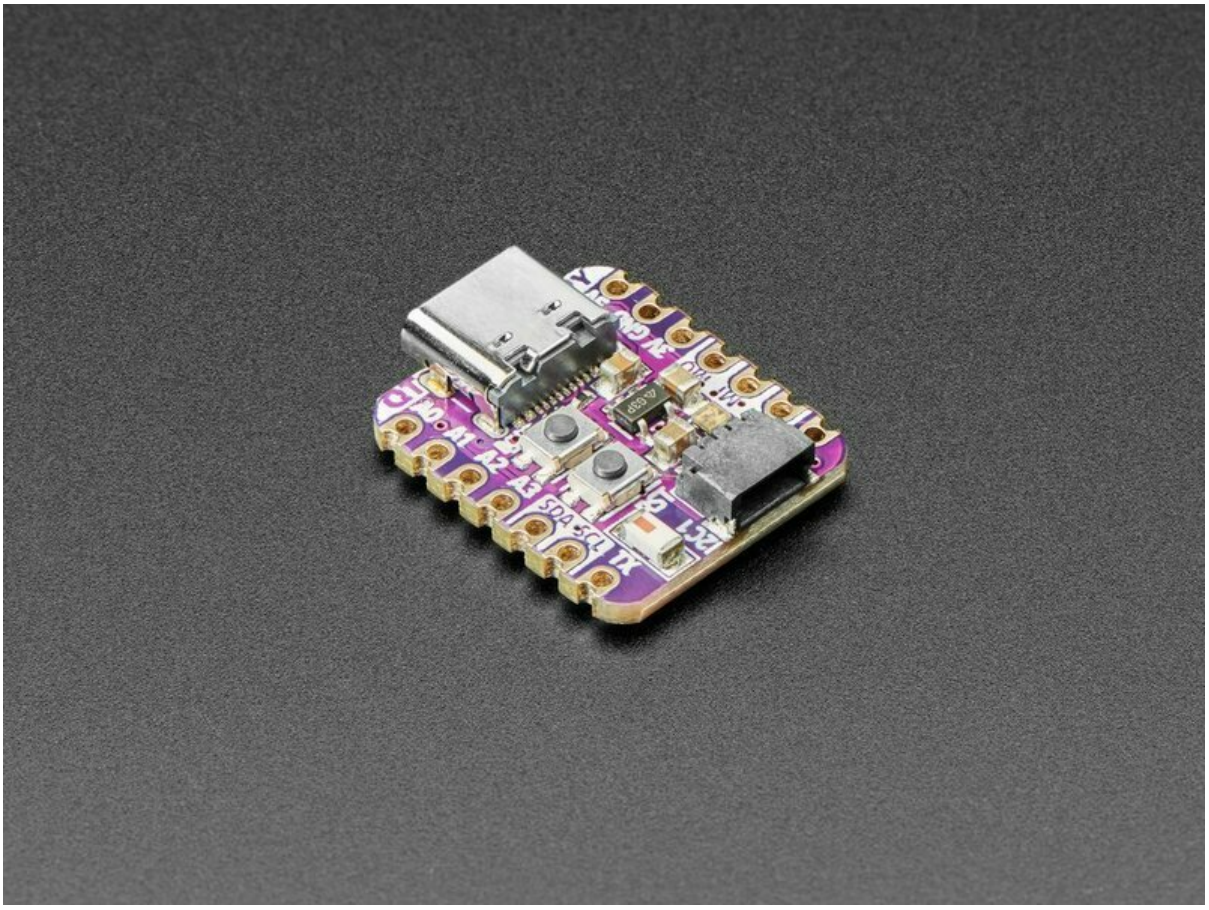
Adafruit QT Py ESP32-S2 and QT Py ESP32-S2 with uFL Antenna

Created by Kattni Rembor



https://learn.adafruit.com/adafruit-qt-py-esp32-s2

Last updated on 2026-10-15 07:13:27 PM UTC

Table of Contents

Overview 9

Pinouts 13

- Power
- ESP32-S2 Chip
- Logic Pins
- STEMMA QT Connector
- NeoPixel LED
- Buttons
- uFL Antenna Port

Update TinyUF2 Bootloader for CircuitPython 10 and Later 20

- Install the Updated TinyUF2 Bootloader on your Board
- Load a CircuitPython 10 (or later) build onto your board

CircuitPython 22

- CircuitPython Quickstart

Install UF2 Bootloader 25

Installing the Mu Editor 27

- Download and Install Mu
- Starting Up Mu
- Using Mu

The CIRCUITPY Drive 29

- Boards Without CIRCUITPY

Creating and Editing Code 30

- Creating Code
- Editing Code
- Back to Editing Code...
- Naming Your Program File

Exploring Your First CircuitPython Program 36

- Imports & Libraries
- Setting Up The LED
- Loop-de-loops
- What Happens When My Code Finishes Running?
- What if I Don't Have the Loop?

Connecting to the Serial Console 39

- Are you using Mu?
- Serial Console Issues or Delays on Linux
- Setting Permissions on Linux
- Using Something Else?

Interacting with the Serial Console 42

The REPL 45

- Entering the REPL
- Interacting with the REPL

• Returning to the Serial Console

CircuitPython Libraries 50

• The Adafruit Learn Guide Project Bundle
• The Adafruit CircuitPython Library Bundle
• Downloading the Adafruit CircuitPython Library Bundle
• The CircuitPython Community Library Bundle
• Downloading the CircuitPython Community Library Bundle
• Understanding the Bundle
• Example Files
• Copying Libraries to Your Board
• Understanding Which Libraries to Install
• Example: ImportError Due to Missing Library
• Library Install on Non-Express Boards
• Updating CircuitPython Libraries and Examples
• CircUp CLI Tool

CircuitPython Documentation 62

• CircuitPython Core Documentation
• CircuitPython Library Documentation

Recommended Editors 69

• Recommended editors
• Recommended only with particular settings or add-ons
• Editors that are NOT recommended
• Moving editor swap files off CIRCUITPY

Advanced Serial Console on Windows 71

• What's the COM?
• Windows Serial Port Terminal Programs
• Install Putty
• Windows 7 and 8.1

Advanced Serial Console on Mac 75

• What's the Port?
• macOS Serial Port Terminal Programs
• Connect with screen

Advanced Serial Console on Linux 77

• What's the Port?
• Linux Serial Port Terminal Programs
• Connect with tio
• Permissions on Linux

Frequently Asked Questions 82

• Using Older Versions
• Python Arithmetic
• Wireless Connectivity
• Asyncio and Interrupts
• Status RGB LED
• Memory Issues
• Unsupported Hardware

ESP32-S2 Bugs & Limitations 92

Troubleshooting 94
• Always Run the Latest Version of CircuitPython and Libraries
• I have to continue using CircuitPython 7.x or earlier. Where can I find compatible libraries?
• macOS Sonoma before 14.4: Errors Writing to CIRCUITPYmacOS 14.4 - 15.1: Slow Writes to CIRCUITPY
• Bootloader (boardnameBOOT) Drive Not Present
• Windows Explorer Locks Up When Accessing boardnameBOOT Drive
• Copying UF2 to boardnameBOOT Drive Hangs at 0% Copied
• CIRCUITPY Drive Does Not Appear or Disappears Quickly
• "M105" Seen on Display, Crashes, Missing CIRCUITPY
• Device Errors or Problems on Windows
• Serial Console in Mu Not Displaying Anything
• code.py Restarts Constantly
• CircuitPython RGB Status Light
• CircuitPython 7.0.0 and Later
• CircuitPython 6.3.0 and earlier
• Serial console showing ValueError: Incompatible .mpy file
• CIRCUITPY Drive Issues
• Safe Mode
• To erase CIRCUITPY: storage.erase_filesystem()
• Erase CIRCUITPY Without Access to the REPL
• For the specific boards listed below:
• For SAMD21 non-Express boards that have a UF2 bootloader:
• For SAMD21 non-Express boards that do not have a UF2 bootloader:
• Running Out of File Space on SAMD21 Non-Express Boards
• Delete something!
• Use tabs
• On macOS?
• Prevent & Remove macOS Hidden Files
• Copy Files on macOS Without Creating Hidden Files
• Other macOS Space-Saving Tips
• Device Locked Up or Boot Looping

Welcome to the Community! 113
• Adafruit Discord
• CircuitPython.org
• Adafruit GitHub
• Adafruit Forums
• Read the Docs

CircuitPython Essentials 123

Blink 124
• NeoPixel Location
• Blinking a NeoPixel LED
• RGB LED Colors

CircuitPython Internet Test 127
• The settings.toml File
• IPv6 Networking

Digital Input 135
• NeoPixel and Button
• Controlling the NeoPixel with a Button

Analog In 138
• Analog to Digital Converter (ADC)

- Potentiometers
- Hardware
- Wire Up the Potentiometer
- Reading Analog Pin Values
- Reading Analog Voltage Values

CPU Temperature 145

- Microcontroller Location
- Reading the Microcontroller Temperature

Storage 147

- The boot.py File
- The code.py File
- Logging the Temperature
- Recovering a Read-Only Filesystem

I2C 153

- I2C and CircuitPython
- Necessary Hardware
- Wiring the MCP9808
- Find Your Sensor
- I2C Sensor Data
- Where's my I2C?

Capacitive Touch 161

- One Capacitive Touch Pin
- Pin Wiring
- Reading Touch on the Pin
- Multiple Capacitive Touch Pins
- Pin Wiring
- Reading Touch on the Pins
- Where are my Touch-Capable pins?

Arduino IDE Setup 168

Arduino NeoPixel Blink 171

- Pre-Flight Check: Get Arduino IDE & Hardware Set Up
- Start up Arduino IDE and Select Board/Port
- Install NeoPixel Library
- New NeoPixel Blink Sketch
- Verify (Compile) Sketch
- Upload Sketch
- Native USB and manual bootloading
- Enter Manual Bootload Mode
- Finally, a Blink!

I2C Scan Test 182

- Common I2C Connectivity Issues
- Perform an I2C scan!
- Wiring the MCP9808

WiFi Test 187

- WiFi Connection Test
- Secure Connection Example

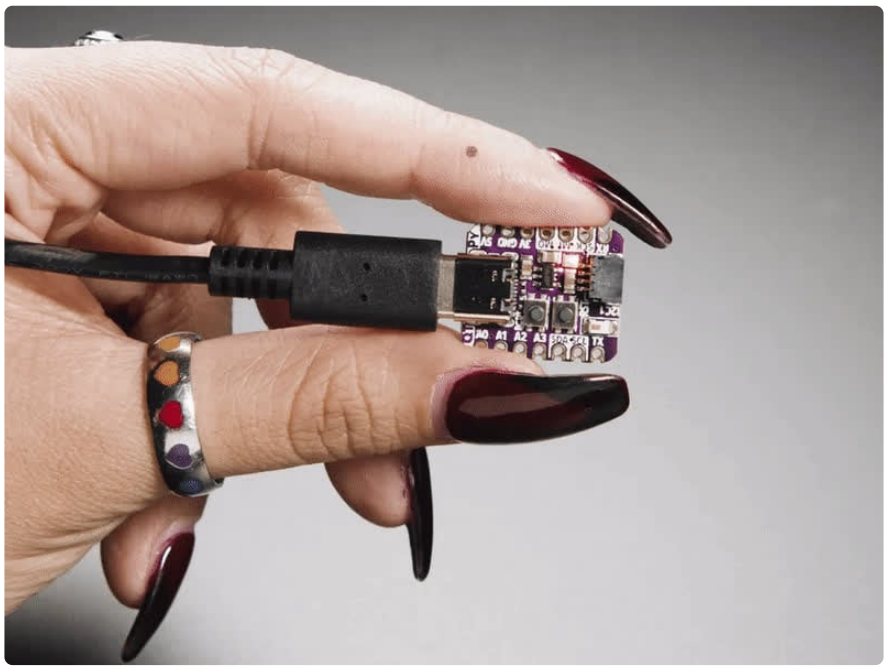
Usage with Adafruit IO 195
• Install Libraries
• Adafruit IO Setup
• Code Usage
WipperSnapper Setup 203
• What is WipperSnapper
• Sign up for Adafruit.io
• Add a New Device to Adafruit IO
• Feedback
• Troubleshooting
• "Uninstalling" WipperSnapper
WipperSnapper Usage 210
• Blink a LED
• Read a Push-Button
• Read an I2C Sensor
• Going Further
WipperSnapper Essentials 224
NeoPixel LED Blink 225
• Where is the NeoPixel on my board?
• Create the NeoPixel Component
• Set the NeoPixel's RGB Color
• Set NeoPixel Brightness
Read a Push-button 231
• Button Location
• Create a Push-button Component on Adafruit IO
Analog Input 236
• Analog to Digital Converter (ADC)
• Potentiometers
• Hardware
• Wire Up the Potentiometer
• Create a Potentiometer Component on Adafruit IO
• Read Analog Pin Values
• Read Analog Pin Voltage Values
I2C Sensor 243
• Parts
• Wiring
• Add an MCP9808 Component
• Read I2C Sensor Values
Factory Reset 248
• Install the Factory Reset Firmware UF2
• UF2 Bootloader Installation and Repair
• OPEN INSTALLER Method for UF2 Bootloader Installation
• Adafruit WebSerial ESPTool and esptool.py Methods for UF2 Bootloader Installation
• Step 1. Download the UF2 Bootloader .bin File for your board
• Step 2. Enter ROM bootloader mode
• Step 3: Alternative A. The Adafruit WebSerial ESPTool Method
• Connect
• Erase the Contents of Flash

• Flash the Bootloader .bin File
• Step 3: Alternative B. The esptool.py Method (for advanced users)
• Install esptool.py
• Test the Installation
• Find the Serial Port
• Connect
• Erase the Flash
• Flash the UF2 Bootloader
• Step 4. Reset the board
• Older Versions of Chrome
• Step 3: Alternative C. The Flash an Arduino Sketch Method
• Arduino IDE Setup
• Load the Blink Sketch

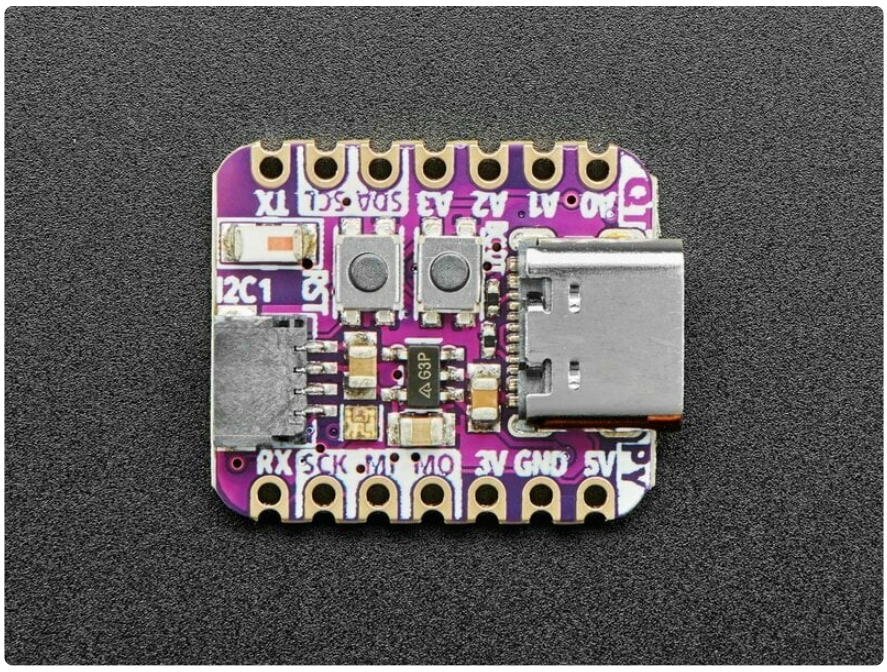
Downloads 266

• Files:
• Schematic and Fab Print Rev C
• Schematic and Fab Print Rev B

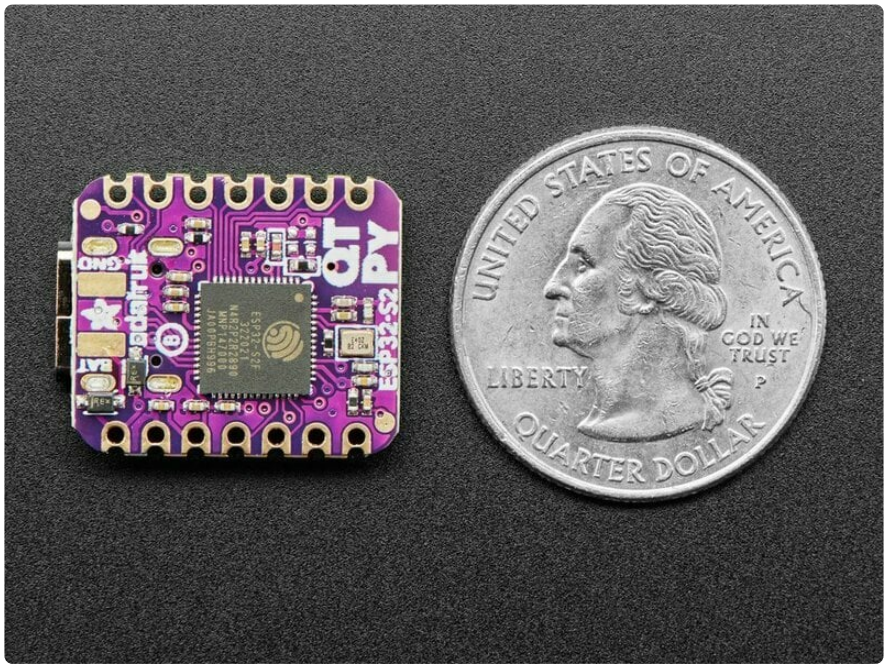
Overview



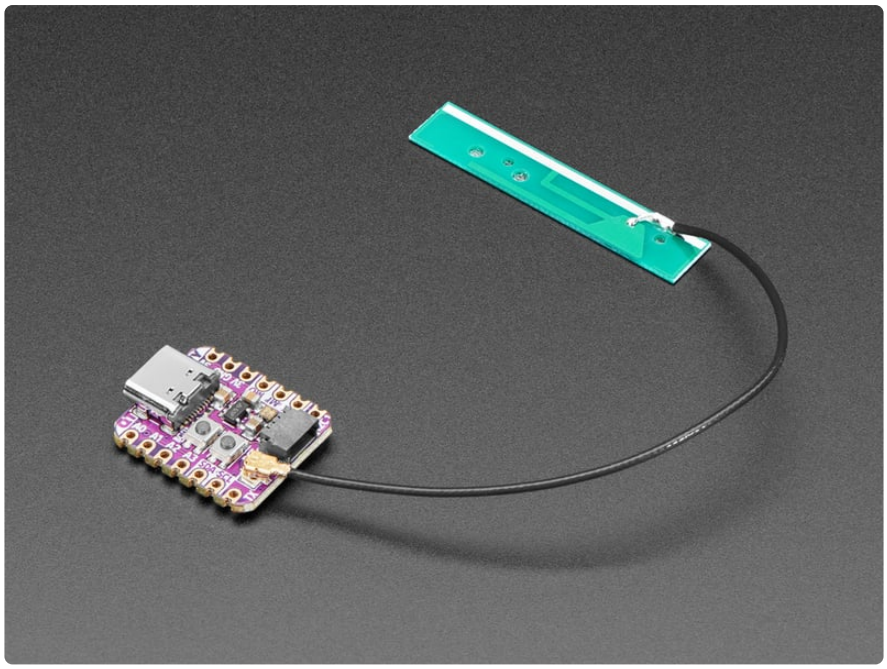
What has your favorite Espressif WiFi microcontroller, comes with our favorite connector - the STEMMA QT (), a chainable I2C port, and has lots of Flash and RAM memory for your next IoT project? What will make your next IoT project flyyyyy? What a cutie pie! Or is it... a **QT Py**? This diminutive dev board comes with one of our new favorite lil chips, the **ESP32-S2**!



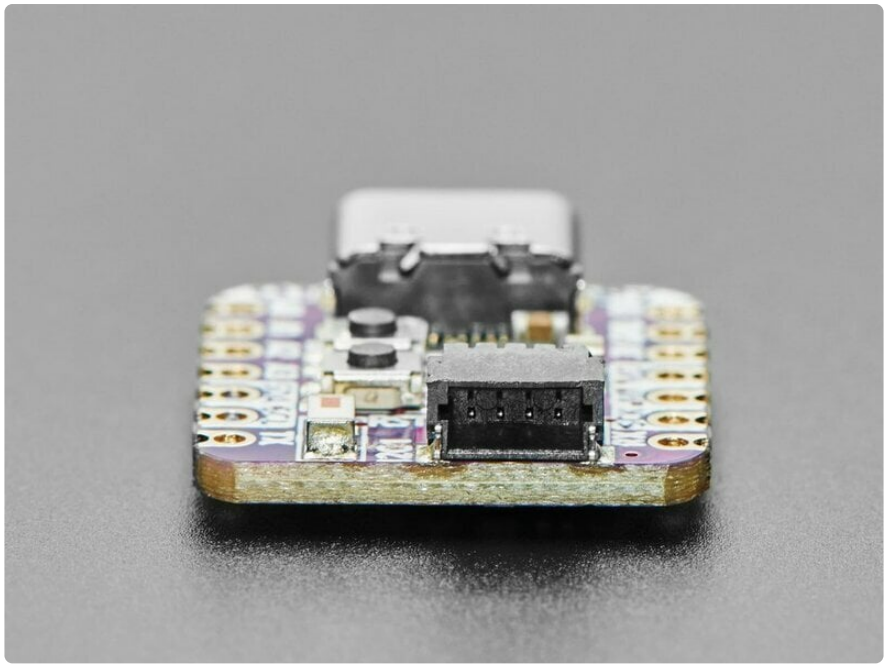
The ESP32-S2 is a highly-integrated, low-power, 2.4 GHz Wi-Fi System-on-Chip (SoC) solution that now has **built-in native USB** as well as some other interesting new technologies like Time of Flight distance measurements. With its state-of-the-art power and RF performance, this SoC is an ideal choice for a wide variety of application scenarios relating to the Internet of Things (IoT) (), wearable electronics (), and smart homes.



Please note the QT Py ESP32-S2 has a single-core 240 MHz chip, so it won't be as fast as ESP32's with dual-core. Also, there is no Bluetooth support. However, we are super excited about the ESP32-S2's native USB which unlocks a lot of capabilities for advanced interfacing! This ESP32-S2 mini-module we are using on the QT Py comes with 4 MB flash and 2 MB PSRAM so you can buffer massive JSON files for parsing!

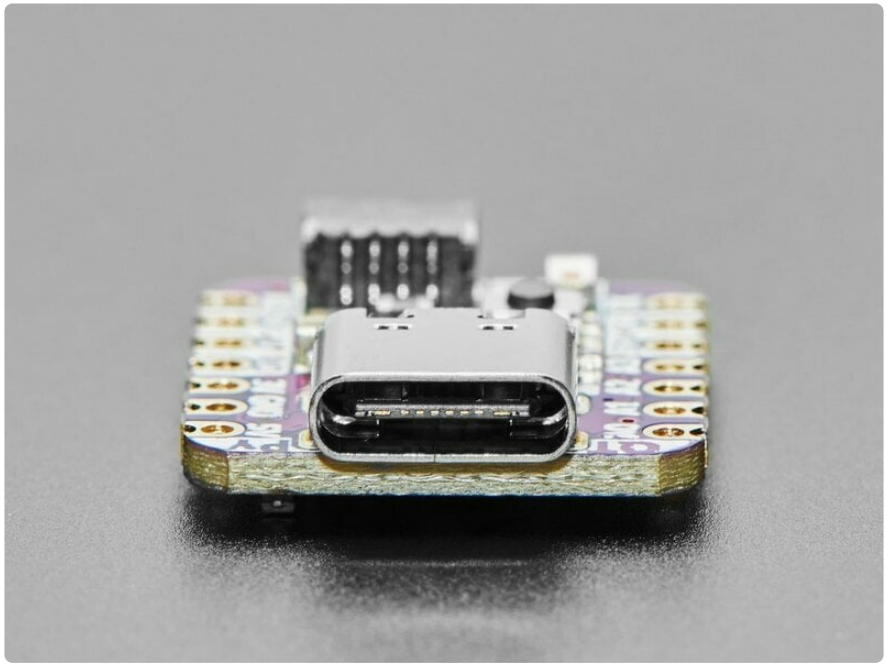


OLEDs ()! Inertial Measurement Units ()! Sensors a-plenty (). All plug-and-play thanks to the innovative chainable design: SparkFun Qwiic ()-compatible STEMMA QT () connectors for the I2C bus so you don't even need to solder! Just plug in a compatible cable and attach it to your MCU of choice, and you're ready to load up some software and measure some light. Seeed Grove I2C boards (http://adafru.it/4528) will also work with this adapter cable.



Pinout and shape are Seeed Xiao () compatible, with castellated pads. In addition to the QT connector, we also added an **RGB NeoPixel** (with controllable power pin to allow for ultra-low-power usage), **a reset button** (great for restarting your program or entering the bootloader) and a button on GPIO 0 for entering the ROM bootloader or for user input

Runs Arduino like a dream, and CircuitPython projects are fantastically fun.

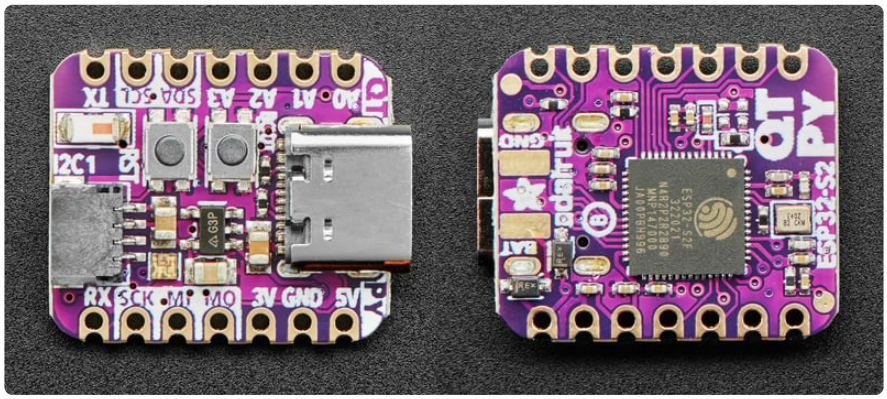


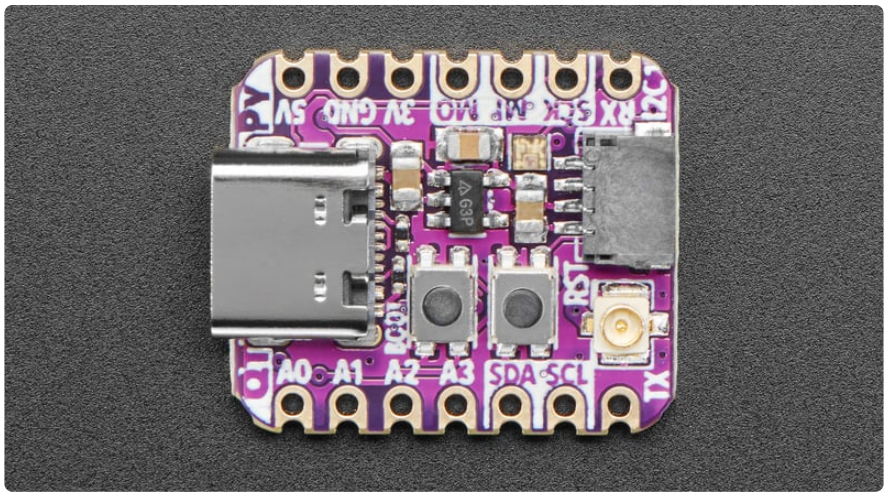
- Same size, form-factor, and pin-out as Seeed Xiao
- **USB Type C connector** - If you have only Micro B cables, this adapter will come in handy (http://adafru.it/4299)!
- **ESP32-S2 240MHz Tensilica processor** - the next generation of ESP32, now with native USB so it can act like a keyboard/mouse, MIDI device, disk drive, etc!
- **4 MB Flash & 2 MB PSRAM**
- Native USB supported by every OS - can be used in Arduino or CircuitPython as USB serial console, MIDI, Keyboard/Mouse HID, even a little disk drive for storing Python scripts.
- Can be used with **Arduino IDE** or **CircuitPython**
- **Built-in RGB NeoPixel LED** with power control to reduce quiescent power in deep sleep
- Battery input pads on underside with diode protection for external battery packs up to 6V input
- **13 GPIO pins:**
    - 11 on breakout pads, 2 more on QT connector

- 10 x 12-bit analog inputs (SPI high speed pads do not have analog inputs)
- 8-bit analog output DAC
- PWM outputs on any pin
- Two I2C ports, one on the breakout pads, and another with STEMMA QT plug-n-play connector
- Hardware UART
- Hardware SPI on the high speed SPI peripheral pins
- Hardware I2S on any pins
- 6 x Capacitive Touch with no additional components required

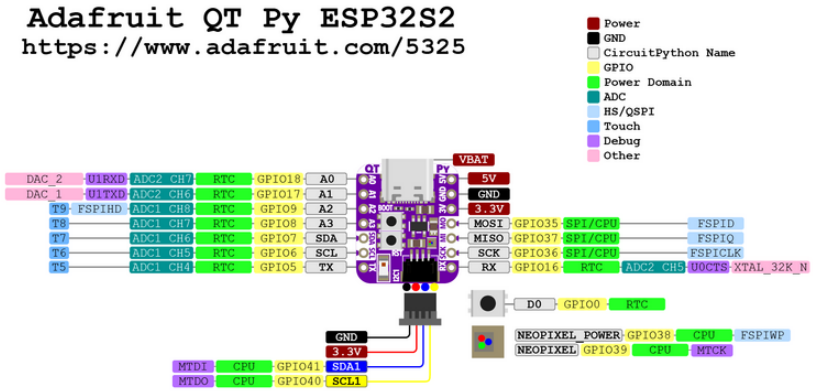
- 3.3V regulator with [600mA peak output]() ()
- Deep sleep at ~70uA
- **Reset switch** for starting your project code over, boot 0 button for entering bootloader mode
- **Really really small**

Pinouts



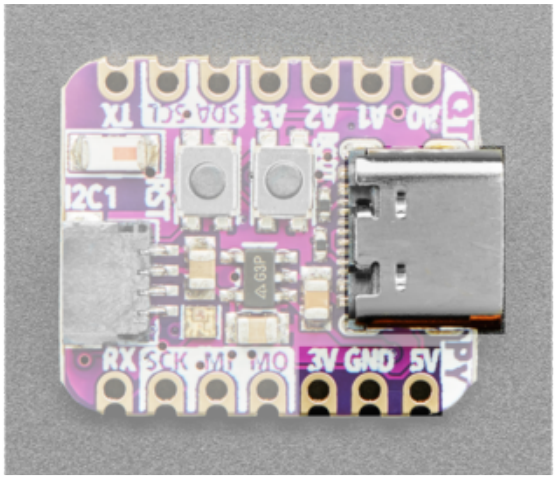


The QT Py ESP32-S2 is tiny but packed with features. Here's a detailed look.



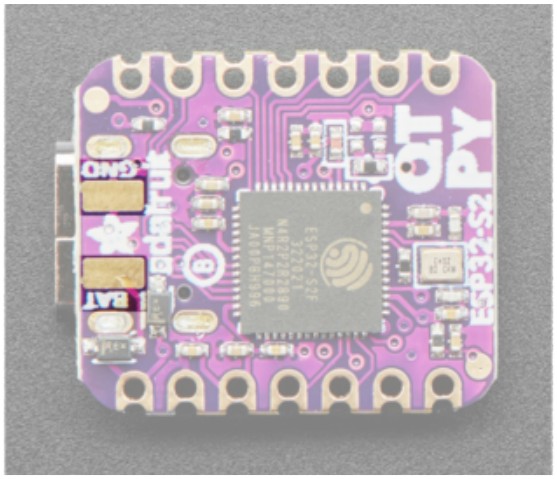
PrettyPins PDF on GitHub ().

Power



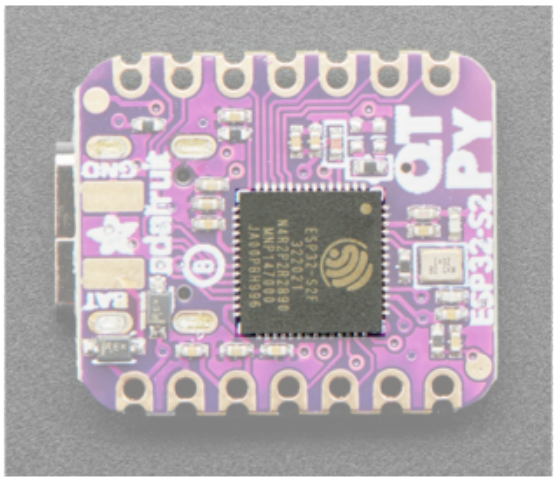
USB-C port - This is used for both powering and programming the board. You can power it with any USB C cable.
3.3V - These pins are the output from the 3.3V regulator, they can supply 600mA peak.
GND - This is the common ground for all power and logic.
5V - This is 5v out from the USB port.

You can also use the 5V pin as a voltage input but you must have some sort of diode (schottky, signal, power, really anything) between your external power source and this pin with anode to battery, cathode to 5V pin. Note that you cannot power the USB port by supplying 5V to this pin: there is a protection diode that prevents the 5V from reaching the USB connector (unless you bridge the jumper on the back!). This is to protect host computer USB ports, etc. You can draw 1A peak through the diode, but we recommend keeping it lower than that, about 500mA.



BAT/GND pads - On the back of the board are two pads labeled BAT and GND. These are the battery input pads with diode protection for external battery packs from 3V to 6V input.

ESP32-S2 Chip



The **ESP32-S2** is a highly-integrated, low-power, 2.4 GHz Wi-Fi System-on-Chip (SoC) solution that now has **built-in native USB** as well as some other interesting new technologies like Time of Flight distance measurements. With its state-of-the-art power and RF performance, this SoC is an ideal choice for a wide variety of application scenarios relating to the Internet of Things (IoT) (), wearable electronics (), and smart homes.

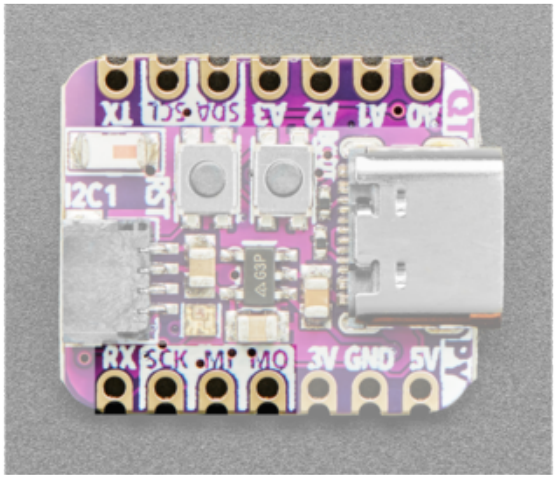
Please note, this is a single-core 240 MHz chip so it won't be as fast as ESP32's with dual-core. The ESP32-S2 supports Wifi, but not does not have Bluetooth support. **This chip comes with 4 MB flash and 2 MB PSRAM.**

The 4 MB of flash is inside the chip and is used for **both** program firmware and filesystem storage. For example, in CircuitPython, we have 3 MB set aside for program firmware (this includes two OTA option spots as well) and a 1MB section for CircuitPython scripts and files.

Text emphasized with a blue exclamation:

The QT Py ESP32-S2 serial input is 3V in max.

Logic Pins



There are eleven GPIO pins broken out to pads. There is hardware I2C, UART, and SPI.

Eight pads are 12-bit analog inputs (SPI high speed pads do not have analog inputs).

You can do PWM output or hardware I2S on any pin.

There are five pins (A2, A3, SCL, SDA, TX) that can do capacitive touch without any external components needed.

That's the general concept. Now for the details!

There are four analog pins.

- **A0 and A1** are the only DAC output pins. These can be used as 8-bit true analog outputs. No other pins can do so. A0 and A1 are on ADC2.
  Note that A0 has a 10K pullup to 3.3V - we're not sure why but this is required by the hardware design notes for the ESP32-S2 (see Fig 1). ()
- **A2 and A3** can also be analog inputs. A2 and A3 are on ADC1. These two pins can do capacitive touch.

Text emphasized with a red exclamation:

For analog to digital inputs (ADC), do not exceed the following values: 0dB Attenuation: 0V to ~0.8V (approx.). 6dB Attenuation: 0V to ~1.3V (approx.). 11dB Attenuation (Default): 0V to ~2.5V (approx.) 12dB Attenuation: 0V to ~2.5V (approx.) To measure voltages higher than 2.5V (like 3.3V or 5V), you must use external resistors to create a voltage divider, ensuring the divided voltage never exceeds the ADC's limit for the chosen attenuation. Applying more than 3.3V (or 3.6V max) directly to an ADC pin can damage the chip, so always use a voltage divider for voltages higher than 2.5V with the default attenuator. See the Espressif Data Sheet for more.

The I2C pins. **These are NOT shared by the STEMMA QT connector!**

- **SCL** - This is the I2C clock pin. There is no pull-up on this pin, so for I2C please add an external pull-up if the breakout doesn't have one already. This pin can do capacitive touch and also analog input
- **SDA** - This is the I2C data pin. There is no pull-up on this pin, so for I2C please add an external pull-up if the breakout doesn't have one already. This pin can do capacitive touch and also analog input

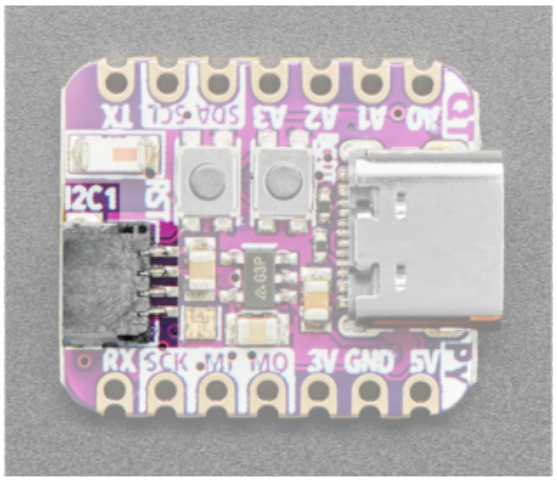
The UART interface.

- **RX** - This is the UART receive pin. Connect to TX (transmit) pin on your sensor or breakout.
- **TX** - This is the UART transmit pin. Connect to RX (receive) pin on your sensor or breakout. This pin can do capacitive touch.

The SPI pins are on the ESP32-S2 high-speed peripheral. You can set any pins to be the low-speed peripheral but you won't get the speedy interface!

- **SCK** - This is the SPI clock pin.
- **MI** - This is the SPI **M**icrocontroller **I**n / **S**ensor **O**ut pin.
- **MO** - This is the SPI **M**icrocontroller **O**ut / **S**ensor **I**n pin.

STEMMA QT Connector

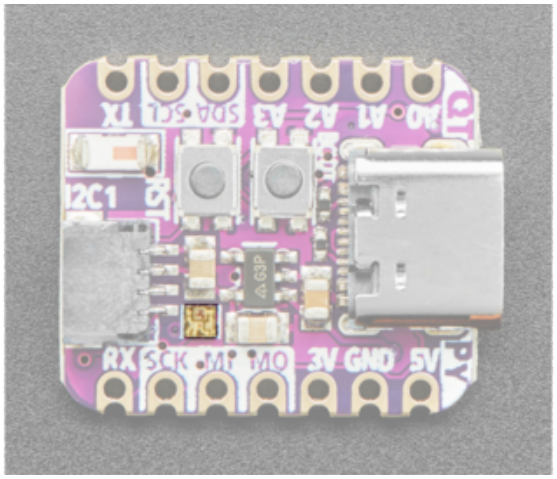


This **JST SH 4-pin** STEMMA QT () **connector** breaks out a second I2C interface (SCL1, SDA1, 3.3V, GND). It allows you to connect to various breakouts and sensors with **STEMMA QT** connectors () or to other things using assorted associated accessories (). It works great with any STEMMA QT or Qwiic sensor/device. You can also use it with Grove I2C devices thanks to this handy cable (http://adafru.it/4528).

In CircuitPython, you can use the STEMMA connector with `board.SCL1` and `board.SDA1`, or `board.STEMMA_I2C()`.

The STEMMA QT connector IO pins in Arduino are `40` (SCL1) and `41` (SDA1) and are available on `Wire1`

NeoPixel LED

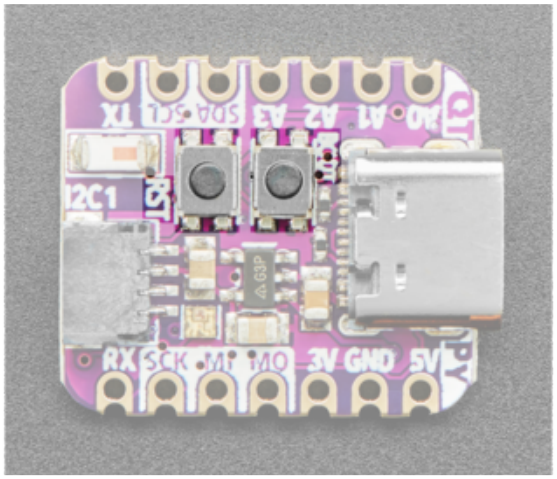


Above the SCK and MI on the silk, is the **RGB NeoPixel LED**. This addressable LED works both as a status LED (in CircuitPython and the bootloader), and can be controlled with code. It is available in CircuitPython as `board.NEOPIXEL`, and in Arduino as `PIN_NEOPIXEL`.
There is a NeoPixel power pin that needs to be pulled high for the NeoPixel to work. **This is done automatically by CircuitPython and Arduino.** It is available in CircuitPython and Arduino as `NEOPIXEL_POWER`.

Text emphasized with a yellow exclamation:

If you run into NeoPixel power issues on Arduino, ensure you are using the latest Espressif board support package. If you are still having issues, you may need to manually pull the pin high in your code.

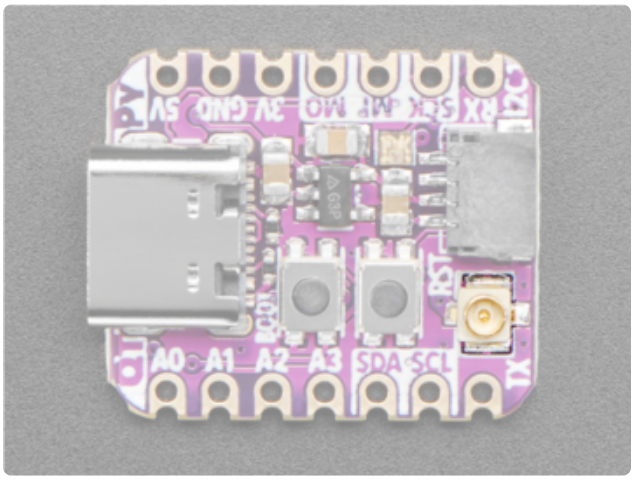
Buttons



There are two buttons.

Reset button - This button restarts the board and helps enter the bootloader. You can click it once to reset the board without unplugging the USB cable or battery. Tap once, and then tap again while the NeoPixel status LED is purple to enter the UF2 bootloader (needed to load CircuitPython).
Boot button - This button can be used to put the board into ROM bootloader mode. To enter ROM bootloader mode, hold down boot button while clicking reset button mentioned above. When in the ROM bootloader, you can upload code and query the chip using `esptool`.
This button can also be read as GPIO 0 (set it to be an input-with-pullup)

uFL Antenna Port



The Adafruit QT Py ESP32-S2 WiFi Dev Board with uFL Antenna Port (http://adafru.it/5348) comes with, you guessed it, a uFL antenna port! **This is the only version that has the port!**

It does NOT come with an antenna, you must purchase one separately. Consider the 2.4GHz Mini Flexible WiFi Antenna **(http://adafru.it/2308)** or a uFL to RP-SMA adapter **(http://adafru.it/852)**.

Update TinyUF2 Bootloader for CircuitPython 10 and Later

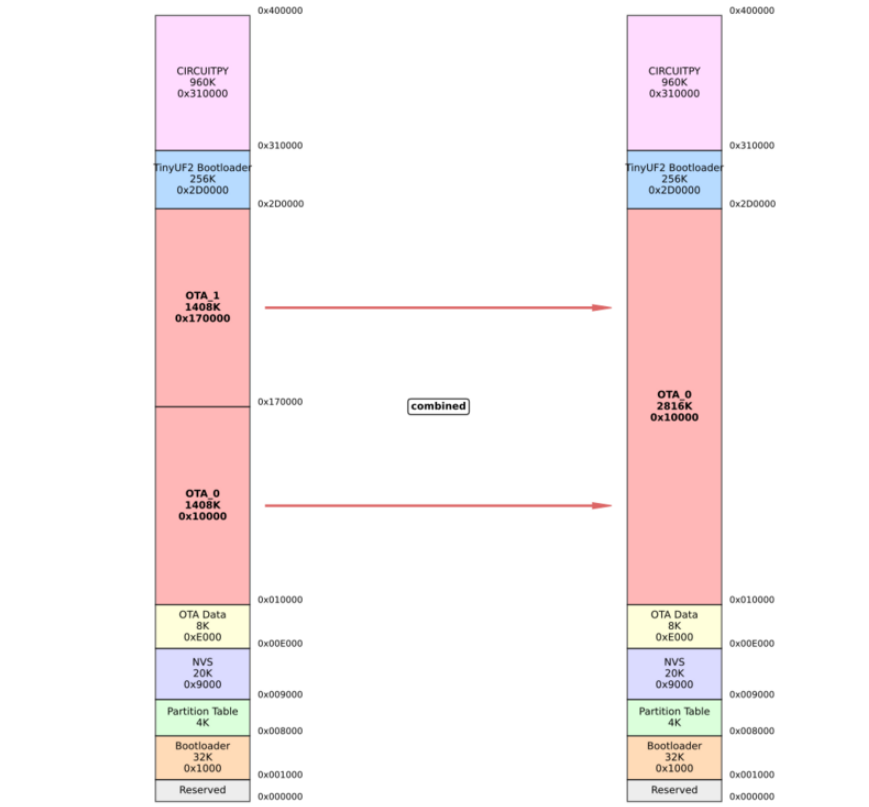
Text emphasized with a blue exclamation:

You don't need to do this if you have a board with 8MB or larger flash. It's only needed for 4MB boards.

CircuitPython 10 changes the flash storage layout on Espressif boards with 4MB flash, to allow for larger CircuitPython builds with more features. This page describes how to update your bootloader to use the new storage layout.

Originally, these 4MB boards had two "ota" (over-the-air) partitions. One contained the CIrcuitPython firmware, and the other was empty, to be used as to update the firmware "over the air". However, that functionality was never fully developed, and the space consumed by the unused partition limited the firmware size of CIrcuitPython. The smaller partition size limited the features that could be included.

This diagram shows the old and new partition layout, and how the two partitions are combined into one.



Text emphasized with a red exclamation:	CIRCUITPY will be erased during this procedure. **Save anything you need that is in CIRCUITPY.**

Install the Updated TinyUF2 Bootloader on your Board

To update the TinyUF2 bootloader for your board, go to the **Factory Reset** section in this guide, and follow the instructions for **UF2 Bootloader Installation/Repair**. You will find a TinyUF2 bootloader listed for 4MB boards for CircuitPython 10.0.0.

Load a CircuitPython 10 (or later) build onto your board

After you've successfully loaded the new bootloader, when you double-click the Reset button, a **...BOOT** drive should appear, such as **FTHRS3BOOT**. The **INFO_UF2.TXT** file in that drive should show that it's version 0.33.0 or later. At this point you can load the CircuitPython **.uf2** file for your board as usual.

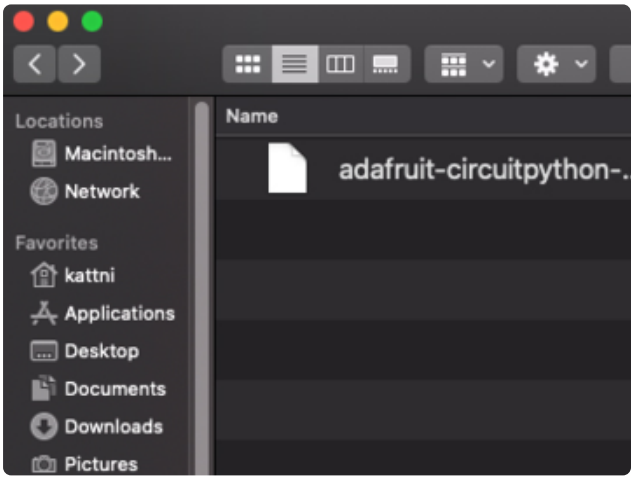
Note that this TinyUF2 upgrade is upward compatible with CircuitPython 9.1.0 and later, so if you need to go back to, say, CircuitPython 9.2.8 from CircuitPython 10, you don't need to reinstall the TinyUF2 bootloader.

CircuitPython

CircuitPython () is a derivative of MicroPython () designed to simplify experimentation and education on low-cost microcontrollers. It makes it easier than ever to get prototyping by requiring no upfront desktop software downloads. Simply copy and edit files on the **CIRCUITPY** drive to iterate.

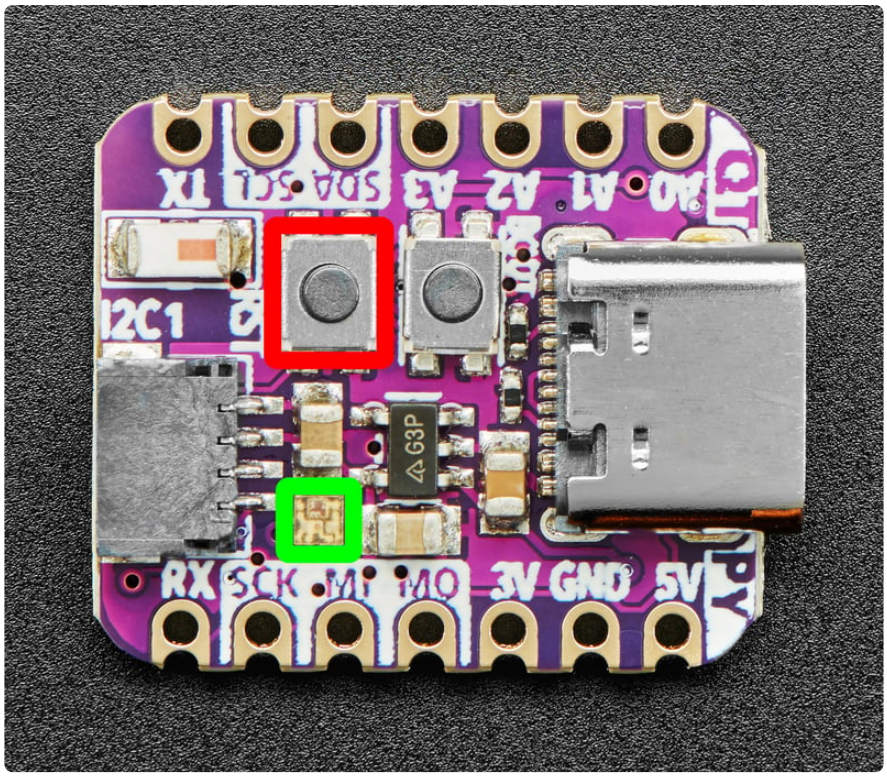
CircuitPython Quickstart

Follow this step-by-step to quickly get CircuitPython running on your board.



Click the link above to download the latest CircuitPython UF2 file.

Save it wherever is convenient for you.



Text emphasized with a blue exclamation:

The board above has a chip antenna, not the u.Fl connector, but the process is the same.

Plug your board into your computer, using a known-good data-sync cable, directly, or via an adapter if needed.

Click the **reset** button once (highlighted in red above), and then click it again when you see the **RGB status LED(s)** (highlighted in green above) turn purple (approximately half a second later). Sometimes it helps to think of it as a "slow double-click" of the reset button.

If you do not see the LED turning purple, you will need to reinstall the UF2 bootloader. See the **Factory Reset** page in this guide for details.

On some very old versions of the UF2 bootloader, the status LED turns red instead of purple.

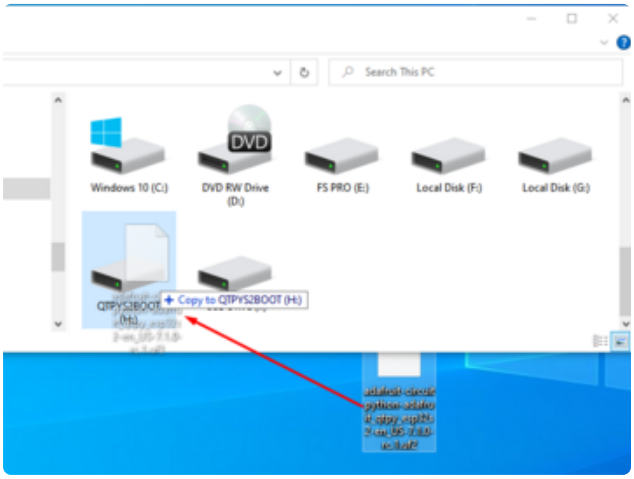
For this board, tap reset and wait for the LED to turn purple, and as soon as it turns purple, tap reset again. The second tap needs to happen while the LED is still purple.

Once successful, you will see the **RGB status LED(s)** turn green (highlighted in green above), and a disk drive ending in "**...BOOT**" should appear on your host computer. If you see red, try another port, or if you're using an adapter or hub, try without the hub, or different adapter or hub.

If double-clicking doesn't work the first time, try again. Sometimes it can take a few tries to get the rhythm right!

A lot of people end up using charge-only USB cables and it is very frustrating! **Make sure you have a USB cable you know is good for data sync.**

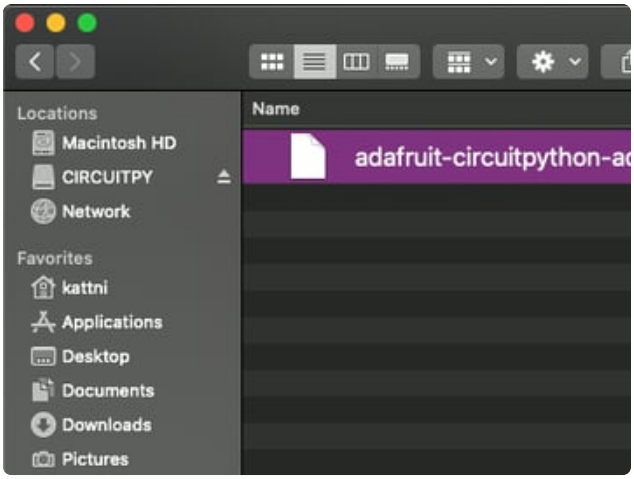
> Text emphasized with a blue exclamation: If after several tries, and verifying your USB cable is data-ready, you still cannot get to the bootloader, it is possible that the bootloader is missing or damaged. Check out the Factory Reset page for details on resolving this issue.



You will see a new disk drive appear called **QTPYS2BOOT**.

Drag the **adafruit_circuitpython_etc.uf2** file to **QTPYS2BOOT**.

Copy or drag the UF2 file you downloaded to the **BOOT** drive.



The **BOOT** drive will disappear and a new disk drive called **CIRCUITPY** will appear.

That's it!

Install UF2 Bootloader

Text emphasized with a yellow exclamation:	If your board already has the UF2 bootloader, installed you do not need to follow the steps on this page. Try to enter the UF2 bootloader before continuing! Double-tap the reset button to do so.

The Adafruit QT Py ESP32-S2 ships with a UF2 bootloader which allows the board to show up as **QTPYS2BOOT** when you double-tap the reset button, and enables you to drag and drop UF2 files to update the firmware.

On ESP32-S2 and ESP32-S3, there is no bootloader protection for the UF2 bootloader. That means it is possible to erase or damage the UF2 bootloader, especially if you upload an Arduino sketch to an ESP32-S2/S3 board that doesn't "know" there's a bootloader it should not overwrite!

It turns out, however, the ESP32-S2/S3 comes with a second bootloader: the ROM bootloader. Thanks to the ROM bootloader, you don't have to worry about damaging the UF2 bootloader. The ROM bootloader can never be disabled or erased, so its always there if you need it! You can simply re-load the UF2 bootloader from the ROM bootloader.

If your UF2 bootloader ends up damaged or overwritten, you can follow the steps found in the Factory Reset and Bootloader Repair () section of the Factory Reset page in this guide.

Once completed, you'll return to where the board was when you opened the package. Then you'll be back in business, and able to continue with your existing plans!

Installing the Mu Editor

Mu is a simple code editor that works with the Adafruit CircuitPython boards. It's written in Python and works on Windows, MacOS, Linux and Raspberry Pi. The serial console is built right in so you get immediate feedback from your board's serial output!

Text emphasized with a blue exclamation:

Mu is our recommended editor - please use it (unless you are an experienced coder with a favorite editor already!). While it has been announced end of life in 2026, it still works fine. You are free to use whatever text editor you wish along with a terminal program to connect to the CircuitPython REPL. Thonny is one such editor.

Download and Install Mu

Download Mu from https://codewith.mu ().

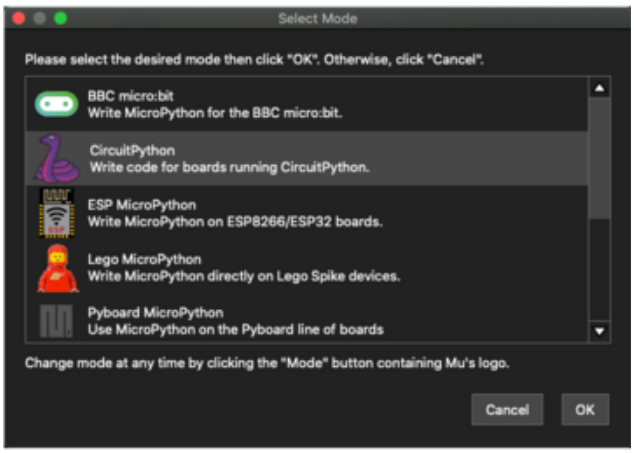
Click the **Download** link for downloads and installation instructions.

Click **Start Here** to find a wealth of other information, including extensive tutorials and and how-to's.

Text emphasized with a yellow exclamation:

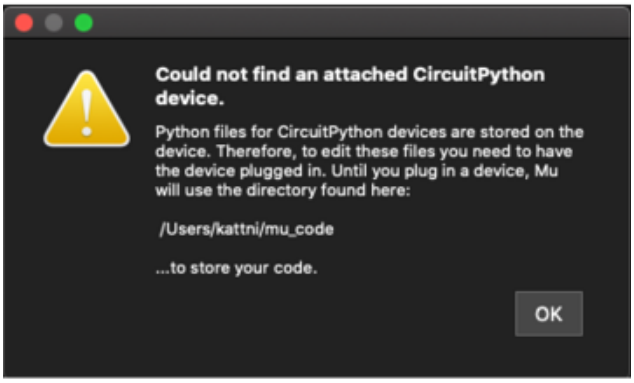
Windows users: due to the nature of MSI installers, please remove old versions of Mu before installing the latest version.

Starting Up Mu



The first time you start Mu, you will be prompted to select your 'mode' - you can always change your mind later. For now please select **CircuitPython**!

The current mode is displayed in the lower right corner of the window, next to the "gear" icon. If the mode says "Microbit" or something else, click the **Mode** button in the upper left, and then choose "CircuitPython" in the dialog box that appears.



Mu attempts to auto-detect your board on startup, so if you do not have a CircuitPython board plugged in with a **CIRCUITPY** drive available, Mu will inform you where it will store any code you save until you plug in a board.

To avoid this warning, plug in a board and ensure that the **CIRCUITPY** drive is mounted before starting Mu.

Using Mu

You can now explore Mu! The three main sections of the window are labeled below; the button bar, the text editor, and the serial console / REPL.

Now you're ready to code! Let's keep going...

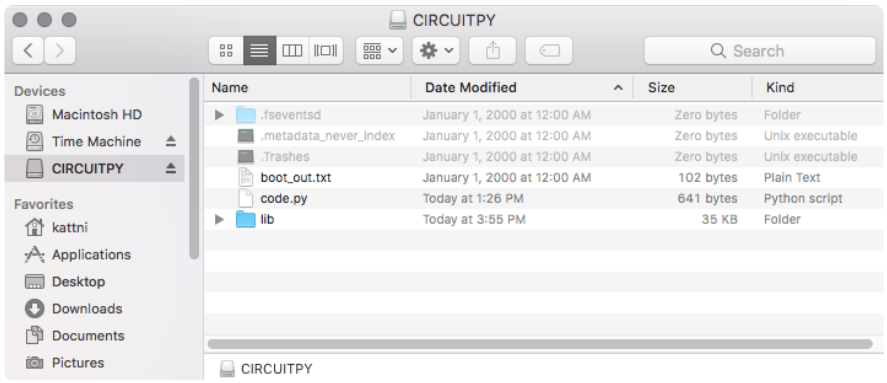
The CIRCUITPY Drive

When CircuitPython finishes installing, or you plug a CircuitPython board into your computer with CircuitPython already installed, the board shows up on your computer as a USB drive called **CIRCUITPY**.

The **CIRCUITPY** drive is where your code and the necessary libraries and files will live. You can edit your code directly on this drive and when you save, it will run automatically. When you create and edit code, you'll save your code in a **code.py** file located on the **CIRCUITPY** drive. If you're following along with a Learn guide, you can paste the contents of the tutorial example into **code.py** on the **CIRCUITPY** drive and save it to run the example.

With a fresh CircuitPython install, on your **CIRCUITPY** drive, you'll find a **code.py** file containing `print("Hello World!")` and an empty **lib** folder. If your **CIRCUITPY** drive does not contain a **code.py** file, you can easily create one and save it to the drive. CircuitPython looks for **code.py** and executes the code within the file automatically when the board starts up or resets. Following a change to the contents of **CIRCUITPY**, such as making a change to the **code.py** file, the board will reset, and the code will be run. You do not need to manually run the code. This is what makes it so easy to get started with your project and update your code!

Note that all changes to the contents of **CIRCUITPY**, such as saving a new file, renaming a current file, or deleting an existing file will trigger a reset of the board.



Boards Without CIRCUITPY

CircuitPython is available for some microcontrollers that do not support native USB. Those boards cannot present a **CIRCUITPY** drive. This includes boards using ESP32 or ESP32-C3 microcontrollers.

On these boards, there are alternative ways to transfer and edit files. You can use the Thonny editor (), which uses hidden commands sent to the REPL to read and write files. Or you can use the CircuitPython web workflow, introduced in Circuitpython 8. The web workflow provides browser-based WiFi access to the CircuitPython filesystem. These guides will help you with the web workflow:

- CircuitPython on ESP32 Quick Start ()
- CircuitPython Web Workflow Code Editor Quick Start ()

Creating and Editing Code

One of the best things about CircuitPython is how simple it is to get code up and running. This section covers how to create and edit your first CircuitPython program.

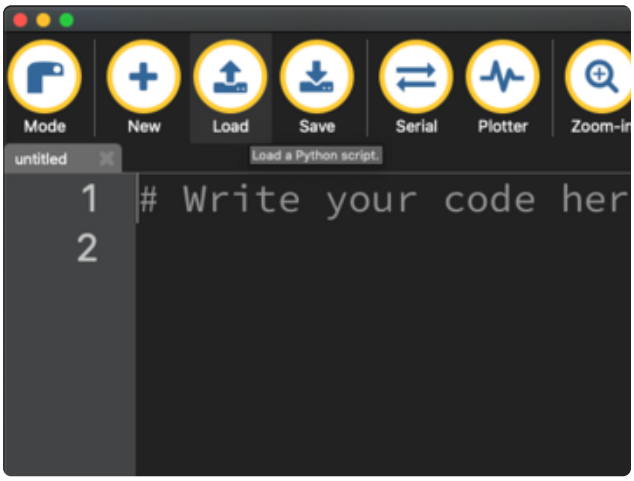
To create and edit code, all you'll need is an editor. There are many options. **Adafruit strongly recommends using Mu! It's designed for CircuitPython, and it's really simple and easy to use, with a built in serial console!**

If you don't or can't use Mu, there are a number of other editors that work quite well. The Recommended Editors page () has more details. Otherwise, make sure you do "Eject" or "Safe Remove" on Windows or "sync" on Linux after writing a file if you aren't using Mu. (This was formerly not a problem on macOS, but see the warning below.)

Text emphasized with a yellow exclamation:

macOS Sonoma 14.1 introduced a bug that delays writes to small drives such as CIRCUITPY drives. This caused errors when saving files to CIRCUITPY. There is a workaround. The bug was fixed in Sonoma 14.4, but at the cost of greatly slowed writes to drives 1GB or smaller.

Creating Code



Installing CircuitPython generates a **code.py** file on your **CIRCUITPY** drive. To begin your own program, open your editor, and load the **code.py** file from the **CIRCUITPY** drive.

If you are using Mu, click the **Load** button in the button bar, navigate to the **CIRCUITPY** drive, and choose **code.py**.

Copy and paste the following code into your editor:

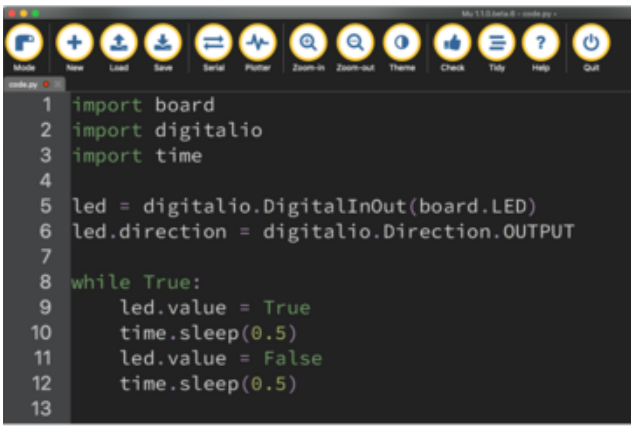
```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

Text emphasized with a red exclamation:

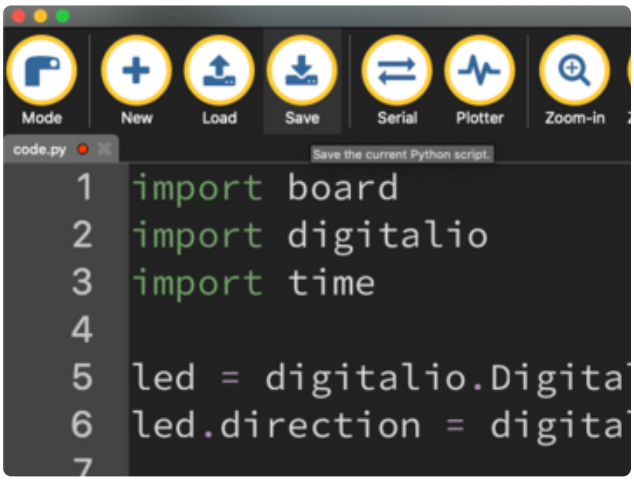
The KB2040, QT Py , Qualia, and the Trinkeys do not have a built-in little red LED! There is an addressable RGB NeoPixel LED. The above example will NOT work on the KB2040, QT Py, Qualia, or the Trinkeys!

If you're using a KB2040, QT Py, Quaila, or a Trinkey, or any other board without a single-color LED that can blink, please download the NeoPixel blink example ().

Text emphasized with a blue exclamation:	The NeoPixel blink example uses the onboard NeoPixel, but the time code is the same. You can use the linked NeoPixel Blink example to follow along with this guide page.



It will look like this. Note that under the `while True:` line, the next four lines begin with four spaces to indent them, and they're indented exactly the same amount. All the lines before that have no spaces before the text.



Save the **code.py** file on your **CIRCUITPY** drive.

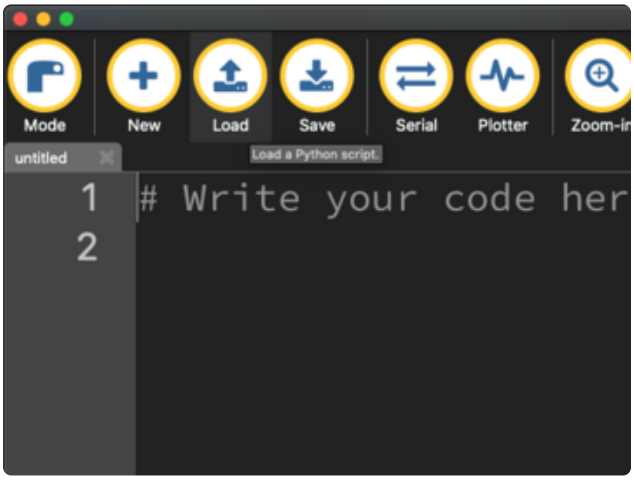
The little LED should now be blinking. Once per half-second.

Congratulations, you've just run your first CircuitPython program!

Text emphasized with a blue exclamation:

On most boards you'll find a tiny red LED. On the ItsyBitsy nRF52840, you'll find a tiny blue LED. On QT Py M0, QT Py RP2040, Qualia, and the Trinkey series, you will find only an RGB NeoPixel LED.

Editing Code



To edit code, open the **code.py** file on your **CIRCUITPY** drive into your editor.

Make the desired changes to your code. Save the file. That's it!

Your code changes are run as soon as the file is done saving.

There's one warning before you continue...

Text emphasized with a red exclamation:

Don't click reset or unplug your board!

The CircuitPython code on your board detects when the files are changed or written and will automatically re-start your code. This makes coding very fast because you save, and it re-runs. If you unplug or reset the board before your computer finishes writing the file to your board, you can corrupt the drive. If this happens, you may lose the code you've written, so it's important to backup your code to your computer regularly.

There are a couple of ways to avoid filesystem corruption.

1. Use an editor that writes out the file completely when you save it.

Check out the Recommended Editors page () for details on different editing options.

Text emphasized with a blue exclamation:	If you are dragging a file from your host computer onto the CIRCUITPY drive, you still need to do step 2. Eject or Sync (below) to make sure the file is completely written.

2. Eject or Sync the Drive After Writing

If you are using one of our not-recommended-editors, not all is lost! You can still make it work.

On Windows, you can Eject or Safe Remove the **CIRCUITPY** drive. It won't actually eject, but it will force the operating system to save your file to disk. On Linux, use the **sync** command in a terminal to force the write to disk.

You also need to do this if you use Windows Explorer or a Linux graphical file manager to drag a file onto **CIRCUITPY**.

Oh No I Did Something Wrong and Now The CIRCUITPY Drive Doesn't Show Up!!!

Don't worry! Corrupting the drive isn't the end of the world (or your board!). If this happens, follow the steps found on the Troubleshooting () page of every board guide to get your board up and running again.

Text emphasized with a blue exclamation:

If you are having trouble saving code on Windows 10, try including this code snippet at the top of code.py:

```
import supervisor
supervisor.runtime.autoreload = False
```

Back to Editing Code...

Now! Let's try editing the program you added to your board. Open your **code.py** file into your editor. You'll make a simple change. Change the first `0.5` to `0.1`. The code should look like this:

```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    led.value = True
    time.sleep(0.1)
    led.value = False
    time.sleep(0.5)
```

Leave the rest of the code as-is. Save your file. See what happens to the LED on your board? Something changed! Do you know why?

You don't have to stop there! Let's keep going. Change the second `0.5` to `0.1` so it looks like this:

```
while True:
    led.value = True
    time.sleep(0.1)
    led.value = False
    time.sleep(0.1)
```

Now it blinks really fast! You decreased the both time that the code leaves the LED on and off!

Now try increasing both of the `0.1` to `1`. Your LED will blink much more slowly because you've increased the amount of time that the LED is turned on and off.

Well done! You're doing great! You're ready to start into new examples and edit them to see what happens! These were simple changes, but major changes are done using the same process. Make your desired change, save it, and get the results. That's really all there is to it!

Naming Your Program File

CircuitPython looks for a code file on the board to run. There are four options: **code.txt**, **code.py**, **main.txt** and **main.py**. CircuitPython looks for those files, in that order, and then runs the first one it finds. While **code.py** is the recommended name for your code file, it is important to know that the other options exist. If your program doesn't seem to be updating as you work, make sure you haven't created another code file that's being read instead of the one you're working on.

Exploring Your First CircuitPython Program

First, you'll take a look at the code you're editing.

Here is the original code again for the LED blink example (if your board doesn't have a single-color LED to blink, look instead at the NeoPixel blink example):

```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

Imports & Libraries

Each CircuitPython program you run needs to have a lot of information to work. The reason CircuitPython is so simple to use is that most of that information is stored in other files and works in the background. The files built into CircuitPython are called

modules, and the files you load separately are called **libraries**. Modules are built into CircuitPython. Libraries are stored on your **CIRCUITPY** drive in a folder called **lib**.

```
import board
import digitalio
import time
```

The `import` statements tells the board that you're going to use a particular library or module in your code. In this example, you imported three modules: `board`, `digitalio`, and `time`. All three of these modules are built into CircuitPython, so no separate library files are needed. That's one of the things that makes this an excellent first example. You don't need anything extra to make it work!

These three modules each have a purpose. The first one, `board`, gives you access to the hardware on your board. The second, `digitalio`, lets you access that hardware as inputs/outputs. The third, `time`, let's you control the flow of your code in multiple ways, including passing time by 'sleeping'.

Setting Up The LED

The next two lines setup the code to use the LED.

```
led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT
```

Your board knows the red LED as `LED`. So, you initialise that pin, and you set it to output. You set `led` to equal the rest of that information so you don't have to type it all out again later in our code.

Loop-de-loops

The third section starts with a `while` statement. `while True:` essentially means, "forever do the following:". `while True:` creates a loop. Code will loop "while" the condition is "true" (vs. false), and as `True` is never False, the code will loop forever. All code that is indented under `while True:` is "inside" the loop.

Inside our loop, you have four items:

```
while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

First, you have `led.value = True`. This line tells the LED to turn on. On the next line, you have `time.sleep(0.5)`. This line is telling CircuitPython to pause running code for 0.5 seconds. Since this is between turning the led on and off, the led will be on for 0.5 seconds.

The next two lines are similar. `led.value = False` tells the LED to turn off, and `time.sleep(0.5)` tells CircuitPython to pause for another 0.5 seconds. This occurs between turning the led off and back on so the LED will be off for 0.5 seconds too.

Then the loop will begin again, and continue to do so as long as the code is running!

So, when you changed the first `0.5` to `0.1`, you decreased the amount of time that the code leaves the LED on. So it blinks on really quickly before turning off!

Great job! You've edited code in a CircuitPython program!

What Happens When My Code Finishes Running?

When your code finishes running, CircuitPython resets your microcontroller board to prepare it for the next run of code. That means any set up you did earlier no longer applies, and the pin states are reset.

For example, try reducing the code snippet above by eliminating the loop entirely, and replacing it with `led.value = True`. The LED will flash almost too quickly to see, and turn off. This is because the code finishes running and resets the pin state, and the LED is no longer receiving a signal.

To that end, most CircuitPython programs involve some kind of loop, infinite or otherwise.

What if I Don't Have the Loop?

If you don't have the loop, the code will run to the end and exit. This can lead to some unexpected behavior in simple programs like this since the "exit" also resets the state of the hardware. This is a different behavior than running commands via REPL. So if you are writing a simple program that doesn't seem to work, you may need to add a loop to the end so the program doesn't exit.

The simplest loop would be:

```
1 while True:
2     pass
```

And remember - you can press CTRL+C to exit the loop.

See also the Behavior section in the docs ().

Connecting to the Serial Console

One of the staples of CircuitPython (and programming in general!) is something called a "print statement". This is a line you include in your code that causes your code to output text. A print statement in CircuitPython (and Python) looks like this:

```
print("Hello, world!")
```

This line in your code.py would result in:

```
Hello, world!
```

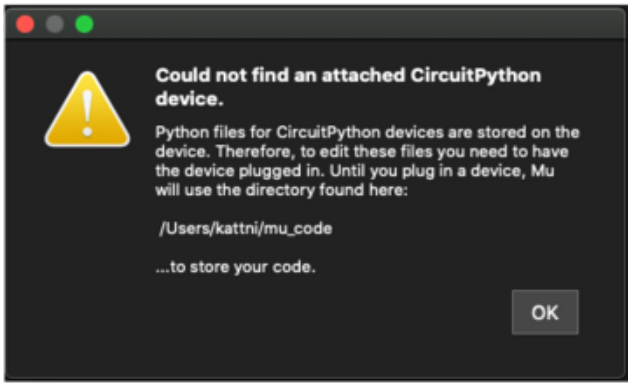
However, these print statements need somewhere to display. That's where the serial console comes in!

The serial console receives output from your CircuitPython board sent over USB and displays it so you can see it. This is necessary when you've included a print statement in your code and you'd like to see what you printed. It is also helpful for troubleshooting errors, because your board will send errors and the serial console will display those too.

The serial console requires an editor that has a built in terminal, or a separate terminal program. A terminal is a program that gives you a text-based interface to perform various tasks.

Are you using Mu?

If so, good news! The serial console **is built into Mu** and will **autodetect your board** making using the serial console really really easy.



First, make sure your CircuitPython board is plugged in.

If you open Mu without a board plugged in, you may encounter the error seen here, letting you know no CircuitPython board was found and indicating where your code will be stored until you plug in a board.

[If you are using Windows 7, make sure you installed the drivers ()](#).

Once you've opened Mu with your board plugged in, look for the **Serial** button in the button bar and click it.

The Mu window will split in two, horizontally, and display the serial console at the bottom.

Text emphasized with a blue exclamation:	If nothing appears in the serial console, it may mean your code is done running or has no print statements in it. Click into the serial console part of Mu, and press CTRL+D to reload.

Serial Console Issues or Delays on Linux

If you're on Linux, and are seeing multi-second delays connecting to the serial console, or are seeing "AT" and other gibberish when you connect, then the `modemmanager` service might be interfering. Just remove it; it doesn't have much use unless you're still using dial-up modems.

To remove `modemmanager`, type the following command at a shell:

```
sudo apt purge modemmanager
```

Setting Permissions on Linux

On Linux, if you see an error box something like the one below when you press the **Serial** button, you need to add yourself to a user group to have permission to connect to the serial console.

On Ubuntu and Debian, add yourself to the **dialout** group by doing:

```
sudo adduser $USER dialout
```

After running the command above, reboot your machine to gain access to the group. On other Linux distributions, the group you need may be different. See the Advanced Serial Console on Linux () for details on how to add yourself to the right group.

Using Something Else?

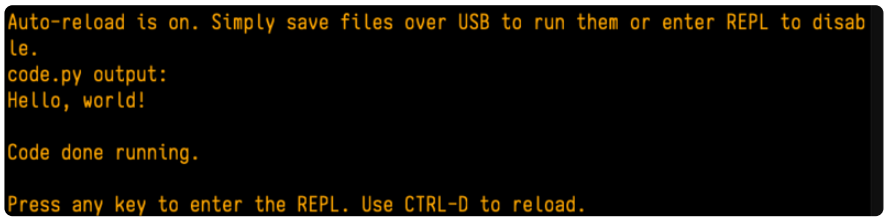
If you're not using Mu to edit, or if for some reason you are not a fan of its built in serial console, you can run the serial console from a separate program.

Windows requires you to download a terminal program. Check out the Advanced Serial Console on Windows page for more details. ()

MacOS has serial connection programs you can run in Terminal. Check the Advanced Serial Console on Mac page for more details. ()

Linux has multiple terminal programs included options are available for download. Check the Advanced Serial Console on Linux page for more details. ()

Once connected, you'll see something like the following.



Interacting with the Serial Console

Once you've successfully connected to the serial console, it's time to start using it.

The code you wrote earlier has no output to the serial console. So, you're going to edit it to create some output.

Open your code.py file into your editor, and include a `print` statement. You can print anything you like! Just include your phrase between the quotation marks inside the parentheses. For example:

```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    print("Hello, CircuitPython!")
    led.value = True
    time.sleep(1)
    led.value = False
    time.sleep(1)
```

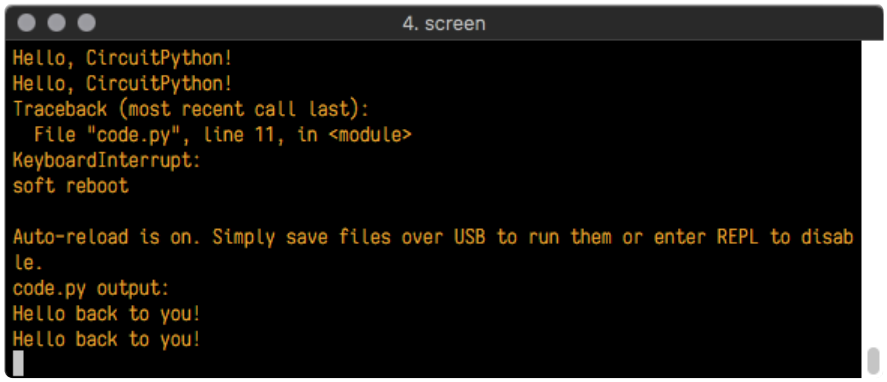
Save your file.

Now, let's go take a look at the window with our connection to the serial console.

Excellent! Our print statement is showing up in our console! Try changing the printed text to something else.

```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    print("Hello back to you!")
    led.value = True
    time.sleep(1)
    led.value = False
    time.sleep(1)
```

Keep your serial console window where you can see it. Save your file. You'll see what the serial console displays when the board reboots. Then you'll see your new change!



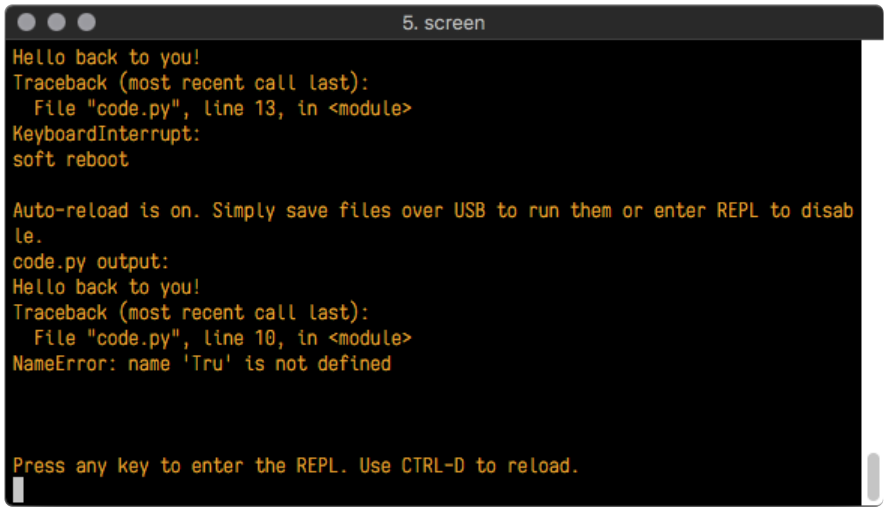
The `Traceback (most recent call last):` is telling you the last thing your board was doing before you saved your file. This is normal behavior and will happen every time the board resets. This is really handy for troubleshooting. Let's introduce an error so you can see how it is used.

Delete the `e` at the end of `True` from the line `led.value = True` so that it says `led.value = Tru`

```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    print("Hello back to you!")
    led.value = Tru
    time.sleep(1)
    led.value = False
    time.sleep(1)
```

Save your file. You will notice that your red LED will stop blinking, and you may have a colored status LED blinking at you. This is because the code is no longer correct and can no longer run properly. You need to fix it!

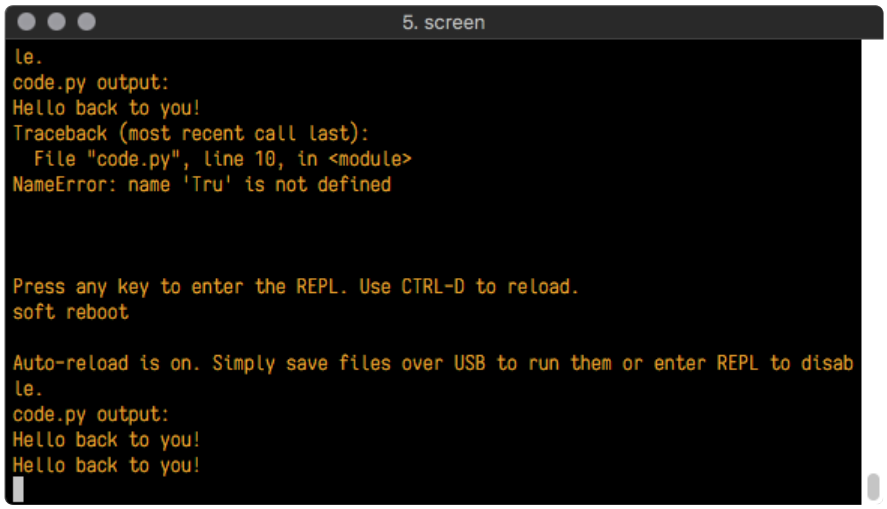
Usually when you run into errors, it's not because you introduced them on purpose. You may have 200 lines of code, and have no idea where your error could be hiding. This is where the serial console can help. Let's take a look!



The `Traceback (most recent call last):` is telling you that the last thing it was able to run was `line 10` in your code. The next line is your error: `NameError: name 'Tru' is not defined`. This error might not mean a lot to you, but combined with knowing the issue is on line 10, it gives you a great place to start!

Go back to your code, and take a look at line 10. Obviously, you know what the problem is already. But if you didn't, you'd want to look at line 10 and see if you could

figure it out. If you're still unsure, try googling the error to get some help. In this case, you know what to look for. You spelled True wrong. Fix the typo and save your file.



Nice job fixing the error! Your serial console is streaming and your red LED Is blinking again.

The serial console will display any output generated by your code. Some sensors, such as a humidity sensor or a thermistor, receive data and you can use print statements to display that information. You can also use print statements for troubleshooting, which is called "print debugging". Essentially, if your code isn't working, and you want to know where it's failing, you can put print statements in various places to see where it stops printing.

The serial console has many uses, and is an amazing tool overall for learning and programming!

The REPL

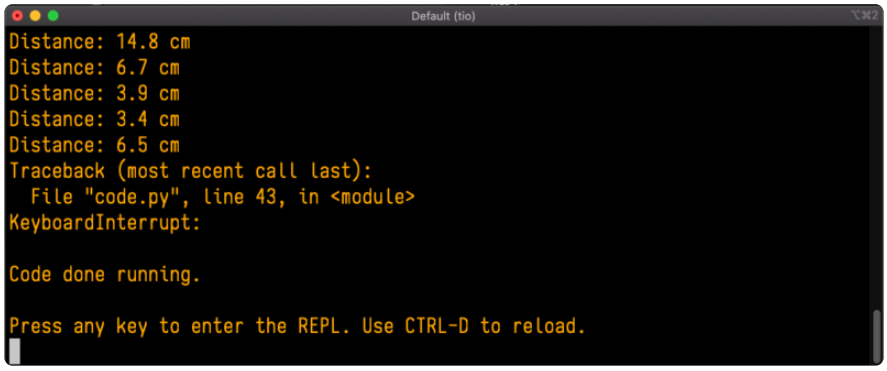
The other feature of the serial connection is the **R**ead-**E**valuate-**P**rint-**L**oop, or REPL. The REPL allows you to enter individual lines of code and have them run immediately. It's really handy if you're running into trouble with a particular program and can't figure out why. It's interactive so it's great for testing new ideas.

Entering the REPL

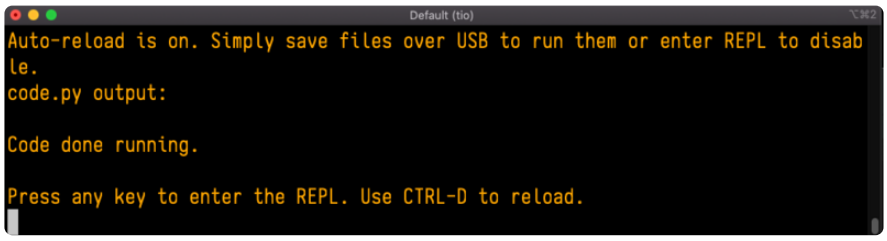
To use the REPL, you first need to be connected to the serial console. Once that connection has been established, you'll want to press **CTRL+C**.

If there is code running, in this case code measuring distance, it will stop and you'll see `Press any key to enter the REPL. Use CTRL-D to reload.` Follow those instructions, and press any key on your keyboard.

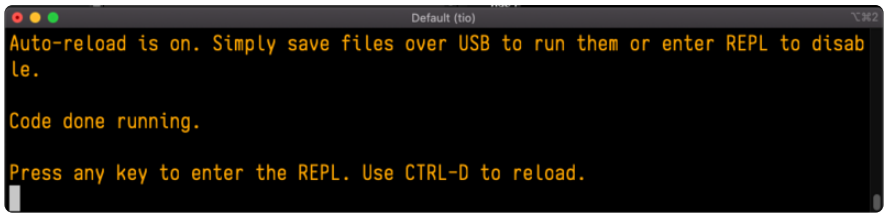
The `Traceback (most recent call last):` is telling you the last thing your board was doing before you pressed Ctrl + C and interrupted it. The `KeyboardInterrupt` is you pressing CTRL+C. This information can be handy when troubleshooting, but for now, don't worry about it. Just note that it is expected behavior.



If your **code.py** file is empty or does not contain a loop, it will show an empty output and `Code done running.`. There is no information about what your board was doing before you interrupted it because there is no code running.



If you have no **code.py** on your **CIRCUITPY** drive, you will enter the REPL immediately after pressing CTRL+C. Again, there is no information about what your board was doing before you interrupted it because there is no code running.



Regardless, once you press a key you'll see a `>>>` prompt welcoming you to the REPL!

If you have trouble getting to the `>>>` prompt, try pressing Ctrl + C a few more times.

The first thing you get from the REPL is information about your board.

```
Adafruit CircuitPython 7.0.0 on 2021-10-26; Adafruit Feather RP2040 with rp2040
```

This line tells you the version of CircuitPython you're using and when it was released. Next, it gives you the type of board you're using and the type of microcontroller the board uses. Each part of this may be different for your board depending on the versions you're working with.

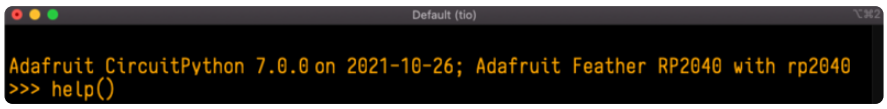
This is followed by the CircuitPython prompt.

```
>>>
```

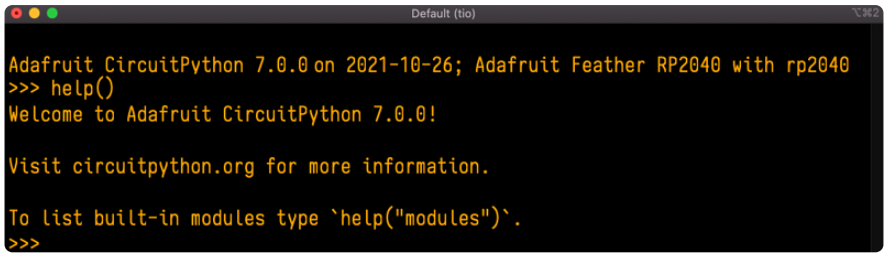
Interacting with the REPL

From this prompt you can run all sorts of commands and code. The first thing you'll do is run `help()`. This will tell you where to start exploring the REPL. To run code in the REPL, type it in next to the REPL prompt.

Type `help()` next to the prompt in the REPL.



Then press enter. You should then see a message.



First part of the message is another reference to the version of CircuitPython you're using. Second, a URL for the CircuitPython related project guides. Then... wait. What's this? To list built-in modules type `help("modules").` Remember the modules you learned about while going through creating code? That's exactly what this is talking about! This is a perfect place to start. Let's take a look!

Type `help("modules")` into the REPL next to the prompt, and press enter.

This is a list of all the core modules built into CircuitPython, including `board`. Remember, `board` contains all of the pins on the board that you can use in your code. From the REPL, you are able to see that list!

Type `import board` into the REPL and press enter. It'll go to a new prompt. It might look like nothing happened, but that's not the case! If you recall, the `import` statement simply tells the code to expect to do something with that module. In this case, it's telling the REPL that you plan to do something with that module.

Next, type `dir(board)` into the REPL and press enter.

This is a list of all of the pins on your board that are available for you to use in your code. Each board's list will differ slightly depending on the number of pins available. Do you see `LED`? That's the pin you used to blink the red LED!

The REPL can also be used to run code. Be aware that **any code you enter into the REPL isn't saved** anywhere. If you're testing something new that you'd like to keep, make sure you have it saved somewhere on your computer as well!

Every programmer in every programming language starts with a piece of code that says, "Hello, World." You're going to say hello to something else. Type into the REPL:

```
print("Hello, CircuitPython!")
```

Then press enter.

That's all there is to running code in the REPL! Nice job!

You can write single lines of code that run stand-alone. You can also write entire programs into the REPL to test them. Remember that nothing typed into the REPL is saved.

There's a lot the REPL can do for you. It's great for testing new ideas if you want to see if a few new lines of code will work. It's fantastic for troubleshooting code by entering it one line at a time and finding out where it fails. It lets you see what modules are available and explore those modules.

Try typing more into the REPL to see what happens!

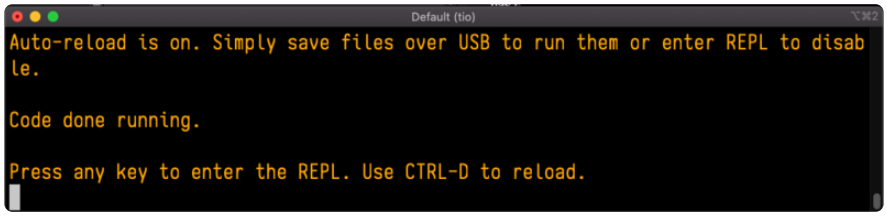
Text emphasized with a yellow exclamation:

Everything typed into the REPL is ephemeral. Once you reload the REPL or return to the serial console, nothing you typed will be retained in any memory space. So be sure to save any desired code you wrote somewhere else, or you'll lose it when you leave the current REPL instance!

Returning to the Serial Console

When you're ready to leave the REPL and return to the serial console, simply press **CTRL+D**. This will reload your board and reenter the serial console. You will restart the program you had running before entering the REPL. In the console window, you'll see any output from the program you had running. And if your program was affecting anything visual on the board, you'll see that start up again as well.

You can return to the REPL at any time!



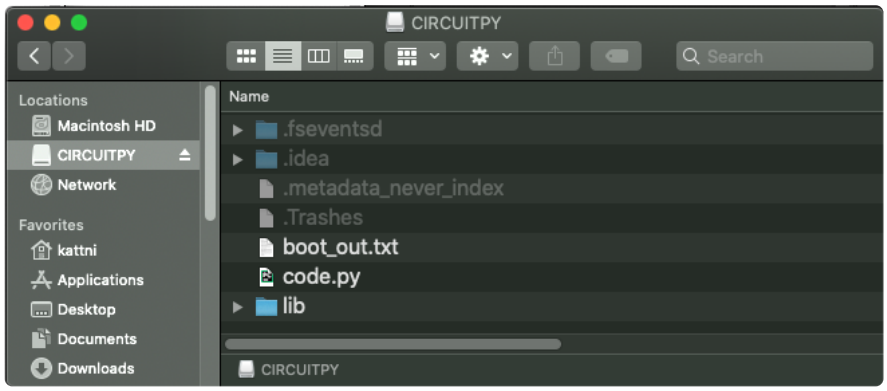
CircuitPython Libraries

Text emphasized with a red exclamation:

As CircuitPython development continues and there are new releases, Adafruit will stop supporting older releases. Visit https://circuitpython.org/downloads to download the latest version of CircuitPython for your board. You must download the CircuitPython Library Bundle that matches your version of CircuitPython. Please update CircuitPython and then visit https://circuitpython.org/libraries to download the latest Library Bundle.

Each CircuitPython program you run needs to have a lot of information to work. The reason CircuitPython is so simple to use is that most of that information is stored in other files and works in the background. These files are called libraries. Some of them are built into CircuitPython. Others are stored on your **CIRCUITPY** drive in a folder called **lib**. Part of what makes CircuitPython so great is its ability to store code separately from the firmware itself. Storing code separately from the firmware makes it easier to update both the code you write and the libraries you depend.

Your board may ship with a **lib** folder already, it's in the base directory of the drive. If not, simply create the folder yourself. When you first install CircuitPython, an empty **lib** directory will be created for you.



CircuitPython libraries work in the same way as regular Python modules so the Python docs () are an excellent reference for how it all should work. In Python terms, you can place our library files in the **lib** directory because it's part of the Python path by default.

One downside of this approach of separate libraries is that they are not built in. To use them, one needs to copy them to the **CIRCUITPY** drive before they can be used. Fortunately, there is a library bundle.

The bundle and the library releases on GitHub also feature optimized versions of the libraries with the **.mpy** file extension. These files take less space on the drive and have a smaller memory footprint as they are loaded.

Due to the regular updates and space constraints, Adafruit does not ship boards with the entire bundle. Therefore, you will need to load the libraries you need when you begin working with your board. You can find example code in the guides for your board that depends on external libraries.

Either way, as you start to explore CircuitPython, you'll want to know how to get libraries on board.

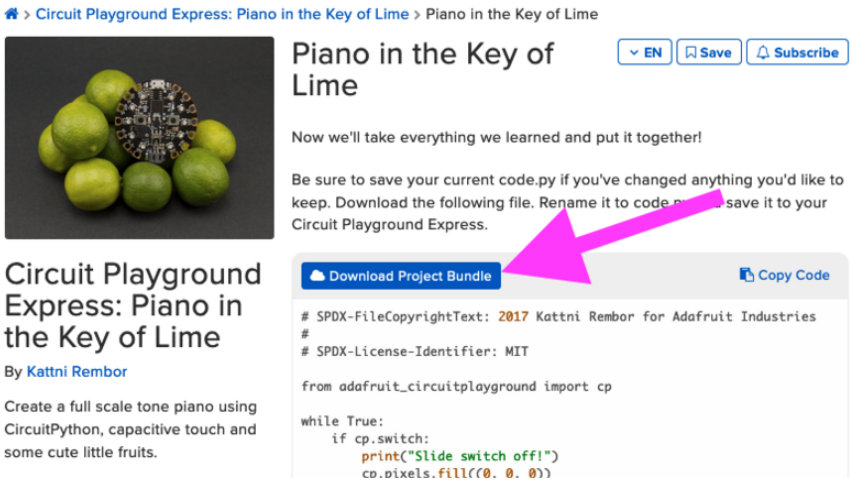
The Adafruit Learn Guide Project Bundle

The quickest and easiest way to get going with a project from the Adafruit Learn System is by utilising the Project Bundle. Most guides now have a **Download Project Bundle** button available at the top of the full code example embed. This button downloads all the necessary files, including images, etc., to get the guide project up and running. Simply click, open the resulting zip, copy over the right files, and you're good to go!

The first step is to find the Download Project Bundle button in the guide you're working on.

Text emphasized with a blue exclamation:

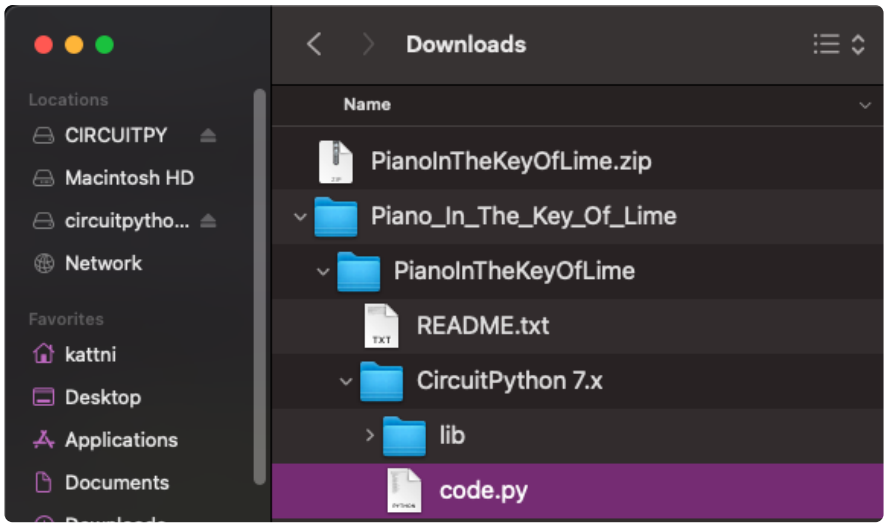
The Download Project Bundle button is only available on full demo code embedded from GitHub in a Learn guide. Code snippets will NOT have the button available.



Text emphasized with a yellow exclamation:

When you copy the contents of the Project Bundle to your CIRCUITPY drive, it will replace all the existing content! If you don't want to lose anything, ensure you copy your current code to your computer before you copy over the new Project Bundle content!

The Download Project Bundle button downloads a zip file. This zip contains a series of directories, nested within which is the **code.py**, any applicable assets like images or audio, and the **lib/** folder containing all the necessary libraries. The following zip was downloaded from the Piano in the Key of Lime guide.



Text emphasized with a blue exclamation:

The Piano in the Key of Lime guide was chosen as an example. That guide is specific to Circuit Playground Express, and cannot be used on all boards. Do not expect to download that exact bundle and have it work on your non-CPX microcontroller.

When you open the zip, you'll find some nested directories. Navigate through them until you find what you need. You'll eventually find a directory for your CircuitPython version (in this case, 7.x). In the version directory, you'll find the file and directory you need: **code.py** and **lib/**. Once you find the content you need, you can copy it all over to your **CIRCUITPY** drive, replacing any files already on the drive with the files from the freshly downloaded zip.

Text emphasized with a blue exclamation:

In some cases, there will be other files such as audio or images in the same directory as code.py and lib/. Make sure you include all the files when you copy things over!

Once you copy over all the relevant files, the project should begin running! If you find that the project is not running as expected, make sure you've copied ALL of the project files onto your microcontroller board.

That's all there is to using the Project Bundle!

The Adafruit CircuitPython Library Bundle

Adafruit provides CircuitPython libraries for much of the hardware they provide, including sensors, breakouts and more. To eliminate the need for searching for each library individually, the libraries are available together in the Adafruit CircuitPython Library Bundle. The bundle contains all the files needed to use each library.

Downloading the Adafruit CircuitPython Library Bundle

You can download the latest Adafruit CircuitPython Library Bundle release by clicking the button below. The libraries are being constantly updated and improved, so you'll always want to download the latest bundle.

Match up the bundle version with the version of CircuitPython you are running. For example, you would download the 6.x library bundle if you're running any version of CircuitPython 6, or the 7.x library bundle if you're running any version of CircuitPython 7, etc. If you mix libraries with major CircuitPython versions, you will get incompatible mpy errors due to changes in library interfaces possible during major version changes.

Download the bundle version that matches your CircuitPython firmware version. If you don't know the version, check the version info in **boot_out.txt** file on the **CIRCUITPY** drive, or the initial prompt in the CircuitPython REPL. For example, if you're running v7.0.0, download the 7.x library bundle.

There's also a **py** bundle which contains the uncompressed python files, you probably don't want that unless you are doing advanced work on libraries.

The CircuitPython Community Library Bundle

The CircuitPython Community Library Bundle is made up of libraries written and provided by members of the CircuitPython community. These libraries are often written when community members encountered hardware not supported in the Adafruit Bundle, or to support a personal project. The authors all chose to submit these libraries to the Community Bundle make them available to the community.

These libraries are maintained by their authors and are not supported by Adafruit. As you would with any library, if you run into problems, feel free to file an issue on the GitHub repo for the library. Bear in mind, though, that most of these libraries are supported by a single person and you should be patient about receiving a response. Remember, these folks are not paid by Adafruit, and are volunteering their personal time when possible to provide support.

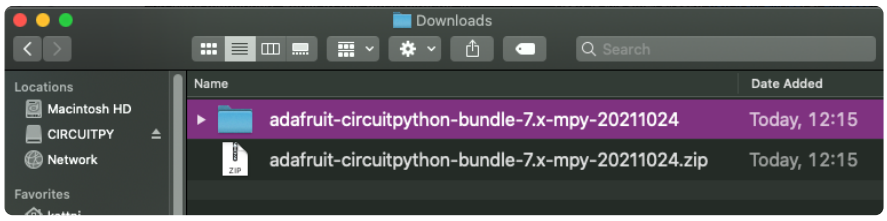
Downloading the CircuitPython Community Library Bundle

You can download the latest CircuitPython Community Library Bundle release by clicking the button below. The libraries are being constantly updated and improved, so you'll always want to download the latest bundle.

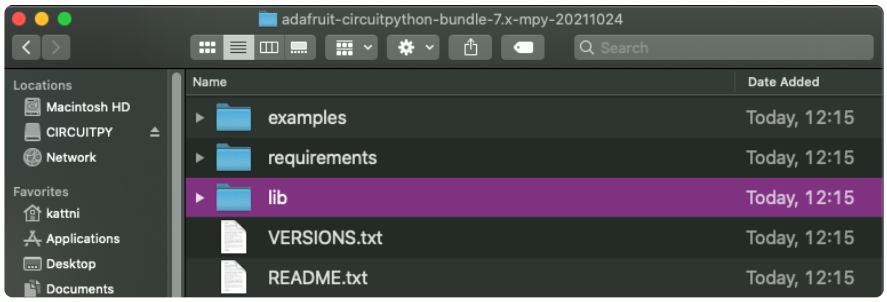
The link takes you to the latest release of the CircuitPython Community Library Bundle on GitHub. There are multiple versions of the bundle available. **Download the bundle version that matches your CircuitPython firmware version.** If you don't know the version, check the version info in **boot_out.txt** file on the **CIRCUITPY** drive, or the initial prompt in the CircuitPython REPL. For example, if you're running v7.0.0, download the 7.x library bundle.

Understanding the Bundle

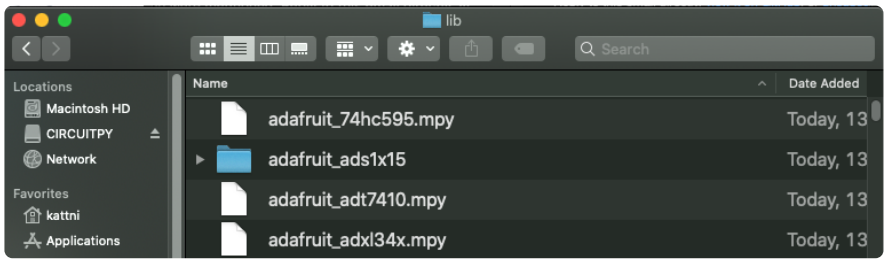
After downloading the zip, extract its contents. This is usually done by double clicking on the zip. On Mac OSX, it places the file in the same directory as the zip.



Open the bundle folder. Inside you'll find two information files, and two folders. One folder is the lib bundle, and the other folder is the examples bundle.



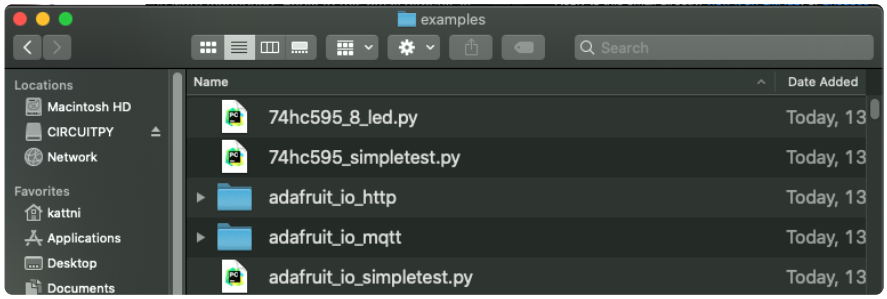
Now open the lib folder. When you open the folder, you'll see a large number of **.mpy** files, and folders.



Example Files

All example files from each library are now included in the bundles in an **examples** directory (as seen above), as well as an examples-only bundle. These are included for two main reasons:

- Allow for quick testing of devices.
- Provide an example base of code, that is easily built upon for individualized purposes.



Copying Libraries to Your Board

First open the **lib** folder on your **CIRCUITPY** drive. Then, open the **lib** folder you extracted from the downloaded zip. Inside you'll find a number of folders and **.mpy** files. Find the library you'd like to use, and copy it to the **lib** folder on **CIRCUITPY**.

If the library is a directory with multiple **.mpy** files in it, be sure to **copy the entire folder to CIRCUITPY/lib**.

This also applies to example files. Open the **examples** folder you extracted from the downloaded zip, and copy the applicable file to your **CIRCUITPY** drive. Then, rename it to **code.py** to run it.

!	Text emphasized with a blue exclamation:	If a library has multiple .mpy files contained in a folder, be sure to copy the entire folder to CIRCUITPY/lib.

Understanding Which Libraries to Install

You now know how to load libraries on to your CircuitPython-compatible microcontroller board. You may now be wondering, how do you know which libraries you need to install? Unfortunately, it's not always straightforward. Fortunately, there is an obvious place to start, and a relatively simple way to figure out the rest. First up: the best place to start.

When you look at most CircuitPython examples, you'll see they begin with one or more `import` statements. These typically look like the following:

- `import library_or_module`

However, `import` statements can also sometimes look like the following:

- `from library_or_module import name`
- `from library_or_module.subpackage import name`
- `from library_or_module import name as local_name`

They can also have more complicated formats, such as including a `try` / `except` block, etc.

The important thing to know is that **an `import` statement will always include the name of the module or library that you're importing**.

Therefore, the best place to start is by reading through the `import` statements.

Here is an example import list for you to work with in this section. There is no setup or other code shown here, as the purpose of this section involves only the import list.

```
import time
import board
import neopixel
import adafruit_lis3dh
import usb_hid
from adafruit_hid.consumer_control import ConsumerControl
from adafruit_hid.consumer_control_code import ConsumerControlCode
```

Keep in mind, not all imported items are libraries. Some of them are almost always built-in CircuitPython modules. How do you know the difference? Time to visit the REPL.

In the Interacting with the REPL section () on The REPL page () in this guide, the `help("modules")` command is discussed. This command provides a list of all of the built-in modules available in CircuitPython for your board. So, if you connect to the serial console on your board, and enter the REPL, you can run `help("modules")` to see what modules are available for your board. Then, as you read through the `import` statements, you can, for the purposes of figuring out which libraries to load, ignore the statement that import modules.

The following is the list of modules built into CircuitPython for the Feather RP2040. Your list may look similar or be anything down to a significant subset of this list for smaller boards.

```
>>> help("modules")
__main__          board           micropython     storage
_bleio            builtins        msgpack         struct
adafruit_bus_device               busio           neopixel_write  supervisor
adafruit_pixelbuf collections     onewireio       synthio
aesio             countio         os              sys
alarm             digitalio       paralleldisplay terminalio
analogio          displayio       pulseio         time
array             errno           pwmio           touchio
atexit            fontio          qrio            traceback
audiobusio        framebufferio   rainbowio       ulab
audiocore         gc              random          usb_cdc
audiomixer        getpass         re              usb_hid
audiomp3          imagecapture    rgbmatrix       usb_midi
audiopwmio        io              rotaryio        vectorio
binascii          json            rp2pio          watchdog
bitbangio         keypad          rtc
bitmaptools       math            sdcardio
bitops            microcontroller sharpdisplay
```

Now that you know what you're looking for, it's time to read through the import statements. The first two, `time` and `board`, are on the modules list above, so they're built-in.

The next one, `neopixel`, is not on the module list. That means it's your first library! So, you would head over to the bundle zip you downloaded, and search for **neopixel**. There is a **neopixel.mpy** file in the bundle zip. Copy it over to the **lib** folder on your **CIRCUITPY** drive. The following one, `adafruit_lis3dh`, is also not on the module list. Follow the same process for **adafruit_lis3dh**, where you'll find **adafruit_lis3dh.mpy**, and copy that over.

The fifth one is `usb_hid`, and it is in the modules list, so it is built in. Often all of the built-in modules come first in the import list, but sometimes they don't! Don't assume that everything after the first library is also a library, and verify each import with the modules list to be sure. Otherwise, you'll search the bundle and come up empty!

The final two imports are not as clear. Remember, when `import` statements are formatted like this, the first thing after the `from` is the library name. In this case, the library name is `adafruit_hid`. A search of the bundle will find an **adafruit_hid folder**. When a library is a folder, you must copy the **entire folder and its contents as it is in the bundle** to the **lib** folder on your **CIRCUITPY** drive. In this case, you would copy the entire **adafruit_hid** folder to your **CIRCUITPY/lib** folder.

Notice that there are two imports that begin with `adafruit_hid`. Sometimes you will need to import more than one thing from the same library. Regardless of how many times you import the same library, you only need to load the library by copying over the **adafruit_hid** folder once.

That is how you can use your example code to figure out what libraries to load on your CircuitPython-compatible board!

There are cases, however, where libraries require other libraries internally. The internally required library is called a dependency. In the event of library dependencies, the easiest way to figure out what other libraries are required is to connect to the serial console and follow along with the `ImportError` printed there. The following is a very simple example of an `ImportError`, but the concept is the same for any missing library.

Example: `ImportError` Due to Missing Library

If you choose to load libraries as you need them, or you're starting fresh with an existing example, you may end up with code that tries to use a library you haven't yet

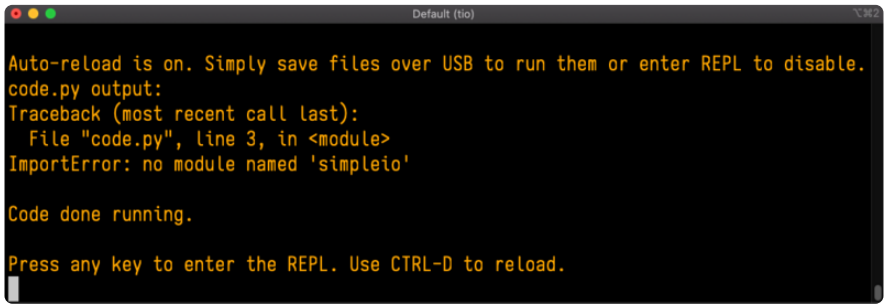
loaded. This section will demonstrate what happens when you try to utilise a library that you don't have loaded on your board, and cover the steps required to resolve the issue.

This demonstration will only return an error if you do not have the required library loaded into the **lib** folder on your **CIRCUITPY** drive.

Let's use a modified version of the Blink example.

```
import board
import time
import simpleio

led = simpleio.DigitalOut(board.LED)

while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

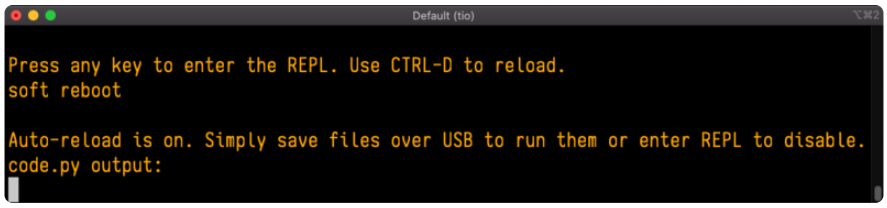
Save this file. Nothing happens to your board. Let's check the serial console to see what's going on.



You have an `ImportError`. It says there is `no module named 'simpleio'`. That's the one you just included in your code!

Click the link above to download the correct bundle. Extract the lib folder from the downloaded bundle file. Scroll down to find **simpleio.mpy**. This is the library file you're looking for! Follow the steps above to load an individual library file.

The LED starts blinking again! Let's check the serial console.



No errors! Excellent. You've successfully resolved an `ImportError`!

If you run into this error in the future, follow along with the steps above and choose the library that matches the one you're missing.

Library Install on Non-Express Boards

If you have an M0 non-Express board such as Trinket M0, Gemma M0, QT Py M0, or one of the M0 Trinkeys, you'll want to follow the same steps in the example above to install libraries as you need them. Remember, you don't need to wait for an `ImportError` if you know what library you added to your code. Open the library bundle you downloaded, find the library you need, and drag it to the **lib** folder on your **CIRCUITPY** drive.

You can still end up running out of space on your M0 non-Express board even if you only load libraries as you need them. There are a number of steps you can use to try to resolve this issue. You'll find suggestions on the Troubleshooting page ().

Updating CircuitPython Libraries and Examples

Libraries and examples are updated from time to time, and it's important to update the files you have on your **CIRCUITPY** drive.

To update a single library or example, follow the same steps above. When you drag the library file to your lib folder, it will ask if you want to replace it. Say yes. That's it!

A new library bundle is released every time there's an update to a library. Updates include things like bug fixes and new features. It's important to check in every so often to see if the libraries you're using have been updated.

CircUp CLI Tool

There is a command line interface (CLI) utility called CircUp () that can be used to easily install and update libraries on your device. Follow the directions on the install page within the CircUp learn guide (). Once you've got it installed you run the

command `circup update` in a terminal to interactively update all libraries on the connected CircuitPython device. See the usage page in the CircUp guide () for a full list of functionality

---

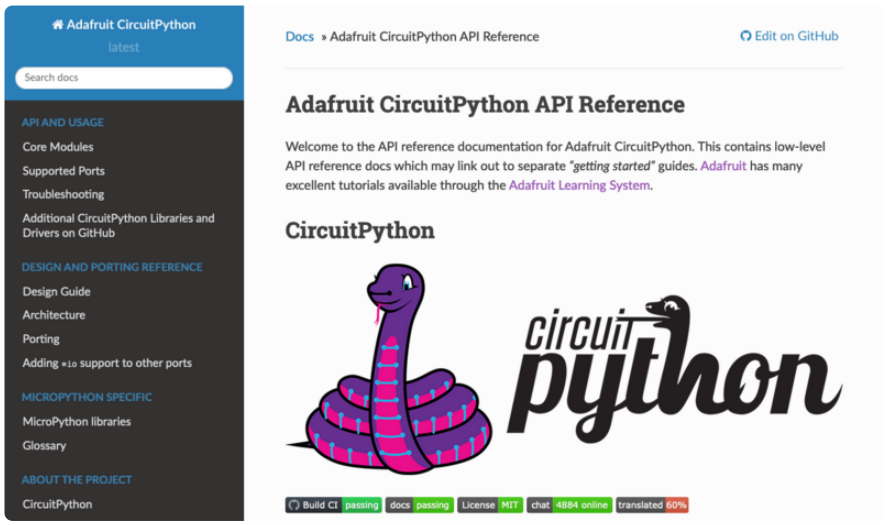
CircuitPython Documentation

You've learned about the CircuitPython built-in modules and external libraries. You know that you can find the modules in CircuitPython, and the libraries in the Library Bundles. There are guides available that explain the basics of many of the modules and libraries. However, there's sometimes more capabilities than are necessarily showcased in the guides, and often more to learn about a module or library. So, where can you find more detailed information? That's when you want to look at the API documentation.

The entire CircuitPython project comes with extensive documentation available on Read the Docs. This includes both the CircuitPython core () and the Adafruit CircuitPython libraries ().

CircuitPython Core Documentation

The CircuitPython core documentation () covers many of the details you might want to know about the CircuitPython core and related topics. It includes API and usage info, a design guide and information about porting CircuitPython to new boards, MicroPython info with relation to CircuitPython, and general information about the project.



The main page covers the basics including where to **download CircuitPython**, how to **contribute**, **differences from MicroPython**, information about the **project structure**, and a **full table of contents** for the rest of the documentation.

The list along the left side leads to more information about specific topics.

The first section is **API and Usage**. This is where you can find information about how to use individual built-in **core modules**, such as `time` and `digitalio`, details about the **supported ports**, suggestions for **troubleshooting**, and basic info and links to the **library bundles**. The **Core Modules** section also includes the **Support Matrix**, which is a table of which core modules are available on which boards.

The second section is **Design and Porting Reference**. It includes a **design guide**, **architecture** information, details on **porting**, and **adding module support** to other ports.

The third section is **MicroPython Specific**. It includes information on **MicroPython and related libraries**, and a **glossary** of terms.

The fourth and final section is **About the Project**. It includes further information including details on **building, testing, and debugging CircuitPython**, along with various other useful links including the **Adafruit Community Code of Conduct**.

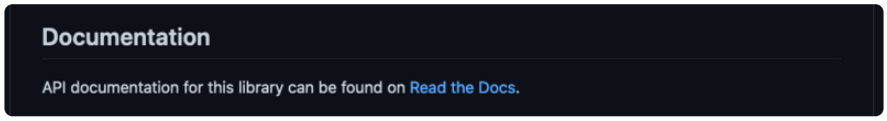
Whether you're a seasoned pro or new to electronics and programming, you'll find a wealth of information to help you along your CircuitPython journey in the documentation!

CircuitPython Library Documentation

The Adafruit CircuitPython libraries are documented in a very similar fashion. Each library has its own page on Read the Docs. There is a comprehensive list available here (). Otherwise, to view the documentation for a specific library, you can visit the GitHub repository for the library, and find the link in the README.

For the purposes of this page, the LED Animation library () documentation will be featured. There are two links to the documentation in each library GitHub repo. The first one is the **docs badge** near the top of the README.

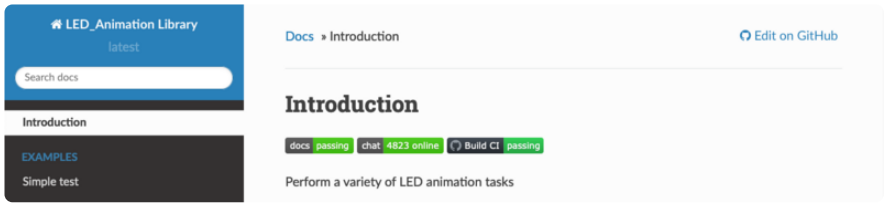
The second place is the **Documentation section** of the README. Scroll down to find it, and click on Read the Docs to get to the documentation.



Now that you know how to find it, it's time to take a look at what to expect.

Text emphasized with a blue exclamation:	Not all library documentation will look exactly the same, but this will give you some idea of what to expect from library docs.

The **Introduction** page is generated from the README, so it includes all the same info, such as PyPI installation instructions, a quick demo, and some build details. It also includes a full table of contents for the rest of the documentation (which is not part of the GitHub README). The page should look something like the following.



The left side contains links to the rest of the documentation, divided into three separate sections: **Examples**, **API Reference**, and **Other Links**.

Examples

The Examples section () is a list of library examples. This list contains anywhere from a small selection to the full list of the examples available for the library.

This section will always contain at least one example - the **simple test** example.

The simple test example is usually a basic example designed to show your setup is working. It may require other libraries to run. Keep in mind, it's simple - it won't showcase a comprehensive use of all the library features.

The LED Animation simple test demonstrates the Blink animation.

Simple test

Ensure your device works with this simple test.

examples/led_animation_simpletest.py

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT

"""
This simpletest example displays the Blink animation.

For NeoPixel FeatherWing. Update pixel_pin and pixel_num to match your wiring if using
a different form of NeoPixels.
"""
import board
import neopixel
from adafruit_led_animation.animation.blink import Blink
from adafruit_led_animation.color import RED

# Update to match the pin connected to your NeoPixels
pixel_pin = board.D6
# Update to match the number of NeoPixels you have connected
pixel_num = 32

pixels = neopixel.NeoPixel(pixel_pin, pixel_num, brightness=0.5, auto_write=False)

blink = Blink(pixels, speed=0.5, color=RED)

while True:
    blink.animate()
```

In some cases, you'll find a longer list, that may include examples that explore other features in the library. The LED Animation documentation includes a series of examples, all of which are available in the library. These examples include demonstrations of both basic and more complex features. Simply click on the example that interests you to view the associated code.

Text emphasized with a blue exclamation:

When there are multiple links in the Examples section, all of the example content is, in actuality, on the same page. Each link after the first is an anchor link to the specified section of the page. Therefore, you can also view all the available examples by scrolling down the page.

You can view the rest of the examples by clicking through the list or scrolling down the page. These examples are fully working code. Which is to say, while they may rely on other libraries as well as the library for which you are viewing the documentation, they should not require modification to otherwise work.

API Reference

The API Reference section () includes a list of the library functions and classes. The API (Application Programming Interface) of a library is the set of functions and classes the library provides. Essentially, the API defines how your program interfaces with the functions and classes that you call in your code to use the library.

There is always at least one list item included. Libraries for which the code is included in a single Python (**.py**) file, will only have one item. Libraries for which the code is multiple Python files in a directory (called subpackages) will have multiple items in this list. The LED Animation library has a series of subpackages, and therefore, multiple items in this list.

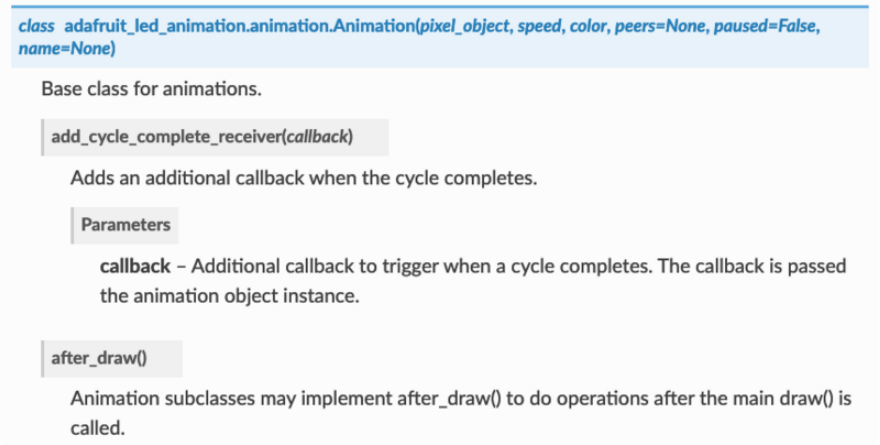
Click on the first item in the list to begin viewing the API Reference section.

Text emphasized with a blue exclamation:

As with the Examples section, all of the API Reference content is on a single page, and the links under API Reference are anchor links to the specified section of the page.

When you click on an item in the API Reference section, you'll find details about the classes and functions in the library. In the case of only one item in this section, all the available functionality of the library will be contained within that first and only subsection. However, in the case of a library that has subpackages, each item will contain the features of the particular subpackage indicated by the link. The documentation will cover all of the available functions of the library, including more complex ones that may not interest you.

The first list item is the animation subpackage. If you scroll down, you'll begin to see the available features of animation. They are listed alphabetically. Each of these things can be called in your code. It includes the name and a description of the specific function you would call, and if any parameters are necessary, lists those with a description as well.



class adafruit_led_animation.animation.Animation(*pixel_object*, *speed*, *color*, *peers=None*, *paused=False*, *name=None*)

Base class for animations.

add_cycle_complete_receiver(*callback*)

Adds an additional callback when the cycle completes.

Parameters

callback – Additional callback to trigger when a cycle completes. The callback is passed the animation object instance.

after_draw()

Animation subclasses may implement after_draw() to do operations after the main draw() is called.

You can view the other subpackages by clicking the link on the left or scrolling down the page. You may be interested in something a little more practical. Here is an example. To use the LED Animation library Comet animation, you would run the following example.

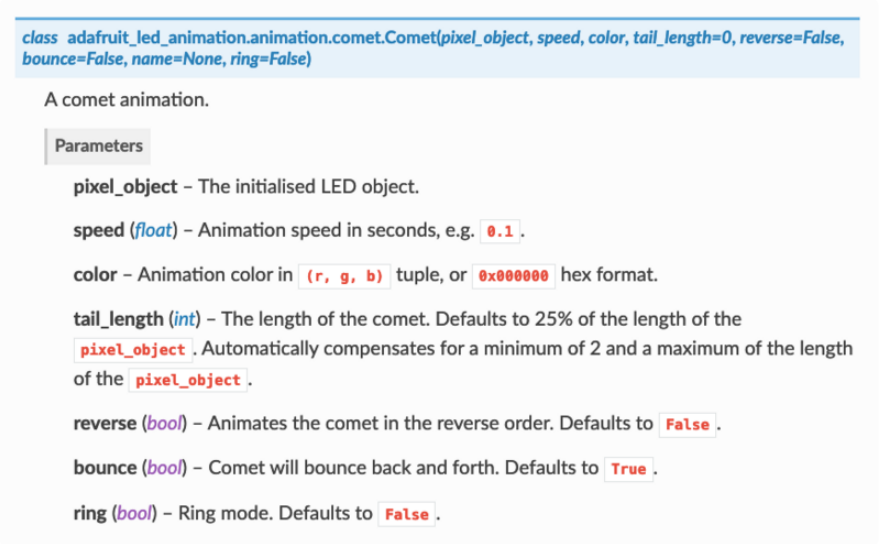
```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT

"""
This example animates a jade comet that bounces from end to end of the strip.

For QT Py Haxpress and a NeoPixel strip. Update pixel_pin and pixel_num to match your wiring if
using a different board or form of NeoPixels.

This example will run on SAMD21 (M0) Express boards (such as Circuit Playground Express or QT Py
Haxpress), but not on SAMD21 non-Express boards (such as QT Py or Trinket).
"""

import board
import neopixel
```

```
16
17  from adafruit_led_animation.animation.comet import Comet
18  from adafruit_led_animation.color import JADE
19
20  # Update to match the pin connected to your NeoPixels
21  pixel_pin = board.A3
22  # Update to match the number of NeoPixels you have connected
23  pixel_num = 30
24
25  pixels = neopixel.NeoPixel(pixel_pin, pixel_num, brightness=0.5, auto_write=False)
26
27  comet = Comet(pixels, speed=0.02, color=JADE, tail_length=10, bounce=True)
28
29  while True:
30      comet.animate()
```

https://github.com/adafruit/Adafruit_CircuitPython_LED_Animation/blob/main/examples/led_animation_comet.py

Note the line where you create the `comet` object. There are a number of items inside the parentheses. In this case, you're provided with a fully working example. But what if you want to change how the comet works? The code alone does not explain what the options mean.

So, in the API Reference documentation list, click the `adafruit_led_animation.animation.comet` link and scroll down a bit until you see the following.



class adafruit_led_animation.animation.comet.Comet(*pixel_object*, *speed*, *color*, *tail_length=0*, *reverse=False*, *bounce=False*, *name=None*, *ring=False*)

A comet animation.

Parameters

pixel_object – The initialised LED object.

speed (*float*) – Animation speed in seconds, e.g. `0.1`.

color – Animation color in `(r, g, b)` tuple, or `0x000000` hex format.

tail_length (*int*) – The length of the comet. Defaults to 25% of the length of the `pixel_object`. Automatically compensates for a minimum of 2 and a maximum of the length of the `pixel_object`.

reverse (*bool*) – Animates the comet in the reverse order. Defaults to `False`.

bounce (*bool*) – Comet will bounce back and forth. Defaults to `True`.

ring (*bool*) – Ring mode. Defaults to `False`.

Look familiar? It is! This is the documentation for setting up the comet object. It explains what each argument provided in the comet setup in the code meant, as well as the other available features. For example, the code includes `speed=0.02`. The documentation clarifies that this is the "Animation speed in seconds". The code

doesn't include `ring`. The documentation indicates this is an available setting that enables "Ring mode".

This type of information is available for any function you would set up in your code. If you need clarification on something, wonder whether there's more options available, or are simply interested in the details involved in the code you're writing, check out the documentation for the CircuitPython libraries!

Other Links

This section is the same for every library. It includes a list of links to external sites, which you can visit for more information about the CircuitPython Project and Adafruit.

That covers the CircuitPython library documentation! When you are ready to go beyond the basic library features covered in a guide, or you're interested in understanding those features better, the library documentation on Read the Docs has you covered!

Recommended Editors

The CircuitPython code on your board detects when the files are changed or written and will automatically re-start your code. This makes coding very fast because you save, and it re-runs.

However, you must wait until the file is done being saved before unplugging or resetting your board! On Windows using some editors this can sometimes take up to 90 seconds, on Linux it can take 30 seconds to complete because the text editor does not save the file completely. On macOS, saves get slower for every swap or backup file an editor writes to CIRCUITPY. See below () to move them off the drive.

This is really important to be aware of. If you unplug or reset the board before your computer finishes writing the file to your board, you can corrupt the drive. If this happens, you may lose the code you've written, so it's important to backup your code to your computer regularly.

To avoid the likelihood of filesystem corruption, use an editor that writes out the file completely when you save it. Check out the list of recommended editors below.

Recommended editors

- **mu** () is an editor that safely writes all changes (it's also our recommended editor!)

- **emacs** () is also an editor that will fully write files on save (). Set it up to keep backup and lock files off **CIRCUITPY**, as shown below ().
- Sublime Text **()** safely writes all changes
- Visual Studio Code **()** appears to safely write all changes
- **gedit** on Linux appears to safely write all changes
- IDLE (), in Python 3.8.1 or later, was fixed () to write all changes immediately
- Thonny () fully writes files on save
- Notepad++ () flushes files after writes, as of several years ago. In addition, you can change the path used for "Enable session snapshot and periodic backup" to write somewhere else than the CIRCUITPY drive. This will save space on CIRCUITPY and reduce writes to the drive.

Recommended only with particular settings or add-ons

- **vim** () / **vi** safely writes all changes. But set up **vim** to not write swapfiles () (.swp files: temporary records of your edits) to **CIRCUITPY**. Otherwise the swapfile writes trigger restarts of your program and make every save slower. Use the `.vimrc` settings below ().
- The PyCharm IDE **()** is safe if "Safe Write" is turned on in Settings->System Settings->Synchronization (true by default).
- If you are using **Atom** (), install the fsync-on-save package () or the language-circuitpython package () so that it will always write out all changes to files on **CIRCUITPY**.
- **SlickEdit** () works only if you add a macro to flush the disk ().

!	Text emphasized with a red exclamation:	The editors listed below are specifically NOT recommended!

Editors that are NOT recommended

- **notepad** (the default Windows editor) can be slow to write, so the editors above are recommended! If you are using notepad, be sure to eject the drive.
- **IDLE** in Python 3.8.0 or earlier does not force out changes immediately. Later versions do force out changes.
- **nano** (on Linux) does not force out changes.
- **geany** (on Linux) does not force out changes.

- **Anything else** - Other editors have not been tested so please use a recommended one!

Moving editor swap files off CIRCUITPY

On macOS, every swap, backup or lock file an editor writes to **CIRCUITPY** adds time to each save. Keep them on your computer instead.

vim

Make a swap folder in your home directory, then add these lines to `~/.vimrc`. Saving `code.py` went from 4.5 seconds to 1.1 seconds.

```
mkdir -p ~/.vim/swap
```

```
" keep swap files on the local disk, not next to the file (slow on CIRCUITPY)
set directory^=~/.vim/swap//
" CIRCUITPY: skip the temporary backup copy on save
autocmd BufReadPre,BufNewFile /Volumes/CIRCUITPY*/* setlocal nowritebackup
```

emacs

Make backup and auto-save folders in your home directory, then add these lines to `~/.emacs.d/init.el`. Saving went from 1.5 seconds to 1.1 seconds, with nothing extra written to **CIRCUITPY**.

```
mkdir -p ~/.emacs.d/backups ~/.emacs.d/auto-save
```

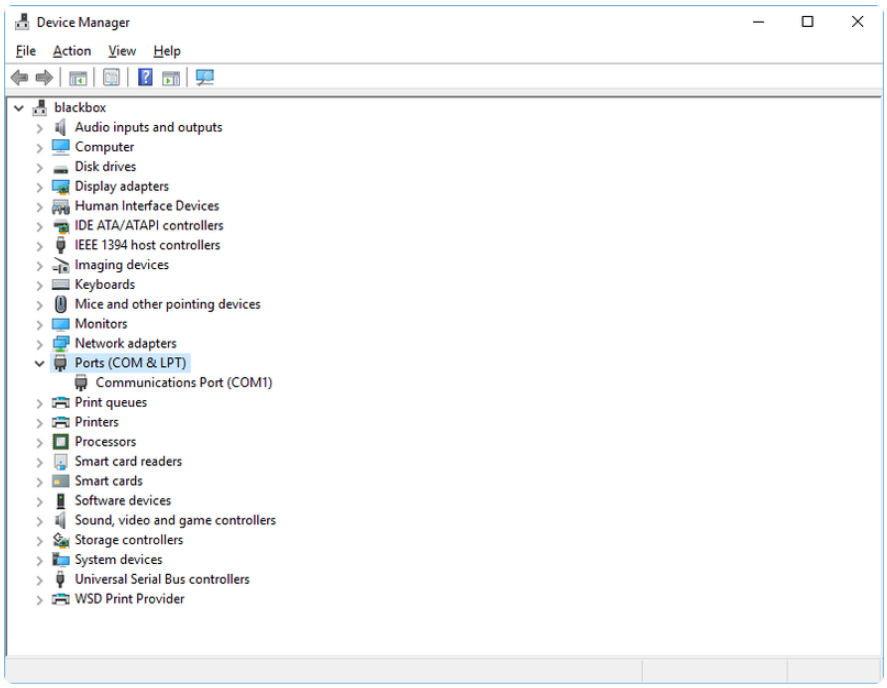
```
;; keep backup, auto-save and lock files off CIRCUITPY
(setq backup-directory-alist '(("." . "~/.emacs.d/backups/")))
(setq auto-save-file-name-transforms '((".*" "~/.emacs.d/auto-save/" t)))
(setq backup-by-copying t)
(setq create-lockfiles nil)
```

Advanced Serial Console on Windows

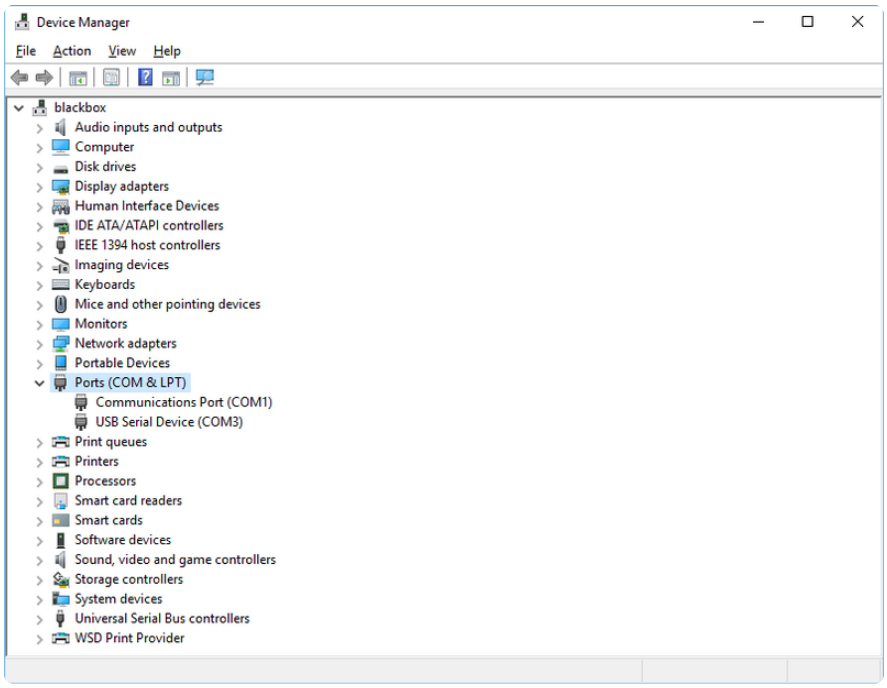
What's the COM?

First, you'll want to find out which serial port your board is using. When you plug your board in to USB on your computer, it connects to a serial port. The port is like a door through which your board can communicate with your computer using USB.

You'll use Windows Device Manager to determine which port the board is using. The easiest way to determine which port the board is using is to first check **without** the board plugged in. Open Device Manager. Click on Ports (COM & LPT). You should find something already in that list with (COM#) after it where # is a number.



Now plug in your board. The Device Manager list will refresh and a new item will appear under Ports (COM & LPT). You'll find a different (COM#) after this item in the list.



Sometimes the item will refer to the name of the board. Other times it may be called something like USB Serial Device, as seen in the image above. Either way, there is a new (COM#) following the name. This is the port your board is using.

Windows Serial Port Terminal Programs

- Putty is a venerable serial port connection program. More details are below.
- Tera Term () is a nice terminal program. It will reconnect automatically after disconnections
- VSCode has a number of serial port extensions, such as Serial Monitor ().
- PyCharm has a Serial Port Monitor () plugin.

Install Putty

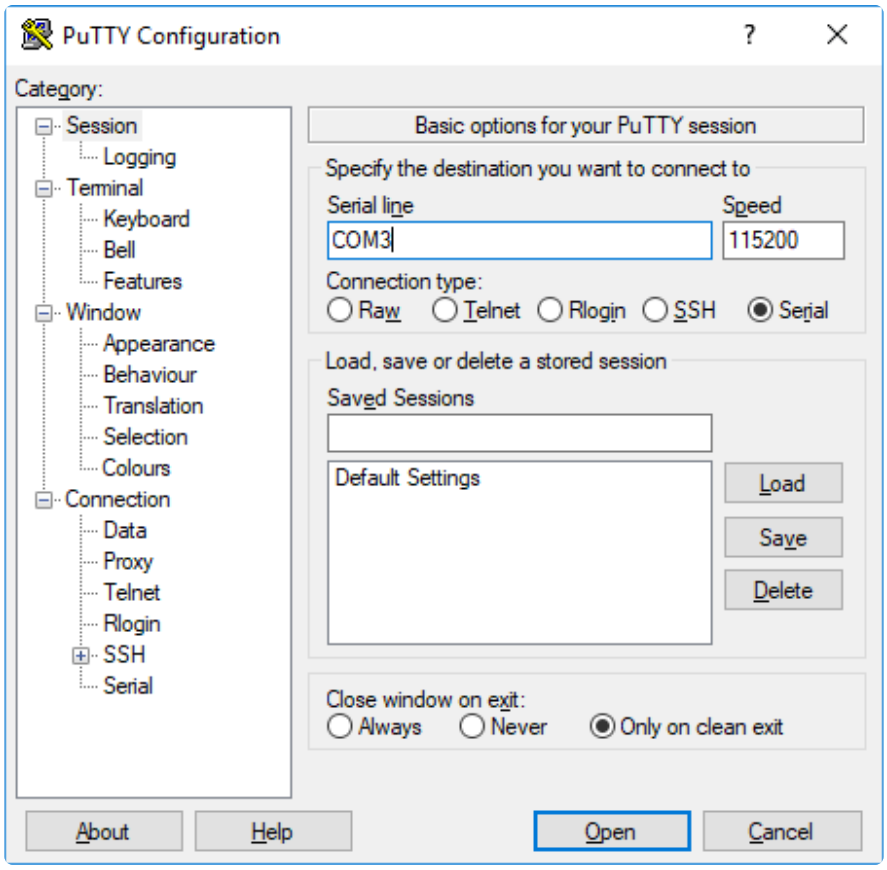
PuTTY is a well-known choice for connecting to serial ports on Windows.

The first thing to do is download the latest version of PuTTY (). You'll want to download the Windows installer file. It is most likely that you'll need the 64-bit version. Download the file and install the program on your machine. If you run into issues, you can try downloading the 32-bit version instead. However, the 64-bit version will work on most PCs.

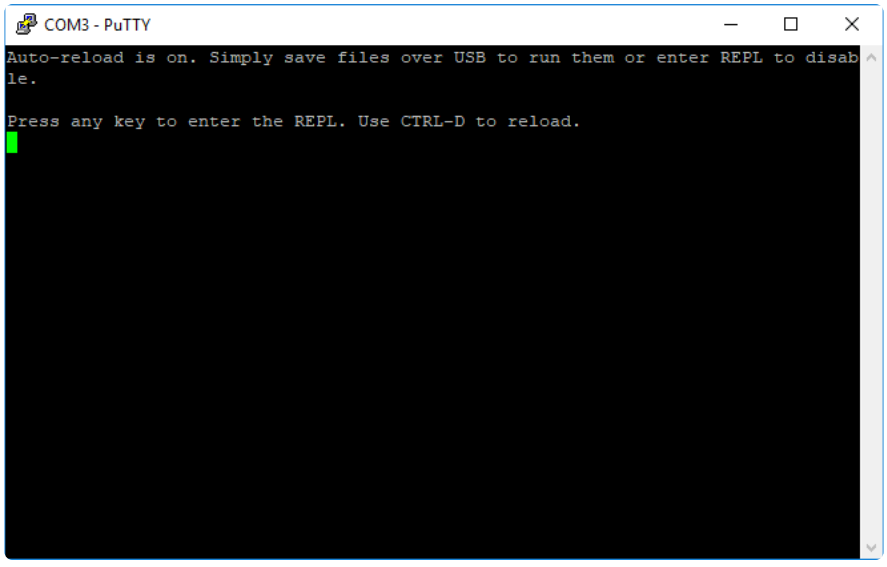
Now you need to open PuTTY.

- Under **Connection type:** choose the button next to **Serial**.
- In the box under **Serial line**, enter the serial port you found that your board is using.
- In the box under **Speed**, enter 115200. This called the baud rate, which is the speed in bits per second that data is sent over the serial connection. For boards with built in USB it doesn't matter so much but for ESP8266 and other board with a separate chip, the speed required by the board is 115200 bits per second. So you might as well just use 115200!

If you want to save those settings for later, use the options under **Load, save or delete a stored session.** Enter a name in the box under **Saved Sessions**, and click the **Save** button on the right.



Once your settings are entered, you're ready to connect to the serial console. Click "Open" at the bottom of the window. A new window will open.



If no code is running, the window will either be blank or will look like the window above. Now you're ready to see the results of your code.

Great job! You've connected to the serial console!

Windows 7 and 8.1

If you're using Windows 7 (or 8 or 8.1), you'll need to install drivers. See the Windows 7 and 8.1 Drivers page () for details. You will not need to install drivers on Mac, Linux or Windows 10 or 11.

Advanced Serial Console on Mac

Connecting to the serial console on Mac does not require installing any drivers or extra software. You'll use a terminal program to find your board, and `screen` to connect to it. Terminal and `screen` both come installed by default.

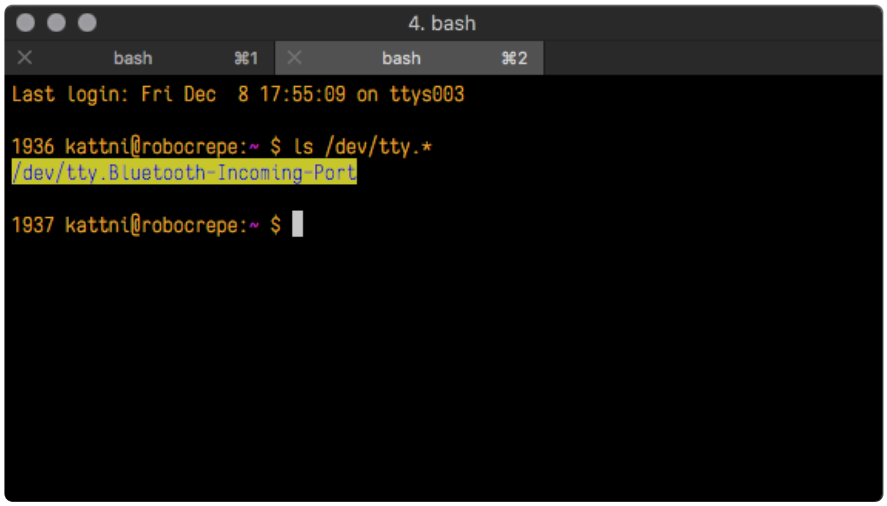
What's the Port?

First you'll want to find out which serial port your board is using. When you plug your board in to USB on your computer, it connects to a serial port. The port is like a door through which your board can communicate with your computer using USB.

The easiest way to determine which port the board is using is to first check **without** the board plugged in. Open Terminal and type the following:

```
ls /dev/tty.*
```

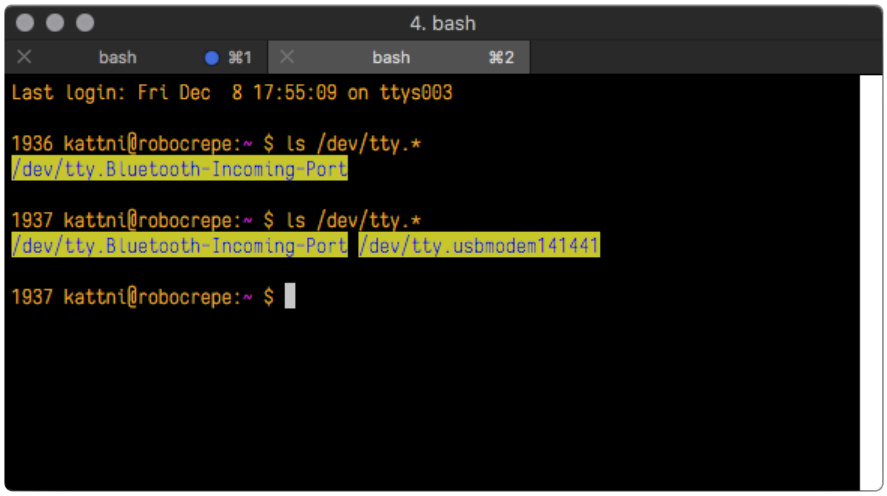
Each serial connection shows up in the `/dev/` directory. It has a name that starts with `tty.`. The command `ls` shows you a list of items in a directory. You can use `*` as a wildcard, to search for files that start with the same letters but end in something different. In this case, you're asking to see all of the listings in `/dev/` that start with `tty.` and end in anything. This will show us the current serial connections.



Now, plug your board. In Terminal, type:

```
ls /dev/tty.*
```

This will show you the current serial connections, which will now include your board.



A new listing has appeared called `/dev/tty.usbmodem141441`.
The `tty.usbmodem141441` part of this listing is the name the example board is using. Yours will be called something similar.

Text emphasized with a red exclamation:

Using the `screen` terminal program can cause your CircuitPython program to hang when trying to print, if you exit `screen` after you've used it to connect.

macOS Serial Port Terminal Programs

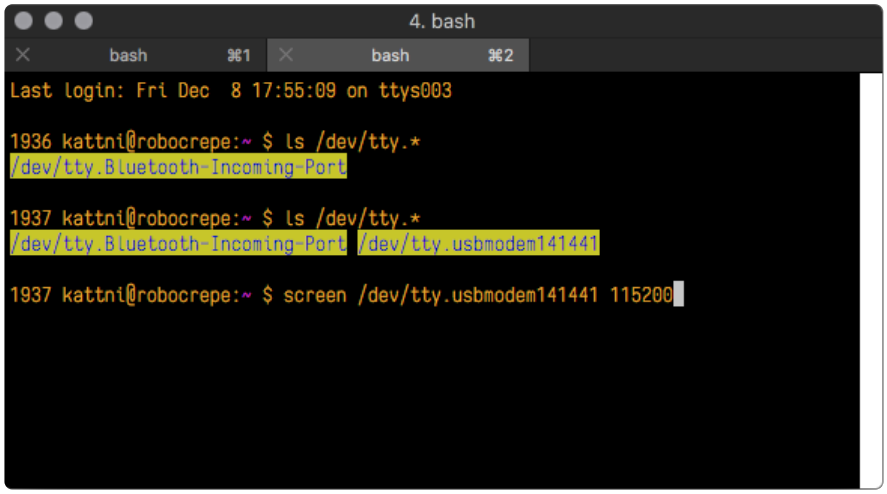
- `screen` is included with macOS. However, it's problematic because when it starts up, it enables the using DTR/RTS flow control signals and does not turn that off when it quits. This causes CircuitPython to block sending output when `screen` has exited, which will cause your program to stall until it is reconnected. See this issue () for a discussion.
- `tio ()` is a nice terminal program that works properly. You can install it with Homebrew ().
- VSCode has a number of serial port extensions, such as Serial Monitor ().
- PyCharm has a Serial Port Monitor () plugin.

Connect with screen

Despite the caveats above, if you can't download a better terminal program, you can use `screen`. The `screen` command is included with MacOS. To connect to the serial console, use Terminal. Type the following command, replacing `board_name` with the name you found your board is using:

```
screen /dev/tty.board_name 115200
```

The first part of this establishes using the `screen` command. The second part tells screen the name of the board you're trying to use. The third part tells screen what baud rate to use for the serial connection. The baud rate is the speed in bits per second that data is sent over the serial connection. In this case, the speed required by the board is 115200 bits per second.



Press enter to run the command. It will open in the same window. If no code is running, the window will be blank. Otherwise, you'll see the output of your code.

Great job! You've connected to the serial console!

Advanced Serial Console on Linux

Connecting to the serial console on Linux does not require installing any drivers, but you may need to install `screen` using your package manager. You'll use a terminal program to find your board, and `screen` to connect to it. There are a variety of terminal programs such as gnome-terminal (called Terminal) or Konsole on KDE.

The `tio` program works as well to connect to your board, and has the benefit of automatically reconnecting. You would need to install it using your package manager.

What's the Port?

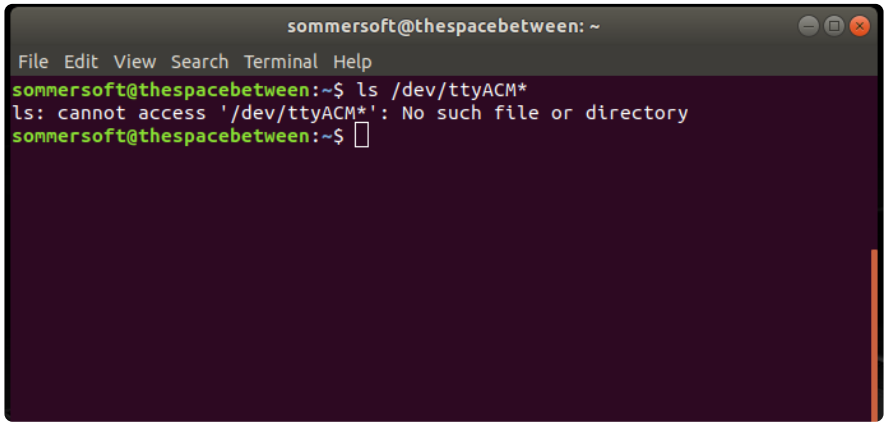
First you'll want to find out which serial port your board is using. When you plug your board in to USB on your computer, it connects to a serial port. The port is like a door through which your board can communicate with your computer using USB.

The easiest way to determine which port the board is using is to first check **without** the board plugged in. Open your terminal program and type the following:

```
ls /dev/ttyACM*
```

Each serial connection shows up in the **/dev/** directory. It has a name that starts with **ttyACM**. The command `ls` shows you a list of items in a directory. You can use `*` as a wildcard, to search for files that start with the same letters but end in something different. In this case, You're asking to see all of the listings in **/dev/** that start with **ttyACM** and end in anything. This will show us the current serial connections.

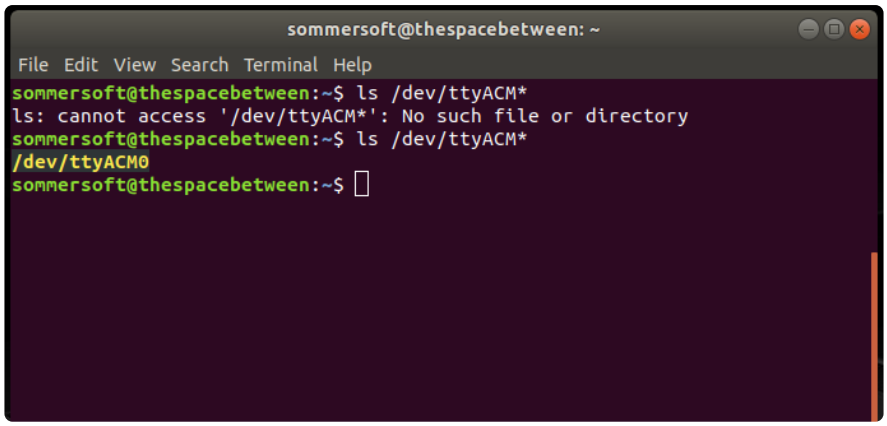
In the example below, the error is indicating that are no current serial connections starting with **ttyACM**.



Now plug in your board. In your terminal program, type:

```
ls /dev/ttyACM*
```

This will show you the current serial connections, which will now include your board.



A new listing has appeared called **/dev/ttyACM0**. The **ttyACM0** part of this listing is the name the example board is using. Yours will be called something similar.

Linux Serial Port Terminal Programs

- `tio ()` is a nice terminal program that works properly. You can install it using your package manager.
- VSCode has a number of serial port extensions, such as Serial Monitor ().
- PyCharm has a Serial Port Monitor () plugin.

Connect with `tio`

Now that you know the name your board is using, you're ready connect to the serial console. Install `tio` using your package manager..

ype the following command, replacing `ttyACMx` with the name you found your board is using:

```
tio /dev/ttyACMx
```

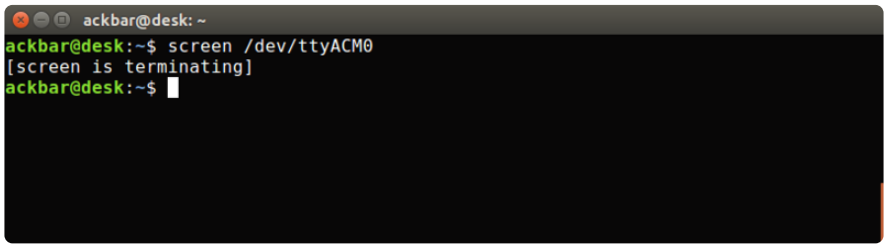
This will connect to the serial port and display a header like this:

```
$ tio /dev/ttyACM0
[11:07:00.578] tio v2.7
[11:07:00.578] Press ctrl-t q to quit
```

Permissions on Linux

If you try to run `tio` and it doesn't work, then you may be running into an issue with permissions. Your Linux distribution may not allow access to serial ports by defaut. You may see something like this; note the "permission denied".

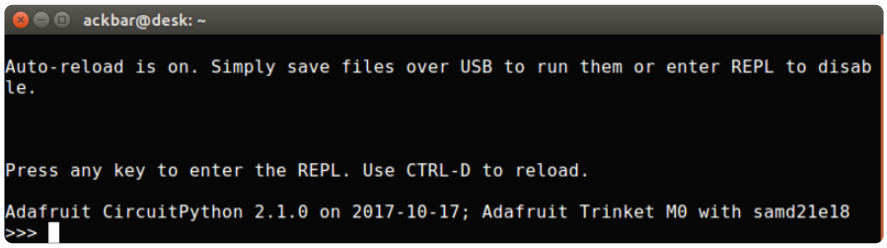
```
[11:13:42.754] tio v2.7
[11:13:42.754] Press ctrl-t q to quit
[11:13:42.754] Warning: Could not open tty device (Permission denied)
[11:13:42.754] Waiting for tty device..
```



then you may need to grant yourself access. There are generally two ways you can do this. The first is to just run `screen` using the `sudo` command, which temporarily gives you elevated privileges.

```
$ sudo tio /dev/ttyACM0
[sudo] password for smith:
```

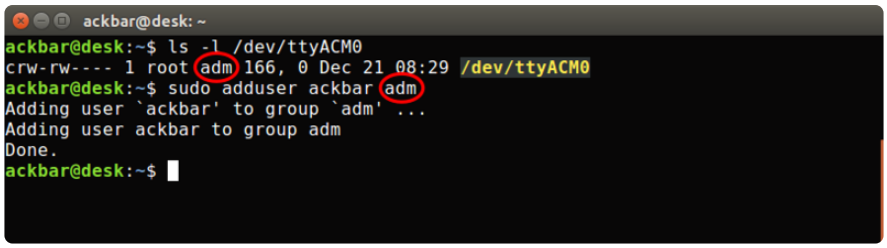
Once you enter your password, you should be in:



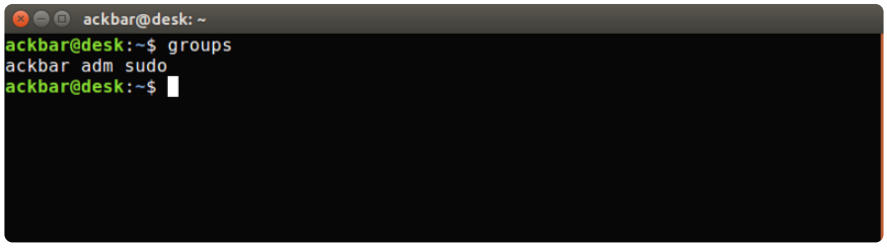
The second way is to add yourself to the user group associated with the hardware. To figure out what that group is, use the command `ls -l` as shown below. The group name is circled in red.

Then use the command `adduser` to add yourself to that group. You need elevated privileges to do this, so you'll need to use `sudo`. In the example below, the group is **adm** and the user is **ackbar**.

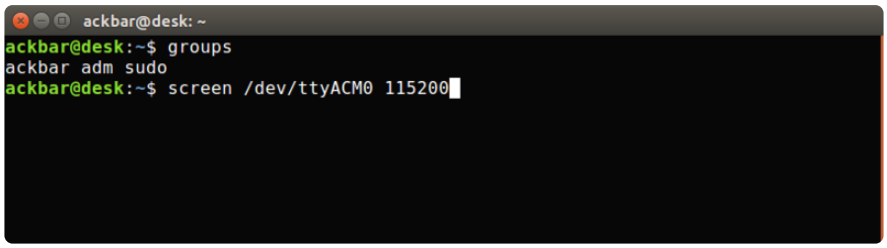
On Debian-based distributions, such as,Ubuntu the group will be **dialout**, not **adm**.



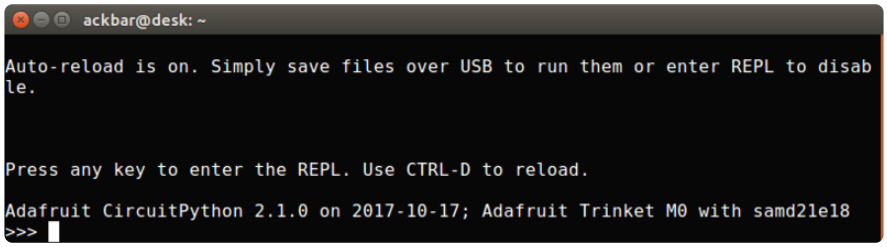
After you add yourself to the group, you'll need to logout and log back in, or in some cases, reboot your machine. After you log in again, verify that you have been added to the group using the command `groups`. If you are still not in the group, reboot and check again.



And now you should be able to run `screen` without using `sudo`.



And you're in:



Frequently Asked Questions

These are some of the common questions regarding CircuitPython and CircuitPython microcontrollers.

What are some common acronyms to know?

CP or CPy = CircuitPython ()
CPC = Circuit Playground Classic (http://adafru.it/3000) (does not run CircuitPython)
CPX = Circuit Playground Express (http://adafru.it/3333)
CPB = Circuit Playground Bluefruit (http://adafru.it/4333)

Using Older Versions

Text emphasized with a red exclamation:

As CircuitPython development continues and there are new releases, Adafruit will stop supporting older releases. Visit https://circuitpython.org/downloads to download the latest version of CircuitPython for your board. You must download the CircuitPython Library Bundle that matches your version of CircuitPython. Please update CircuitPython and then visit https://circuitpython.org/libraries to download the latest Library Bundle.

I have to continue using CircuitPython 8.x or earlier. Where can I find compatible libraries?

We are no longer building or supporting the CircuitPython 8.x or earlier library bundles. We highly encourage you to update CircuitPython to the latest version **() and use** the current version of the libraries **()**. However, if for some reason you cannot update, here are the last available library bundles for older versions:

- 2.x bundle ()
- 3.x bundle ()
- 4.x bundle ()
- 5.x bundle ()
- 6.x bundle ()
- 7.x bundle ()
- 8.x bundle ()

Python Arithmetic

Does CircuitPython support floating-point numbers?

All CircuitPython boards support floating point arithmetic, even if the microcontroller chip does not support floating point in hardware. Floating point numbers are stored in 30 bits, with an 8-bit exponent and a 22-bit mantissa. Note that this is two bits less than standard

32-bit single-precision floats. You will get about 5-1/2 digits of decimal precision.

(The **broadcom** port may provide 64-bit floats in some cases.)

Does CircuitPython support long integers, like regular Python?

Python long integers (integers of arbitrary size) are available on most builds, except those on boards with the smallest available firmware size. On these boards, integers are stored in 31 bits.

Boards without long integer support are mostly SAMD21 ("M0") boards without an external flash chip, such as the Adafruit Gemma M0, Trinket M0, QT Py M0, and the Trinkey series. There are also a number of third-party boards in this category. There are also a few small STM third-party boards without long integer support.

`time.localtime()`, `time.mktime()`, `time.time()`, and `time.monotonic_ns()` are available only on builds with long integers.

Wireless Connectivity

How do I connect to the Internet with CircuitPython?

If you'd like to include WiFi in your project, your best bet is to use a board that is running natively on ESP32 chipsets - those have WiFi built in!

If your development board has an SPI port and at least 4 additional pins, you can check out this guide () on using AirLift with CircuitPython - extra wiring is required and some boards like the MacroPad or NeoTrellis do not have enough available pins to add the hardware support.

For further project examples, and guides about using AirLift with specific hardware, check out the Adafruit Learn System ().

How do I do BLE (Bluetooth Low Energy) with CircuitPython?

nRF52840, nRF52833, and **as of CircuitPython 9.1.0**, ESP32, ESP32-C3, and ESP32-S3 boards (with 8MB) have the most complete BLE implementation. Your program can act as both a BLE central and peripheral. As a central, you can scan for advertisements, and connect to an advertising board. As a peripheral, you can advertise, and you can create services

available to a central. Pairing and bonding are supported.

Most Espressif boards with only 4MB of flash do not have enough room to include BLE in CircuitPython 9. Check the Module Support Matrix () to see if your board has support for `_bleio`. CircuitPython 10 is planned to support `_bleio` on Espressif boards with 4MB flash.

Note that the ESP32-S2 does not have Bluetooth capability.

On most other boards with adequate firmware space, BLE is available for use with AirLift () or other NINA-FW-based co-processors. Some boards have this coprocessor on board, such as the PyPortal (). Currently, this implementation only supports acting as a BLE peripheral. Scanning and connecting as a central are not yet implemented. Bonding and pairing are not supported.

Are there other ways to communicate by radio with CircuitPython?

Check out Adafruit's RFM boards ()for simple radio communication supported by CircuitPython, which can be used over distances of 100m to over a km, depending on the version. The RFM SAMD21 M0 boards can be used, but they were not designed for CircuitPython, and have limited RAM and flash

space; using the RFM breakouts or FeatherWings with more capable boards will be easier.

Asyncio and Interrupts

Is there asyncio support in CircuitPython?

There is support for asyncio starting with CircuitPython 7.1.0, on all boards except the smallest SAMD21 builds. Read about using it in the Cooperative Multitasking in CircuitPython () Guide.

Does CircuitPython support interrupts?

No. CircuitPython does not currently support interrupts - please use asyncio for multitasking / 'threaded' control of your code

Status RGB LED

My RGB NeoPixel/ DotStar LED is blinking funny colors - what does it mean?

The status LED can tell you what's going on with your CircuitPython board. Read more here for what the colors mean! ()

Memory Issues

What is a MemoryError?

Memory allocation errors happen when you're trying to store too much on the board. The CircuitPython microcontroller boards have a limited amount of memory available. You can have about 250 lines of code on the M0 Express boards. If you try to `import` too many libraries, a combination of large libraries, or run a program with too many lines of code, your code will fail to run and you will receive a `MemoryError` in the serial console.

What do I do when I encounter a MemoryError?

Try resetting your board. Each time you reset the board, it reallocates the memory. While this is unlikely to resolve your issue, it's a simple step and is worth trying.

Make sure you are using **.mpy** versions of libraries. All of the CircuitPython libraries are available in the bundle in a **.mpy** format which takes up less memory than **.py** format. Be sure that you're using the latest library bundle () for your version of CircuitPython.

If that does not resolve your issue, try shortening your code. Shorten comments, remove extraneous or unneeded code, or any other clean up you can do to shorten your code. If you're using a lot of functions, you could try moving those into a separate library, creating a **.mpy** of that library, and importing it into your code.

You can turn your entire file into a **.mpy** and `import` that into **code.py**. This means you will be unable to edit your code live on the board, but it can save you space.

Can the order of my <code class=

It can because the memory gets fragmented differently depending on allocation order and the size of objects. Loading **.mpy** files uses less memory so its recommended

to do that for files you aren't editing.

How can I create my own .mpy files?

You can make your own **.mpy** versions of files with `mpy-cross`.

You can download **mpy-cross** for your operating system from here (). Builds are available for Windows, macOS, x64 Linux, and Raspberry Pi Linux. Choose the latest **mpy-cross** whose version matches the version of CircuitPython you are using.

On macOS and Linux, after you download **mpy-cross**, you must make the the file executable by doing `chmod +x name-of-the-mpy-cross-executable`.

To make a **.mpy** file, run `./mpy-cross path/to/yourfile.py` to create a **yourfile.mpy** in the same directory as the original file.

How much memory I have free?

Run the following to see the number of bytes available for use:

```
import gc
gc.mem_free()
```

Unsupported Hardware

Is ESP8266 or ESP32 supported in CircuitPython? Why not?

We dropped ESP8266 support as of 4.x - For more information please read about it here ()!

As of CircuitPython 8.x we have started to support ESP32 and ESP32-C3 and have added a WiFi workflow for wireless coding! ()

We also support ESP32-S2 & ESP32-S3, which have native USB.

Does Feather M0 support WINC1500?

No, WINC1500 will not fit into the M0 flash space.

Can AVRs such as ATmega328 or ATmega2560 run CircuitPython?

No.

ESP32-S2 Bugs & Limitations

Nobody likes bugs, but all nontrivial software and hardware has some. The master list of problems is the Issues list on github ().

I2C at 100 kHz bus frequency runs slowly

The default I2C bus clock speed is 100 kHz (100000) . At that rate, the ESP32-S2 will leave 10ms () gaps between I2C transactions. This can slow down your I2C interactions considerably, such as when you are controlling a stepper motor with a PCA9685 controller.

Raising the I2C bus frequency to 125 kHz (125000) or higher fixes this problem. If your I2C peripheral can handle higher frequencies, you can use 400 kHz (400000) or even in some cases 1 MHz (1000000).

Note that `board.I2C()` creates an I2C bus that runs at 100 kHz. The bus frequency cannot be changed.. To create

an I2C bus on the default I2C pins that runs at a different frequency, you must use `busio.I2C(board.SCL, board.SDA, frequency=)`.

No DAC-based audio output

Current versions of the ESP-IDF SDK do not have the required APIs for DAC-based audio output. Once a future version of ESP-IDF that adds it, it will be possible to implement DAC-based AudioOut in CircuitPython.

Workaround: PWMOut can create tones and buzzes.

Workaround: I2SOut audio is currently being developed and will work with boards such as the I2S 3W Class D Amplifier Breakout - MAX98357A (http://adafru.it/3006).

Deep Sleep & Wake-up sources

ESP32-S2 has hardware limitations on what kind of "pin alarms" can

wake it. The following combinations are possible:

- EITHER one or two pins that wake from deep sleep when they are pulled LOW
- OR an arbitrary number of pins that wake from deep sleep when they are pulled HIGH, and optionally one pin that wakes from deep sleep when pulled LOW

This means that "wake" buttons should be wired so that pressing them pulls HIGH and a pull DOWN resistor is used with the pin. However, in some hardware designs including the original MagTag, the integrated buttons are pulled LOW when pressed and so only 1 or 2 buttons can be selected to wake the MagTag.

Troubleshooting

From time to time, you will run into issues when working with CircuitPython. Here are a few things you may encounter and how to resolve them.

Text emphasized with a red exclamation:

As CircuitPython development continues and there are new releases, Adafruit will stop supporting older releases. Visit https://circuitpython.org/downloads to download the latest version of CircuitPython for your board. You must download the CircuitPython Library Bundle that matches your version of CircuitPython. Please update CircuitPython and then visit https://circuitpython.org/libraries to download the latest Library Bundle.

Always Run the Latest Version of CircuitPython and Libraries

As CircuitPython development continues and there are new releases, Adafruit will stop supporting older releases. **You need to** update to the latest CircuitPython **().**

You need to download the CircuitPython Library Bundle that matches your version of CircuitPython. **Please update CircuitPython and then** download the latest bundle **()**.

As new versions of CircuitPython are released, Adafruit will stop providing the previous bundles as automatically created downloads on the Adafruit CircuitPython Library Bundle repo. If you must continue to use an earlier version, you can still download the appropriate version of `mpy-cross` from the particular release of CircuitPython on the CircuitPython repo and create your own compatible .mpy library files. **However, it is best to update to the latest for both CircuitPython and the library bundle.**

I have to continue using CircuitPython 7.x or earlier. Where can I find compatible libraries?

Adafruit is no longer building or supporting the CircuitPython 7.x or earlier library bundles. You are highly encourged to update CircuitPython to the latest version **()** **and use** the current version of the libraries **().** However, if for some reason you cannot update, links to the previous bundles are available in the FAQ ().

macOS Sonoma before 14.4: Errors Writing to CIRCUITPY
macOS 14.4 - 15.1: Slow Writes to CIRCUITPY

macOS Sonoma before 14.4 took many seconds to complete writes to small FAT drives, 8MB or smaller. This causes errors when writing to CIRCUITPY. The best solution was to remount the CIRCUITPY drive after it is automatically mounted. Or consider downgrading back to Ventura if that works for you. This problem was tracked in CircuitPython GitHub issue 8449 ().

Below is a shell script to do this remount conveniently (courtesy @czei in GitHub ()). Copy the code here into a file named, say, **remount-CIRCUITPY.sh**. Place the file in a directory on your PATH, or in some other convenient place.

macOS Sonoma 14.4 and versions of macOS before Sequoia 15.2 did not have the problem above, but did take an inordinately long time to write to FAT drives of size 1GB or less (40 times longer than 2GB drives). As of macOS 15.2, writes are no longer very slow. This problem was tracked in CircuitPython GitHub issue 8918 ().

```
#!/bin/sh
#
# This works around bug where, by default,
# macOS 14.x before 14.4 writes part of a file immediately,
# and then doesn't update the directory for 20-60 seconds, causing
# the file system to be corrupted.
#

disky=`df | grep CIRCUITPY | cut -d" " -f1`
sudo umount /Volumes/CIRCUITPY
sudo mkdir /Volumes/CIRCUITPY
sleep 2
sudo mount -v -o noasync -t msdos $disky /Volumes/CIRCUITPY
```

Then in a Terminal window, do this to make this script executable:

```
chmod +x remount-CIRCUITPY.sh
```

Place the file in a directory on your `PATH`, or in some other convenient place.

Now, each time you plug in or reset your CIRCUITPY board, run the file **remount-CIRCUITPY.sh**. You can run it in a Terminal window or you may be able to place it on the desktop or in your dock to run it just by double-clicking.

This will be something of a nuisance but it is the safest solution.

This problem is being tracked in this CircuitPython issue ().

Bootloader (boardnameBOOT) Drive Not Present

You may have a different board.

Only Adafruit Express boards and the SAMD21 non-Express boards ship with the UF2 bootloader ()installed. The Feather M0 Basic, Feather M0 Adalogger, and similar boards use a regular Arduino-compatible bootloader, which does not show a **boardnameBOOT** drive.

MakeCode

If you are running a MakeCode () program on Circuit Playground Express, press the reset button just once to get the **CPLAYBOOT** drive to show up. Pressing it twice will not work.

macOS

DriveDx and its accompanything **SAT SMART Driver** can interfere with seeing the BOOT drive. See this forum post () for how to fix the problem.

Windows 10 or later

Did you install the Adafruit Windows Drivers package by mistake, or did you upgrade to Windows 10 or later with the driver package installed? You don't need to install this package on Windows 10 or 11for most Adafruit boards. The old version (v1.5) can interfere with recognizing your device. Go to **Settings** -> **Apps** and uninstall all the "Adafruit" driver programs.

Windows 7 or 8.1

Windows 7 and 8.1 have reached end of life. It is recommended () that you upgrade to Windows 10 or 11 if possible. Drivers are available for some older CircuitPython boards, but there are no plans to release drivers for newer boards.

Text emphasized with a blue exclamation:	The Windows Drivers installer was last updated in November 2020 (v2.5.0.0) . Windows 7 drivers for CircuitPython boards released since then, including RP2040 boards, are not available. There are no plans to release drivers for newer boards. The boards work fine on Windows 10 and later.

You should now be done! Test by unplugging and replugging the board. You should see the **CIRCUITPY** drive, and when you double-click the reset button (single click on Circuit Playground Express running MakeCode), you should see the appropriate **boardnameBOOT** drive.

Let us know in the Adafruit support forums () or on the Adafruit Discord () if this does not work for you!

Windows Explorer Locks Up When Accessing **boardnameBOOT** Drive

On Windows, several third-party programs that can cause issues. The symptom is that you try to access the **boardnameBOOT** drive, and Windows or Windows Explorer seems to lock up. These programs are known to cause trouble:

- **AIDA64**: to fix, stop the program. This problem has been reported to AIDA64. They acquired hardware to test, and released a beta version that fixes the problem. This may have been incorporated into the latest release. Please let us know in the forums if you test this.
- **BitDefender anti-virus**
- **Hard Disk Sentinel**
- **Kaspersky anti-virus**: To fix, you may need to disable Kaspersky completely. Disabling some aspects of Kaspersky does not always solve the problem. This problem has been reported to Kaspersky.
- **ESET NOD32 anti-virus**: There have been problems with at least version 9.0.386.0, solved by uninstallation.

Copying UF2 to **boardnameBOOT** Drive Hangs at 0% Copied

On Windows, a **Western DIgital (WD) utility** that comes with their external USB drives can interfere with copying UF2 files to the **boardnameBOOT** drive. Uninstall that utility to fix the problem.

CIRCUITPY Drive Does Not Appear or Disappears Quickly

BitDefender anti-virus has been reported to block access to CIRCUITPY. You can set an exception for the drive letter.

Kaspersky **anti-virus** can block the appearance of the **CIRCUITPY** drive. There has not yet been settings change discovered that prevents this. Complete uninstallation of Kaspersky fixes the problem.

Norton anti-virus can interfere with **CIRCUITPY**. A user has reported this problem on Windows 7. The user turned off both Smart Firewall and Auto Protect, and **CIRCUITPY** then appeared.

Sophos Endpoint security software can cause CIRCUITPY to disappear () and the BOOT drive to reappear. It is not clear what causes this behavior.

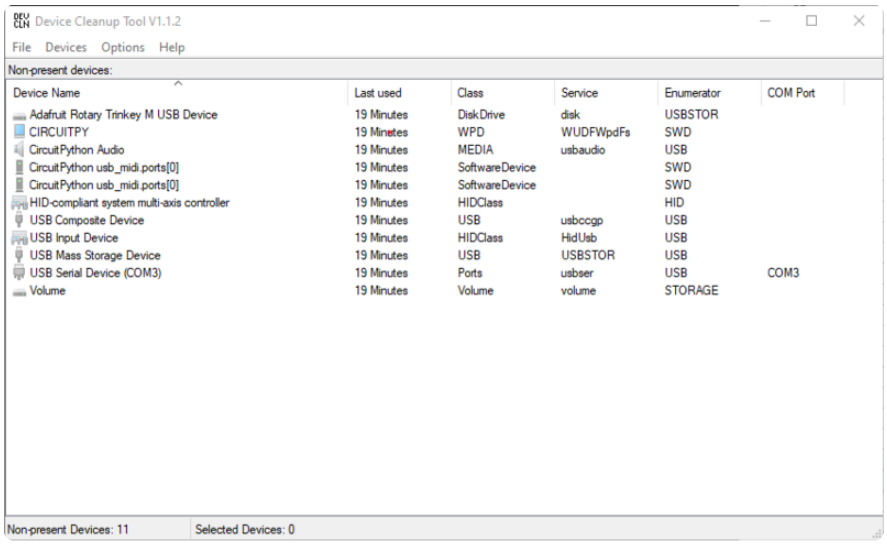
Samsung Magician can cause CIRCUITPY to disappear (reported here () and here ()).

"M105" Seen on Display, Crashes, Missing CIRCUITPY

The **Cura** 3D printing program sends 3D printing GCODE commands to unused serial ports to try to find 3D printers connected over serial. This causes a variety of problems. Disable (uncheck) **USB Printing** in Cura in the **Market** -> **Installed** menu, or uninstall Cura. For more information see this forum post (), this CircuitPython issue (), and this Cura issue ().

Device Errors or Problems on Windows

Windows can become confused about USB device installations. Try cleaning up your USB devices. Use Uwe Sieber's Device Cleanup Tool () (on that page, scroll down to "Device Cleanup Tool"). Download and unzip the tool. Unplug all the boards and other USB devices you want to clean up. Run the tool as Administrator. You will see a listing like this, probably with many more devices. It is listing all the USB devices that are not currently attached.



Select all the devices you want to remove, and then press Delete. It is usually safe just to select everything. Any device that is removed will get a fresh install when you plug it in. Using the Device Cleanup Tool also discards all the COM port assignments for the unplugged boards. If you have used many Arduino and CircuitPython boards,

you have probably seen higher and higher COM port numbers used, seemingly without end. This will fix that problem.

Serial Console in Mu Not Displaying Anything

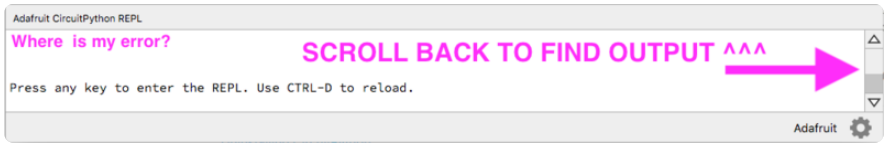
There are times when the serial console will accurately not display anything, such as, when no code is currently running, or when code with no serial output is already running before you open the console. However, if you find yourself in a situation where you feel it should be displaying something like an error, consider the following.

Depending on the size of your screen or Mu window, when you open the serial console, the serial console panel may be very small. This can be a problem. A basic CircuitPython error takes 10 lines to display!

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Traceback (most recent call last):
  File "code.py", line 7
SyntaxError: invalid syntax



Press any key to enter the REPL. Use CTRL-D to reload.

```

More complex errors take even more lines!

Therefore, if your serial console panel is five lines tall or less, you may only see blank lines or blank lines followed by `Press any key to enter the REPL. Use CTRL-D to reload.`. If this is the case, you need to either mouse over the top of the panel to utilise the option to resize the serial panel, or use the scrollbar on the right side to scroll up and find your message.



This applies to any kind of serial output whether it be error messages or print statements. So before you start trying to debug your problem on the hardware side, be sure to check that you haven't simply missed the serial messages due to serial output panel height.

code.py Restarts Constantly

CircuitPython will restart **code.py** if you or your computer writes to something on the CIRCUITPY drive. This feature is called auto-reload, and lets you test a change to your program immediately.

Some utility programs, such as backup, anti-virus, or disk-checking apps, will write to the CIRCUITPY as part of their operation. Sometimes they do this very frequently, causing constant restarts.

Acronis True Image and related Acronis programs on Windows are known to cause this problem. It is possible to prevent this by disabling the " ()Acronis Managed Machine Service Mini" ().

If you cannot stop whatever is causing the writes, you can disable auto-reload by putting this code in **boot.py** or **code.py**:

```
import supervisor

supervisor.runtime.autoreload = False
```

CircuitPython RGB Status Light

Nearly all CircuitPython-capable boards have a single NeoPixel or DotStar RGB LED on the board that indicates the status of CircuitPython. A few boards designed before CircuitPython existed, such as the Feather M0 Basic, do not.

Circuit Playground Express and Circuit Playground Bluefruit have multiple RGB LEDs, but do NOT have a status LED. The LEDs are all green when in the bootloader. In versions before 7.0.0, they do NOT indicate any status while running CircuitPython.

CircuitPython 7.0.0 and Later

The status LED blinks were changed in CircuitPython 7.0.0 in order to save battery power and simplify the blinks. These blink patterns will occur on single color LEDs when the board does not have any RGB LEDs. Speed and blink count also vary for this reason.

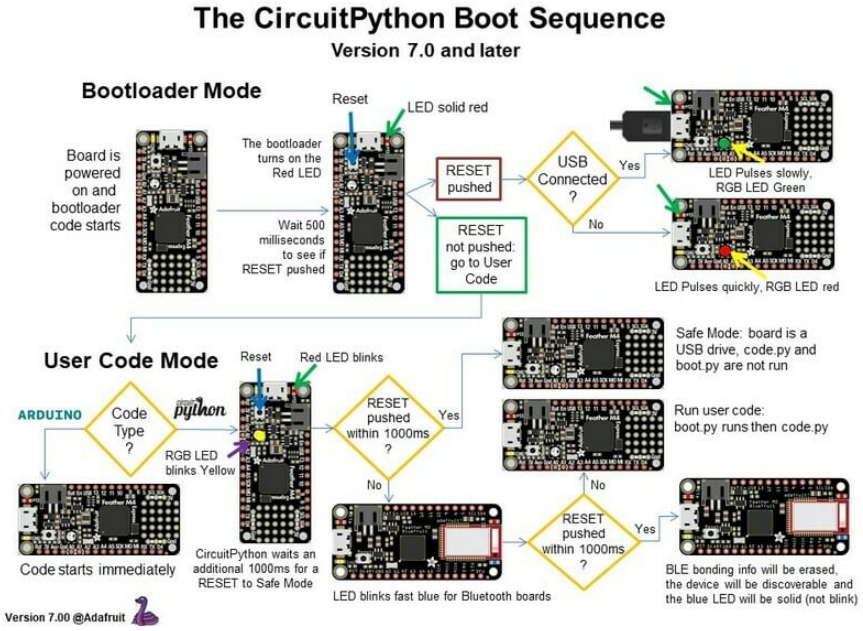
On start up, the LED will blink **YELLOW** multiple times for 1 second. Pressing the RESET button (or on Espressif, the BOOT button) during this time will restart the board and then enter safe mode. On Bluetooth capable boards, after the yellow blinks, there will be a set of faster blue blinks. Pressing reset during the **BLUE** blinks will clear

Bluetooth information and start the device in discoverable mode, so it can be used with a BLE code editor.

Once started, CircuitPython will blink a pattern every 5 seconds when no user code is running to indicate why the code stopped:

- 1 **GREEN** blink: Code finished without error.
- 2 **RED** blinks: Code ended due to an exception. Check the serial console for details.
- 3 **YELLOW** blinks: CircuitPython is in safe mode. No user code was run. Check the serial console for safe mode reason.

When in the REPL, CircuitPython will set the status LED to **WHITE**. You can change the LED color from the REPL. The status indicator will not persist on non-NeoPixel or DotStar LEDs.



CircuitPython 6.3.0 and earlier

Here's what the colors and blinking mean:

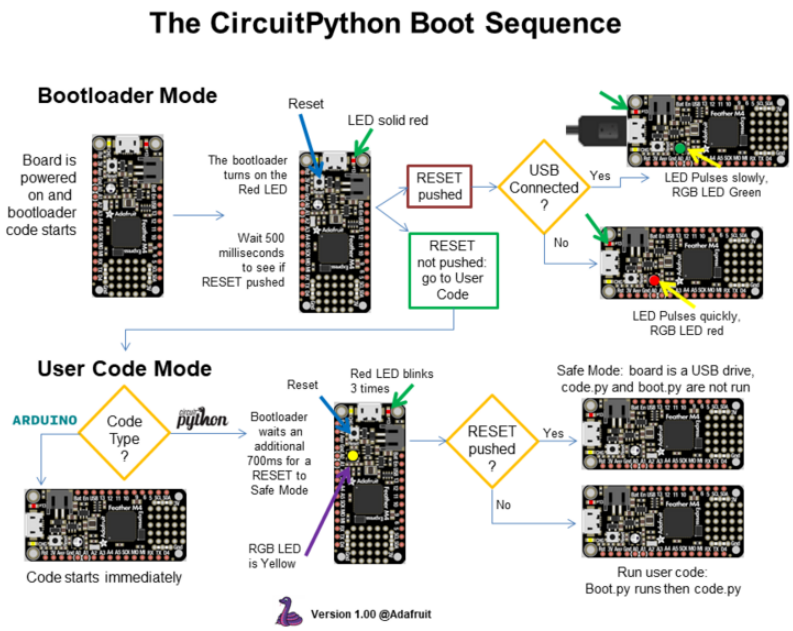
- steady **GREEN**: **code.py** (or **code.txt**, **main.py**, or **main.txt**) is running
- pulsing **GREEN**: **code.py** (etc.) has finished or does not exist
- steady **YELLOW** at start up: (4.0.0-alpha.5 and newer) CircuitPython is waiting for a reset to indicate that it should start in safe mode
- pulsing **YELLOW**: Circuit Python is in safe mode: it crashed and restarted

- steady **WHITE**: REPL is running
- steady **BLUE**: **boot.py** is running

Colors with multiple flashes following indicate a Python exception and then indicate the line number of the error. The color of the first flash indicates the type of error:

- **GREEN**: IndentationError
- **CYAN**: SyntaxError
- **WHITE**: NameError
- **ORANGE**: OSError
- **PURPLE**: ValueError
- **YELLOW**: other error

These are followed by flashes indicating the line number, including place value. **WHITE** flashes are thousands' place, **BLUE** are hundreds' place, **YELLOW** are tens' place, and **CYAN** are one's place. So for example, an error on line 32 would flash **YELLOW** three times and then **CYAN** two times. Zeroes are indicated by an extra-long dark gap.



Serial console showing `ValueError: Incompatible .mpy file`

This error occurs when importing a module that is stored as a **.mpy** binary file that was generated by a different version of CircuitPython than the one its being loaded

into. In particular, the mpy binary format changed between CircuitPython versions 6.x and 7.x, 2.x and 3.x, and 1.x and 2.x.

So, for instance, if you upgraded to CircuitPython 7.x from 6.x you'll need to download a newer version of the library that triggered the error on `import`. All libraries are available in the Adafruit bundle ().

CIRCUITPY Drive Issues

You may find that you can no longer save files to your **CIRCUITPY** drive. You may find that your **CIRCUITPY** stops showing up in your file explorer, or shows up as **NO_NAME**. These are indicators that your filesystem has issues. When the **CIRCUITPY** disk is not safely ejected before being reset by the button or being disconnected from USB, it may corrupt the flash drive. It can happen on Windows, Mac or Linux, though it is more common on Windows.

Be aware, if you have used Arduino to program your board, CircuitPython is no longer able to provide the USB services. You will need to reload CircuitPython to resolve this situation.

The easiest first step is to reload CircuitPython. Double-tap reset on the board so you get a **boardnameBOOT** drive rather than a **CIRCUITPY** drive, and copy the latest version of CircuitPython (**.uf2**) back to the board. This may restore **CIRCUITPY** functionality.

If reloading CircuitPython does not resolve your issue, the next step is to try putting the board into safe mode.

Safe Mode

Whether you've run into a situation where you can no longer edit your **code.py** on your **CIRCUITPY** drive, your board has gotten into a state where **CIRCUITPY** is read-only, or you have turned off the **CIRCUITPY** drive altogether, safe mode can help.

Safe mode in CircuitPython does not run any user code on startup, and disables auto-reload. This means a few things. First, safe mode bypasses any code in **boot.py** (where you can set **CIRCUITPY** read-only or turn it off completely). Second, it does not run the code in **code.py**. And finally, it does not automatically soft-reload when data is written to the **CIRCUITPY** drive.

Therefore, whatever you may have done to put your board in a non-interactive state, safe mode gives you the opportunity to correct it without losing all of the data on the **CIRCUITPY** drive.

Entering Safe Mode in CircuitPython 7.x and Later

You can enter safe by pressing reset during the right time when the board boots. Immediately after the board starts up or resets, it waits one second. On some boards, the onboard status LED will blink yellow during that time. If you press reset during that one second period, the board will start up in safe mode. It can be difficult to react to the yellow LED, so you may want to think of it simply as a "slow" double click of the reset button. (Remember, a fast double click of reset enters the bootloader.)

Entering Safe Mode in CircuitPython 6.x

You can enter safe by pressing reset during the right time when the board boots.. Immediately after the board starts up or resets, it waits 0.7 seconds. On some boards, the onboard status LED (highlighted in green above) will turn solid yellow during this time. If you press reset during that 0.7 seconds, the board will start up in safe mode. It can be difficult to react to the yellow LED, so you may want to think of it simply as a slow double click of the reset button. (Remember, a fast double click of reset enters the bootloader.)

In Safe Mode

Once you've entered safe mode successfully in CircuitPython 6.x, the LED will pulse yellow.

If you successfully enter safe mode on CircuitPython 7.x, the LED will intermittently blink yellow three times.

If you connect to the serial console, you'll find the following message.

```
Auto-reload is off.
Running in safe mode! Not running saved code.

CircuitPython is in safe mode because you pressed the reset button during boot. Press again to

Press any key to enter the REPL. Use CTRL-D to reload.
```

You can now edit the contents of the **CIRCUITPY** drive. Remember, your code will not run until you press the reset button, or unplug and plug in your board, to get out of safe mode.

At this point, you'll want to remove any user code in **code.py** and, if present, the **boot.py** file from **CIRCUITPY**. Once removed, tap the reset button, or unplug and plug in your board, to restart CircuitPython. This will restart the board and may resolve your drive issues. If resolved, you can begin coding again as usual.

If safe mode does not resolve your issue, the board must be completely erased and CircuitPython must be reloaded onto the board.

Text emphasized with a red exclamation:	You WILL lose everything on the board when you complete the following steps. If possible, make a copy of your code before continuing.

To erase CIRCUITPY: `storage.erase_filesystem()`

CircuitPython includes a built-in function to erase and reformat the filesystem. If you have a version of CircuitPython older than 2.3.0 on your board, you can update to the newest version () to do this.

1. Connect to the CircuitPython REPL () using Mu or a terminal program.
2. Type the following into the REPL:

```
>>> import storage
>>> storage.erase_filesystem()
```

CIRCUITPY will be erased and reformatted, and your board will restart. That's it!

Erase CIRCUITPY Without Access to the REPL

If you can't access the REPL, or you're running a version of CircuitPython previous to 2.3.0 and you don't want to upgrade, there are options available for some specific boards.

The options listed below are considered to be the "old way" of erasing your board. The method shown above using the REPL is highly recommended as the best method for erasing your board.

Text emphasized with a yellow exclamation:	If at all possible, it is recommended to use the REPL to erase your CIRCUITPY drive. The REPL method is explained above.

For the specific boards listed below:

If the board you are trying to erase is listed below, follow the steps to use the file to erase your board.

1. Download the correct erase file:

2. Double-click the reset button on the board to bring up the **boardnameBOOT** drive.
3. Drag the erase **.uf2** file to the **boardnameBOOT** drive.
4. The status LED will turn yellow or blue, indicating the erase has started.
5. After approximately 15 seconds, the status LED will light up green. On the NeoTrellis M4 this is the first NeoPixel on the grid
6. Double-click the reset button on the board to bring up the **boardnameBOOT** drive.
7. Drag the appropriate latest release of CircuitPython () **.uf2** file to the **boardnameBOOT** drive.

It should reboot automatically and you should see **CIRCUITPY** in your file explorer again.

If the LED flashes red during step 5, it means the erase has failed. Repeat the steps starting with 2.

If you haven't already downloaded the latest release of CircuitPython for your board, check out the installation page (). You'll also need to load your code and reinstall your libraries!

For SAMD21 non-Express boards that have a UF2 bootloader:

Any SAMD21-based microcontroller that does not have external flash available is considered a SAMD21 non-Express board. Non-Express boards that have a UF2 bootloader include Trinket M0, GEMMA M0, QT Py M0, and the SAMD21-based Trinkey boards.

If you are trying to erase a SAMD21 non-Express board, follow these steps to erase your board.

1. Download the erase file:

2. Double-click the reset button on the board to bring up the **boardnameBOOT** drive.
3. Drag the erase **.uf2** file to the **boardnameBOOT** drive.
4. The boot LED will start flashing again, and the **boardnameBOOT** drive will reappear.

5. Drag the appropriate latest release CircuitPython () **.uf2** file to the boardnameBOOT drive.

It should reboot automatically and you should see **CIRCUITPY** in your file explorer again.

If you haven't already downloaded the latest release of CircuitPython for your board, check out the installation page () YYou'll also need to load your code and reinstall your libraries!

For SAMD21 non-Express boards that do not have a UF2 bootloader:

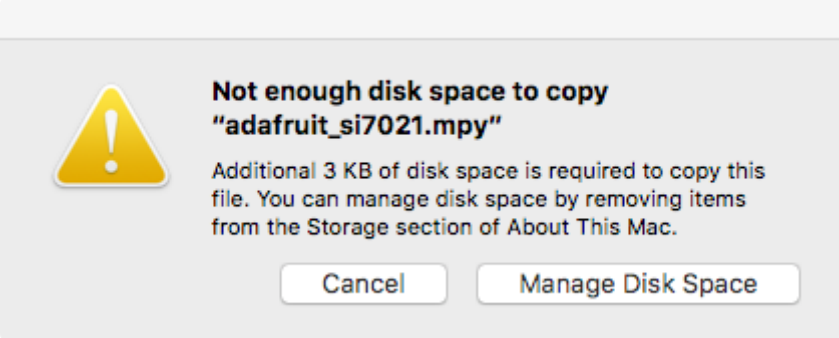
Any SAMD21-based microcontroller that does not have external flash available is considered a SAMD21 non-Express board. Non-Express boards that do **not** have a UF2 bootloader include the Feather M0 Basic Proto, Feather Adalogger, or the Arduino Zero.

If you are trying to erase a non-Express board that does not have a UF2 bootloader, follow these directions to reload CircuitPython using `bossac` (), which will erase and re-create **CIRCUITPY**.

Running Out of File Space on SAMD21 Non-Express Boards

Any SAMD21-based microcontroller that does not have external flash available is considered a SAMD21 non-Express board. This includes boards like the Trinket M0, GEMMA M0, QT Py M0, and the SAMD21-based Trinkey boards.

The file system on the board is very tiny. (Smaller than an ancient floppy disk.) So, its likely you'll run out of space but don't panic! There are a number of ways to free up space.



Delete something!

The simplest way of freeing up space is to delete files from the drive. Perhaps there are libraries in the **lib** folder that you aren't using anymore or test code that isn't in use. Don't delete the **lib** folder completely, though, just remove what you don't need.

The board ships with the Windows 7 serial driver too! Feel free to delete that if you don't need it or have already installed it. It's ~12KiB or so.

Use tabs

One unique feature of Python is that the indentation of code matters. Usually the recommendation is to indent code with four spaces for every indent. In general, that is recommended too. **However**, one trick to storing more human-readable code is to use a single tab character for indentation. This approach uses 1/4 of the space for indentation and can be significant when you're counting bytes.

On macOS?

MacOS loves to generate hidden files. Luckily you can disable some of the extra hidden files that macOS adds by running a few commands to disable search indexing and create zero byte placeholders. Follow the steps below to maximize the amount of space available on macOS.

Prevent & Remove macOS Hidden Files

First find the volume name for your board. With the board plugged in run this command in a terminal to list all the volumes:

```
1 ls -l /Volumes
```

Look for a volume with a name like **CIRCUITPY** (the default for CircuitPython). The full path to the volume is the **/Volumes/CIRCUITPY** path.

Now follow the steps from this question () to run these terminal commands that stop hidden files from being created on the board:

```
mdutil -i off /Volumes/CIRCUITPY
cd /Volumes/CIRCUITPY
rm -rf .{,_.}{fseventsd,Spotlight-V*,Trashes}
mkdir .fseventsd
touch .fseventsd/no_log .metadata_never_index .Trashes
cd -
```

Replace **/Volumes/CIRCUITPY** in the commands above with the full path to your board's volume if it's different. At this point all the hidden files should be cleared from the board and some hidden files will be prevented from being created.

Alternatively, with CircuitPython 4.x and above, the special files and folders mentioned above will be created automatically if you erase and reformat the filesystem. **WARNING: Save your files first!** Do this in the REPL:

```
>>> import storage
>>> storage.erase_filesystem()
```

However there are still some cases where hidden files will be created by MacOS. In particular if you copy a file that was downloaded from the internet it will have special metadata that MacOS stores as a hidden file. Luckily you can run a copy command from the terminal to copy files **without** this hidden metadata file. See the steps below.

Copy Files on macOS Without Creating Hidden Files

Once you've disabled and removed hidden files with the above commands on macOS you need to be careful to copy files to the board with a special command that prevents future hidden files from being created. Unfortunately you **cannot** use drag and drop copy in Finder because it will still create these hidden extended attribute files in some cases (for files downloaded from the internet, like Adafruit's modules).

To copy a file or folder use the **-X** option for the **cp** command in a terminal. For example to copy a **file_name.mpy** file to the board use a command like:

```
cp -X file_name.mpy /Volumes/CIRCUITPY
```

(Replace **file_name.mpy** with the name of the file you want to copy.)

Or to copy a folder and all of the files and folders contained within, use a command like:

```
cp -rX folder_to_copy /Volumes/CIRCUITPY
```

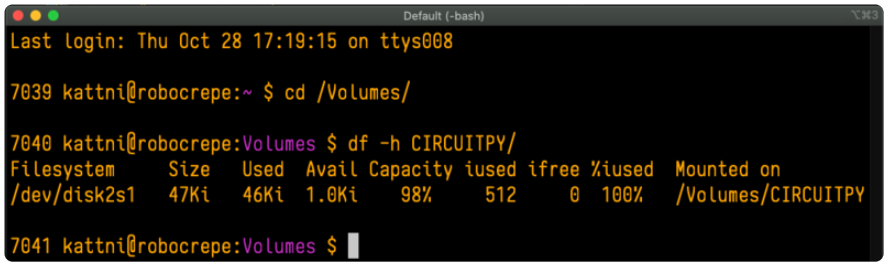
If you are copying to the **lib** folder, or another folder, make sure it exists before copying.

```
# if lib does not exist, you'll create a file named lib !
cp -X file_name.mpy /Volumes/CIRCUITPY/lib
```

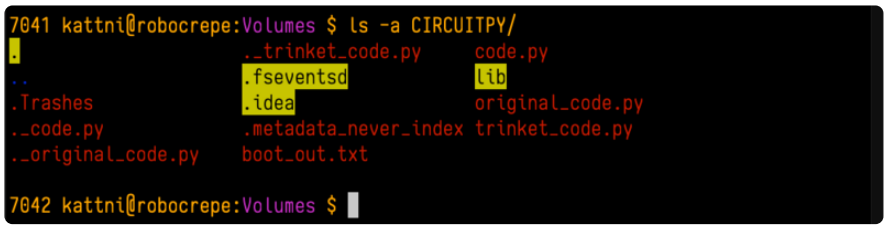
```
# This is safer, and will complain if a lib folder does not exist.
cp -X file_name.mpy /Volumes/CIRCUITPY/lib/
```

Other macOS Space-Saving Tips

If you'd like to see the amount of space used on the drive and manually delete hidden files here's how to do so. First, move into the **Volumes/** directory with `cd /Volumes/`, and then list the amount of space used on the **CIRCUITPY** drive with the `df` command.



That's not very much space left! The next step is to show a list of the files currently on the **CIRCUITPY** drive, including the hidden files, using the `ls` command. You cannot use Finder to do this, you must do it via command line!



There are a few of the hidden files that MacOS loves to generate, all of which begin with a ._ before the file name. Remove the ._ files using the `rm` command. You can remove them all once by running `rm CIRCUITPY/._*`. The `*` acts as a wildcard to apply the command to everything that begins with ._ at the same time.

Finally, you can run `df` again to see the current space used.

```
7043 kattni@robocrepe:Volumes $ df -h CIRCUITPY/
Filesystem     Size   Used  Avail Capacity iused ifree %iused  Mounted on
/dev/disk2s1   47Ki   34Ki   13Ki    73%     512     0  100%   /Volumes/CIRCUITPY

7044 kattni@robocrepe:Volumes $
```

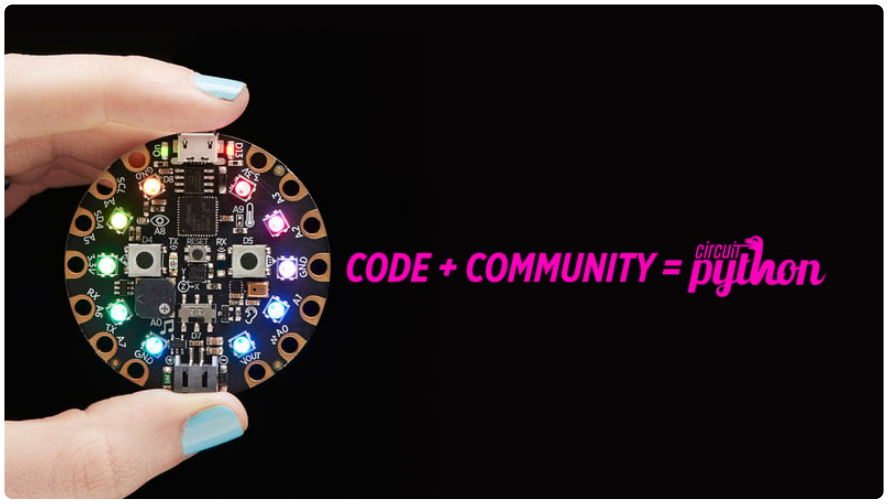
Nice! You have 12Ki more than before! This space can now be used for libraries and code!

Device Locked Up or Boot Looping

In rare cases, it may happen that something in your **code.py** or **boot.py** files causes the device to get locked up, or even go into a boot loop. A boot loop occurs when the board reboots repeatedly and never fully loads. These are not caused by your everyday Python exceptions, typically it's the result of a deeper problem within CircuitPython. In this situation, it can be difficult to recover your device if **CIRCUITPY** is not allowing you to modify the **code.py** or **boot.py** files. Safe mode is one recovery option. When the device boots up in safe mode it will not run the **code.py** or **boot.py** scripts, but will still connect the **CIRCUITPY** drive so that you can remove or modify those files as needed.

For more information on safe mode and how to enter safe mode, see the Safe Mode section on this page ().

Welcome to the Community!

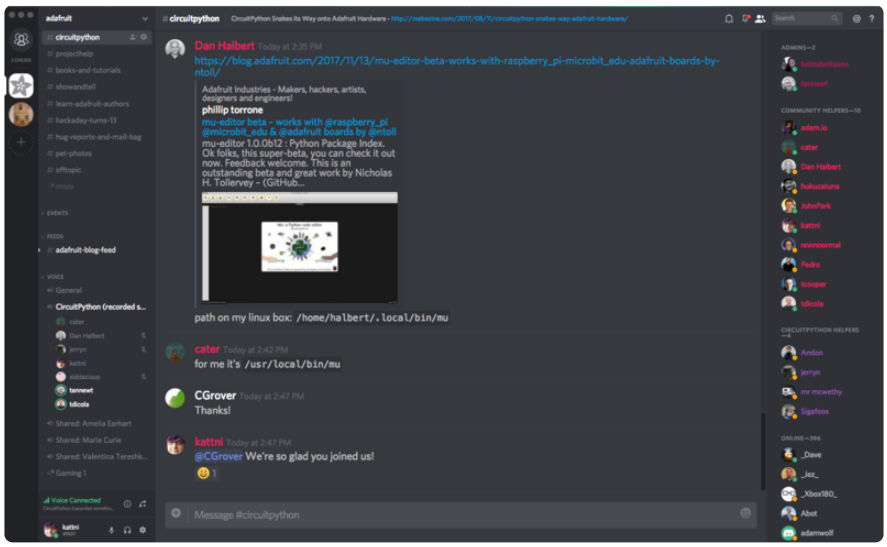


CircuitPython is a programming language that's super simple to get started with and great for learning. It runs on microcontrollers and works out of the box. You can plug it

in and get started with any text editor. The best part? CircuitPython comes with an amazing, supportive community.

Everyone is welcome! CircuitPython is Open Source. This means it's available for anyone to use, edit, copy and improve upon. This also means CircuitPython becomes better because of you being a part of it. Whether this is your first microcontroller board or you're a seasoned software engineer, you have something important to offer the Adafruit CircuitPython community. This page highlights some of the many ways you can be a part of it!

Adafruit Discord



The Adafruit Discord server is the best place to start. Discord is where the community comes together to volunteer and provide live support of all kinds. From general discussion to detailed problem solving, and everything in between, Discord is a digital maker space with makers from around the world.

There are many different channels so you can choose the one best suited to your needs. Each channel is shown on Discord as "#channelname". There's the #help-with-projects channel for assistance with your current project or help coming up with ideas for your next one. There's the #show-and-tell channel for showing off your newest creation. Don't be afraid to ask a question in any channel! If you're unsure, #general is a great place to start. If another channel is more likely to provide you with a better answer, someone will guide you.

The help with CircuitPython channel is where to go with your CircuitPython questions. #help-with-circuitpython is there for new users and developers alike so feel free to ask a question or post a comment! Everyone of any experience level is welcome to

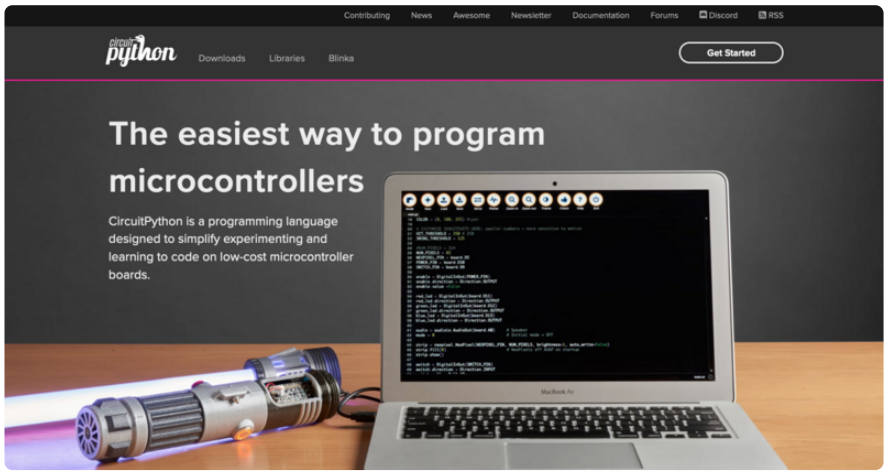
join in on the conversation. Your contributions are important! The #circuitpython-dev channel is available for development discussions as well.

The easiest way to contribute to the community is to assist others on Discord. Supporting others doesn't always mean answering questions. Join in celebrating successes! Celebrate your mistakes! Sometimes just hearing that someone else has gone through a similar struggle can be enough to keep a maker moving forward.

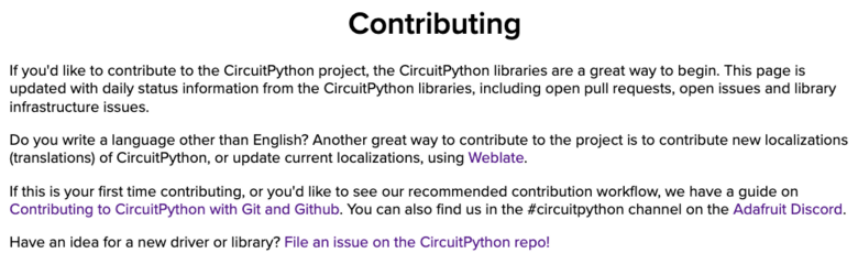
The Adafruit Discord is the 24x7x365 hackerspace that you can bring your granddaughter to.

Visit https://adafru.it/discord ()to sign up for Discord. Everyone is looking forward to meeting you!

CircuitPython.org



Beyond the Adafruit Learn System, which you are viewing right now, the best place to find information about CircuitPython is circuitpython.org (). Everything you need to get started with your new microcontroller and beyond is available. You can do things like download CircuitPython for your microcontroller () or download the latest CircuitPython Library bundle (), or check out which single board computers support Blinka (). You can also get to various other CircuitPython related things like Awesome CircuitPython or the Python for Microcontrollers newsletter. This is all incredibly useful, but it isn't necessarily community related. So why is it included here? The Contributing page ().



CircuitPython itself is written in C. However, all of the Adafruit CircuitPython libraries are written in Python. If you're interested in contributing to CircuitPython on the Python side of things, check out circuitpython.org/contributing (). You'll find information pertaining to every Adafruit CircuitPython library GitHub repository, giving you the opportunity to join the community by finding a contributing option that works for you.

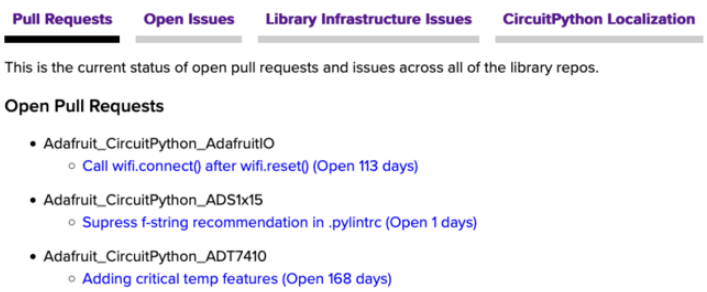
Note the date on the page next to **Current Status for**:

If you submit any contributions to the libraries, and do not see them reflected on the Contributing page, it could be that the job that checks for new updates hasn't yet run for today. Simply check back tomorrow!

Now, a look at the different options.

Pull Requests

The first tab you'll find is a list of **open pull requests**.

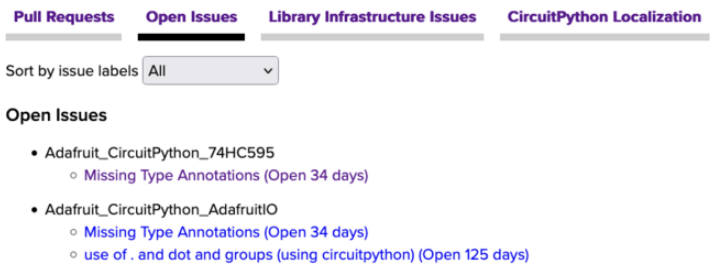


GitHub pull requests, or PRs, are opened when folks have added something to an Adafruit CircuitPython library GitHub repo, and are asking for Adafruit to add, or

merge, their changes into the main library code. For PRs to be merged, they must first be reviewed. Reviewing is a great way to contribute! Take a look at the list of open pull requests, and pick one that interests you. If you have the hardware, you can test code changes. If you don't, you can still check the code updates for syntax. In the case of documentation updates, you can verify the information, or check it for spelling and grammar. Once you've checked out the update, you can leave a comment letting us know that you took a look. Once you've done that for a while, and you're more comfortable with it, you can consider joining the CircuitPythonLibrarians review team. The more reviewers we have, the more authors we can support. Reviewing is a crucial part of an open source ecosystem, CircuitPython included.

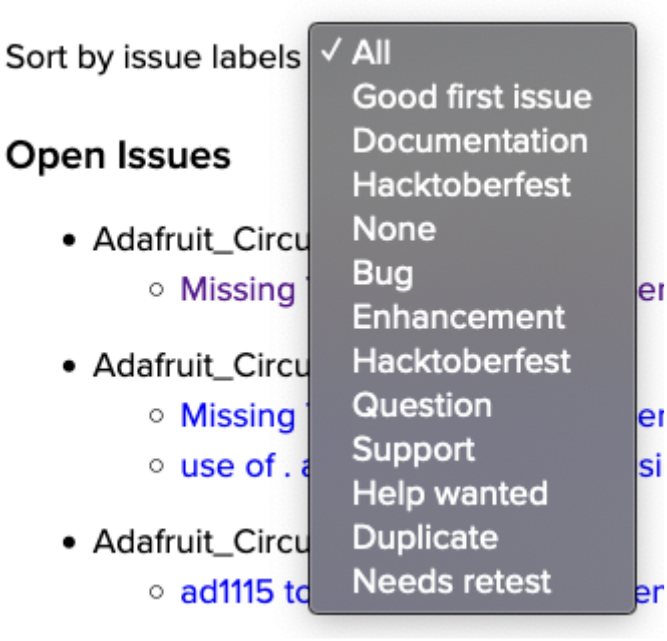
Open Issues

The second tab you'll find is a list of **open issues**.



GitHub issues are filed for a number of reasons, including when there is a bug in the library or example code, or when someone wants to make a feature request. Issues are a great way to find an opportunity to contribute directly to the libraries by updating code or documentation. If you're interested in contributing code or documentation, take a look at the open issues and find one that interests you.

If you're not sure where to start, you can search the issues by label. Labels are applied to issues to make the goal easier to identify at a first glance, or to indicate the difficulty level of the issue. Click on the dropdown next to "Sort by issue labels" to see the list of available labels, and click on one to choose it.



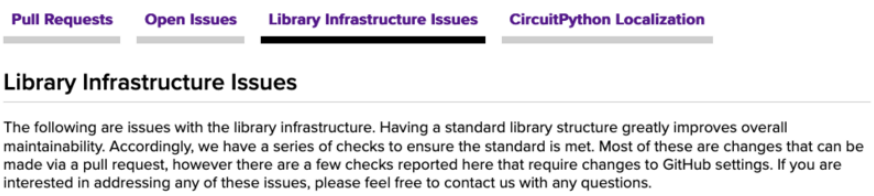
If you're new to everything, new to contributing to open source, or new to contributing to the CircuitPython project, you can choose "Good first issue". Issues with that label are well defined, with a finite scope, and are intended to be easy for someone new to figure out.

If you're looking for something a little more complicated, consider "Bug" or "Enhancement". The Bug label is applied to issues that pertain to problems or failures found in the library. The Enhancement label is applied to feature requests.

Don't let the process intimidate you. If you're new to Git and GitHub, there is a guide () to walk you through the entire process. As well, there are always folks available on Discord () to answer questions.

Library Infrastructure Issues

The third tab you'll find is a list of **library infrastructure issues**.

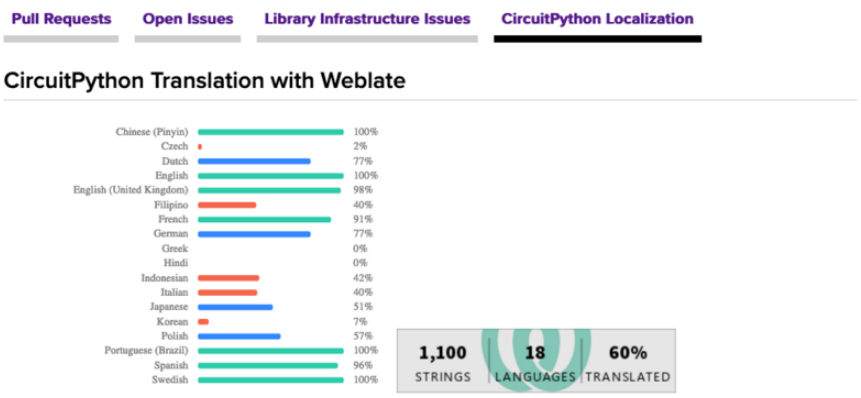


This section is generated by a script that runs checks on the libraries, and then reports back where there may be issues. It is made up of a list of subsections each containing links to the repositories that are experiencing that particular issue. This

page is available mostly for internal use, but you may find some opportunities to contribute on this page. If there's an issue listed that sounds like something you could help with, mention it on Discord, or file an issue on GitHub indicating you're working to resolve that issue. Others can reply either way to let you know what the scope of it might be, and help you resolve it if necessary.

CircuitPython Localization

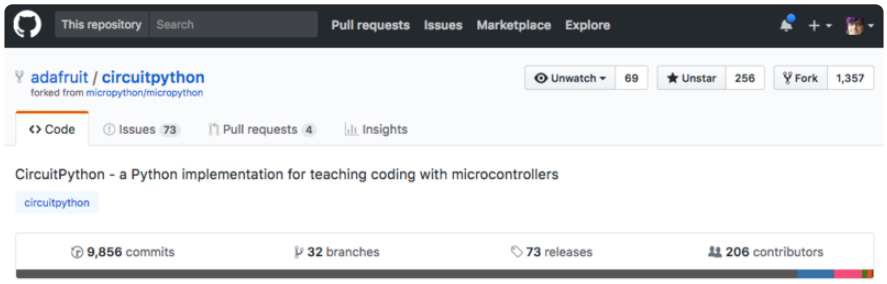
The fourth tab you'll find is the **CircuitPython Localization** tab.



If you speak another language, you can help translate CircuitPython! The translations apply to informational and error messages that are within the CircuitPython core. It means that folks who do not speak English have the opportunity to have these messages shown to them in their own language when using CircuitPython. This is incredibly important to provide the best experience possible for all users. CircuitPython uses Weblate to translate, which makes it much simpler to contribute translations. You will still need to know some CircuitPython-specific practices and a few basics about coding strings, but as with any CircuitPython contributions, folks are there to help.

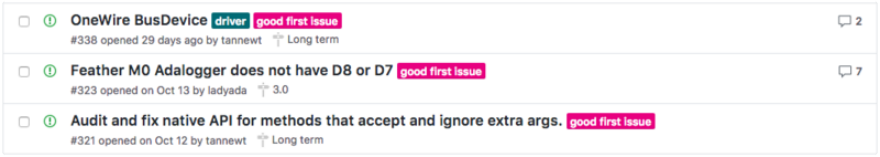
Regardless of your skill level, or how you want to contribute to the CircuitPython project, there is an opportunity available. The Contributing page () is an excellent place to start!

Adafruit GitHub



Whether you're just beginning or are life-long programmer who would like to contribute, there are ways for everyone to be a part of the CircuitPython project. The CircuitPython core is written in C. The libraries are written in Python. GitHub is the best source of ways to contribute to the CircuitPython core (), and the CircuitPython libraries (). If you need an account, visit https://github.com/ () and sign up.

If you're new to GitHub or programming in general, there are great opportunities for you. For the CircuitPython core, head over to the CircuitPython repository on GitHub, click on "Issues ()", and you'll find a list that includes issues labeled "good first issue ()". For the libraries, head over to the Contributing page Issues list (), and use the drop down menu to search for "good first issue ()". These issues are things that have been identified as something that someone with any level of experience can help with. These issues include options like updating documentation, providing feedback, and fixing simple bugs. If you need help getting started with GitHub, there is an excellent guide on Contributing to CircuitPython with Git and GitHub ().



Already experienced and looking for a challenge? Checkout the rest of either issues list and you'll find plenty of ways to contribute. You'll find all sorts of things, from new driver requests, to library bugs, to core module updates. There's plenty of opportunities for everyone at any level!

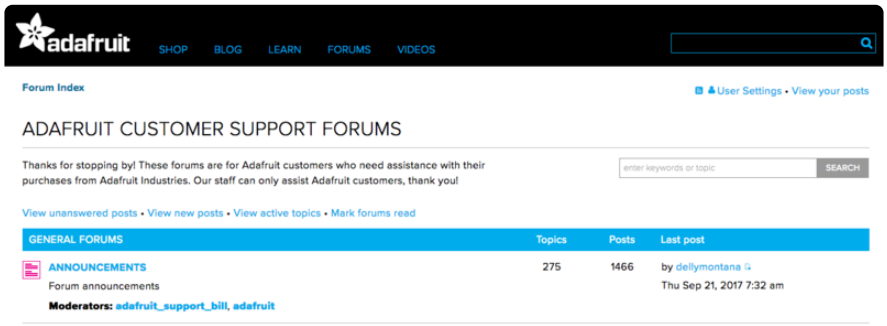
When working with or using CircuitPython or the CircuitPython libraries, you may find problems. If you find a bug, that's great! The team loves bugs! Posting a detailed issue to GitHub is an invaluable way to contribute to improving CircuitPython. For CircuitPython itself, file an issue here (). For the libraries, file an issue on the specific

library repository on GitHub. Be sure to include the steps to replicate the issue as well as any other information you think is relevant. The more detail, the better!

Testing new software is easy and incredibly helpful. Simply load the newest version of CircuitPython or a library onto your CircuitPython hardware, and use it. Let us know about any problems you find by posting a new issue to GitHub. Software testing on both stable and unstable releases is a very important part of contributing CircuitPython. The developers can't possibly find all the problems themselves! They need your help to make CircuitPython even better.

On GitHub, you can submit feature requests, provide feedback, report problems and much more. If you have questions, remember that Discord and the Forums are both there for help!

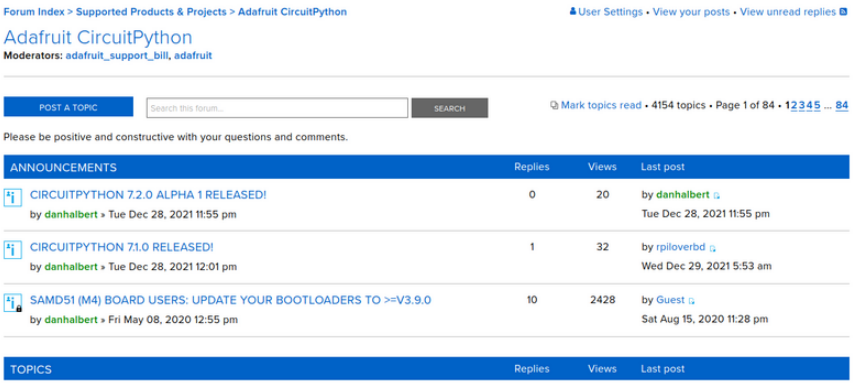
Adafruit Forums



The Adafruit Forums () are the perfect place for support. Adafruit has wonderful paid support folks to answer any questions you may have. Whether your hardware is giving you issues or your code doesn't seem to be working, the forums are always there for you to ask. You need an Adafruit account to post to the forums. You can use the same account you use to order from Adafruit.

While Discord may provide you with quicker responses than the forums, the forums are a more reliable source of information. If you want to be certain you're getting an Adafruit-supported answer, the forums are the best place to be.

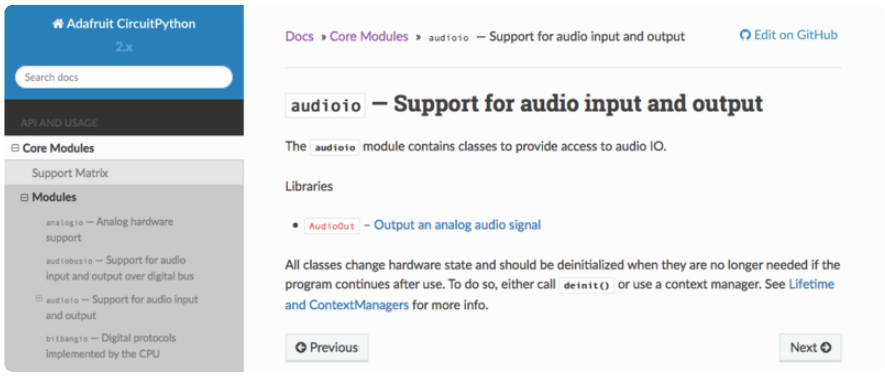
There are forum categories that cover all kinds of topics, including everything Adafruit. The Adafruit CircuitPython () category under "Supported Products & Projects" is the best place to post your CircuitPython questions.



Be sure to include the steps you took to get to where you are. If it involves wiring, post a picture! If your code is giving you trouble, include your code in your post! These are great ways to make sure that there's enough information to help you with your issue.

You might think you're just getting started, but you definitely know something that someone else doesn't. The great thing about the forums is that you can help others too! Everyone is welcome and encouraged to provide constructive feedback to any of the posted questions. This is an excellent way to contribute to the community and share your knowledge!

Read the Docs



Read the Docs () is a an excellent resource for a more detailed look at the CircuitPython core and the CircuitPython libraries. This is where you'll find things like API documentation and example code. For an in depth look at viewing and understanding Read the Docs, check out the CircuitPython Documentation () page!

Here is blinky:

```
import time
import digitalio
import board

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT
while True:
    led.value = True
    time.sleep(0.1)
    led.value = False
    time.sleep(0.1)
```

CircuitPython Essentials

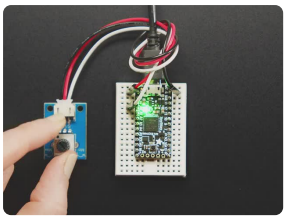
You've been introduced to CircuitPython, and worked through getting everything set up. What's next? CircuitPython Essentials!

There are a number of core modules built into CircuitPython, which can be used along side the many CircuitPython libraries available. The following pages demonstrate some of these modules. Each page presents a different concept including a code example with an explanation. All of the examples are designed to work with your microcontroller board.

Time to get started learning the CircuitPython essentials!

Some examples require external components, such as switches or sensors. You'll find wiring diagrams where applicable to show you how to wire up the necessary components to work with each example.

The following components are needed to complete all of the examples:



Featured

STEMMA Wired Potentiometer Breakout Board - 10K ohm Linear

For the easiest way possible to measure twists, turn to this STEMMA potentiometer breakout (ha!). This plug-n-play pot comes with a JST-PH 2mm connector and a matching

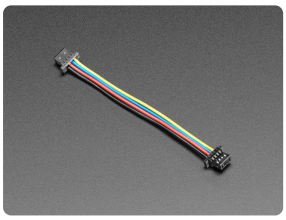
https://www.adafruit.com/product/4493

Featured

Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/ precise we've ever seen, with a typical accuracy of ±0.25°C over the sensor's -40°C to...

https://www.adafruit.com/product/5027



Featured

STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long

This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

https://www.adafruit.com/product/4399

Blink

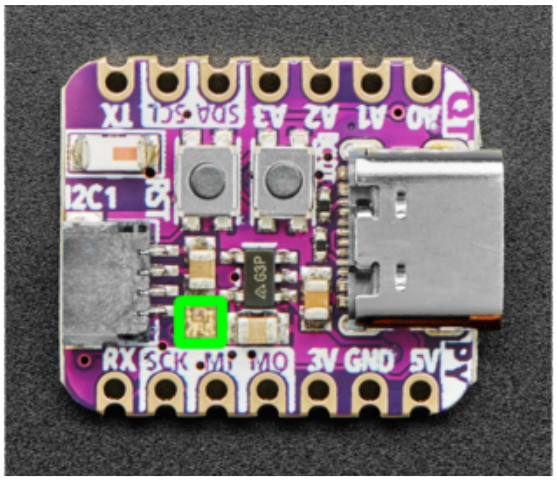
In learning any programming language, you often begin with some sort of `Hello, World!` program. In CircuitPython, Hello, World! is blinking an LED. Blink is one of the simplest programs in CircuitPython. Despite its simplicity, it shows you many of the basic concepts needed for most CircuitPython programs, and provides a solid basis for more complex projects. Your board has a built-in NeoPixel LED that is great this example.

A NeoPixel is what Adafruit calls the WS281x family of addressable RGB LEDs. The built-in status LED on your board is a NeoPixel! It contains three LEDs - a red one, a green one and a blue one - along side a driver chip in a tiny package controlled by a single pin. They can be used individually (as in the built-in LED on your board), or chained together in strips or other creative form factors. NeoPixels do not light up on

their own; they require a microcontroller. So, it's super convenient that the NeoPixel is built in to your microcontroller board!

Time to get blinky!

NeoPixel Location



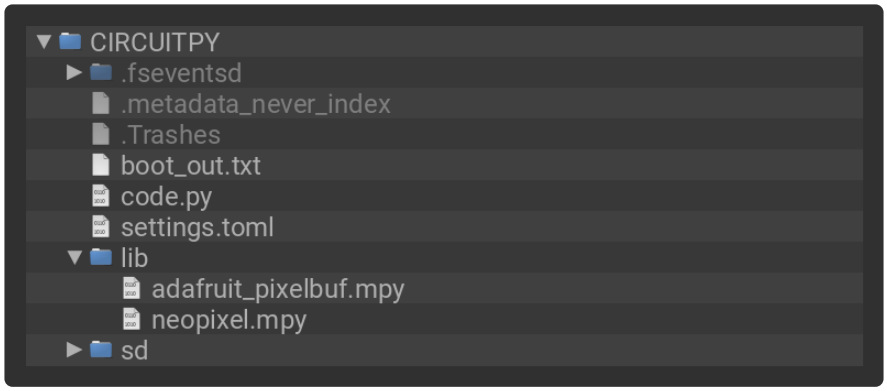
The **RGB NeoPixel LED** (highlighted in green) is located near the SCK/MI on the board silk, at the corner of the STEMMA QT sensor.

Blinking a NeoPixel LED

To use with CircuitPython, you need to first install a few libraries, into the lib folder on your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/neopixel_blink_one_pixel/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

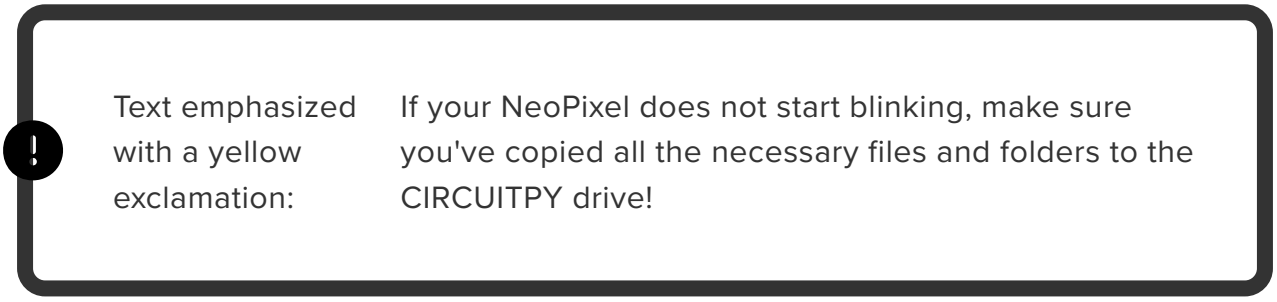
Your **CIRCUITPY** drive should now look similar to the following image:



```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython blink example for built-in NeoPixel LED"""
import time
import board
import neopixel

pixel = neopixel.NeoPixel(board.NEOPIXEL, 1)

while True:
    pixel.fill((255, 0, 0))
    time.sleep(0.5)
    pixel.fill((0, 0, 0))
    time.sleep(0.5)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/neopixel_blink_one_pixel/code.py

The built-in NeoPixel LED begins blinking!



It's important to understand what is going on in this program.

First you `import` three modules: `time`, `board` and `neopixel`. This makes these modules and libraries available for use in your code. The first two are modules built-in to CircuitPython, so you don't need to download anything to use those. The `neopixel` library is separate, which is why you needed to install it before getting started.

Next, you set up the NeoPixel LED. To interact with hardware in CircuitPython, your code must let the board know where to look for the hardware and what to do with it. So, you create a `neopixel.NeoPixel()` object, provide it the NeoPixel LED pin using the `board` module, and tell it the number of LEDs. You save this object to the variable `pixel`.

Finally, you create a `while True:` loop. This means all the code inside the loop will repeat indefinitely. Inside the loop, you "fill" the pixel with red using the RGB tuple `(255, 0, 0)`. (For more information on how RGB tuples work, see the next section!) Then, you use `time.sleep(0.5)` to tell the code to wait half a second before

moving on to the next line. The next fills the pixel with "black", which turns it off. Then you use another `time.sleep(0.5)` to wait half a second before starting the loop over again.

With only a small update, you can control the blink speed. The blink speed is controlled by the amount of time you tell the code to wait before moving on using `time.sleep()`. The example uses `0.5`, which is one half of one second. Try increasing or decreasing these values to see how the blinking changes.

That's all there is to blinking a built-in NeoPixel LED using CircuitPython!

RGB LED Colors

RGB LED colors are set using a combination of **r**ed, **g**reen, and **b**lue, in the form of an (**R**, **G**, **B**) tuple. Each member of the tuple is set to a number between 0 and 255 that determines the amount of each color present. Red, green and blue in different combinations can create all the colors in the rainbow! So, for example, to set an LED to red, the tuple would be (255, 0, 0), which has the maximum level of red, and no green or blue. Green would be (0, 255, 0), etc. For the colors between, you set a combination, such as cyan which is (0, 255, 255), with equal amounts of green and blue. If you increase all values to the same level, you get white! If you decrease all the values to 0, you turn the LED off.

Common colors include:

- red: (255, 0, 0)
- green: (0, 255, 0)
- blue: (0, 0, 255)
- cyan: (0, 255, 255)
- purple: (255, 0, 255)
- yellow: (255, 255, 0)
- white: (255, 255, 255)
- black (off): (0, 0, 0)

CircuitPython Internet Test

One of the great things about most Espressif microcontrollers are their built-in WiFi capabilities. This page covers the basics of getting connected using CircuitPython.

The first thing you need to do is update your **code.py** to the following (it will error until WiFi details are added). Click the **Download Project Bundle** button to download the

necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py** file to your **CIRCUITPY** drive.

```
# SPDX-FileCopyrightText: 2020 Brent Rubell for Adafruit Industries
#
# SPDX-License-Identifier: MIT

import os
import ipaddress
import ssl
import wifi
import socketpool
import adafruit_requests

# URLs to fetch from
TEXT_URL = "http://wifitest.adafruit.com/testwifi/index.html"
JSON_QUOTES_URL = "https://www.adafruit.com/api/quotes.php"
JSON_STARS_URL = "https://api.github.com/repos/adafruit/circuitpython"

print("ESP32-S2 WebClient Test")

print(f"My MAC address: {[hex(i) for i in wifi.radio.mac_address]}")

print("Available WiFi networks:")
for network in wifi.radio.start_scanning_networks():
    print("\t%s\t\tRSSI: %d\tChannel: %d" % (str(network.ssid, "utf-8"),
                                             network.rssi, network.channel))
wifi.radio.stop_scanning_networks()

print(f"Connecting to {os.getenv('CIRCUITPY_WIFI_SSID')}")
wifi.radio.connect(os.getenv("CIRCUITPY_WIFI_SSID"), os.getenv("CIRCUITPY_WIFI_PASSWORD"))
print(f"Connected to {os.getenv('CIRCUITPY_WIFI_SSID')}")
print(f"My IP address: {wifi.radio.ipv4_address}")

ping_ip = ipaddress.IPv4Address("8.8.8.8")
ping = wifi.radio.ping(ip=ping_ip)

# retry once if timed out
if ping is None:
    ping = wifi.radio.ping(ip=ping_ip)

if ping is None:
    print("Couldn't ping 'google.com' successfully")
else:
    # convert s to ms
    print(f"Pinging 'google.com' took: {ping * 1000} ms")

pool = socketpool.SocketPool(wifi.radio)
requests = adafruit_requests.Session(pool, ssl.create_default_context())

print(f"Fetching text from {TEXT_URL}")
response = requests.get(TEXT_URL)
print("-" * 40)
print(response.text)
print("-" * 40)

```

```
print(f"Fetching json from {JSON_QUOTES_URL}")
response = requests.get(JSON_QUOTES_URL)
print("-" * 40)
print(response.json())
print("-" * 40)

print()

print(f"Fetching and parsing json from {JSON_STARS_URL}")
response = requests.get(JSON_STARS_URL)
print("-" * 40)
print(f"CircuitPython GitHub Stars: {response.json()['stargazers_count']}")
print("-" * 40)

print("Done")
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/ESP32_S2_WiFi_Tests/CPy_Native_WiFi_Test/code.py

Your **CIRCUITPY** drive should resemble the following.

To get connected, the next thing you need to do is update the **settings.toml** file.

The settings.toml File

We expect people to share tons of projects as they build CircuitPython WiFi widgets. What we want to avoid is people accidentally sharing their passwords or secret tokens and API keys. So, we designed all our examples to use a **settings.toml** file, that is on your **CIRCUITPY** drive, to hold secret/private/custom data. That way you can share your main project without worrying about accidentally sharing private stuff.

If you have a fresh install of CircuitPython on your board, the initial **settings.toml** file on your **CIRCUITPY** drive is empty.

To get started, you can update the **settings.toml** on your **CIRCUITPY** drive to contain the following code.

```
# SPDX-FileCopyrightText: 2023 Adafruit Industries
#
# SPDX-License-Identifier: MIT

# This is where you store the credentials necessary for your code.
# The associated demo only requires WiFi, but you can include any
# credentials here, such as Adafruit IO username and key, etc.
CIRCUITPY_WIFI_SSID = "your-wifi-ssid"
CIRCUITPY_WIFI_PASSWORD = "your-wifi-password"
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/ESP32_S2_WiFi_Tests/CPy_Native_WiFi_Test/settings.toml

This file should contain a series of Python variables, each assigned to a string. Each variable should describe what it represents (say `wifi_ssid`), followed by an = (equals sign), followed by the data in the form of a Python string (such as `"my-wifi-password"` including the quote marks).

At a minimum you'll need to add/update your WiFi SSID and WiFi password, so do that now!

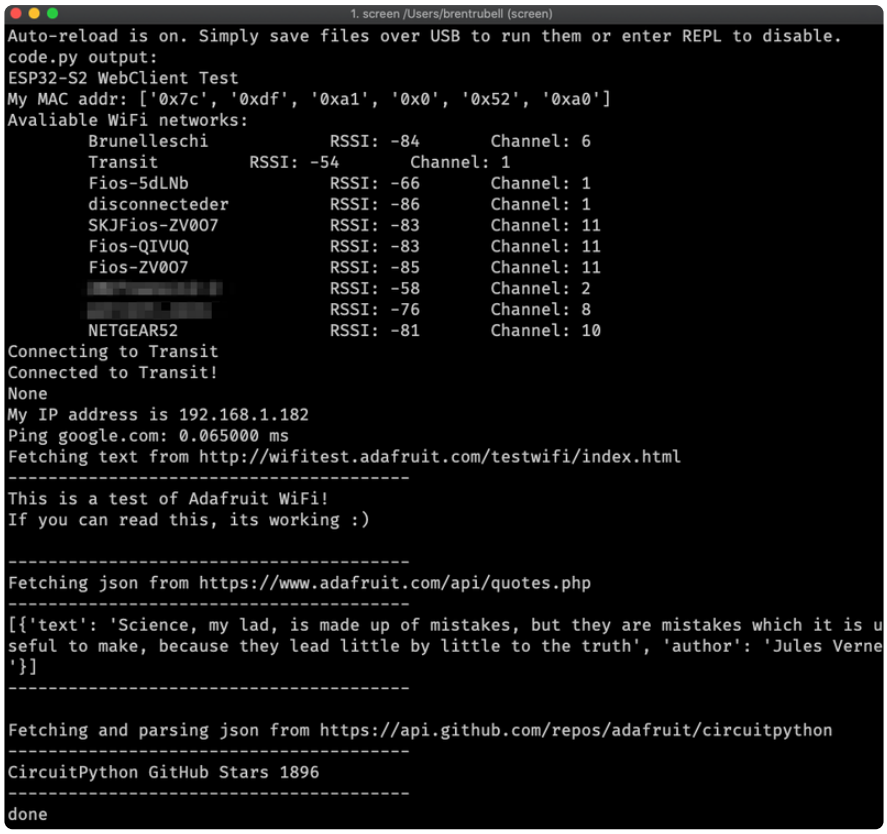
As you make projects you may need more tokens and keys, just add them one line at a time. See for example other tokens such as one for accessing GitHub or the Hackaday API. Other non-secret data like your timezone can also go here.

For the correct time zone string, look at http://worldtimeapi.org/timezones () and remember that if your city is not listed, look for a city in the same time zone, for example Boston, New York, Philadelphia, Washington DC, and Miami are all on the same time as New York.

Of course, don't share your **settings.toml** - keep that out of GitHub, Discord or other project-sharing sites.

Text emphasized with a yellow exclamation:	Don't share your settings.toml file! It has your passwords and API keys in it!

If you connect to the serial console, you should see something like the following:



In order, the example code...

Checks the ESP32's MAC address.

```
print(f"My MAC address: {[hex(i) for i in wifi.radio.mac_address]}")
```

Performs a scan of all access points and prints out the access point's name (SSID), signal strength (RSSI), and channel.

```
print("Available WiFi networks:")
for network in wifi.radio.start_scanning_networks():
    print("\t%s\t\tRSSI: %d\tChannel: %d" % (str(network.ssid, "utf-8"),
                                    network.rssi, network.channel))
wifi.radio.stop_scanning_networks()
```

Connects to the access point you defined in the **settings.toml** file, and prints out its local IP address.

```
print(f"Connecting to {os.getenv('WIFI_SSID')}")
wifi.radio.connect(os.getenv("WIFI_SSID"), os.getenv("WIFI_PASSWORD"))
print(f"Connected to {os.getenv('WIFI_SSID')}")
print(f"My IP address: {wifi.radio.ipv4_address}")
```

Attempts to ping a Google DNS server to test connectivity. If a ping fails, it returns `None`. Initial pings can sometimes fail for various reasons. So, if the initial ping is successful (`is not None`), it will print the echo speed in ms. If the initial ping fails, it will try one more time to ping, and then print the returned value. If the second ping fails, it will result in `"Ping google.com: None ms"` being printed to the serial console. Failure to ping does not always indicate a lack of connectivity, so the code will continue to run.

```
ping_ip = ipaddress.IPv4Address("8.8.8.8")
ping = wifi.radio.ping(ip=ping_ip) * 1000
if ping is not None:
    print(f"Ping google.com: {ping} ms")
else:
    ping = wifi.radio.ping(ip=ping_ip)
    print(f"Ping google.com: {ping} ms")
```

The code creates a socketpool using the wifi radio's available sockets. This is performed so we don't need to re-use sockets. Then, it initializes a a new instance of the requests () interface - which makes getting data from the internet really really easy.

```
pool = socketpool.SocketPool(wifi.radio)
requests = adafruit_requests.Session(pool, ssl.create_default_context())
```

To read in plain-text from a web URL, call `requests.get` - you may pass in either a http, or a https url for SSL connectivity.

```
print(f"Fetching text from {TEXT_URL}")
response = requests.get(TEXT_URL)
print("-" * 40)
print(response.text)
print("-" * 40)
```

Requests can also display a JSON-formatted response from a web URL using a call to `requests.get`.

```
print(f"Fetching json from {JSON_QUOTES_URL}")
response = requests.get(JSON_QUOTES_URL)
print("-" * 40)
print(response.json())
print("-" * 40)
```

Finally, you can fetch and parse a JSON URL using `requests.get`. This code snippet obtains the `stargazers_count` field from a call to the GitHub API.

```
print(f"Fetching and parsing json from {JSON_STARS_URL}")
response = requests.get(JSON_STARS_URL)
print("-" * 40)
print(f"CircuitPython GitHub Stars: {response.json()['stargazers_count']}")
print("-" * 40)
```

OK you now have your ESP32 board set up with a proper **settings.toml** file and can connect over the Internet. If not, check that your **settings.toml** file has the right SSID and password and retrace your steps until you get the Internet connectivity working!

IPv6 Networking

Starting in CircuitPython 9.2, IPv6 networking is available on most Espressif wifi boards. Socket-using libraries like **adafruit_requests** and **adafruit_ntp** will need to be updated to use the new APIs and for now can only connect to services on IPv4.

IPv6 connectivity & privacy

IPv6 addresses are divided into many special kinds, and many of those kinds (like those starting with **FC**, **FD**, **FE**) are private or local; Addresses starting with other prefixes like **2002:** and **2001:** are globally routable. In 2024, far from all ISPs and home networks support IPv6 internet connectivity. For more info consult resources like Wikipedia (). If you're interested in global IPv6 connectivity you can use services like Hurricane Electric () to create an "IPv6 tunnel" (free as of 2024, but requires expertise and a compatible router or host computer to set up)

It's also important to be aware that, as currently implemented by Espressif, there are privacy concerns especially when these devices operate on the global IPv6 network: The device's unique identifier (its EUI-64 or MAC address) is used by default as part of its IPv6 address. This means that the device identity can be tracked across multiple networks by any service it connects to.

Enable IPv6 networking

Due to the privacy consideration, IPv6 networking is not automatically enabled. Instead, it must be explicitly enabled by a call to `start_dhcp_client` with the `ipv6=True` argument specified:

```
wifi.start_dhcp_client(ipv6=True)
```

Check IP addresses

The read-only `addresses` property of the `wifi.radio` object holds all addresses, including IPv4 and IPv6 addresses:

```
>>> wifi.radio.addresses
('FE80::7EDF:A1FF:FE00:518C', 'FD5F:3F5C:FE50:0:7EDF:A1FF:FE00:518C', '10.0.3.96')
```

The `wifi.radio.dns` servers can be IPv4 or IPv6:

```
>>> wifi.radio.dns
('FD5F:3F5C:FE50::1',)
>>> wifi.radio.dns = ("1.1.1.1",)
>>> wifi.radio.dns
('1.1.1.1',)
```

Ping v6 networks

`wifi.radio.ping` accepts v6 addresses and names:

```
>>> wifi.radio.ping("google.com")
0.043
>>> wifi.radio.ping("ipv6.google.com")
0.048
```

Create & use IPv6 sockets

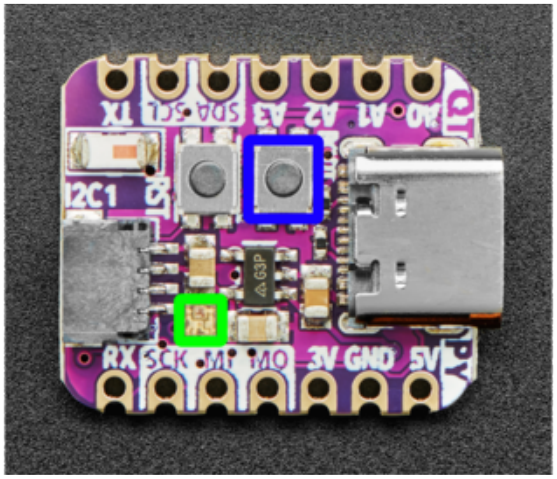
Use the address family `socket.AF_INET6`. After the socket is created, use methods like `connect`, `send`, `recfrom_into`, etc just like for IPv4 sockets. This code snippet shows communicating with a private-network NTP server; this IPv6 address will not work on your network:

```
>>> ntp_addr = ("fd5f:3f5c:fe50::20e", 123)
>>> PACKET_SIZE = 48
>>>
>>> buf = bytearray(PACKET_SIZE)
>>> with socket.socket(socket.AF_INET6, socket.SOCK_DGRAM) as s:
...     s.settimeout(1)
...     buf[0] = 0b0010_0011
...     s.sendto(buf, ntp_addr)
...     print(s.recvfrom_into(buf))
...     print(buf)
...
48
(48, ('fd5f:3f5c:fe50::20e', 123))
bytearray(b'$\x01\x03\xeb\x00\x00\x00\x00\x00\x00\x00GGPS\x00\xeaA0h\x07s;\xc0\x00\x00\x00\x00\x00\x00\x00\x
```

Digital Input

The CircuitPython `digitalio` module has many applications. You can easily set up a **digital input** such as a button to control the NeoPixel LED. This example builds on the basic Blink example, but now includes setup for a button switch. Instead of using the `time` module to blink the LED, it uses the status of the button switch to control whether the LED is turned on or off.

NeoPixel and Button



The **RGB NeoPixel LED** (highlighted in green) is located near the SCK/MI on the board silk, at the corner of the STEMMA QT sensor.
The **Boot button** (highlighted in blue) is located next to the Reset button, at the corner of the USB C connector.

Controlling the NeoPixel with a Button

To use the built-in NeoPixel on your board, you need to first install the NeoPixel library into the **lib** folder on your **CIRCUITPY** drive.

Then you need to update **code.py**.

Click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py file** to your **CIRCUITPY** drive.

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""
CircuitPython Digital Input example - Blinking a built-in NeoPixel LED using a button switch.
"""
import board
import digitalio
import neopixel

```

```
pixel = neopixel.NeoPixel(board.NEOPIXEL, 1)

button = digitalio.DigitalInOut(board.BUTTON)
button.switch_to_input(pull=digitalio.Pull.UP)

while True:
    if not button.value:
        pixel.fill((255, 0, 0))
    else:
        pixel.fill((0, 0, 0))
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/Adafruit_QT_Py_ESP32-S2/digital_input/code.py

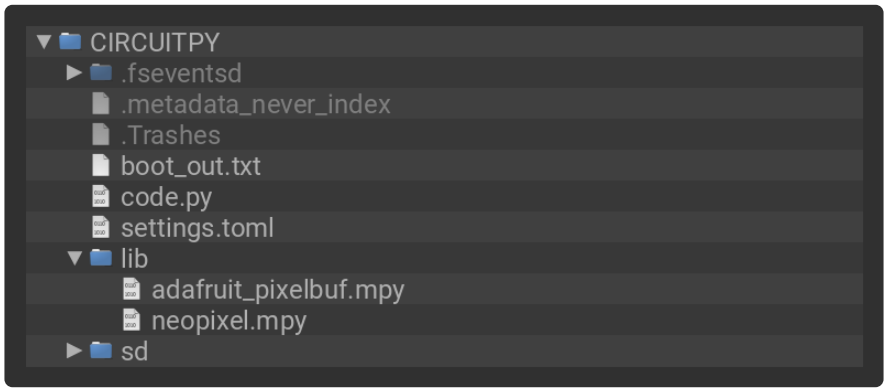
Your **CIRCUITPY** drive contents should resemble the image below.

You should have in **Root Folder** / of the **CIRCUITPY** drive:

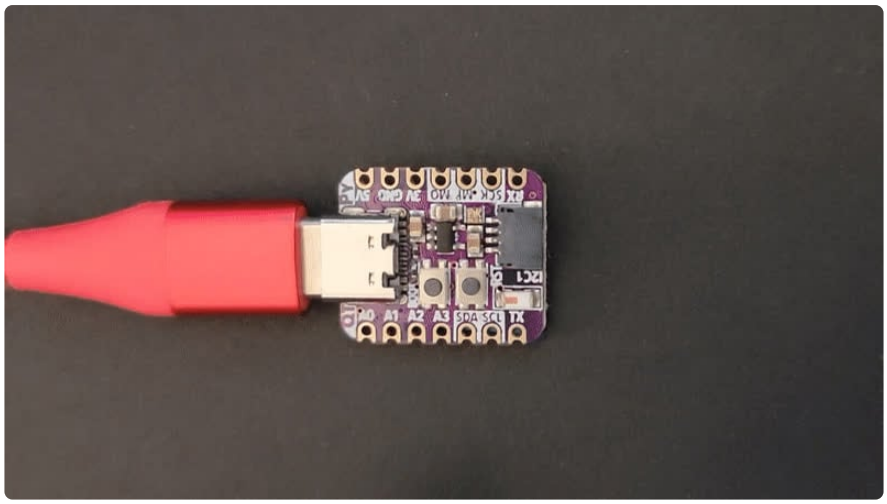
- **code.py**

And in the **lib** folder on your **CIRCUITPY** drive:

- **adafruit_pixelbuf.mpy**
- **neopixel.mpy**



Now, press the button. The NeoPixel lights up! Let go of the button and the NeoPixel turns off.



Text emphasized with a yellow exclamation:

If your NeoPixel does not light up when you press the button, make sure you've copied all the necessary files and folders to the CIRCUITPY drive!

First you `import` two modules, `board` and `digitalio`, and one library, `neopixel`. This makes these modules available for use in your code.

Next, you set up the NeoPixel. To interact with hardware in CircuitPython, your code must let the board know where to look for the hardware and what to do with it. So, you create a `neopixel.NeoPixel()` object, provide it the NeoPixel LED pin using the `board` module, and tell it the number of NeoPixels, `1`. You save this object to the variable `pixel`.

Then, you create a `digitalio.DigitalInOut()` object, provide it the button pin using the `board` module, and save it to the variable `button`. You tell the pin to act as an `INPUT` and provide a pull up.

Inside the loop, you check to see if the button is pressed, and if so, turn the NeoPixel red. Otherwise the NeoPixel is off.

That's all there is to controlling a NeoPixel LED with a button switch!

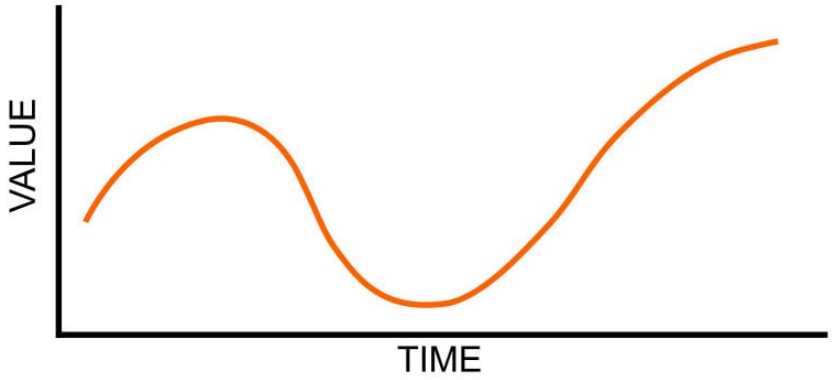
Analog In

Your microcontroller board has both digital and analog signal capabilities. Some pins are analog, some are digital, and some are capable of both. Check the **Pinouts** page in this guide for details about your board.

Analog signals are different from digital signals in that they can be any voltage and can vary continuously and smoothly between voltages. An analog signal is like a dimmer switch on a light, whereas a digital signal is like a simple on/off switch.

Digital signals only can ever have two states, they are either are **on** (high logic level voltage like 3.3V) or **off** (low logic level voltage like 0V / ground).

By contrast, analog signals can be any voltage in-between on and off, such as 1.8V or 0.001V or 2.98V and so on.



Analog signals are continuous values which means they can be an infinite number of different voltages. Think of analog signals like a floating point or fractional number, they can smoothly transiting to any in-between value like 1.8V, 1.81V, 1.801V, 1.8001V, 1.80001V and so forth to infinity.

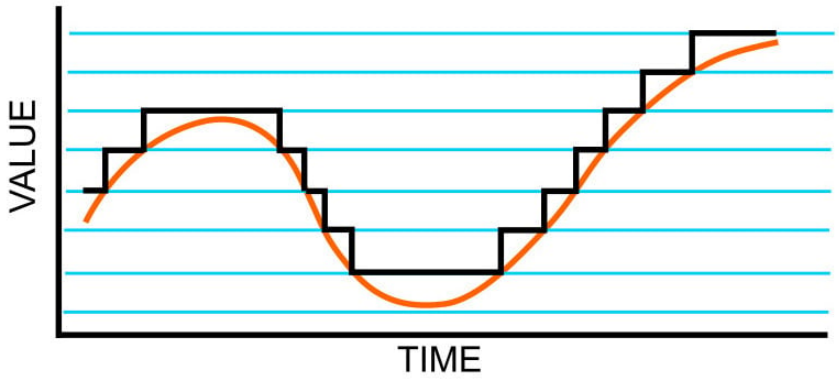
Many devices use analog signals, in particular sensors typically output an analog signal or voltage that varies based on something being sensed like light, heat, humidity, etc.

Analog to Digital Converter (ADC)

An analog-to-digital-converter, or ADC, is the key to reading analog signals and voltages with a microcontroller. An ADC is a device that reads the voltage of an

analog signal and converts it into a digital, or numeric, value. The microcontroller can't read analog signals directly, so the analog signal is first converted into a numeric value by the ADC.

The black line below shows a digital signal over time, and the red line shows the converted analog signal over the same amount of time.

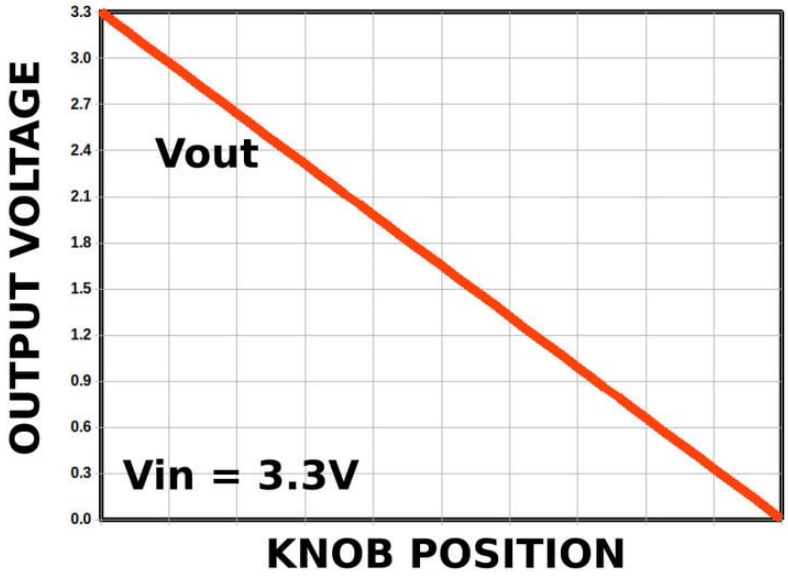


Once that analog signal has been converted by the ADC, the microcontroller can use those digital values any way you like!

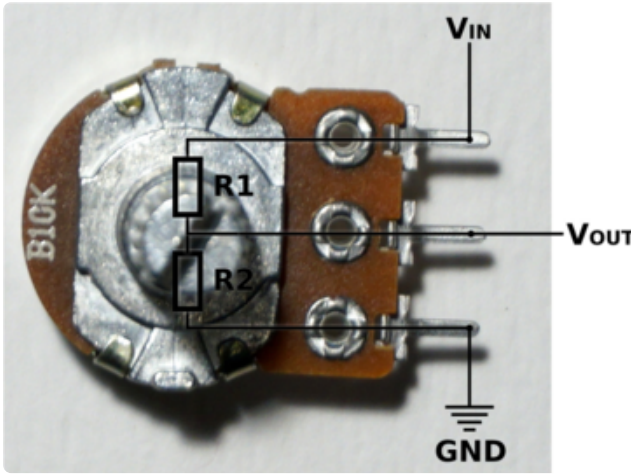
Potentiometers

A potentiometer is a small variable resistor that you can twist a knob or shaft to change its resistance. It has three pins. By twisting the knob on the potentiometer you can change the resistance of the middle pin (called the wiper) to be anywhere within the range of resistance of the potentiometer.

By wiring the potentiometer to your board in a special way (called a voltage divider) you can turn the change in resistance into a change in voltage that your board's analog to digital converter can read.



To wire up a potentiometer as a voltage divider:

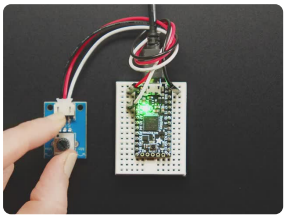


Connect **one outside pin** to **ground**
Connect **the other outside pin** to **voltage in (e.g. 3.3V)**
Connect **the middle pin** to **an analog pin (e.g. A0)**

Hardware

In addition to your microcontroller board, you will need the following hardware to follow along with this example.

Potentiometer



Featured

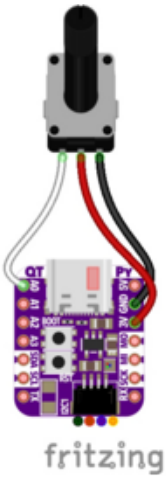
STEMMA Wired Potentiometer Breakout Board - 10K ohm Linear

For the easiest way possible to measure twists, turn to this STEMMA potentiometer breakout (ha!). This plug-n-play pot comes with a JST-PH 2mm connector and a matching

https://www.adafruit.com/product/4493

Wire Up the Potentiometer

Connect the potentiometer to your board as follows.



Potentiometer left pin (white wire) to **QT Py ESP32-S2 A0**
Potentiometer center pin (red wire) to **QT Py ESP32-S2 3.3V**
Potentiometer right pin (black wire) to **QT Py ESP32-S2 GND**

Reading Analog Pin Values

CircuitPython makes it easy to read analog pin values. Simply import two modules, set up the pin, and then print the value inside a loop.

You'll need to connect to the serial console () to see the values printed out.

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/analog_pin_values/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython analog pin value example"""
import time
import board
import analogio

analog_pin = analogio.AnalogIn(board.A0)

while True:
    print(analog_pin.value)
    time.sleep(0.1)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/analog_pin_values/code.py

Now, rotate the potentiometer to see the values change.

What do these values mean? In CircuitPython ADC values are put into the range of 16-bit unsigned values. This means the possible values you'll read from the ADC fall within the range of 0 to 65535 (or 2^16 - 1). When you twist the potentiometer knob to be near ground, or as far to the left as possible, you see a value close to zero.

When you twist it to the right, the value gets bigger up to some value that is dependent on the microcontroller. Many microcontrollers get a value very close to 65535. Some, such as the ESP32-S2, have a smaller limit of about 51000 or 2.57 volts.

The code is simple. You begin by importing three modules: `time`, `board` and `analogio`. All three modules are built into CircuitPython, so you don't need to download anything to get started.

Then, you set up the analog pin by creating an `analogio.AnalogIn()` object, providing it the desired pin using the `board` module, and saving it to the variable `analog_pin`.

Finally, in the loop, you print out the analog value with `analog_pin.value`, including a `time.sleep()` to slow down the values to a human-readable rate.

Reading Analog Voltage Values

These values don't necessarily equate to anything obvious. You can get an idea of the rotation of the potentiometer based on where in the range the value falls, but not without doing some math. Remember, you wired up the potentiometer as a voltage divider. By adding a simple function to your code, you can get a more human-readable value from the potentiometer.

You'll need to connect to the serial console () to see the values printed out.

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/analog_voltage_values_ESP32-S2/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```
# SPDX-FileCopyrightText: 2022 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython Analog In Voltage Example for ESP32-S2"""
import time
import board
import analogio
```

```
7
8   analog_pin = analogio.AnalogIn(board.A0)
9
10
11  def get_voltage(pin):
12      return (pin.value * 2.57) / 51000
13
14
15  while True:
16      print(get_voltage(analog_pin))
17      time.sleep(0.1)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/analog_voltage_values_ESP32-S2/code.py

Now, rotate the potentiometer to see the values change.

Now the values range from around 0 to 2.57! Note that due to variations in each chip, you may not get all the way to 0 or 2.57, and in some cases, you may exceed 2.57. Both of these possibilities are normal.

The example code begins with the same imports and pin setup.

This time, you include the `get_voltage` helper. This function requires that you provide an analog pin. It then maps the raw analog values, `0` to `51000`, to the voltage values, `0` to `2.57`. It does the math so you don't have to!

Finally, inside the loop, you provide the function with your `analog_pin`, and print the resulting values.

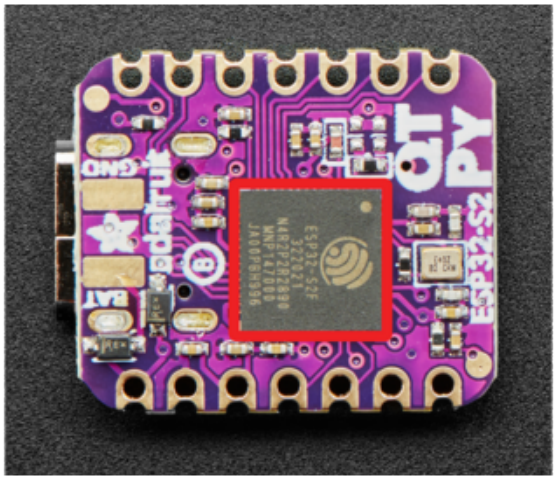
That's all there is to reading analog voltage values using CircuitPython!

CPU Temperature

There is a temperature sensor built into the CPU on your microcontroller board. It reads the internal CPU temperature, which varies depending on how long the board has been running or how intense your code is.

CircuitPython makes it really simple to read this data from the temperature sensor built into the microcontroller. Using the built-in `microcontroller` module, you can easily read the temperature.

Microcontroller Location



The **ESP32-S2 microcontroller** (highlighted in red) is located in the center of the back of the QT Py.

Reading the Microcontroller Temperature

The data is read using two lines of code. All necessary modules are built into CircuitPython, so you don't need to download any extra files to get started.

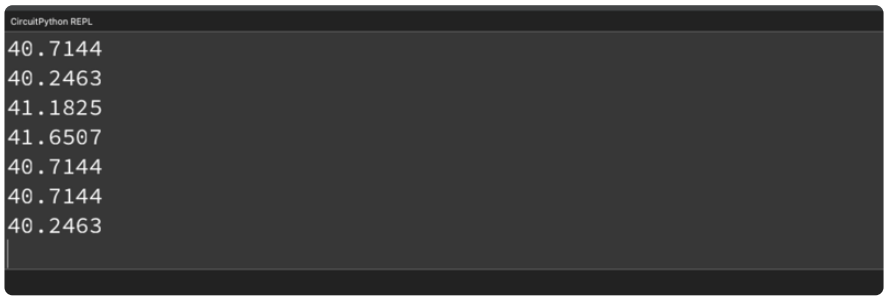
Connect to the serial console (), and then update your **code.py** to the following.

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/cpu_temperature/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython CPU temperature example in Celsius"""
import time
import microcontroller

while True:
    print(microcontroller.cpu.temperature)
    time.sleep(0.15)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/cpu_temperature/code.py



The CPU temperature in Celsius is printed out to the serial console!

Try putting your finger on the microcontroller to see the temperature change.

The code is simple. First you import two modules: `time` and `microcontroller`. Then, inside the loop, you print the microcontroller CPU temperature, and the `time.sleep()` slows down the print enough to be readable. That's it!

You can easily print out the temperature in Fahrenheit by adding a little math to your code, using this simple formula: Celsius * (9/5) + 32.

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/cpu_temperature_f/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython CPU temperature example in Fahrenheit"""
import time
import microcontroller

while True:
    print(microcontroller.cpu.temperature * (9 / 5) + 32)
    time.sleep(0.15)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/cpu_temperature_f/code.py

The CPU temperature in Fahrenheit is printed out to the serial console!

That's all there is to reading the CPU temperature using CircuitPython!

Storage

CircuitPython-compatible microcontrollers show up as a **CIRCUITPY** drive when plugged into your computer, allowing you to edit code directly on the board. Perhaps you've wondered whether or not you can write data from CircuitPython directly to the board to act as a data logger. The answer is **yes**!

The `storage` module in CircuitPython enables you to write code that allows CircuitPython to write data to the **CIRCUITPY** drive. This process requires you to include a **boot.py** file on your **CIRCUITPY** drive, along side your **code.py** file.

The **boot.py** file is special - the code within it is executed when CircuitPython starts up, either from a hard reset or powering up the board. It is not run on soft reset, for example, if you reload the board from the serial console or the REPL. This is in contrast to the code within **code.py**, which is executed after CircuitPython is already running.

The **CIRCUITPY** drive is typically writable by your computer; this is what allows you to edit your code directly on the board. The reason you need a **boot.py** file is that you have to set the filesystem to be read-only by your computer to allow it to be writable by CircuitPython. This is because CircuitPython cannot write to the filesystem at the same time as your computer. Doing so can lead to filesystem corruption and loss of all content on the drive, so CircuitPython is designed to only allow one at at time.

Text emphasized with a red exclamation:	You can only have EITHER your computer edit files on the CIRCUITPY drive, OR have CircuitPython edit files. You cannot have both writing to the CIRCUITPY drive at the same time. CircuitPython doesn't allow it!

The boot.py File

Text emphasized with a blue exclamation:	The filesystem will NOT automatically be set to read-only on creation of this file! You'll still be able to edit files on CIRCUITPY after saving this boot.py.

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""
CircuitPython Essentials Storage CP Filesystem boot.py file
"""
import time
import board
import digitalio
import storage
import neopixel

pixel = neopixel.NeoPixel(board.NEOPIXEL, 1)

button = digitalio.DigitalInOut(board.BUTTON)
```

```
15  button.switch_to_input(pull=digitalio.Pull.UP)
16
17  # Turn the NeoPixel blue for one second to indicate when to press the boot button.
18  pixel.fill((255, 255, 255))
19  time.sleep(1)
20
21  # If the button is connected to ground, the filesystem is writable by CircuitPython
22  storage.remount("/", readonly=button.value)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/Adafruit_QT_Py_ESP32-S2/Storage/boot.py

The `storage.remount()` command has a `readonly` keyword argument. **This argument refers to the read/write state of CircuitPython.** It does NOT refer to the read/write state of your computer.

When the button is pressed, it returns `False`. The `readonly` argument in **boot.py** is set to the `value` of the button. When the `value=True`, the **CIRCUITPY** drive is read-only to CircuitPython (and writable by your computer). When the `value=False`, the **CIRCUITPY** drive is writable by CircuitPython (and read-only by your computer).

The code.py File

Save the following as **code.py** on your **CIRCUITPY** drive.

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **Adafruit_QT_Py_ESP32-S2/Storage/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""
CircuitPython Essentials Storage CP Filesystem code.py file
"""
import time
import board
import microcontroller
import neopixel

pixel = neopixel.NeoPixel(board.NEOPIXEL, 1)

try:
    with open("/temperature.txt", "a") as temp_log:
        while True:
            # The microcontroller temperature in Celsius. Include the
            # math to do the C to F conversion here, if desired.
            temperature = microcontroller.cpu.temperature

            # Write the temperature to the temperature.txt file every 10 seconds.
            temp_log.write('{0:.2f}\n'.format(temperature))
            temp_log.flush()

            # Blink the NeoPixel on every write...
            pixel.fill((255, 0, 0))
            time.sleep(1)  # ...for one second.
            pixel.fill((0, 0, 0))  # Then turn it off...
            time.sleep(9)  # ...for the other 9 seconds.

except OSError as e:  # When the filesystem is NOT writable by CircuitPython...
    delay = 0.5  # ...blink the NeoPixel every half second.
    if e.args[0] == 28:  # If the file system is full...
        delay = 0.15  # ...blink the NeoPixel every 0.15 seconds!
    while True:
        pixel.fill((255, 0, 0))
        time.sleep(delay)
        pixel.fill((0, 0, 0))
        time.sleep(delay)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/Adafruit_QT_Py_ESP32-S2/Storage/code.py

First you import the necessary modules to make them available to your code, and you set up the LED.

Next you have a `try` / `except` block, which is used to handle the three potential states of the board: read/write, read-only, or filesystem full. The code in the `try` block will run if the filesystem is writable by CircuitPython. The code in the `except` block will run if the filesystem is read-only to CircuitPython OR if the filesystem is full.

Under the `try`, you open a **temperature.txt** log file. If it is the first time, it will create the file. For all subsequent times, it opens the file and appends data. Inside the loop, you get the microcontroller temperature value and assign it to a `temperature`

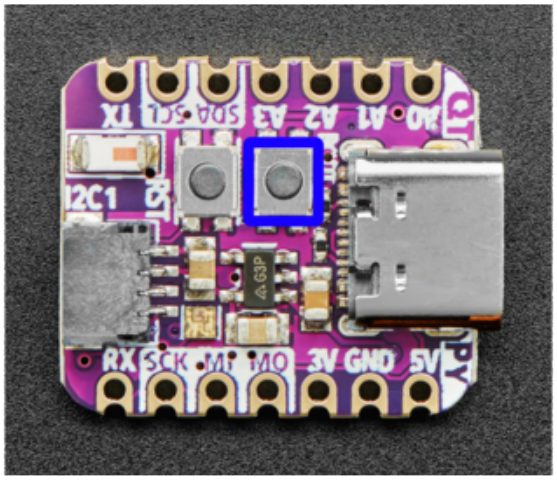
variable. Then, you write the temperature value to the log file, followed by clearing the buffer for the next time through the loop. The temperature data is limited to two decimal points to save space for more data. Finally, you turn the LED on for one second, and then turn it off for the next nine seconds. Essentially, you blink the LED for one second every time the temperature is logged to the file which happens every ten seconds.

Next you `except` an `OSError`. An `OSError` number 30 is raised when trying to create, open or write to a file on a filesystem that is read-only to CircuitPython. If any `OSError` other than 28 is raised (e.g. 30), the `delay` is set to 0.5 seconds. If the filesystem fills up, CircuitPython raises `OSError` number 28. If `OSError` number 28 is raised, the `delay` is set to 0.15 seconds. Inside the loop, the LED is turned on for the duration of the `delay`, and turned off for the duration of the `delay`, effectively blinking the LED at the speed of the `delay`.

Logging the Temperature

At the moment, the LED on your board should be blinking once every half second. This indicates that the board is currently read-only to CircuitPython, and writable to your computer, allowing you to update the files on your CIRCUITPY drive as needed.

The way the code in **boot.py** works is, it checks to see if the button is pressed when the board is powered on and boot.py is run. To begin logging the temperature, you must press the button.



The **boot button** (highlighted in blue) is labeled Boot on the silk, and is located at the corner of the USB connector.

While holding down the button, you need to either hard reset the board by pressing the reset button, or by unplugging the USB cable and plugging it back in. This will run the code within **boot.py** and set your board to writable by CircuitPython, and therefore, read-only by the computer.

Text emphasized with a blue exclamation:	For the QT Py ESP32-S2, the button-press timing is a little different. Press it when the NeoPixel LED turns white!

For the QT Py ESP32-S2, it's difficult to get the timing right for when to press the boot button. So, the **boot.py** file includes turning the NeoPixel on bright white for one second. Press the boot button when the NeoPixel is white!

The red blinking will slow down to one second long, every 10 seconds. This indicates that the board is currently logging the temperature, once every 10 seconds.

As long as the button is pressed, you can plug the board in anywhere you have USB power, and log the temperature in that location! The temperature is not the ambient temperature; it is the temperature inside the microcontroller, which will typically be higher than ambient temperature. However, running only this code, once the microcontroller temperature stabilises, it should at least be consistent, and therefore usable for tracking changes in ambient temperature.

If the LED starts blinking really quickly, it means the filesystem is full! You'll need to get your temperature data and delete the temperature log file to begin again.

That's all there is to logging the temperature using CircuitPython!

Recovering a Read-Only Filesystem

In the event that you make your **CIRCUITPY** drive read-only to your computer, and for some reason, it doesn't easily switch back to writable, there are a couple of things you can do to recover the filesystem.

Even when the **CIRCUITPY** drive is read-only to your computer, you can still access the serial console and REPL. If you connect to the serial console and enter the REPL, you can run either of the following two sets of commands at the `>>>` prompt. You do not need to run both.

First, you can rename your **boot.py** file to something other than **boot.py**.

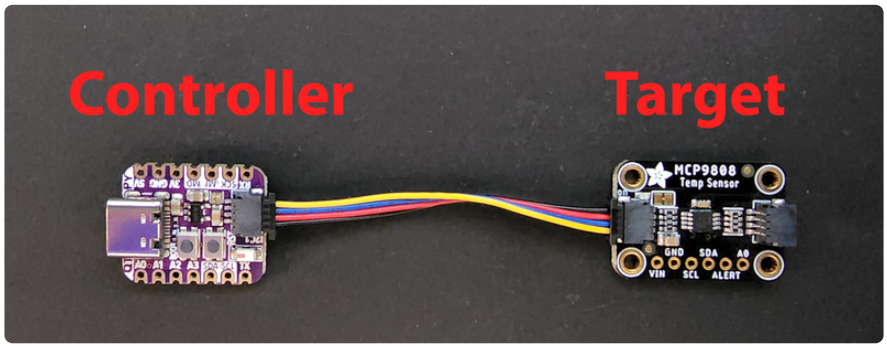
```
import os
os.rename("boot.py", "something_else.py")
```

Alternatively, you can remove the **boot.py** file altogether.

```
import os
os.remove("boot.py")
```

Then, restart the board by either hitting the reset button or unplugging USB and plugging it back in. **CIRCUITPY** should show up on your computer as usual, but now it should be writable by your computer.

I2C



The **I2C**, or inter-integrated circuit (), is a 2-wire protocol for communicating with simple sensors and devices, which means it uses two connections, or wires, for transmitting and receiving data. One connection is a clock, called **SCL**. The other is the data line, called **SDA**. Each pair of clock and data pins are referred to as a **bus**.

Typically, there is a device that acts as a **controller** and sends requests to the **target** devices on each bus. In this case, your microcontroller board acts as the controller, and the sensor breakout acts as the target. Historically, the controller is referred to as the master, and the target is referred to as the slave, so you may run into that terminology elsewhere. The official terminology is controller and target ().

Multiple I2C devices can be connected to the same clock and data lines. Each I2C device has an address, and as long as the addresses are different, you can connect them at the same time. This means you can have many different sensors and devices all connected to the same two pins.

Both I2C connections require pull-up resistors, and most Adafruit I2C sensors and breakouts have pull-up resistors built in. If you're using one that does not, you'll need to add your own 2.2-10kΩ pull-up resistors from SCL and SDA to 3.3V.

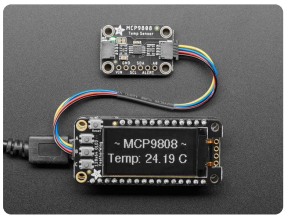
I2C and CircuitPython

CircuitPython supports many I2C devices, and makes it super simple to interact with them. There are libraries available for many I2C devices in the CircuitPython Library Bundle (). (If you don't see the sensor you're looking for, keep checking back, more are being written all the time!)

In this section, you'll learn how to scan the I2C bus for all connected devices. Then you'll learn how to interact with an I2C device.

Necessary Hardware

You'll need the following additional hardware to complete the examples on this page.

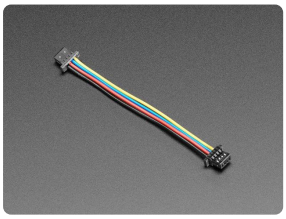


Featured

Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/ precise we've ever seen, with a typical accuracy of ±0.25°C over the sensor's -40°C to...

https://www.adafruit.com/product/5027



Featured

STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long

This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

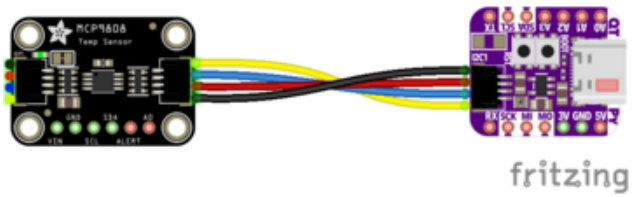
https://www.adafruit.com/product/4399

While the examples here will be using the Adafruit MCP9808 (http://adafru.it/5027), a high accuracy temperature sensor, the overall process is the same for just about any I2C sensor or device.

The first thing you'll want to do is get the sensor connected so your board has I2C to talk to.

Wiring the MCP9808

The MCP9808 comes with a STEMMA QT connector, which makes wiring it up quite simple and solder-free.



Simply connect the STEMMA QT cable from **the STEMMA QT port on your board** to **the STEMMA QT port on the MCP9808**.

Find Your Sensor

The first thing you'll want to do after getting the sensor wired up, is make sure it's wired correctly. You're going to do an I2C scan to see if the board is detected, and if it is, print out its I2C address.

Save the following to your **CIRCUITPY** drive as **code.py**.

Click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, find your CircuitPython version, and copy the matching **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython I2C Device Address Scan"""
import time
import board
```

```
6
7  i2c = board.I2C()  # uses board.SCL and board.SDA
8  # i2c = board.STEMMA_I2C()  # For using the built-in STEMMA QT connector on a microcontroller
9
10 # To create I2C bus on specific pins
11 # import busio
12 # i2c = busio.I2C(board.GP1, board.GP0)    # Pi Pico RP2040
13
14 while not i2c.try_lock():
15     pass
16
17 try:
18     while True:
19         print(
20             "I2C addresses found:",
21             [hex(device_address) for device_address in i2c.scan()],
22         )
23         time.sleep(2)
24
25 finally:  # unlock the i2c bus when ctrl-c'ing out of the loop
26     i2c.unlock()
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/i2c_scan/code.py

If you run this and it seems to hang, try manually unlocking your I2C bus by running the following two commands from the REPL.

```
import board
board.I2C().unlock()
```

First you create the `i2c` object, using `board.I2C()`. This convenience routine creates and saves a `busio.I2C` object using the default pins `board.SCL` and `board.SDA`. If the object has already been created, then the existing object is returned. No matter how many times you call `board.I2C()`, it will return the same object. This is called a singleton.

To be able to scan it, you need to lock the I2C down so the only thing accessing it is the code. So next you include a loop that waits until I2C is locked and then continues on to the scan function.

Last, you have the loop that runs the actual scan, `i2c_scan()`. Because I2C typically refers to addresses in hex form, the example includes this bit of code that formats the

results into hex format: `[hex(device_address) for device_address in i2c.scan()]`.

Open the serial console to see the results! The code prints out an array of addresses. You've connected the MCP9808 which has a 7-bit I2C address of 0x18. The result for this sensor is `I2C addresses found: ['0x18']`. If no addresses are returned, refer back to the wiring diagrams to make sure you've wired up your sensor correctly.

I2C Sensor Data

Now you know for certain that your sensor is connected and ready to go. Time to find out how to get the data from the sensor!

Save the following to your **CIRCUITPY** drive as **code.py**.

Click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, find your CircuitPython version, and copy the matching **entire lib folder** and **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython I2C MCP9808 Temperature Sensor Example"""
import time
import board
import adafruit_mcp9808

i2c = board.I2C()  # uses board.SCL and board.SDA
# i2c = board.STEMMA_I2C()  # For using the built-in STEMMA QT connector on a microcontroller
# import busio
# i2c = busio.I2C(board.SCL1, board.SDA1) # For QT Py RP2040, QT Py ESP32-S2
mcp9808 = adafruit_mcp9808.MCP9808(i2c)

while True:
```

```
    temperature_celsius = mcp9808.temperature
    temperature_fahrenheit = temperature_celsius * 9 / 5 + 32
    print("Temperature: {:.2f} C {:.2f} F ".format(temperature_celsius, temperature_fahrenheit))
    time.sleep(2)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/i2c_mcp9808/code.py

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disab
le.
code.py output:
Temperature: 23.38 C 74.07 F
```

Text emphasized with a blue exclamation:	For the QT Py ESP32-S2, you'll need to change the I2C setup to the commented out setup included in the code above.

The QT Py ESP32-S2 STEMMA QT connector uses `board.SCL1` and `board.SDA1` for the I2C pins. Comment out the current `i2c` setup line, and uncomment the two lines following it to use the STEMMA QT connector on your board.

This code begins the same way as the scan code, except this time, you create your sensor object using the sensor library. You call it `mcp9808` and provide it the `i2c` object.

Then you have a simple loop that prints out the temperature reading using the sensor object you created. Finally, there's a `time.sleep(2)`, so it only prints once every two seconds. Connect to the serial console to see the results. Try touching the MCP9808 with your finger to see the values change!

Where's my I2C?

On many microcontrollers, you have the flexibility of using a wide range of pins for I2C. On some types of microcontrollers, any pin can be used for I2C! Other chips require using bitbangio, but can also use any pins for I2C. There are further microcontrollers that may have fixed I2C pins.

Given the many different types of microcontroller boards available, it's impossible to guarantee anything other than the labeled 'SDA' and 'SCL' pins. So, if you want some

other setup, or multiple I2C interfaces, how will you find those pins? Easy! Below is a handy script.

Save the following to your **CIRCUITPY** drive as **code.py**.

Click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, find your CircuitPython version, and copy the matching **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```
# SPDX-FileCopyrightText: 2021-2023 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython I2C possible pin-pair identifying script"""
import board
import busio
from microcontroller import Pin


def is_hardware_i2c(scl, sda):
    try:
        p = busio.I2C(scl, sda)
        p.deinit()
        return True
    except ValueError:
        return False


def get_unique_pins():
    exclude = [
        getattr(board, p)
        for p in [
            # This is not an exhaustive list of unexposed pins. Your results
            # may include other pins that you cannot easily connect to.
            "NEOPIXEL",
            "DOTSTAR_CLOCK",
            "DOTSTAR_DATA",
            "APA102_SCK",
            "APA102_MOSI",
            "LED",
            "SWITCH",
            "BUTTON",
```

```
            "ACCELEROMETER_INTERRUPT",
            "VOLTAGE_MONITOR",
            "MICROPHONE_CLOCK",
            "MICROPHONE_DATA",
            "RFM_RST",
            "RFM_CS",
            "RFM_IO0",
            "RFM_IO1",
            "RFM_IO2",
            "RFM_IO3",
            "RFM_IO4",
            "RFM_IO5",
            "TFT_I2C_POWER",
            "NEOPIXEL_POWER",
        ]
        if p in dir(board)
    ]
    pins = [
        pin
        for pin in [getattr(board, p) for p in dir(board)]
        if isinstance(pin, Pin) and pin not in exclude
    ]
    unique = []
    for p in pins:
        if p not in unique:
            unique.append(p)
    return unique


for scl_pin in get_unique_pins():
    for sda_pin in get_unique_pins():
        if scl_pin is sda_pin:
            continue
        if is_hardware_i2c(scl_pin, sda_pin):
            print("SCL pin:", scl_pin, "\t SDA pin:", sda_pin)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/i2c_find_pins/code.py

Now, connect to the serial console and check out the output! The results print out a nice handy list of SCL and SDA pin pairs that support I2C.

Text emphasized with a green exclamation:	The output for the QT Py ESP32-S2 is extremely long! The screenshot shows only the beginning. Run the script yourself to see the full output!

Text emphasized with a blue exclamation:	This example only runs once, so if you do not see any output when you connect to the serial console, try CTRL+D to reload.

Capacitive Touch

Your microcontroller board has capacitive touch capabilities on multiple pins. The CircuitPython `touchio` module makes it simple to detect when you touch a pin, enabling you to use it as an input.

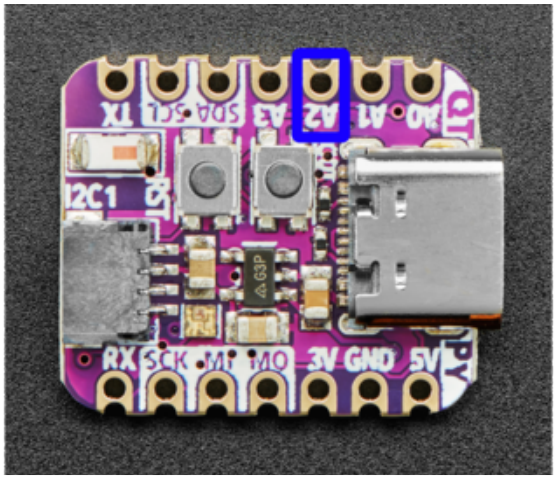
This section first covers using the `touchio` module to read touches on one pin. You'll learn how to setup the pin in your program, and read the touch status. Then, you'll learn how to read touches on multiple pins in a single example. Time to get started!

One Capacitive Touch Pin

The first example covered here will show you how to read touches on one pin.

Pin Wiring

Capacitive touch always benefits from the use of a 1MΩ pulldown resistor. Some microcontrollers have pulldown resistors built in, but using the built-in ones can yield unexpected results. Other microcontrollers to not have built-in pulldowns, and require an external pulldown resistor. Therefore, the best option is to include one regardless.



A2 (highlighted in blue) is the first pin used for capacitive touch.

Reading Touch on the Pin

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/cap_touch_one_pin_ESP32-S2/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

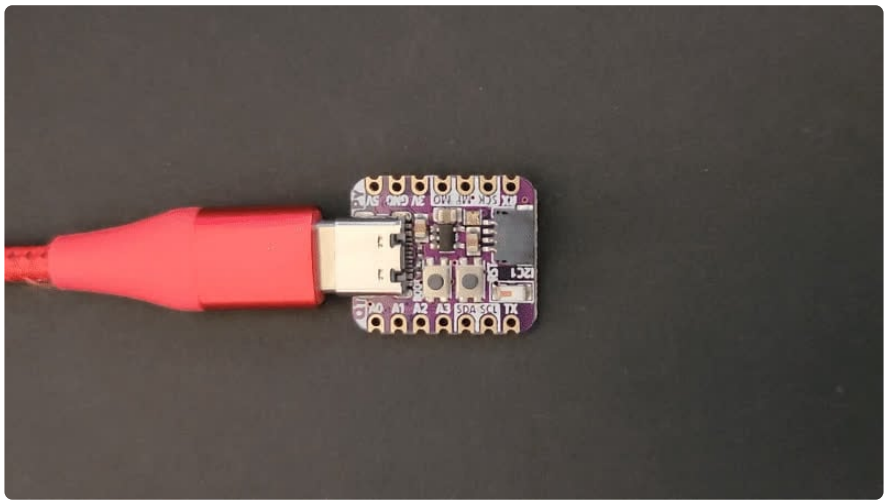
Then, connect to the serial console ().

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""
CircuitPython Capacitive Touch Pin Example for ESP32-S2.
Print to the serial console when one pin is touched.
"""
import time
import board
import touchio

touch = touchio.TouchIn(board.D8)
```

```
12
13  while True:
14      if touch.value:
15          print("Pin touched!")
16      time.sleep(0.1)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/cap_touch_one_pin_ESP32-S2/code.py

Now touch the pin indicated in the diagram above. You'll see `Pin touched!` printed to the serial console!



First you `import` three modules: `time`, `board` and `touchio`. This makes these modules available for use in your code. All three are built-in to CircuitPython, so you don't find any library files in the Project Bundle.

Next, you create the `touchio.TouchIn()` object, and provide it the pin name using the `board` module. You save that to the `touch` variable.

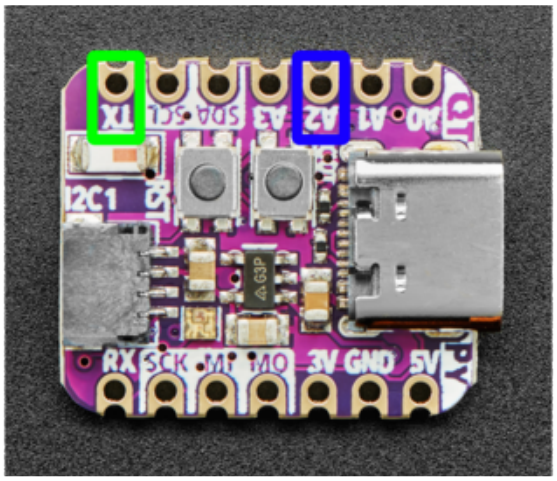
Inside the loop, you check to see if the pin is touched. If so, you print to the serial console. Finally, you include a `time.sleep()` to slow it down a bit so the output is readable.

That's all there is to reading touch on a single pin using `touchio` in CircuitPython!

Multiple Capacitive Touch Pins

The next example shows you how to read touches on multiple pins in a single program.

Pin Wiring



A2 (highlighted in blue) is the first pin used for capacitive touch.
TX (highlighted in green) is the second pin used for capacitive touch.

Reading Touch on the Pins

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/cap_touch_two_pins_ESP32-S2/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

Then, connect to the serial console ().

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""
CircuitPython Capacitive Two Touch Pin Example for ESP32-S2
Print to the serial console when a pin is touched.
"""
import time
import board
```

```
import touchio

touch_one = touchio.TouchIn(board.D8)
touch_two = touchio.TouchIn(board.D5)

while True:
    if touch_one.value:
        print("Pin one touched!")
    if touch_two.value:
        print("Pin two touched!")
    time.sleep(0.1)
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/cap_touch_two_pins_ESP32-S2/code.py

Touch the pins to see the messages printed to the serial console!

This example builds on the first. The imports remain the same.

The `touchio.TouchIn()` object is created, but is instead saved to `touch_one`. A second `touchio.TouchIn()` object is also created, the second pin is provided to it using the `board` module, and is saved to `touch_two`.

Inside the loop, we check to see if pin one and pin two are touched, and if so, print to the serial console `Pin one touched!` and `Pin two touched!`, respectively. The same `time.sleep()` is included.

If more touch-capable pins are available on your board, you can easily add them by expanding on this example!

Where are my Touch-Capable pins?

There are specific pins on a microcontroller that support capacitive touch. How do you know which pins will work? Easy! Run the script below to get a list of all the pins that are available.

Save the following to your **CIRCUITPY** drive as **code.py**.

Click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, find your CircuitPython version, and copy the matching **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

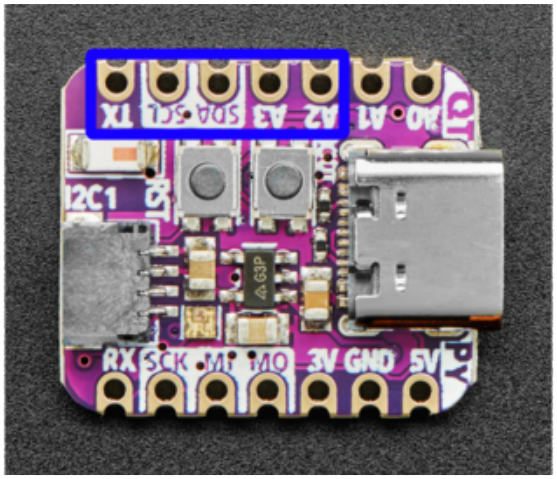
```
# SPDX-FileCopyrightText: 2021-2023 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""
CircuitPython Touch-Compatible Pin Identification Script

Depending on the order of the pins in the CircuitPython pin definition, some inaccessible pins
may be returned in the script results. Consult the board schematic and use your best judgement.

In some cases, such as LED, the associated pin, such as D13, may be accessible. The LED pin
name is first in the list in the pin definition, and is therefore printed in the results. The
pin name "LED" will work in code, but "D13" may be more obvious. Use the schematic to verify.
"""
import board
import touchio
from microcontroller import Pin


def get_pin_names():
    """
    Gets all unique pin names available in the board module, excluding a defined list.
    This list is not exhaustive, and depending on the order of the pins in the CircuitPython
    pin definition, some of the pins in the list may still show up in the script results.
    """
    exclude = [
        "NEOPIXEL",
        "APA102_MOSI",
        "APA102_SCK",
        "LED",
        "NEOPIXEL_POWER",
        "BUTTON",
        "BUTTON_UP",
        "BUTTON_DOWN",
        "BUTTON_SELECT",
        "DOTSTAR_CLOCK",
        "DOTSTAR_DATA",
        "IR_PROXIMITY",
        "SPEAKER_ENABLE",
        "BUTTON_A",
        "BUTTON_B",
        "POWER_SWITCH",
        "SLIDE_SWITCH",
        "TEMPERATURE",
        "ACCELEROMETER_INTERRUPT",
        "ACCELEROMETER_SDA",
        "ACCELEROMETER_SCL",
        "MICROPHONE_CLOCK",
```

```
        "MICROPHONE_DATA",
        "RFM_RST",
        "RFM_CS",
        "RFM_IO0",
        "RFM_IO1",
        "RFM_IO2",
        "RFM_IO3",
        "RFM_IO4",
        "RFM_IO5",
    ]
    pins = [
        pin
        for pin in [getattr(board, p) for p in dir(board) if p not in exclude]
        if isinstance(pin, Pin)
    ]
    pin_names = []
    for pin_object in pins:
        if pin_object not in pin_names:
            pin_names.append(pin_object)
    return pin_names


for possible_touch_pin in get_pin_names():  # Get the pin name.
    try:
        touch_pin_object = touchio.TouchIn(
            possible_touch_pin
        )  # Attempt to create the touch object on each pin.
        # Print the touch-capable pins that do not need, or already have, an external pulldown.
        print("Touch on:", str(possible_touch_pin).replace("board.", ""))
    except ValueError as error:  # A ValueError is raised when a pin is invalid or needs a pulldown.
        # Obtain the message associated with the ValueError.
        error_message = getattr(error, "message", str(error))
        if (
            "pulldown" in error_message  # If the ValueError is regarding needing a pulldown...
        ):
            print(
                "Touch on:", str(possible_touch_pin).replace("board.", "")
            )
        else:
            print("No touch on:", str(possible_touch_pin).replace("board.", ""))
    except TypeError:  # Error returned when checking a non-pin object in dir(board).
        pass  # Passes over non-pin objects in dir(board).
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/CircuitPython_Templates/cap_touch_pin_script/code.py

Now, connect to the serial console and check out the output! The results print out a nice handy list of pins that support capacitive touch.

There are five capacitive touch capable pins on the QT Py ESP32-S2.



A2 - CircuitPython `board.A2`. Arduino `9`.
A3 - CircuitPython `board.A3`. Arduino `8`.
SDA - CircuitPython `board.SDA`. Arduino `7`.
SCL - CircuitPython `board.SCL`. Arduino `6`.
TX - CircuitPython `board.TX`. Arduino `5`.

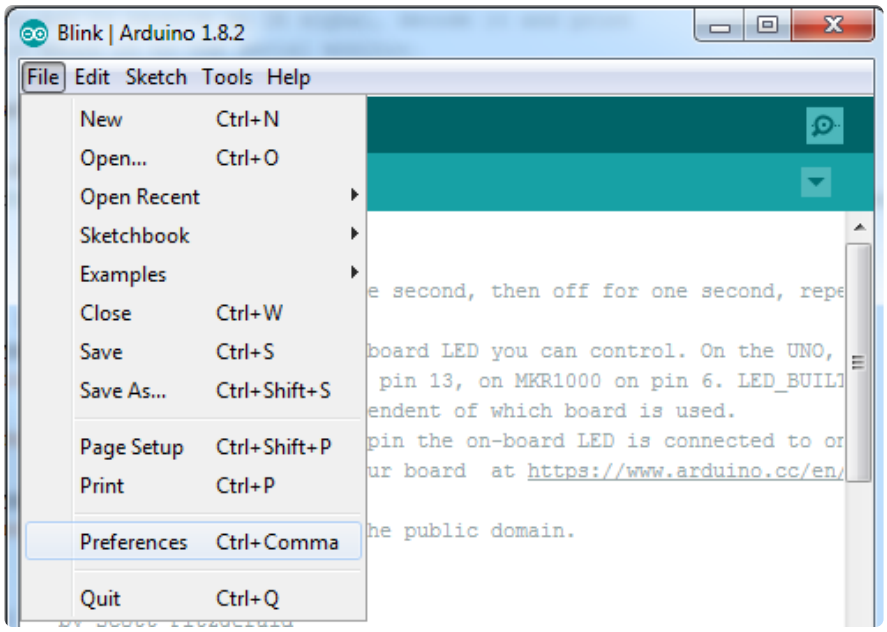
Arduino IDE Setup

Text emphasized with a yellow exclamation:	The ESP32-S2/S3 bootloader does not have USB serial support for Windows 7 or 8. (See https://github.com/espressif/arduino-esp32/issues/5994) please update to version 10 which is supported by espressif! Alternatively you can try this community-crafted Windows 7 driver (https://github.com/kutukvpavel/Esp32-Win7-VCP-drivers)

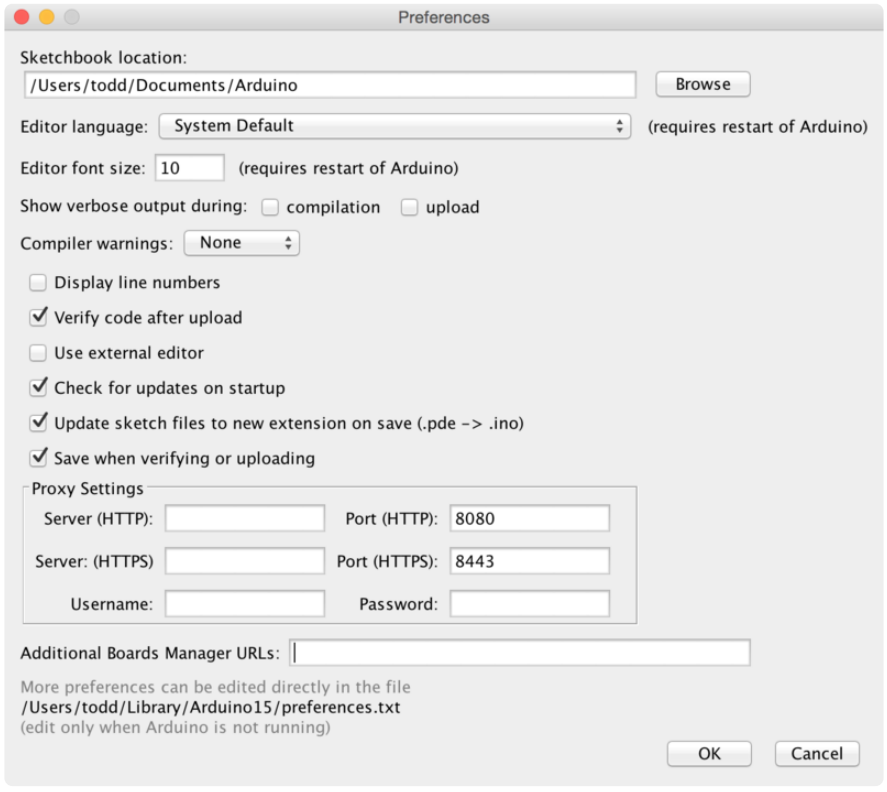
The first thing you will need to do is to download the latest release of the Arduino IDE. You will need to be using **version 1.8** or higher for this guide

To use the ESP32-S2/S3 with Arduino, you'll need to follow the steps below for your operating system. You can also check out the Espressif Arduino repository for the most up to date details on how to install it ().

After you have downloaded and installed **the latest version of Arduino IDE**, you will need to start the IDE and navigate to the **Preferences** menu. You can access it from the **File** menu in Windows or Linux, or the **Arduino** menu on OS X.



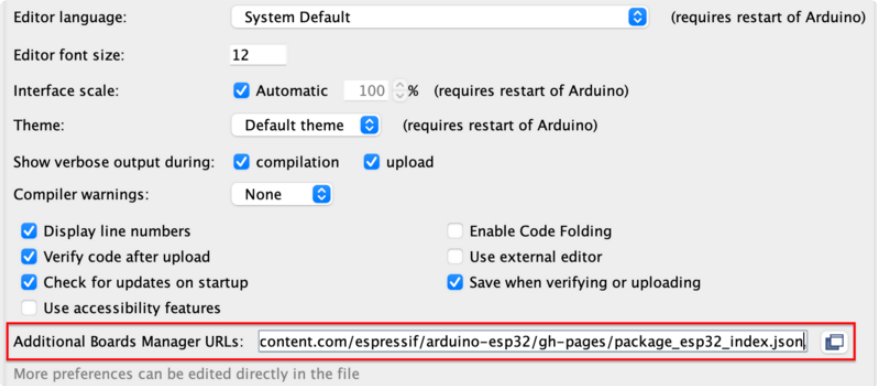
A dialog will pop up just like the one shown below.



We will be adding a URL to the new **Additional Boards Manager URLs** option. The list of URLs is comma separated, and you will only have to add each URL once. New Adafruit boards and updates to existing boards will automatically be picked up by the Board Manager each time it is opened. The URLs point to index files that the Board Manager uses to build the list of available & installed boards.

To find the most up to date list of URLs you can add, you can visit the list of [third party board URLs on the Arduino IDE wiki]() (). We will only need to add one URL to the IDE in this example, but **you can add multiple URLS by separating them with commas**. Copy and paste the link below into the **Additional Boards Manager URLs** option in the Arduino IDE preferences.

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```



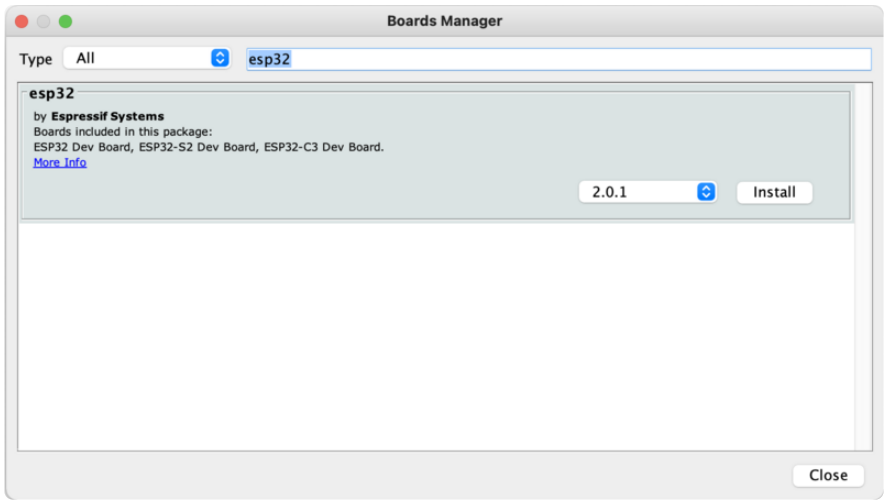
If you're an advanced hacker and want the 'bleeding edge' release that may have fixes (or bugs!) you can check out the dev url instead:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_dev_index.json
```

If you have multiple boards you want to support, say ESP8266 and Adafruit, have both URLs in the text box separated by a comma (,)

Once done click **OK** to save the new preference settings.

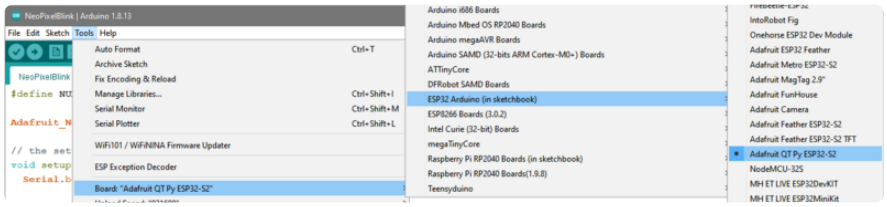
The next step is to actually install the Board Support Package (BSP). Go to the **Tools → Board → Board Manager** submenu. A dialog should come up with various BSPs. Search for **esp32**. Choose the latest version, which may be later than the version shown in the screenshot below.



Click the **Install** button and wait for it to finish. Once it is finished, you can close the dialog.

In the **Tools → Board** submenu you should see **ESP32 Arduino** and in that dropdown it should contain the ESP32 boards along with all the latest ESP32-S2/S3 boards.

In the **Tools > Boards** menu you should see the **ESP32 Arduino** menu. In the expanded menu, look for the menu option for the **Adafruit QT Py ESP32-S2**, and click on it to choose it.



Arduino NeoPixel Blink

The first and most basic program you can upload to your Arduino is the classic Blink sketch. This takes something on the board and makes it, well, blink! On and off. It's a great way to make sure everything is working and you're uploading your sketch to the right board and right configuration.

When all else fails, you can always come back to Blink!

Now traditionally you would use an onboard LED to make a blink occur. **However, this board does not have an onboard single-color LED**, so we will 'mimic' the same blink

sketch but instead of using a digital output pin, we will use NeoPixel support to blink the onboard RGB LED!

Pre-Flight Check: Get Arduino IDE & Hardware Set Up

This lesson assumes you have Arduino IDE set up. This is a generalized checklist, some elements may not apply to your hardware. If you haven't yet, check the previous steps in the guide to make sure you:

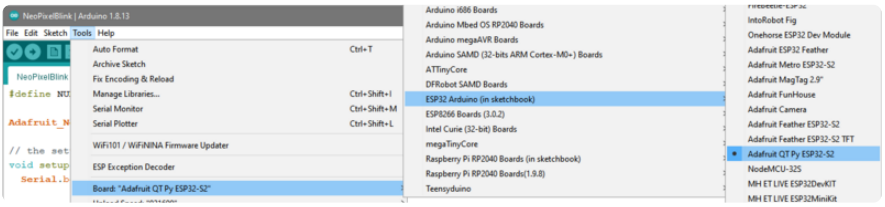
- **Install the very latest Arduino IDE for Desktop** (not all boards are supported by the Web IDE so we don't recommend it)
- **Install any board support packages (BSP) required for your hardware.** Some boards are built in defaults on the IDE, but lots are not! You may need to install plug-in support which is called the BSP.
- **Get a Data/Sync USB cable for connecting your hardware.** A significant amount of problems folks have stem from not having a USB cable with data pins. Yes, these cursed cables roam the land, making your life hard. If you find a USB cable that doesn't work for data/sync, throw it away immediately! There is no need to keep it around, cables are very inexpensive these days.
- **Install any drivers required** - If you have a board with a FTDI or CP210x chip, you may need to get separate drivers. If your board has native USB, it probably doesn't need anything. After installing, reboot to make sure the driver sinks in.
- **Connect the board to your computer.** If your board has a power LED, make sure its lit. Is there a power switch? Make sure its turned On!

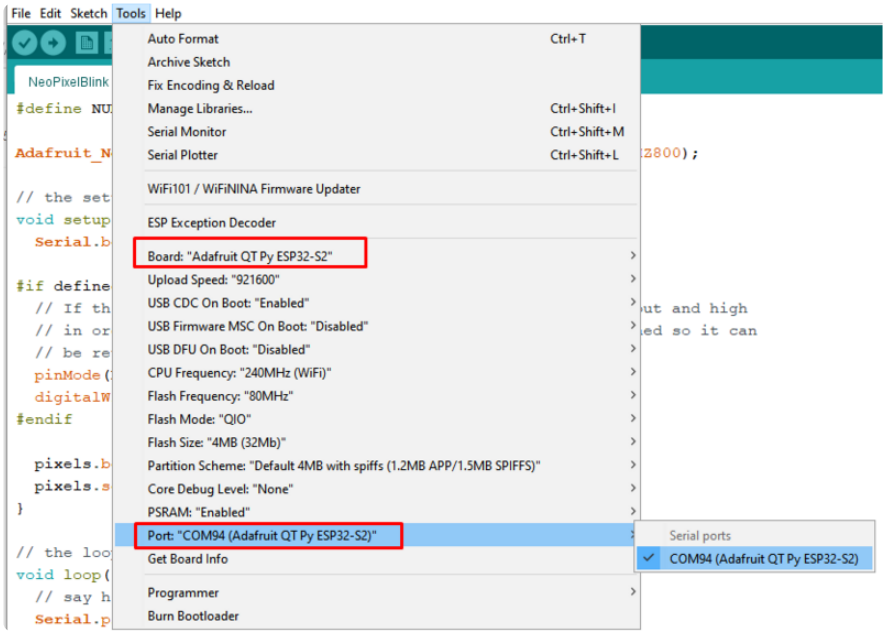
Text emphasized with a yellow exclamation:	The ESP32-S2 QT Py does not require any drivers and does not have a power LED so you may not see anything glowing when you plug it in, thats ok!

Start up Arduino IDE and Select Board/Port

OK, now you are prepared! Open the Arduino IDE on your computer. Now you have to tell the IDE what board you are using, and how you want to connect to it.

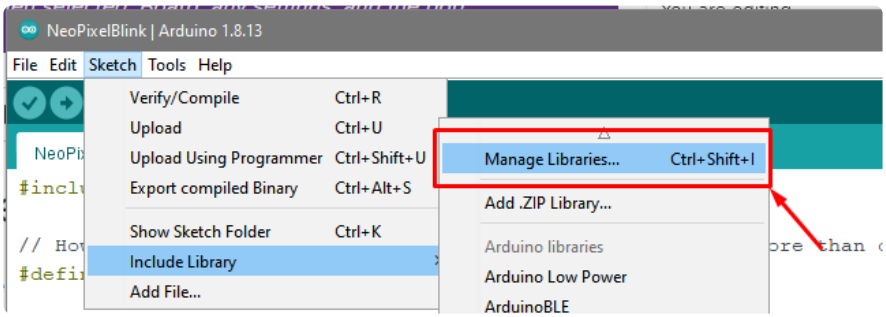
In the IDE find the **Tools** menu. You will use this to select the board. If you switch boards, you must switch the selection! So always double-check before you upload code in a new session.



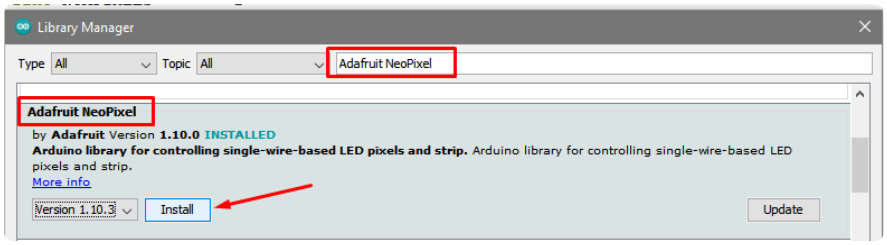


Install NeoPixel Library

Despite their popularity, NeoPixel RGB LEDs are not supported 'out of the box' in Arduino IDE! You will need to add support by installing the library. Good news it is very easy to do it. Go to the **Library Manager** here

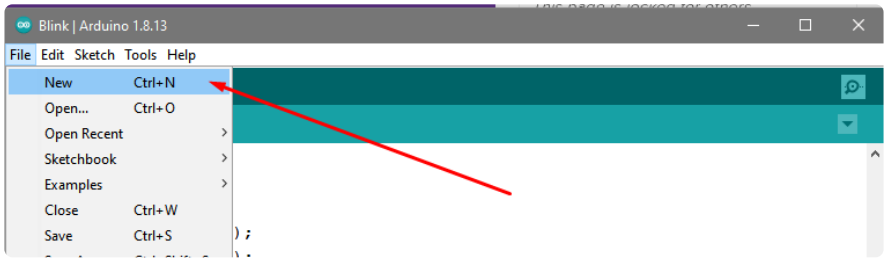


Search for and install the **Adafruit NeoPixel** library. It might not be first in the list so make sure you get the name matched up right!



New NeoPixel Blink Sketch

OK lets make a new blink sketch! From the **File** menu, select **New**



Then in the new window, copy and paste this text:

```
#include <Adafruit_NeoPixel.h>

// How many internal neopixels do we have? some boards have more than one!
#define NUMPIXELS       1

Adafruit_NeoPixel pixels(NUMPIXELS, PIN_NEOPIXEL, NEO_GRB + NEO_KHZ800);

// the setup routine runs once when you press reset:
void setup() {
  Serial.begin(115200);

#if defined(NEOPIXEL_POWER)
  // If this board has a power control pin, we must set it to output and high
  // in order to enable the NeoPixels. We put this in an #if defined so it can
  // be reused for other boards without compilation errors
  pinMode(NEOPIXEL_POWER, OUTPUT);
  digitalWrite(NEOPIXEL_POWER, HIGH);
#endif

  pixels.begin(); // INITIALIZE NeoPixel strip object (REQUIRED)
  pixels.setBrightness(20); // not so bright
}

```

```
// the loop routine runs over and over again forever:
void loop() {
  // say hi
  Serial.println("Hello!");

  // set color to red
  pixels.fill(0xFF0000);
  pixels.show();
  delay(500); // wait half a second

  // turn off
  pixels.fill(0x000000);
  pixels.show();
  delay(500); // wait half a second
}
```

Text emphasized with a blue exclamation:

Note that in this example, we are not only blinking the NeoPixel LED but also printing to the Serial monitor. Think of it as a little bonus to test the serial connection.

One note you'll see is that we reference the LED with the constant `PIN_NEOPIXEL` rather than a number. That's because each board could have the built in NeoPixels on a different pin and this makes the code a little more portable!

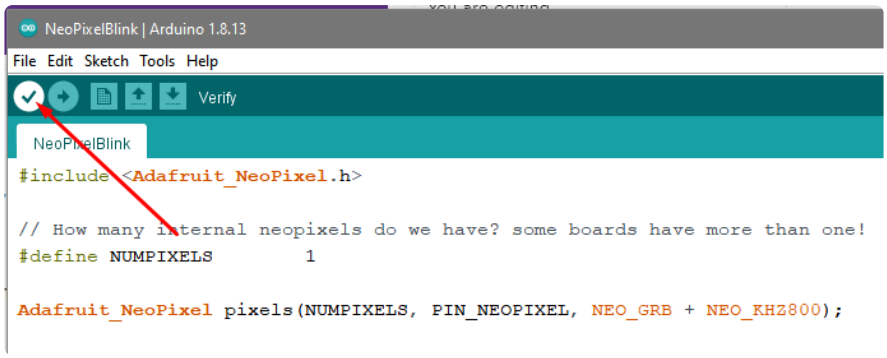
Text emphasized with a blue exclamation:

On the ESP32-S2 QT Py the NeoPixel is on pin/GPIO #39 and the Power pin is on GPIO #38 - the power pin MUST be set to an OUTPUT and HIGH before the internal NeoPixel can be initialized/displayed

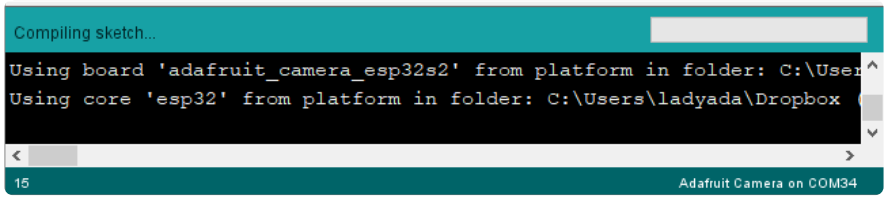
Verify (Compile) Sketch

OK now you can click the Verify button to convert the sketch into binary data to be uploaded to the board.

Note that Verifying a sketch is the same as Compiling a sketch - so these terms will be used interchangeably.

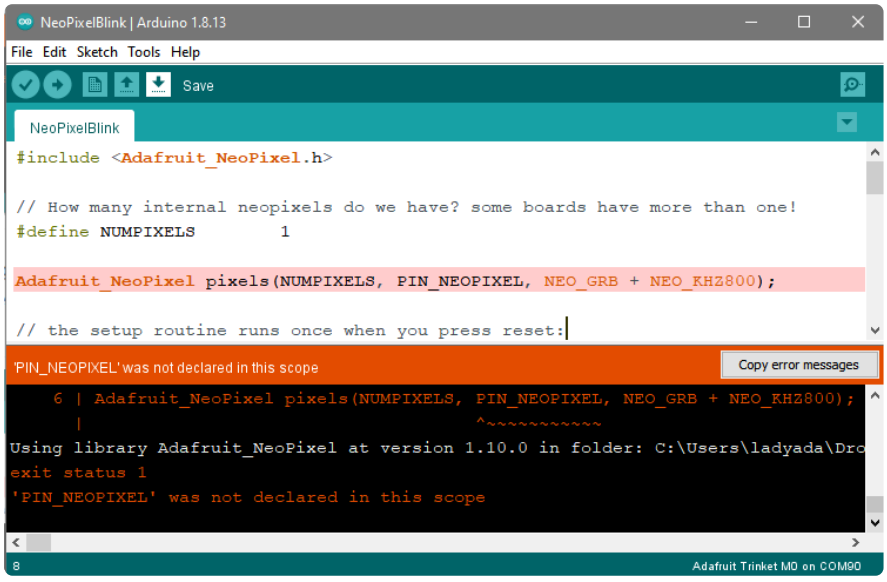


During verification/compilation, the computer will do a bunch of work to collect all the libraries and code and the results will appear in the bottom window of the IDE.

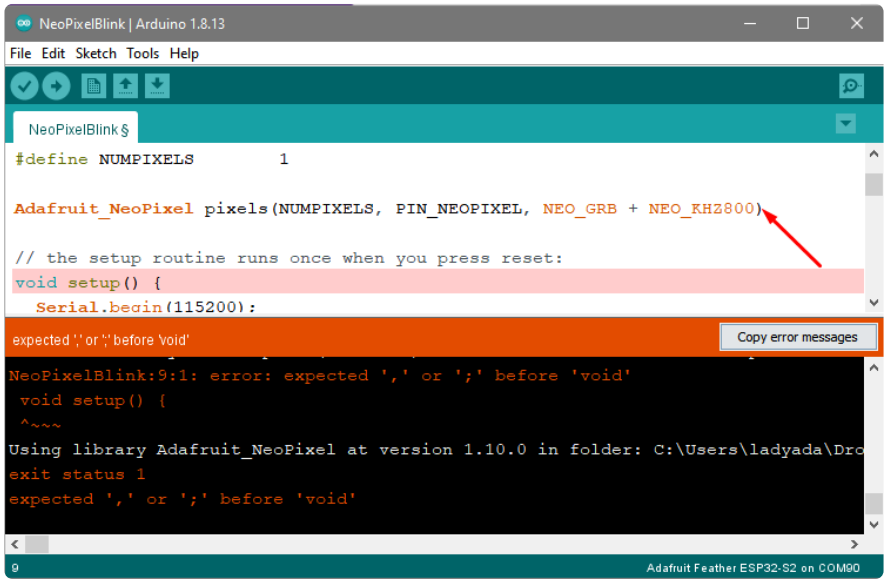


If something went wrong with compilation, you will get red warning/error text in the bottom window letting you know what the error was. It will also highlight the line with an error.

For example, here I had the wrong board selected - and the selected board does not have a built-in NeoPixel pin defined!



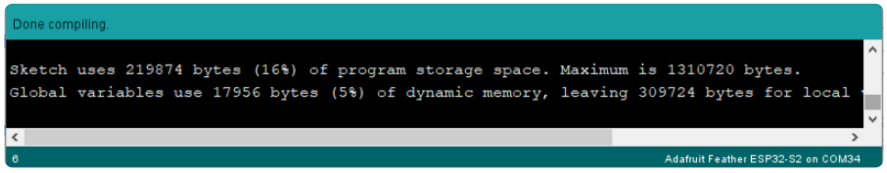
Here's another common error, in my haste I forgot to add a `;` at the end of a line. The compiler warns me that it's looking for one - note that the error is actually a few lines up!



Text emphasized with a blue exclamation:

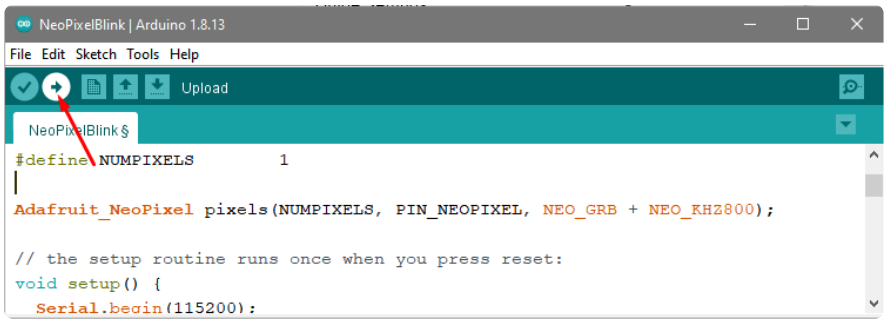
Turning on detailed compilation warnings and output can be very helpful sometimes - Its in Preferences under "Show Verbose Output During:" and check the Compilation button. If you ever need to get help from others, be sure to do this and then provide all the text that is output. It can assist in nailing down what happened!

On success you will see something like this white text output and the message **Done compiling.** in the message area.



Upload Sketch

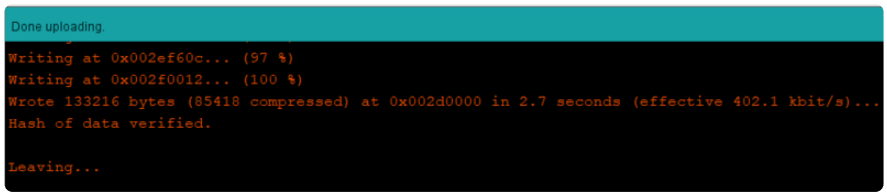
Once the code is verified/compiling cleanly you can upload it to your board. Click the **Upload** button:



The IDE will try to compile the sketch again for good measure, then it will try to connect to the board and upload a the file.

This is actually one of the hardest parts for beginners because it's where a lot of things can go wrong.

However, start with what it looks like: success! Here's what your board upload process looks like when it goes right:



Often times you will get a warning like this, which is kind of vague:

`No device found on COM66` (or whatever port is selected)
`An error occurred while uploading the sketch`

This could be a few things.

First up, check again that you have the correct board selected! Many electronics boards have very similar names or look, and often times folks grab a board different from what they thought.

If you're positive the right board is selected, we recommend the next step is to put the board into manual bootloading mode.

Native USB and manual bootloading

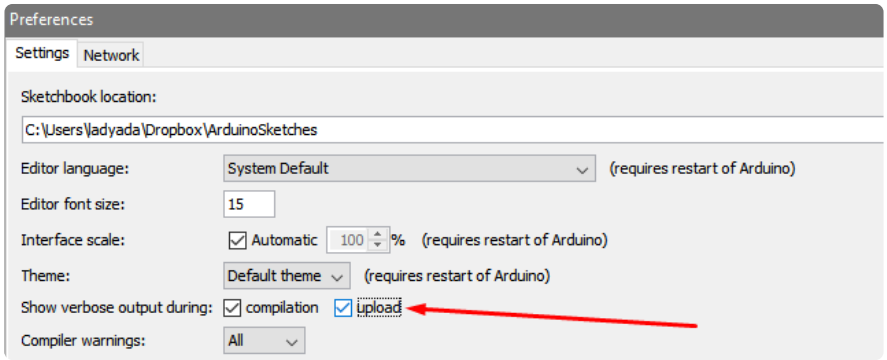
Historically, microcontroller boards contained two chips: the main micro chip (say, ATmega328 or ESP8266 or ESP32) and a separate chip for USB interface that would be used for bootloading (a CH430, FT232, CP210x, etc). With these older designs, the microcontroller is put into a bootloading state for uploading code by the separate chip. It allows for easier uploading but is more expensive as two chips are needed, and also the microcontroller can't act like a keyboard or disk drive.

Modern chips often have 'native' USB - that means that there is no separate chip for USB interface. It's all in one! Great for cost savings, simplicity of design, reduced size and more control. However, it means the chip must be self-aware enough to be able to put itself into bootload/upload mode on its own. That's fine 99% of the time but is very likely you will at some point get the board into an odd state that makes it too confused to bootload.

Text emphasized with a yellow exclamation:

A lot of beginners have a little freakout the first time this happens, they think the board is ruined or 'bricked' - it's almost certainly not, it is just crashed and/or confused. You may need to perform a little trick to get the board back into a good state, at which point you won't need to manually bootload again.

Before continuing we really really suggest turning on **Verbose Upload** messages, it will help in this process because you will be able to see what the IDE is trying to do. It's a checkbox in the **Preferences** menu.



Enter Manual Bootload Mode

OK now you know it's probably time to try manual bootloading. No problem! Here is how you do that for this board:

Entering the ROM bootloader is easy. Complete the following steps.

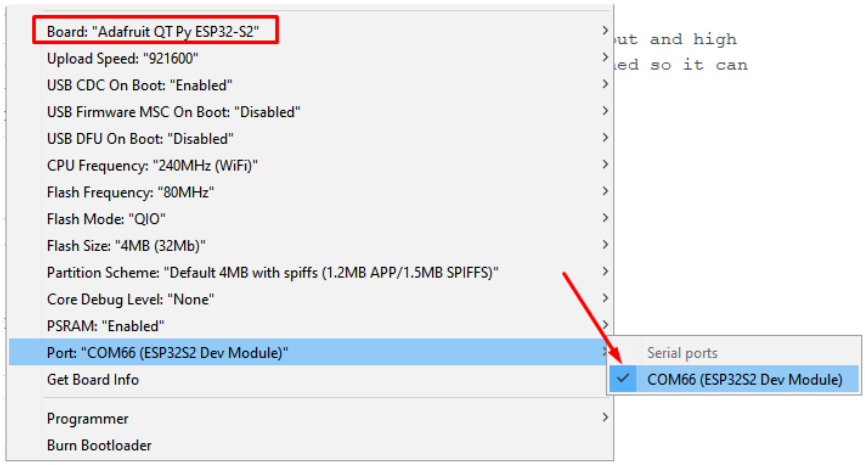
Before you start, make sure your QT Py ESP32-S2 is plugged into USB port to your computer using a data/sync cable. Charge-only cables will not work!

To enter the bootloader:

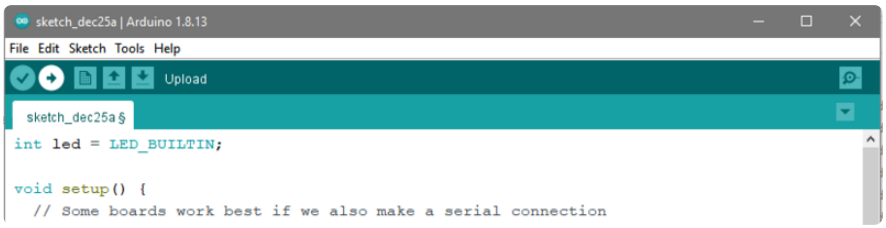
1. **Press and hold the Boot button down.** Don't let go of it yet!
2. **Press and release the Reset button.** You should still have the Boot button pressed while you do this.
3. **Now you can release the Boot button.**

No USB drive will appear when you've entered the ROM bootloader. This is normal!

Once you are in manual bootload mode, go to the Tools menu, and make sure you have selected the bootloader serial port. **It is almost certain that the serial port has changed now that the bootloader is enabled.**



Now you can try uploading again!



Text emphasized with a blue exclamation: Did you remember to select the new Port in the Tools menu since the bootloader port has changed?

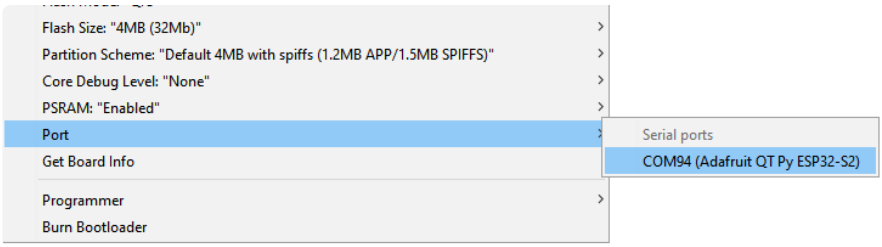
This time, you should have success!

After uploading this way, be sure to **click the reset button** - it sort of makes sure that the board got a good reset and will come back to life nicely.

Text emphasized with a yellow exclamation: After uploading with Manual Bootloader - don't forget to re-select the old Port again

It's also a good idea to try to re-upload the sketch again now that you've performed a manual bootload to get the chip into a good state. It should perform an auto-reset the second time, so you don't have to manually bootload again.

Once you've finished the re-upload, don't forget to press RESET and then select the old Port again!



Finally, a Blink!

OK it was a journey but now we're here and you can enjoy your blinking LED. Next up, try to change the delay between blinks and re-upload. It's a good way to make sure your upload process is smooth and practiced.

I2C Scan Test

A lot of sensors, displays, and devices can connect over I2C. I2C is a 2-wire 'bus' that allows multiple devices to all connect on one set of pins so it's very convenient for wiring!

When using your board, you'll probably want to connect up I2C devices, and it can be a little tricky the first time. The best way to debug I2C is go through a checklist and then perform an I2C scan

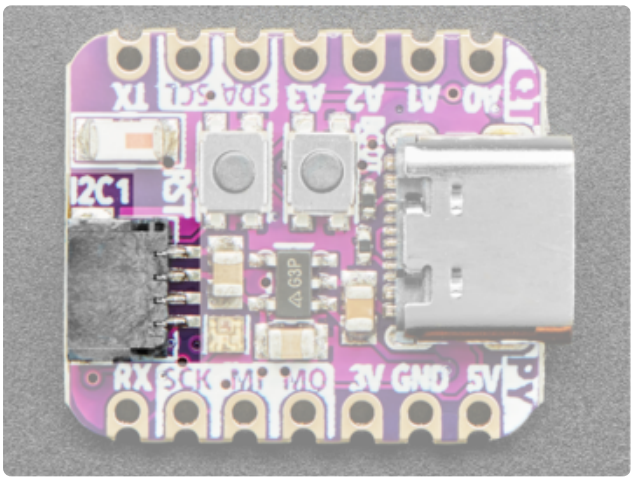
Common I2C Connectivity Issues

- **Have you connected four wires (at a minimum) for each I2C device?** Power the device with whatever is the logic level of your microcontroller board (probably 3.3V), then a ground wire, and a SCL clock wire, and and a SDA data wire.
- **If you're using a STEMMA QT board - check if the power LED is lit.** It's usually a green LED to the left side of the board.
- **Does the STEMMA QT/I2C port have switchable power or pullups?** To reduce power, some boards have the ability to cut power to I2C devices or the pullup resistors. Check the documentation if you have to do something special to turn on the power or pullups.
- **If you are using a DIY I2C device, do you have pullup resistors?** Many boards do not have pullup resistors built in and they are required! We suggest any common 2.2K to 10K resistors. You'll need two: one each connects from SDA to positive power, and SCL to positive power. Again, positive power (a.k.a VCC, VDD or V+) is often 3.3V
- **Do you have an address collision?** You can only have one board per address. So you cannot, say, connect two AHT20's to one I2C port because they have the

same address and will interfere. Check the sensor or documentation for the address. Sometimes there are ways to adjust the address.

- **Does your board have multiple I2C ports?** Historically, boards only came with one. But nowadays you can have two or even three! This can help solve the "hey, but what if I want two devices with the same address" problem: just put one on each bus.
- **Are you hot-plugging devices?** I2C does not support dynamic re-connection, you cannot connect and disconnect sensors as you please. They should all be connected on boot and not change. (Only exception is if you're using a hot-plug assistant but that'll cost you (http://adafru.it/5159)).
- **Are you keeping the total bus length reasonable?** I2C was designed for maybe 6" max length. We like to push that with plug-n-play cables, but really please keep them as short as possible! (Only exception is if you're using an active bus extender (http://adafru.it/4756)).

QT Py ESP32-S2 I2C Configurations



The QT Py ESP32-S2 has **TWO** I2C ports!

`Wire` is the default port, and is available on the **SDA** and **SCL** pins on the castellated side pads

`Wire1` is the secondary port, and is available on the **STEMMA QT** port on the end of the board

Neither of the I2C ports have pullup resistors installed. You will need to either add external pull ups or verify that any sensors include them!

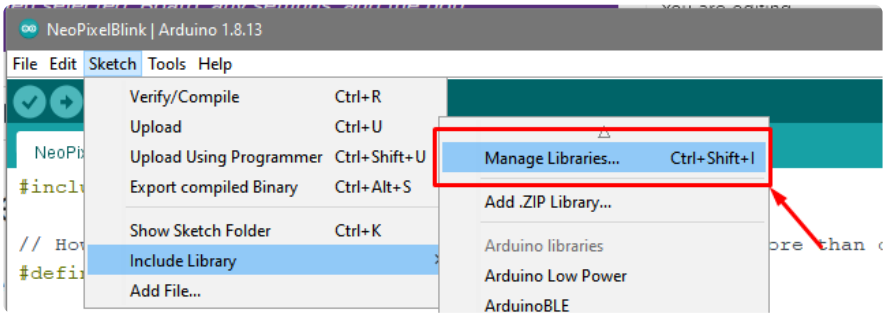
The STEMMA QT port has another thing to watch for! You need to manually set the pins for the wire peripheral early in your code. So add this line to your sketch in the `setup()` part, it is not optional at this time.

```
Wire1.setPins(SDA1, SCL1);
```

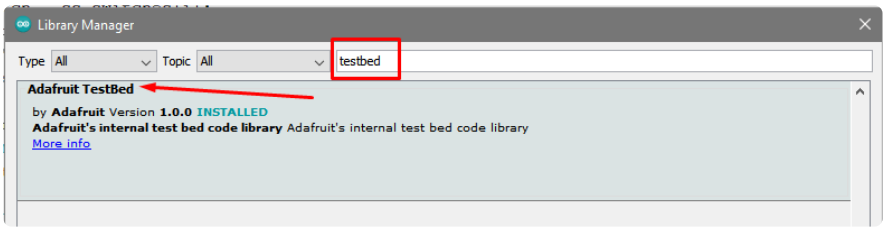
Perform an I2C scan!

Install TestBed Library

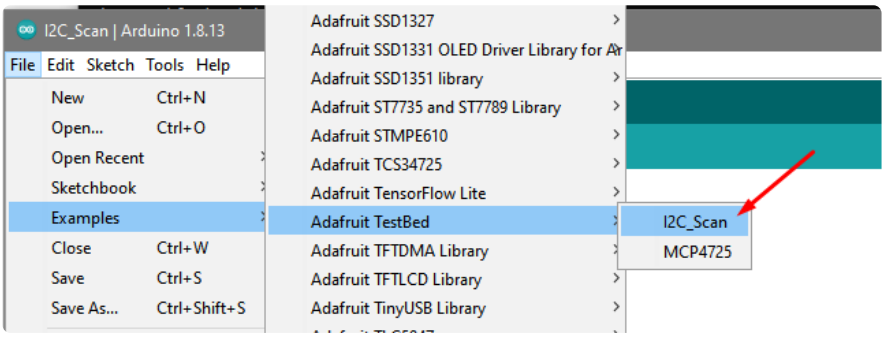
To scan I2C, the Adafruit TestBed library is used. This library and example just makes the scan a little easier to run because it takes care of some of the basics. You will need to add support by installing the library. Good news: it is very easy to do it. Go to the **Arduino Library Manager.**



Search for **TestBed** and install the **Adafruit TestBed** library



Now open up the I2C Scan example




```
1  // SPDX-FileCopyrightText: 2023 Carter Nelson for Adafruit Industries
2  //
3  // SPDX-License-Identifier: MIT
4  // ------------------------------------
5  // i2c_scanner
6  //
```

```
// Modified from https://playground.arduino.cc/Main/I2cScanner/
// --------------------------------------

#include <Wire.h>

// Set I2C bus to use: Wire, Wire1, etc.
#define WIRE Wire

void setup() {
  WIRE.begin();

  Serial.begin(9600);
  while (!Serial)
    delay(10);
  Serial.println("\nI2C Scanner");
}


void loop() {
  byte error, address;
  int nDevices;

  Serial.println("Scanning...");

  nDevices = 0;
  for(address = 1; address < 127; address++ )
  {
    // The i2c_scanner uses the return value of
    // the Write.endTransmisstion to see if
    // a device did acknowledge to the address.
    WIRE.beginTransmission(address);
    error = WIRE.endTransmission();

    if (error == 0)
    {
      Serial.print("I2C device found at address 0x");
      if (address<16)
        Serial.print("0");
      Serial.print(address,HEX);
      Serial.println("  !");

      nDevices++;
    }
    else if (error==4)
    {
      Serial.print("Unknown error at address 0x");
      if (address<16)
        Serial.print("0");
      Serial.println(address,HEX);
    }
  }
  if (nDevices == 0)
    Serial.println("No I2C devices found\n");
  else
    Serial.println("done\n");

```

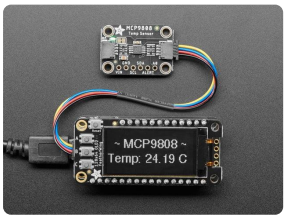
```
63    delay(5000);          // wait 5 seconds for next scan
64  }
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/I2C_Scanners/arduino/i2c_scanner/i2c_scanner.ino

Wire up I2C device

While the examples here will be using the Adafruit MCP9808 (http://adafru.it/5027), a high accuracy temperature sensor, the overall process is the same for just about any I2C sensor or device.

The first thing you'll want to do is get the sensor connected so your board has I2C to talk to.

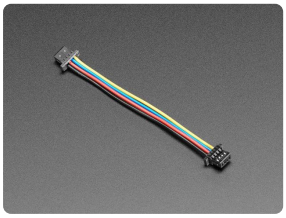


Featured

Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of ±0.25°C over the sensor's -40°C to...

https://www.adafruit.com/product/5027



Featured

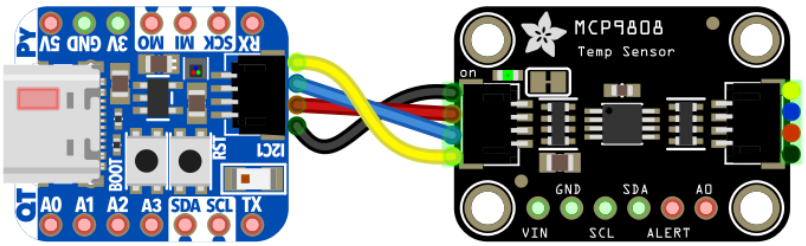
STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long

This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

https://www.adafruit.com/product/4399

Wiring the MCP9808

The MCP9808 comes with a STEMMA QT connector, which makes wiring it up quite simple and solder-free.

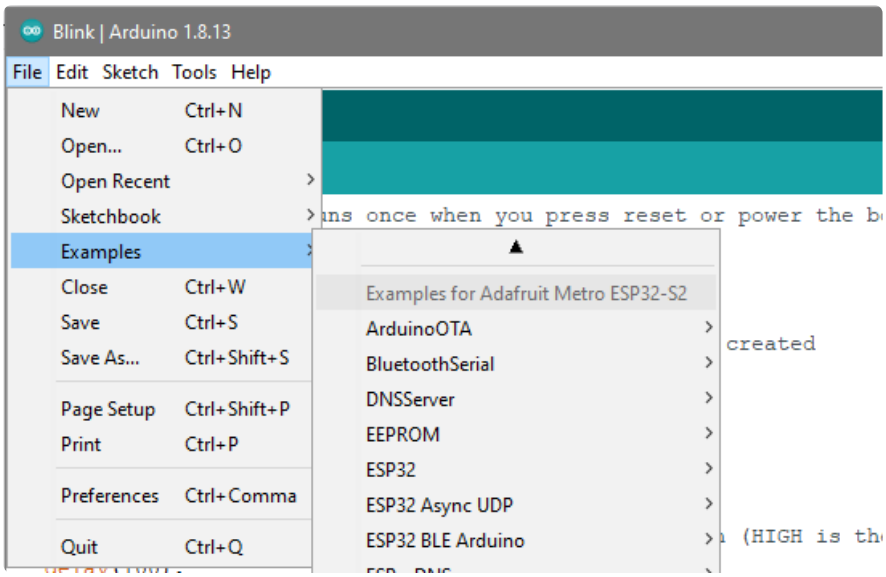


!	Text emphasized with a yellow exclamation:	Note we are using the STEMMA QT port here, that means we need to reference Wire1 for I2C communication and also add Wire1.setPins(SDA1, SCL1) in our code in setup()

Now upload the scanning sketch to your microcontroller and open the serial port to see the output. You should see something like this:

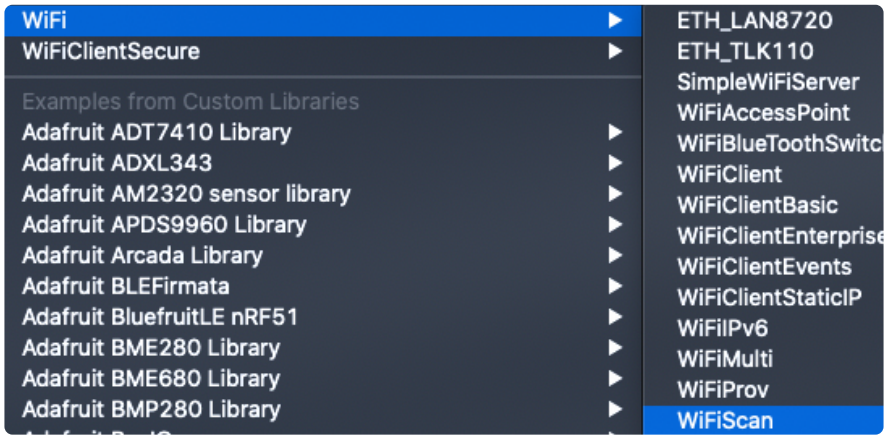
WiFi Test

Thanksfully if you have ESP32 sketches, they'll 'just work' with variations of ESP32. You can find a wide range of examples in the **File->Examples->Examples for Adafruit Metro ESP32-S2** subheading (the name of the board may vary so it could be **"Examples for Adafruit Feather ESP32 V2"** etc)



Let's start by scanning the local networks.

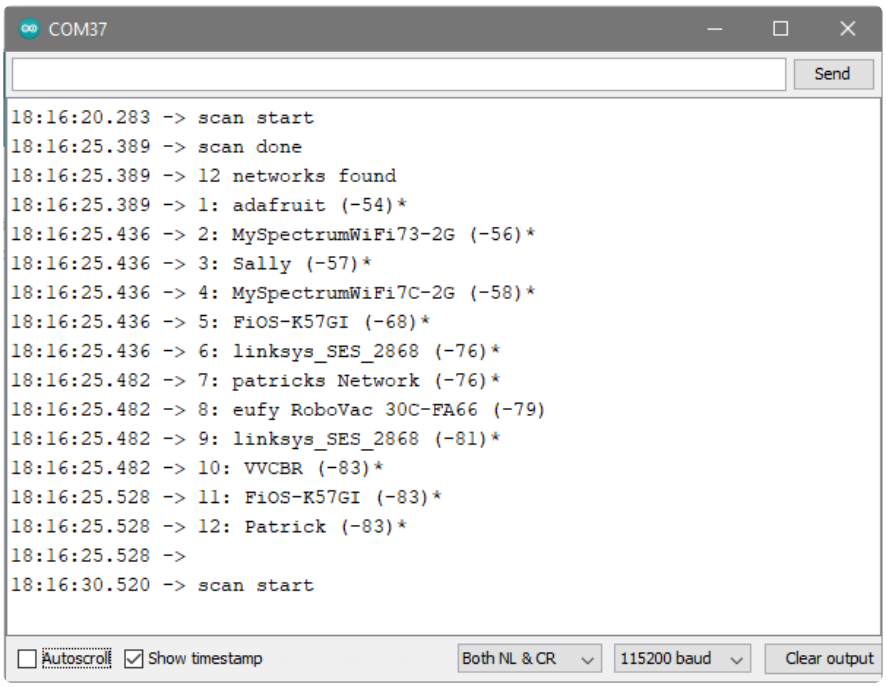
Load up the WiFiScan example under **Examples->Examples for YOUR BOARDNAME->WiFi->WiFiScan**



And **upload this example to your board**. The ESP32 should scan and find WiFi networks around you.

For ESP32, open the serial monitor, to see the scan begin.

For ESP32-S2, -S3 and -C3, don't forget you have to click Reset after uploading through the ROM bootloader. Then select the new USB Serial port created by the ESP32. It will take a few seconds for the board to complete the scan.



If you can not scan any networks, check your power supply. You need a solid power supply in order for the ESP32 to not brown out. A skinny USB cable or drained battery can cause issues.

WiFi Connection Test

Now that you can scan networks around you, its time to connect to the Internet!

Copy the example below and paste it into the Arduino IDE:

```
// SPDX-FileCopyrightText: 2020 Brent Rubell for Adafruit Industries
//
// SPDX-License-Identifier: MIT


/*
  Web client

 This sketch connects to a website (wifitest.adafruit.com/testwifi/index.html)
 using the WiFi module.

 This example is written for a network using WPA encryption. For
 WEP or WPA, change the Wifi.begin() call accordingly.

 This example is written for a network using WPA encryption. For
 WEP or WPA, change the Wifi.begin() call accordingly.

 created 13 July 2010
 by dlf (Metodo2 srl)
 modified 31 May 2012
 by Tom Igoe
```

```
 */

#include <WiFi.h>

// Enter your WiFi SSID and password
char ssid[] = "YOUR_SSID";            // your network SSID (name)
char pass[] = "YOUR_SSID_PASSWORD";    // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                     // your network key Index number (needed only for WEP)

int status = WL_IDLE_STATUS;
// if you don't want to use DNS (and reduce your sketch size)
// use the numeric IP instead of the name for the server:
//IPAddress server(74,125,232,128);  // numeric IP for Google (no DNS)

char server[] = "wifitest.adafruit.com";    // name address for adafruit test
char path[]   = "/testwifi/index.html";

// Initialize the Ethernet client library
// with the IP address and port of the server
// that you want to connect to (port 80 is default for HTTP):
WiFiClient client;

void setup() {
  //Initialize serial and wait for port to open:
  Serial.begin(115200);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }

  // attempt to connect to Wifi network:
  Serial.print("Attempting to connect to SSID: ");
  Serial.println(ssid);

  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) {
     delay(500);
     Serial.print(".");
  }

  Serial.println("");
  Serial.println("Connected to WiFi");
  printWifiStatus();

  Serial.println("\nStarting connection to server...");
  // if you get a connection, report back via serial:
  if (client.connect(server, 80)) {
    Serial.println("connected to server");
    // Make a HTTP request:
    client.print("GET "); client.print(path); client.println(" HTTP/1.1");
    client.print("Host: "); client.println(server);
    client.println("Connection: close");
    client.println();
  }
}

void loop() {
```

```
78    // if there are incoming bytes available
79    // from the server, read them and print them:
80    while (client.available()) {
81      char c = client.read();
82      Serial.write(c);
83    }
84
85    // if the server's disconnected, stop the client:
86    if (!client.connected()) {
87      Serial.println();
88      Serial.println("disconnecting from server.");
89      client.stop();
90
91      // do nothing forevermore:
92      while (true) {
93        delay(100);
94      }
95    }
96  }
97
98
99  void printWifiStatus() {
100   // print the SSID of the network you're attached to:
101   Serial.print("SSID: ");
102   Serial.println(WiFi.SSID());
103
104   // print your board's IP address:
105   IPAddress ip = WiFi.localIP();
106   Serial.print("IP Address: ");
107   Serial.println(ip);
108
109   // print the received signal strength:
110   long rssi = WiFi.RSSI();
111   Serial.print("signal strength (RSSI):");
112   Serial.print(rssi);
113   Serial.println(" dBm");
114 }
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/ESP32_S2_WiFi_Tests/WiFiWebClient/WiFiWebClient.ino

NOTE: You must **change the `SECRET_SSID` and `SECRET_PASS` in the example code to your WiFi SSID and password** before uploading this to your board.

```
// Enter your WiFi SSID and password
char ssid[] = "YOUR_SSID";            // your network SSID (name)
char pass[] = "YOUR_SSID_PASSWORD";   // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                     // your network key Index number (needed only for WEP)
```

After you've set it correctly, upload and check the serial monitor. You should see the following. If not, go back, check wiring, power and your SSID/password

```
Attempting to connect to SSID: Transit
........
Connected to WiFi
SSID: Transit
IP Address: 192.168.1.182
signal strength (RSSI):-57 dBm

Starting connection to server...
connected to server
HTTP/1.1 200 OK
Server: nginx/1.10.3 (Ubuntu)
Date: Wed, 11 Nov 2020 20:51:30 GMT
Content-Type: text/html
Content-Length: 70
Last-Modified: Thu, 16 May 2019 18:21:16 GMT
Connection: close
ETag: "5cddaa1c-46"
Accept-Ranges: bytes

This is a test of Adafruit WiFi!
If you can read this, its working :)

disconnecting from server.
```

Text emphasized with a yellow exclamation: If you have a newer QT Py ESP32-S3 and experience WiFi issues, you can try changing the WiFi TX power from the default 20 dBm to 15 dBm. You can do this in `setup()` with: `WiFi.setTxPower(WIFI_POWER_15dBm);` and confirm it with: `WiFi.getTxPower();`

Secure Connection Example

Many servers today do not allow non-SSL connectivity. Lucky for you the ESP32 has a great TLS/SSL stack so you can have that all taken care of for you. Here's an example of a using a secure WiFi connection to connect to the Twitter API.

Copy and paste it into the Arduino IDE:

```
1  // SPDX-FileCopyrightText: 2015 Arturo Guadalupi
2  // SPDX-FileCopyrightText: 2020 Brent Rubell for Adafruit Industries
3  //
4  // SPDX-License-Identifier: MIT
5
6  /*
```

```
 7  This example creates a client object that connects and transfers
 8  data using always SSL.
 9
10  It is compatible with the methods normally related to plain
11  connections, like client.connect(host, port).
12
13  Written by Arturo Guadalupi
14  last revision November 2015
15
16  */
17
18  #include <WiFiClientSecure.h>
19  #include <WiFi.h>
20
21  // Enter your WiFi SSID and password
22  char ssid[] = "YOUR_SSID";            // your network SSID (name)
23  char pass[] = "YOUR_SSID_PASSWORD";   // your network password (use for WPA, or use as key for WEP)
24  int keyIndex = 0;                     // your network key Index number (needed only for WEP)
25
26  int status = WL_IDLE_STATUS;
27  // if you don't want to use DNS (and reduce your sketch size)
28  // use the numeric IP instead of the name for the server:
29  //IPAddress server(74,125,232,128);  // numeric IP for Google (no DNS)
30
31  #define SERVER "cdn.syndication.twimg.com"
32  #define PATH   "/widgets/followbutton/info.json?screen_names=adafruit"
33
34  // Initialize the SSL client library
35  // with the IP address and port of the server
36  // that you want to connect to (port 443 is default for HTTPS):
37  WiFiClientSecure client;
38
39  void setup() {
40    //Initialize serial and wait for port to open:
41    Serial.begin(115200);
42    while (!Serial) {
43      ; // wait for serial port to connect. Needed for native USB port only
44    }
45
46    // attempt to connect to Wifi network:
47    Serial.print("Attempting to connect to SSID: ");
48    Serial.println(ssid);
49
50    WiFi.begin(ssid, pass);
51    while (WiFi.status() != WL_CONNECTED) {
52       delay(500);
53       Serial.print(".");
54    }
55
56    Serial.println("");
57    Serial.println("Connected to WiFi");
58    printWifiStatus();
59
60    client.setInsecure(); // don't use a root cert
61
62    Serial.println("\nStarting connection to server...");
```

```
  // if you get a connection, report back via serial:
  if (client.connect(SERVER, 443)) {
    Serial.println("connected to server");
    // Make a HTTP request:
    client.println("GET " PATH " HTTP/1.1");
    client.println("Host: " SERVER);
    client.println("Connection: close");
    client.println();
  }
}

uint32_t bytes = 0;

void loop() {
  // if there are incoming bytes available
  // from the server, read them and print them:
  while (client.available()) {
    char c = client.read();
    Serial.write(c);
    bytes++;
  }

  // if the server's disconnected, stop the client:
  if (!client.connected()) {
    Serial.println();
    Serial.println("disconnecting from server.");
    client.stop();
    Serial.print("Read "); Serial.print(bytes); Serial.println(" bytes");

    // do nothing forevermore:
    while (true);
  }
}


void printWifiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your board's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // print the received signal strength:
  long rssi = WiFi.RSSI();
  Serial.print("signal strength (RSSI):");
  Serial.print(rssi);
  Serial.println(" dBm");
}
```

https://github.com/adafruit/Adafruit_Learning_System_Guides/blob/main/ESP32_S2_WiFi_Tests/WiFiSSLClient/WiFiSSLClient.ino

As before, **update the ssid and password first**, then upload the example to your board.

Note we use `WiFiClientSecure client` instead of `WiFiClient client;` to require a SSL connection!

```
Attempting to connect to SSID: Transit
......
Connected to WiFi
SSID: Transit
IP Address: 192.168.1.182
signal strength (RSSI):-52 dBm

Starting connection to server...
connected to server
HTTP/1.1 200 OK
Accept-Ranges: bytes
Access-Contol-Allow-Origin: platform.twitter.com
Access-Control-Allow-Methods: GET
Age: 12
cache-control: must-revalidate, max-age=600
content-disposition: attachment; filename=json.json
Content-Type: application/json;charset=utf-8
Date: Wed, 11 Nov 2020 20:58:39 GMT
expires: Wed, 11 Nov 2020 21:08:39 GMT
Last-Modified: Wed, 11 Nov 2020 20:58:27 GMT
Server: ECS (agb/52BA)
strict-transport-security: max-age=631138519
timing-allow-origin: *
X-Cache: HIT
x-connection-hash: a50988a9020759ec70520caef6c38bcf
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
x-response-time: 12
x-transaction: 003d88570028acec
x-tw-cdn: VZ
x-tw-cdn: VZ
x-xss-protection: 0
Content-Length: 197
Connection: close

[{"following":false,"id":"20731304","screen_name":"adafruit","name":"adafruit industries","
disconnecting from server.
Read 966 bytes
```

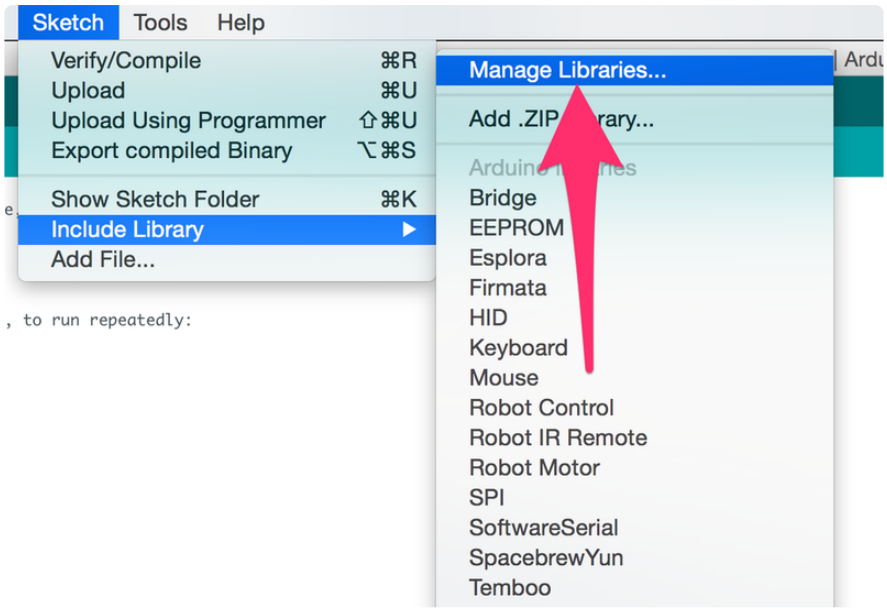
Usage with Adafruit IO

The ESP32-S2/S3 is an affordable, all-in-one, option for connecting your projects to the internet using our IoT platform, Adafruit IO ().

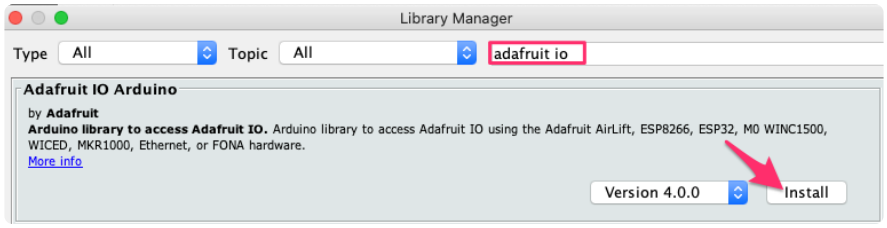
- For more information and guides about Adafruit IO, check out the Adafruit IO Basics Series. ()

Install Libraries

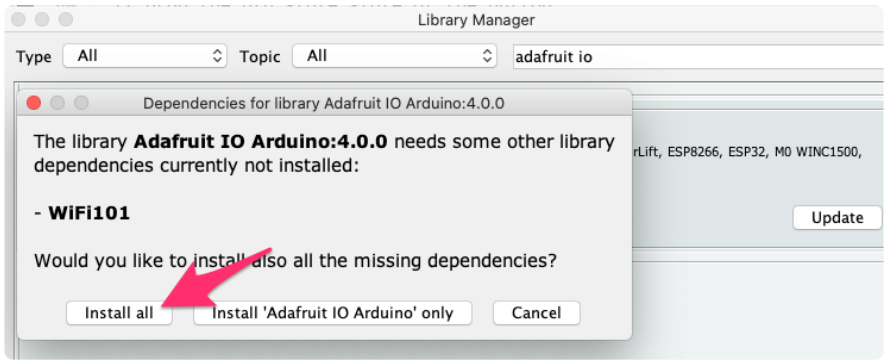
In the Arduino IDE, navigate to **Sketch -> Include Library->Manage Libraries...**



Enter **Adafruit IO Arduino** into the search box, and click **Install** on the **Adafruit IO Arduino** library option to install version 4.0.0 or higher.



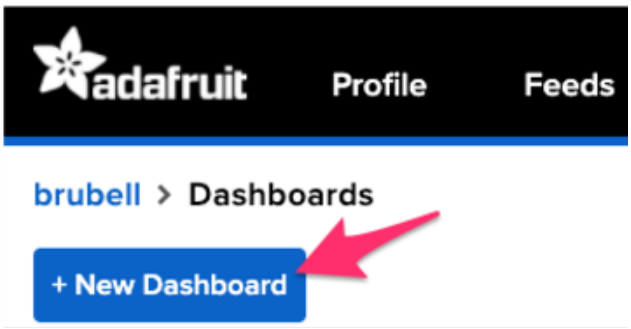
When asked to install dependencies, **click Install all**.

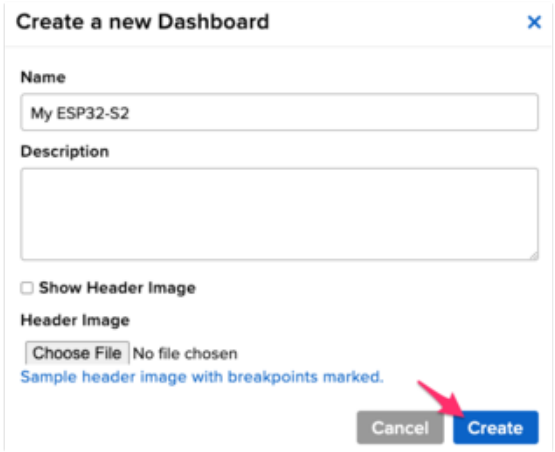


Adafruit IO Setup

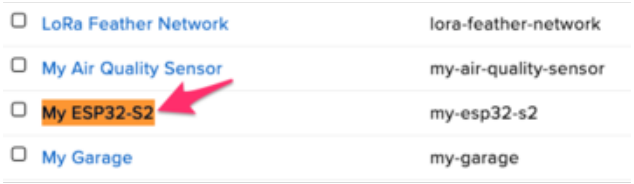
If you do not already have an Adafruit IO account, create one now (). Next, **navigate to the Adafruit IO Dashboards page**.

We'll create a dashboard to visualize and interact with the data being sent between your ESP32-S2/S3 board and Adafruit IO.

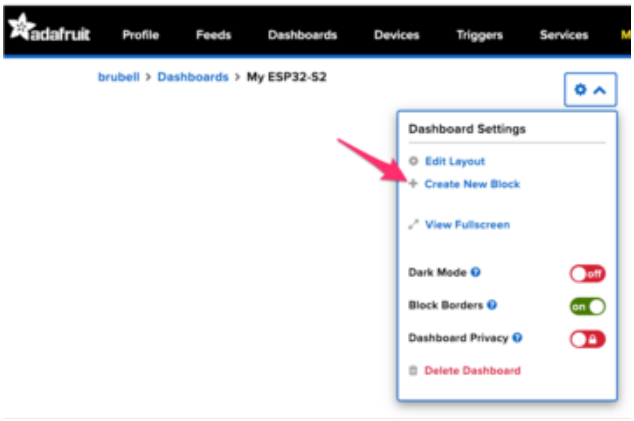


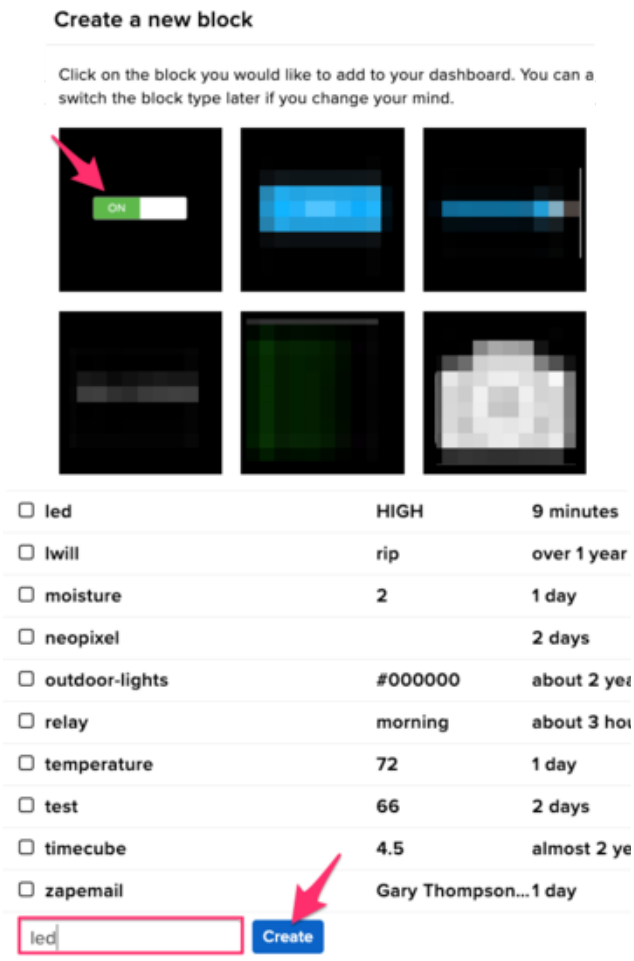


Click the New Dashboard button.
Name your dashboard My ESP32-S2 or My ESP32-S3 depending on your board.
Your new dashboard should appear in the list.
Click the link to be brought to your new dashboard.

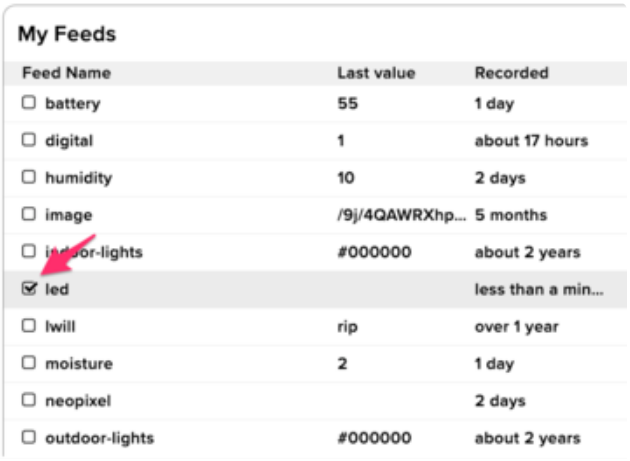


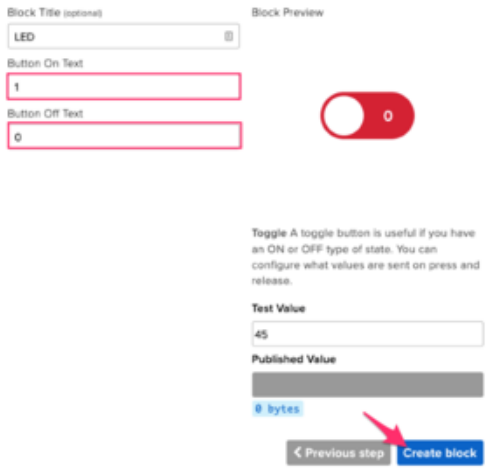
We'll want to turn the board's LED on or off from Adafruit IO. To do this, **we'll need to add a toggle button to our dashboard**.





Click the cog at the top right hand corner of your dashboard.
In the dashboard settings dropdown, **click Create New Block.**
Select the toggle block.
Under My Feeds, **enter led as a feed name**. **Click Create**.
Choose the led feed to connect it to the toggle block. **Click Next step**.

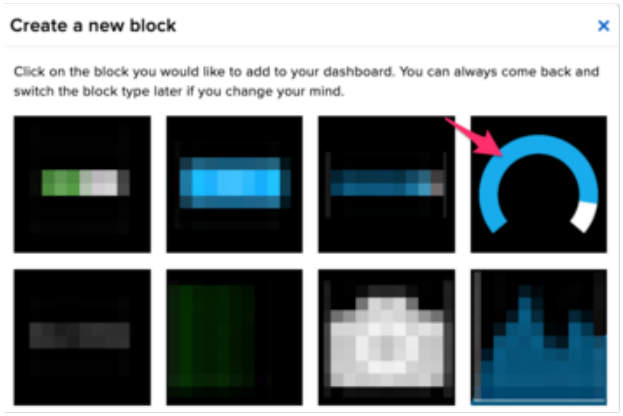


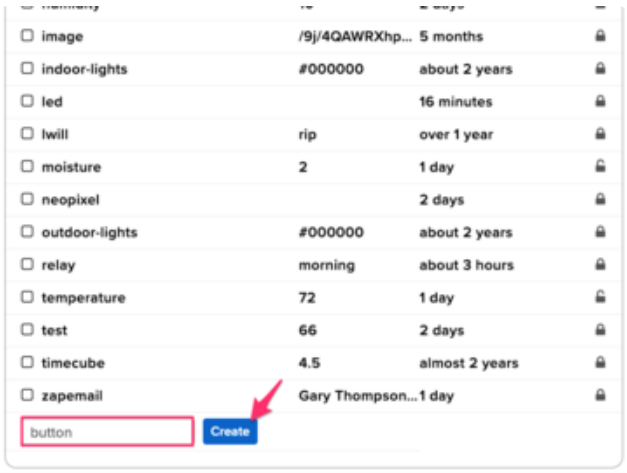


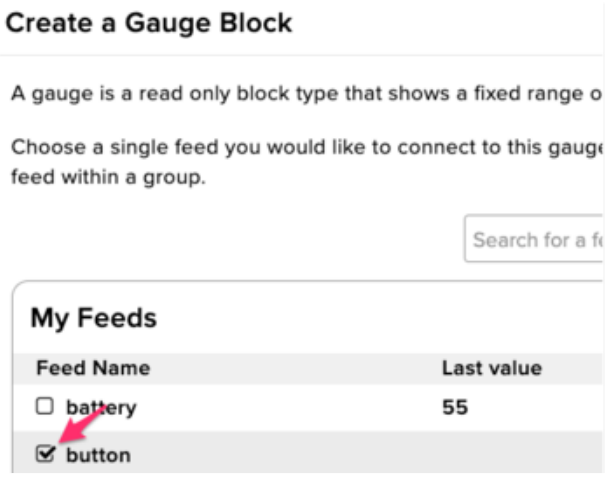
Under Block Settings,

Change Button On Text to **1**
Change Button Off Text to **0**
Click Create block

Next up, we'll want to display button press data from your board on Adafruit IO. To do this, **we'll add a gauge block to the Adafruit IO dashboard**. A gauge is a read only block type that shows a fixed range of values.





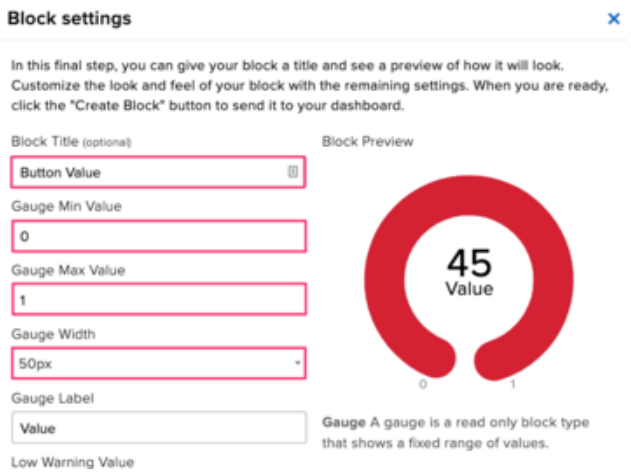


Click the cog at the top right hand corner of your dashboard.
In the dashboard settings dropdown, **click Create New Block.**
Select the gauge block.
Under My Feeds, **enter button as a feed name**.

Click Create.

Choose the button feed to connect it to the toggle block.

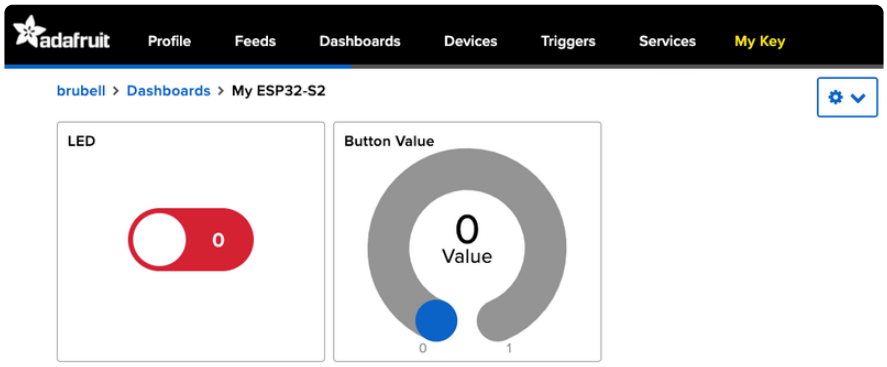
Click Next step.



Under block settings,

Change Block Title to **Button Value**
Change Gauge Min Value to **0**, the button's state when it's off
Change Gauge Max Value to **1**, the button's state when it's on
Click Create block

Your dashboard should look like the following:



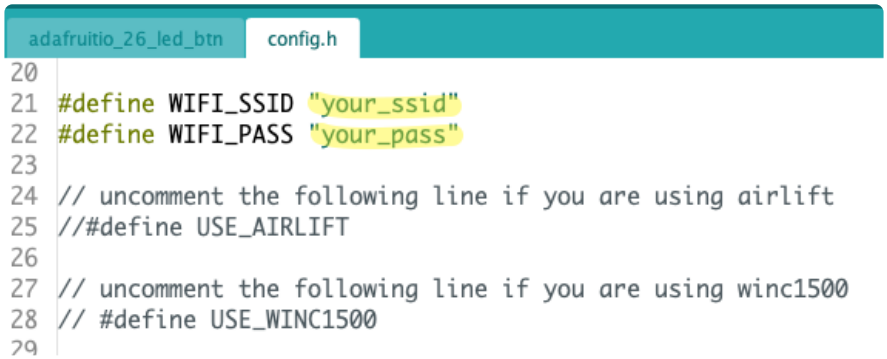
Code Usage

For this example, you will need to open the **adafruitio_26_led_btn** example included with the **Adafruit IO Arduino** library. In the Arduino IDE, navigate to **File -> Examples -> Adafruit IO Arduino -> adafruitio_26_led_btn**.

Before uploading this code to the ESP32-S2/S3, you'll need to add your network and Adafruit IO credentials. **Click on the config.h tab** in the sketch.

Obtain your Adafruit IO Credentials from navigating to io.adafruit.com and clicking My Key (). Copy and paste these credentials next to `IO_USERNAME` and `IO_KEY`.

Enter your network credentials next to `WIFI_SSID` and `WIFI_PASS`.

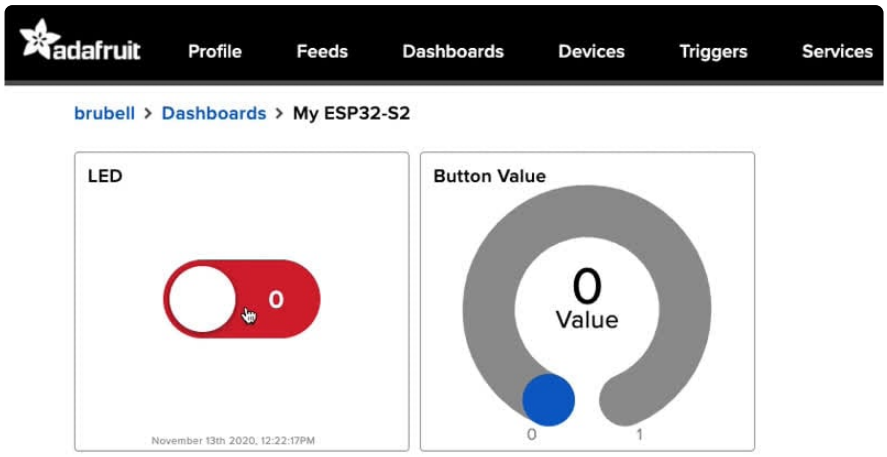


Text emphasized with a yellow exclamation:

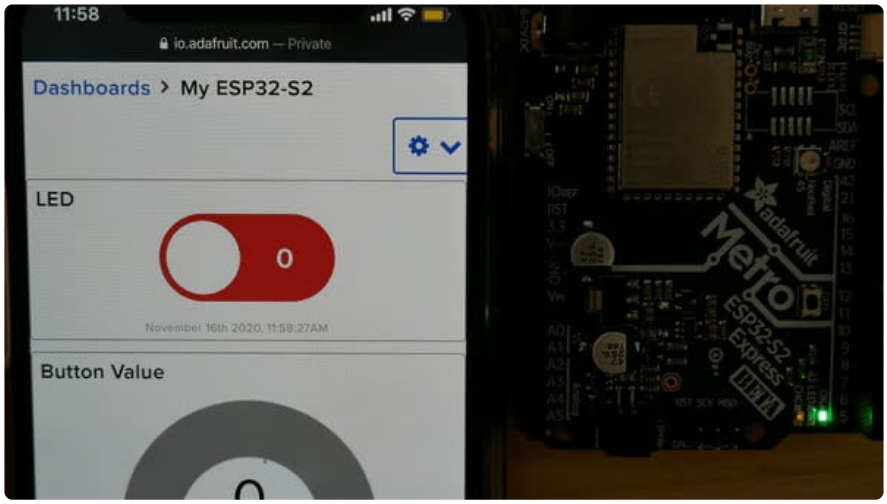
If you have a newer QT Py ESP32-S3 and experience WiFi issues, you can try changing the WiFi TX power from the default 20 dBm to 15 dBm. You can do this in `setup()` with: `WiFi.setTxPower(WIFI_POWER_15dBm);` and confirm it with: `WiFi.getTxPower();`

Click the Upload button to upload your sketch to the ESP32-S2/S3. After uploading, **press the RESET button on your board to launch the sketch.**

Open the Arduino Serial monitor and **navigate to the Adafruit IO dashboard you created**. You should see the gauge response to button press and the board's LED light up in response to the Toggle Switch block.



You should also see the ESP32-S2/S3's LED turning on and off when the LED is toggled:



WipperSnapper Setup

Text emphasized with a yellow exclamation:

The WipperSnapper firmware and ecosystem are in BETA and are actively being developed to add functionality, more boards, more sensors, and fix bugs. We encourage you to try out WipperSnapper with the understanding that it is not final release software and is still in development. If you encounter any bugs, glitches, or difficulties during the beta period, or with this guide, please contact us via http://io.adafruit.com/support

What is WipperSnapper

WipperSnapper is a firmware designed to turn any WiFi-capable board into an Internet-of-Things device without programming a single line of code. WipperSnapper connects to Adafruit IO (), a web platform designed (by Adafruit! ()) to display, respond, and interact with your project's data.

Simply load the WipperSnapper firmware onto your board, add credentials, and plug it into power. Your board will automatically register itself with your Adafruit IO account.

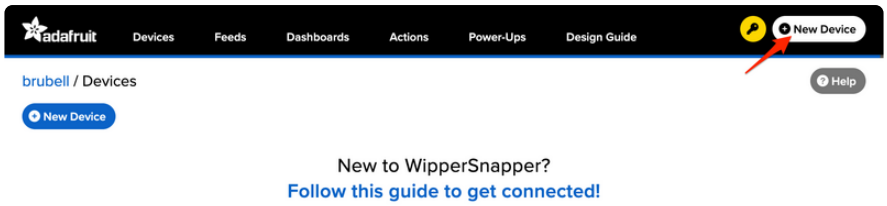
From there, you can add components to your board such as buttons, switches, potentiometers, sensors, and more! Components are dynamically added to hardware, so you can immediately start interacting, logging, and streaming the data your projects produce without writing code.

Sign up for Adafruit.io

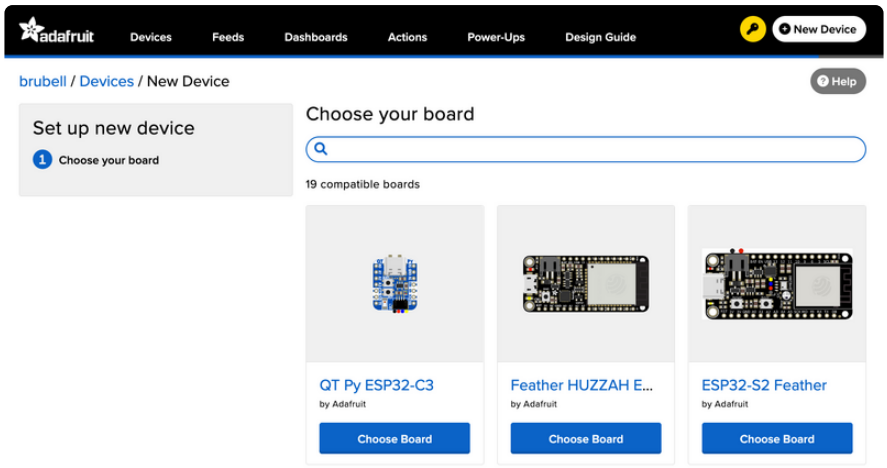
You will need an Adafruit IO account to use WipperSnapper on your board. If you do not already have one, head over to io.adafruit.com () to create a free account.

Add a New Device to Adafruit IO

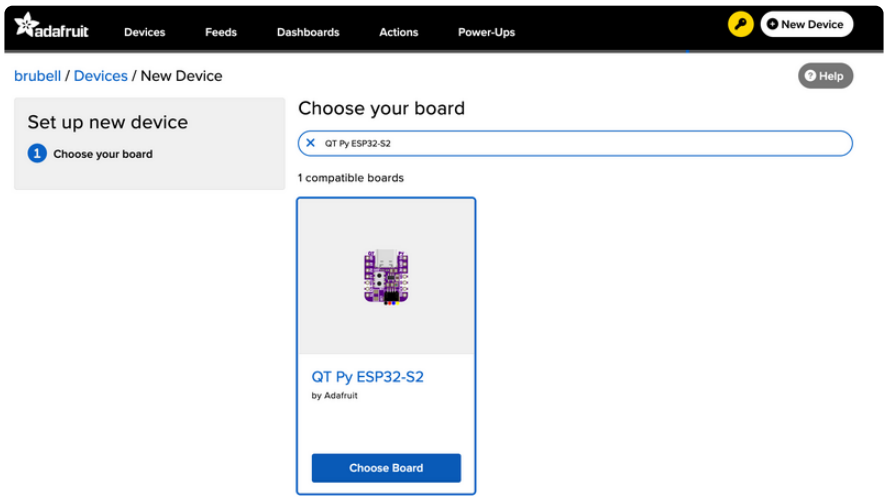
Log into your Adafruit IO () account. Click the New Device button at the top of the page.



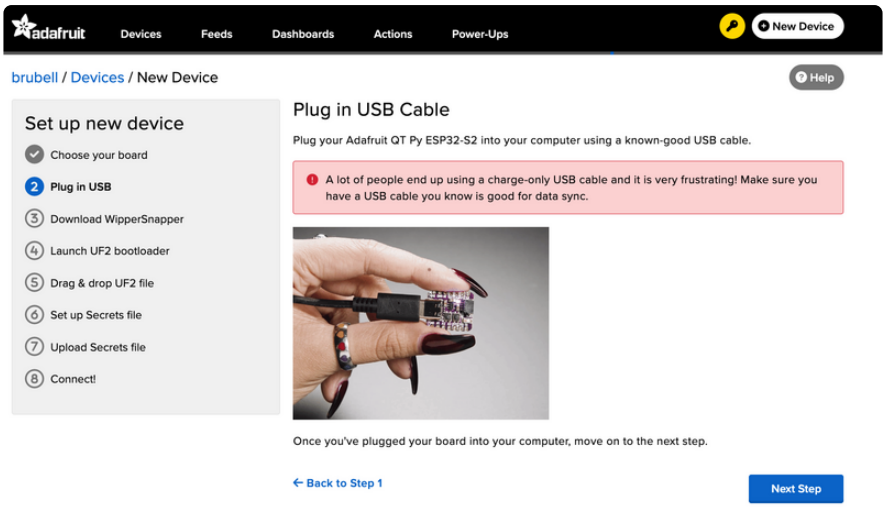
After clicking New Device, you should be on the board selector page. This page displays every board that is compatible with the WipperSnapper firmware.



In the board selector page's search bar, search for the QT Py ESP32-S2. Once you've located the board you'd like to install WipperSnapper on, click the Choose Board button to bring you to the self-guided installation wizard.

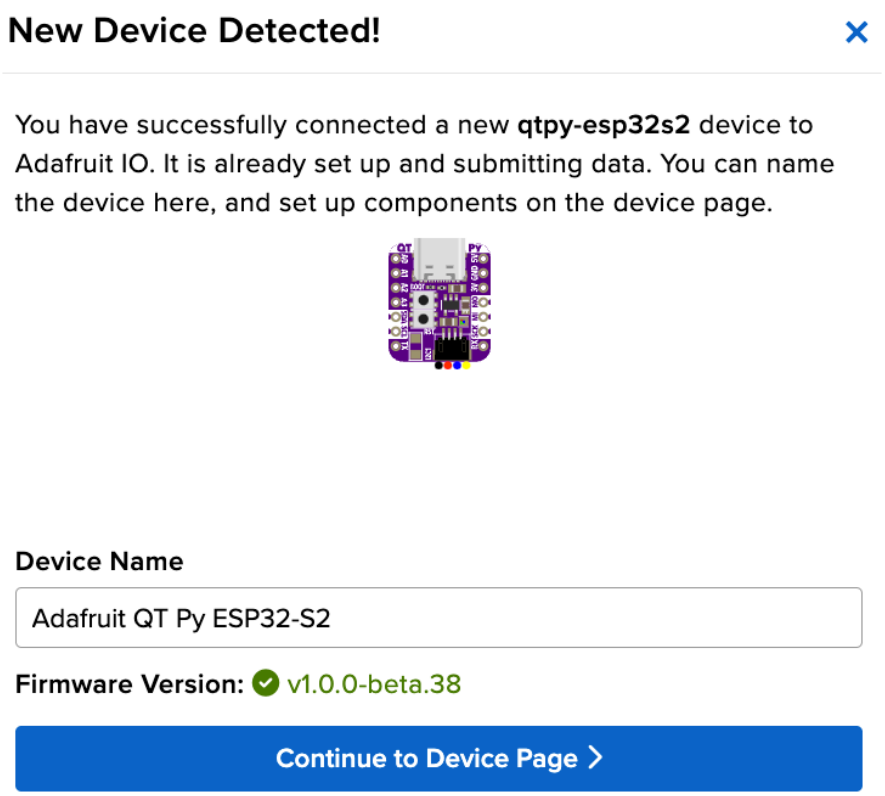


Follow the step-by-step instructions on the page to install Wippersnapper on your device and connect it to Adafruit IO.

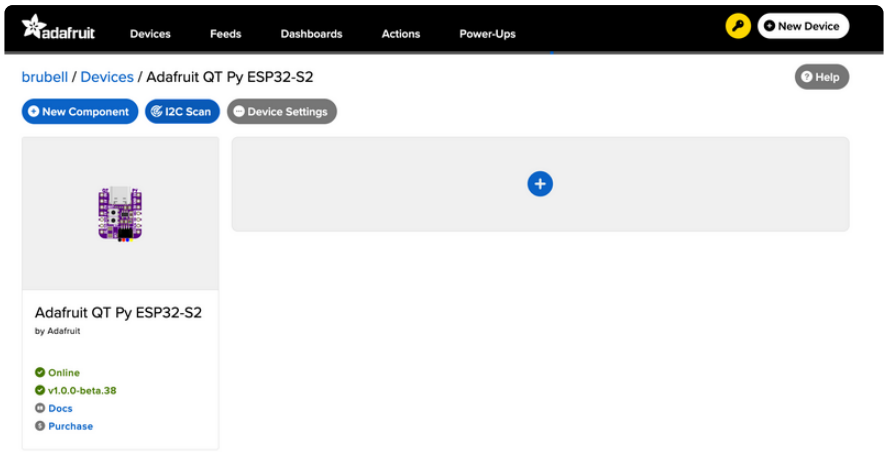


If the installation was successful, a popover should appear displaying that your board has successfully been detected by Adafruit IO.

Give your board a name and click "Continue to Device Page".



You should be brought to your board's device page.



Feedback

Adafruit.io WipperSnapper is in **beta** and you can help improve it!

If you have suggestions or general feedback about the installation process - visit https://io.adafruit.com/support (), click "Contact Adafruit IO Support" and select "I have feedback or suggestions for the WipperSnapper Beta".

Troubleshooting

If you encountered an issue during installation, please try the steps below first.

If you're still unable to resolve the issue, or if your issue is not listed below, get in touch with us directly at https://io.adafruit.com/support (). Make sure to click "Contact Adafruit IO Support" and select "There is an issue with WipperSnapper. Something is broken!"

I don't see my board on Adafruit IO, it is stuck connecting to WiFi

First, make sure that you selected the correct board on the board selector.

Next, please make sure that you entered your WiFi credentials properly,

there are no spaces/special characters in either your network name (SSID) or password, and that you are connected to a 2.4GHz wireless network.

If you're still unable to connect your board to WiFi, please make a new post on the WipperSnapper technical support forum with the error you're experiencing, the LED colors which are blinking, and the board you're using. ()

I don't see my board on Adafruit IO, it is stuck "Registering with Adafruit IO"

Try hard-resetting your board by unplugging it from USB power and plugging it back in.

If the error is still occurring, please make a new post on the WipperSnapper technical support forum with information about what you're experiencing, the LED colors which are blinking (if applicable), and the board you're using. ()

"Uninstalling" WipperSnapper

WipperSnapper firmware is an application that is loaded onto your board. There is nothing to "uninstall". However, you may want to "move" your board from running WipperSnapper to running Arduino or CircuitPython. You also may need to restore your board to the state it was shipped to you from the Adafruit factory.

Moving from WipperSnapper to CircuitPython

Follow the steps on the Installing CircuitPython page () to install CircuitPython on your board running WipperSnapper.

- If you are unable to double-tap the RST button to enter the UF2 bootloader, follow the "Factory Resetting a WipperSnapper Board" instructions below.

Uploading this sketch will overwrite WipperSnapper. If you want to re-install WipperSnapper, follow the instructions at the top of this page.

Moving from WipperSnapper to Arduino

If you want to use your board with Arduino, you will use the Arduino IDE to load any sketch onto your board.

First, follow the page below to set up your Arduino IDE environment for use with your board.

Then, follow the page below to upload the "Arduino Blink" sketch to your board.

Uploading this sketch will overwrite WipperSnapper. If you want to re-install WipperSnapper, follow the instructions at the top of this page.

Factory Resetting a WipperSnapper Board

Sometimes, hardware gets into a state that requires it to be "restored" to the original state it shipped in. If you'd like to get your board back to its original factory state, follow the guide below.

WipperSnapper Usage

Now that you've installed WipperSnapper on your board - let's learn how to use Adafruit IO to interact with your board!

There's a large number of components (physical parts like buttons, switches, sensors, actuators) supported by the WipperSnapper firmware, this page will get you started with the core concepts to build an IoT project with Adafruit IO.

Text emphasized with a yellow exclamation:	This page assumes that you have installed WipperSnapper on your QT Py and registered it with the Adafruit.io WipperSnapper web page. If you have not done this yet, please go back to the previous page in this guide and connect your QT Py.

Blink a LED

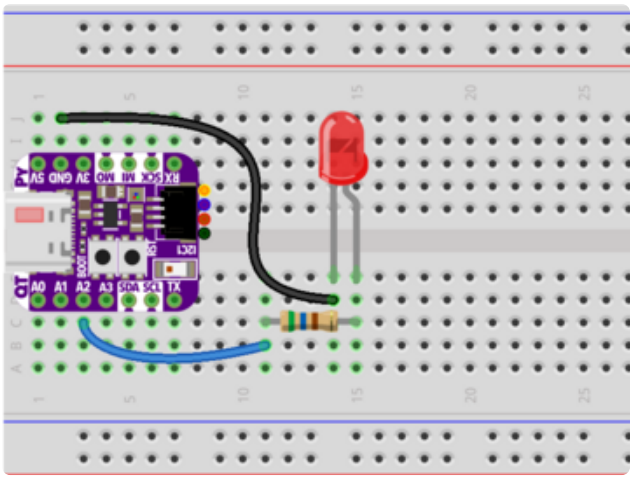
One of the first programs you typically write to get used to embedded programming is a sketch that repeatably blinks an LED. IoT projects are wireless so after completing this section, you'll be able to turn on (or off) the LED connected to your QT Py from anywhere in the world.

Text emphasized with a blue exclamation:	In this demo, we show controlling an LED from Adafruit IO. But the same kind of control can be used for relays, lights, motors, or solenoids.

Wiring

While the QT Py has a built-in NeoPixel, WipperSnapper does not (yet) support NeoPixels (we'll update this guide when it does).

To work around this, you'll build an external LED circuit connected to your QT Py.

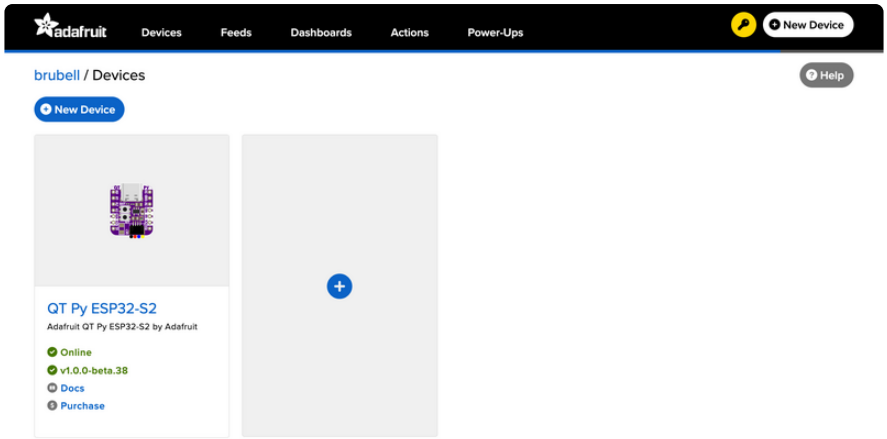


You will need to connect the following pins to the LED and resistor:

QT PY **GND** to LED **cathode** (short leg)
QT PY **Pin A2** to one leg of the **560-ohm resistor**
LED **anode** (long leg) to the second leg of the **560-ohm resistor**

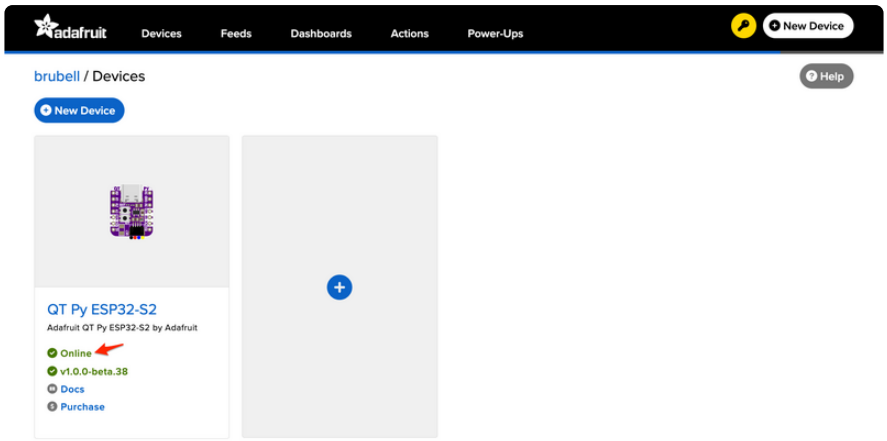
Usage

Navigate to the device page, io.adafruit.com/wippersnapper (). You should see the QT Py you just connected to Adafruit IO listed on this page.

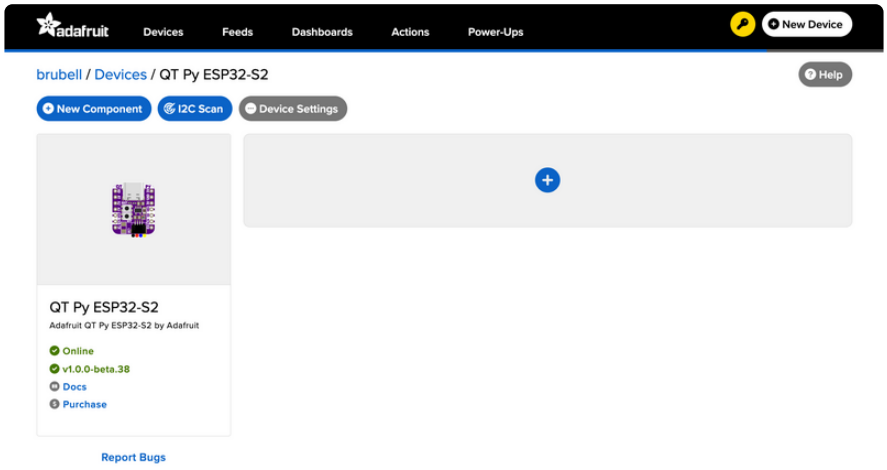


Verify that your QT Py is online and ready to communicate with Adafruit IO by checking that the device tile says Online in green text.

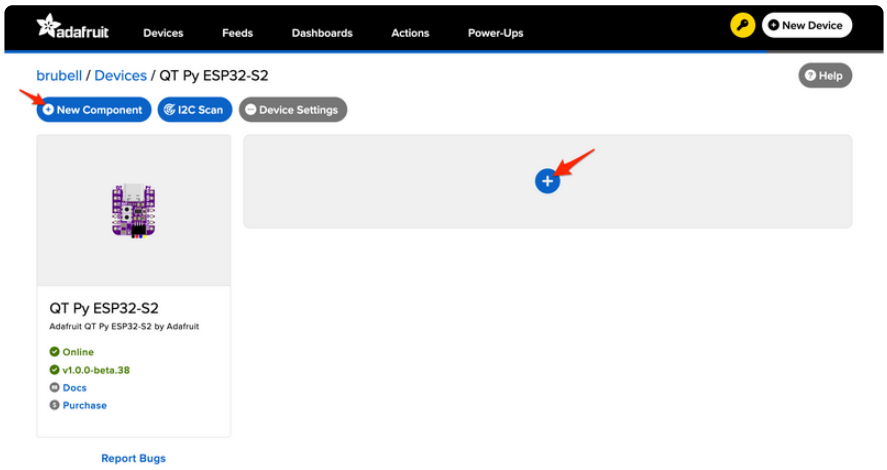
- If the QT Py appears offline on the website but was previously connected, press the Reset (RST) button to force the board to reboot.



Once verified that the device is online, click the device tile to navigate to the board page.

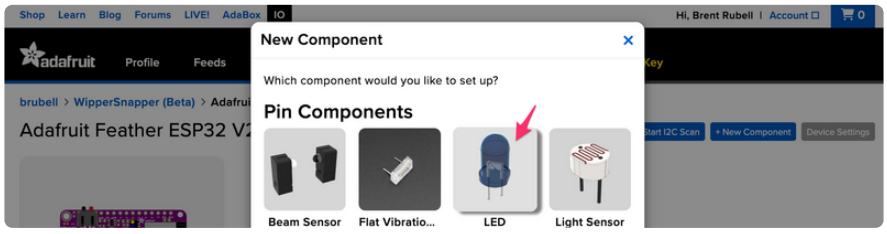


Add a new component to your QT Py by clicking the + **button** or the + **New Component** button.



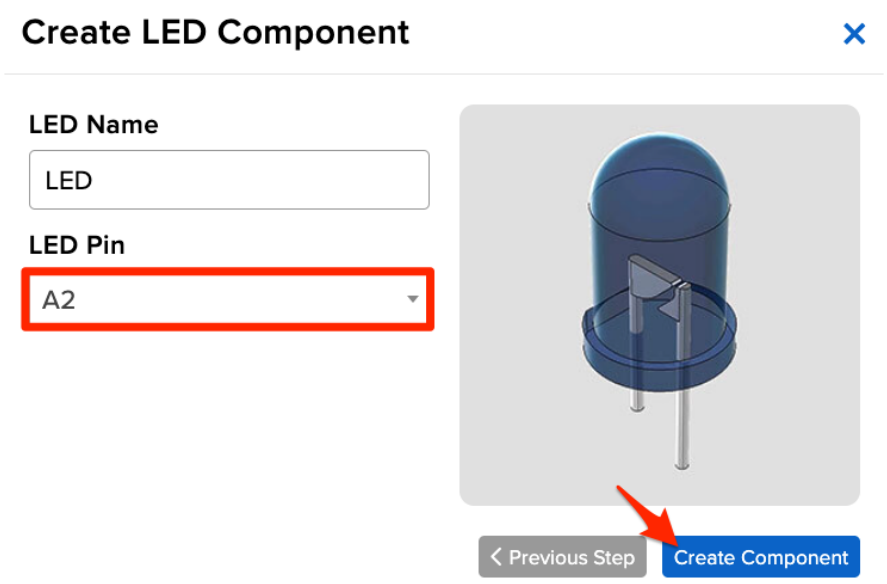
The Component Picker lists all the available components, sensors, and parts, which can be used with the WipperSnapper firmware.

Click the LED component.

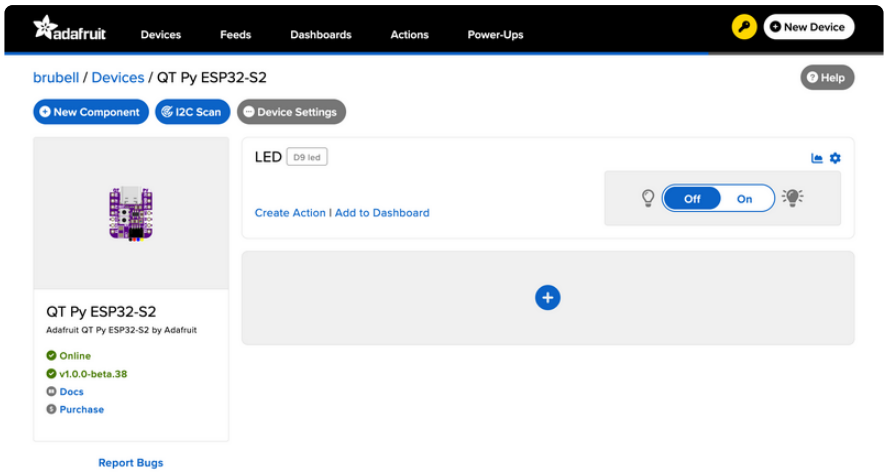


The QT Py contains GPIO pins that can be configured either as an input or an output. The "Create LED Component" screen tells Adafruit IO to configure a general-purpose output pin connected to the LED on your QT Py as a digital output so you can turn the LED on or off.

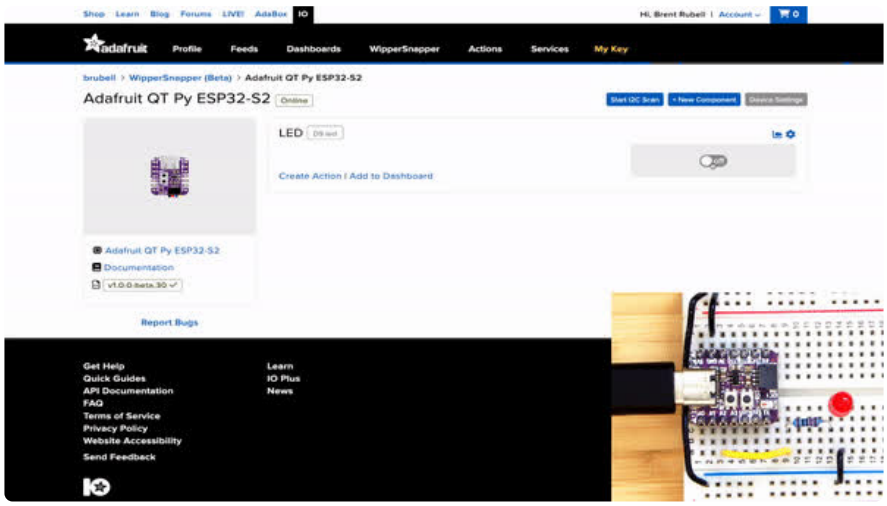
Select the A2 pin as the LED Pin and click **Create Component**



Adafruit IO will send a command to your board (running WipperSnapper) telling it to configure the GPIO pin as a digital output. Your QT Py's board page displays the new LED component.



On the board page, **toggle the LED component by clicking the toggle switch.** This should toggle the LED connected to your QT Py on or off.



Read a Push-Button

You can also configure a board running WipperSnapper to wirelessly read data from standard input buttons, switches, or digital sensors, and send the state to Adafruit IO.

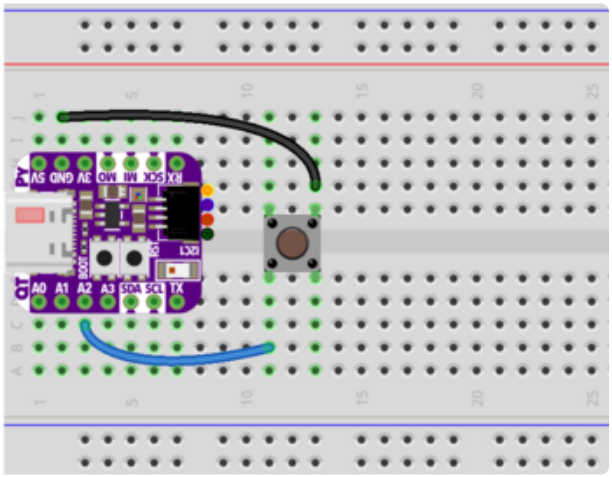
Let's wire up a push button to a QT Py and configure it with WipperSnapper to publish a value to Adafruit IO when the button has been pressed or depressed.

Text
emphasized

with a blue exclamation: In this demo, we show reading the state of a push-button from WipperSnapper. But the same kind of control can be used for reading switches, break beam sensors, and other digital sensors.

Wiring

We'll be using the QT Py's internal pull-up resistors instead of a physical resistor.

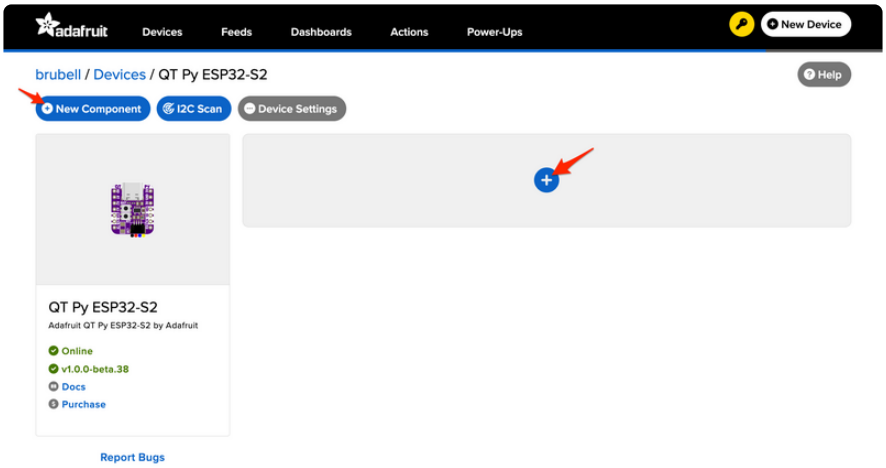


Make the following connections between the QT-PY and the button:

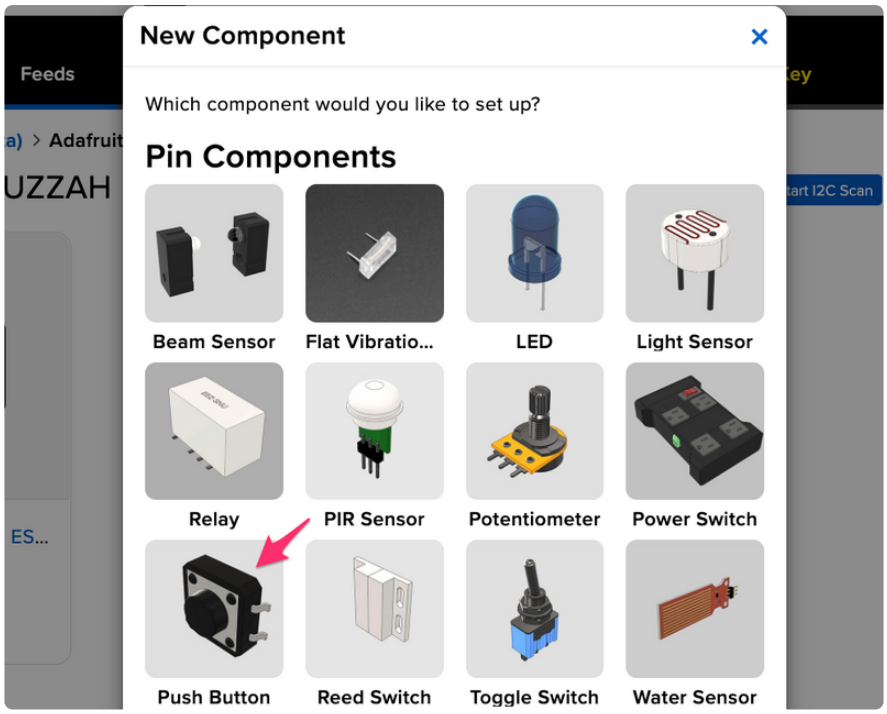
QT Py GND to one leg of the **Push Button**
QT Py GPIO A2 to the other leg of the **Push Button**

Usage

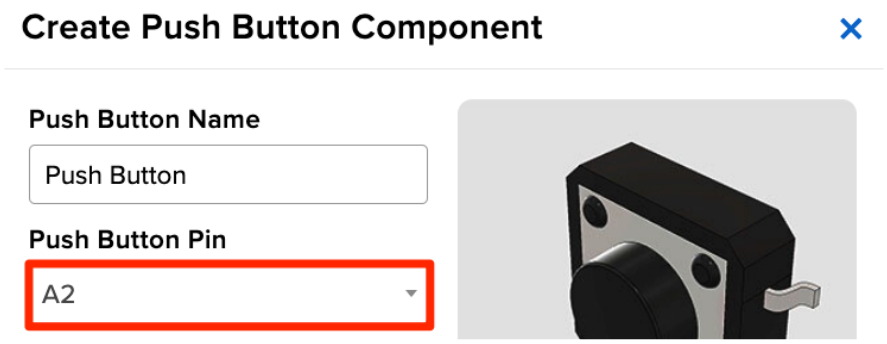
On the board page, add a new component to your QT Py by clicking the + **button** or the + **New Component** button.



From the component picker, select the **Push Button**.

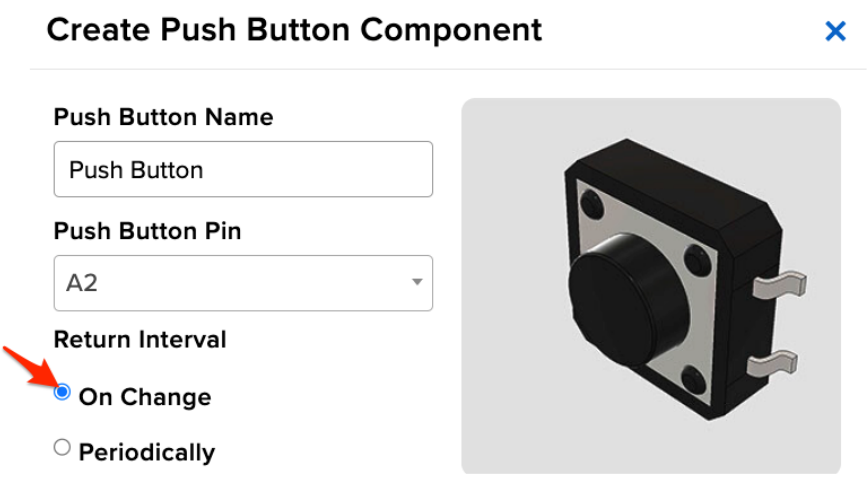


The next screen presents you with options for configuring the push button. Start by selecting the QT Py's digital pin you connected to the push button.

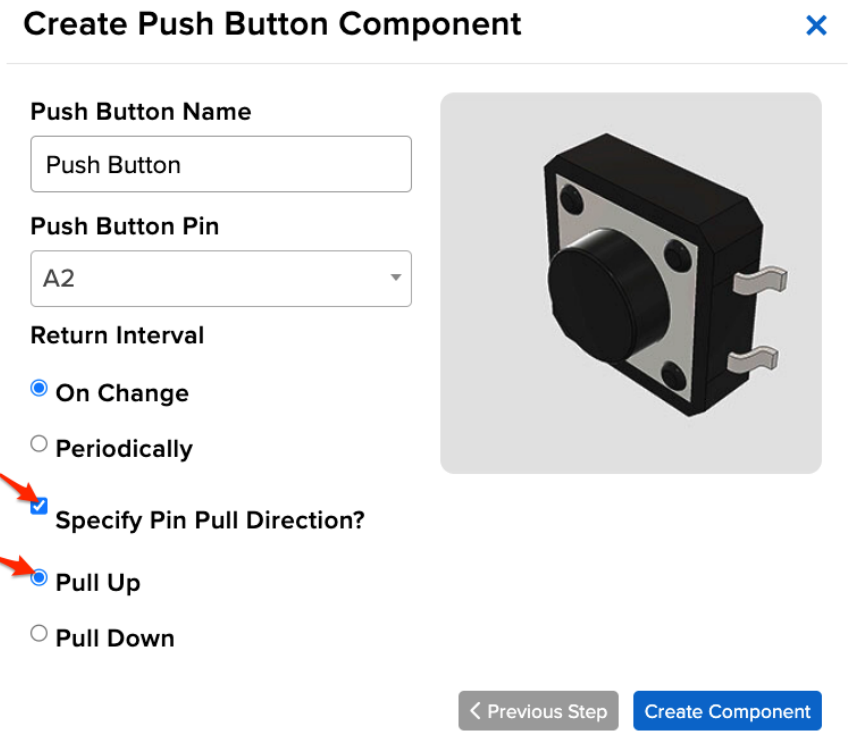


The Return Interval dictates how frequently the value of the push-button will be sent from the QT Py to Adafruit IO. For this example, the push-button's value should only be sent when it's pressed.

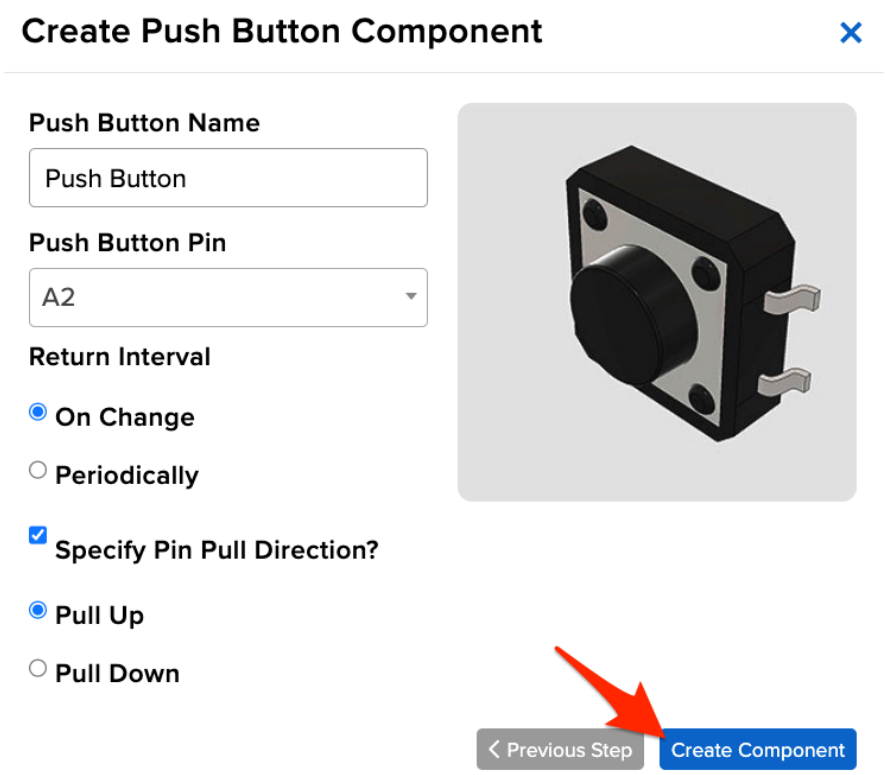
Select **On Change**



Finally, check the **Specify Pin Pull Direction** checkbox and select **Pull Up** to turn on the QT Py's internal pull-up resistor.

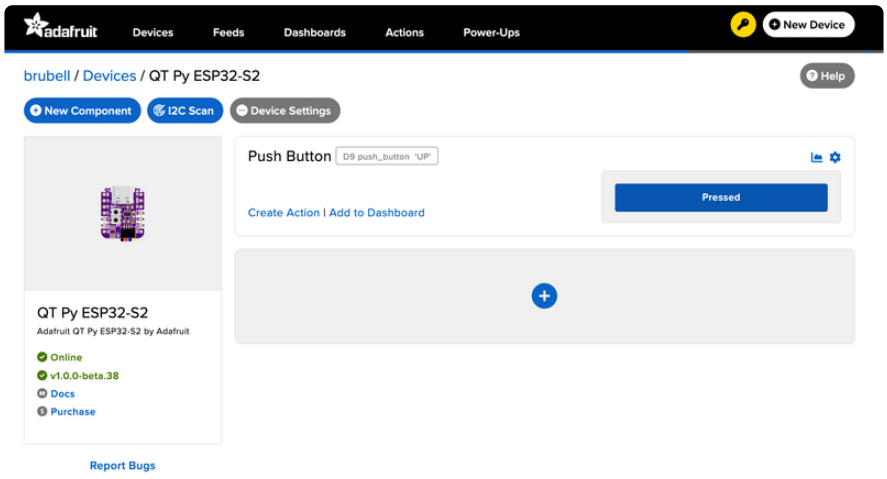


Make sure the form's settings look like the following screenshot. Then, click **Create Component**.

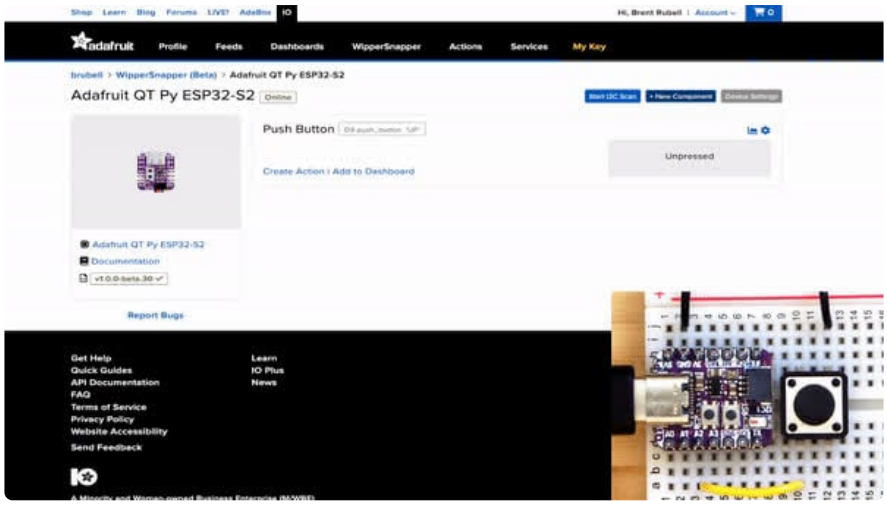


Adafruit IO should send a command to your board (running WipperSnapper), telling it to configure the GPIO pin you selected to behave as a digital input pin and to enable it to pull up the internal resistor.

Your QT Py's board page should also show the new push-button component.



Press the button to change the value of the push button component on Adafruit IO.



Read an I2C Sensor

Inter-Integrate Circuit, aka **I2C**, is a two-wire protocol for connecting sensors and "devices" to a microcontroller. A large number of sensors, including the ones sold by Adafruit, use I2C to communicate.

Typically, using I2C with a microcontroller involves programming. WipperSnapper lets you configure a microcontroller to read data from an I2C sensor and publish that data to the internet without writing code.

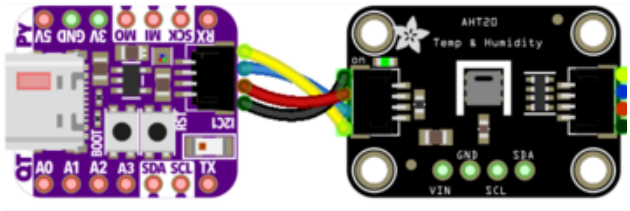
WipperSnapper supports a number of I2C sensors, viewable in list format here (). If you do not see the I2C sensor you're attempting to use with WipperSnapper - we have a guide on adding a component to WipperSnapper here ().

The process of adding an I2C component to your WipperSnapper board is similar between most sensors. For this section, we're using the Adafruit AHT20 (http://adafru.it/4566), an inexpensive sensor that can read ambient temperature and humidity.

Text emphasized with a yellow exclamation:	There are TWO I2C Ports on the QT Py S2. One port is located on the SDA/SCL pins and the other is located in the STEMMA QT connector. WipperSnapper ONLY supports I2C sensors attached to the STEMMA QT connector at this time!

Wiring

First, wire up an AHT20 sensor to your board exactly as follows. Here is an example of the AHT20 wired to a QT Py using I2C [with a STEMMA QT cable (no soldering required) (http://adafru.it/4209)](http://adafru.it/4209).



Make the following connections between the QT Py ESP32-S2 and the AHT20 Sensor

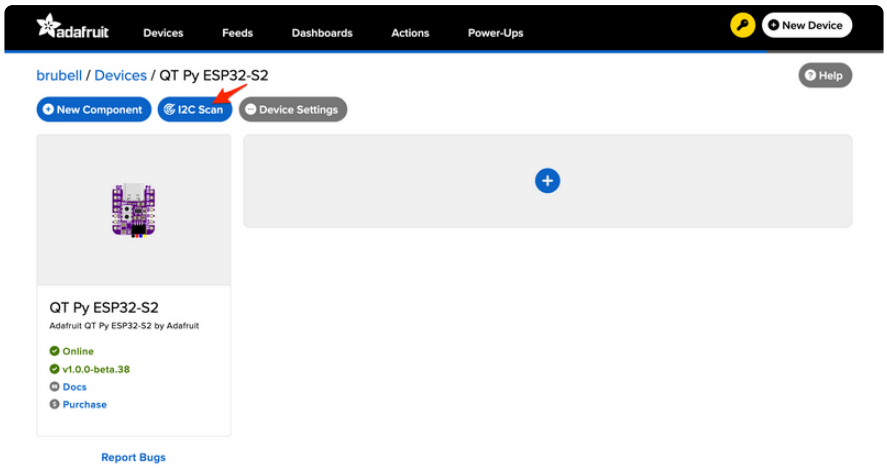
Board 3V to **sensor VIN (red wire on STEMMA QT)**
Board GND to **sensor GND (black wire on STEMMA QT)**
Board SCL to **sensor SCL (yellow wire on STEMMA QT)**
Board SDA to **sensor SDA (blue wire on STEMMA QT)**

Scan I2C Bus

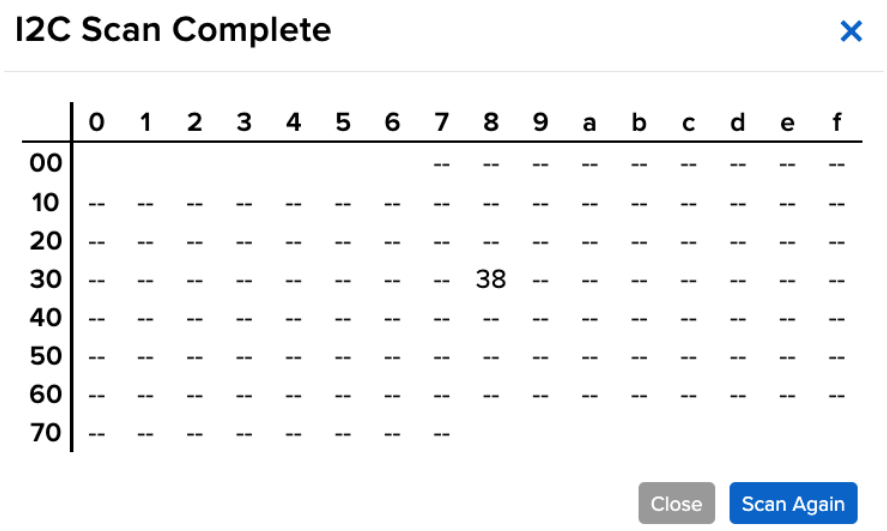
First, ensure that you've correctly wired the AHT20 sensor to your QT Py by performing an I2C scan to detect the I2C device on the bus.

On the upper right-hand corner of the board page, click **Start I2C Scan**.

- If you do not see this button, double-check that your QT Py shows as Online.



You should see a new pop-up showing a list of the I2C addresses detected by an I2C scan. If wired correctly, the AHT20's default I2C address of `0x38` appear in the I2C scan list.



I don't see the I2C sensor's address in the list

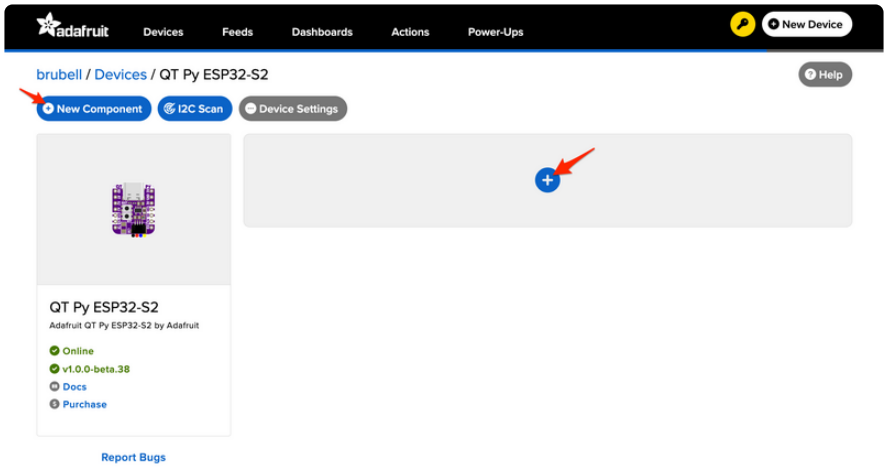
First, double-check the connection and/or wiring between the sensor and the board.

Then, reset the board and let it re-connect to Adafruit IO WipperSnapper.

Create the Sensor Component

Now that you know the sensor can be detected by the QT Py, it's time to configure and create the sensor on WipperSnapper.

Start by clicking +**New Component**



The Component Picker lists all the available components, sensors, and parts that can be used with WipperSnapper.

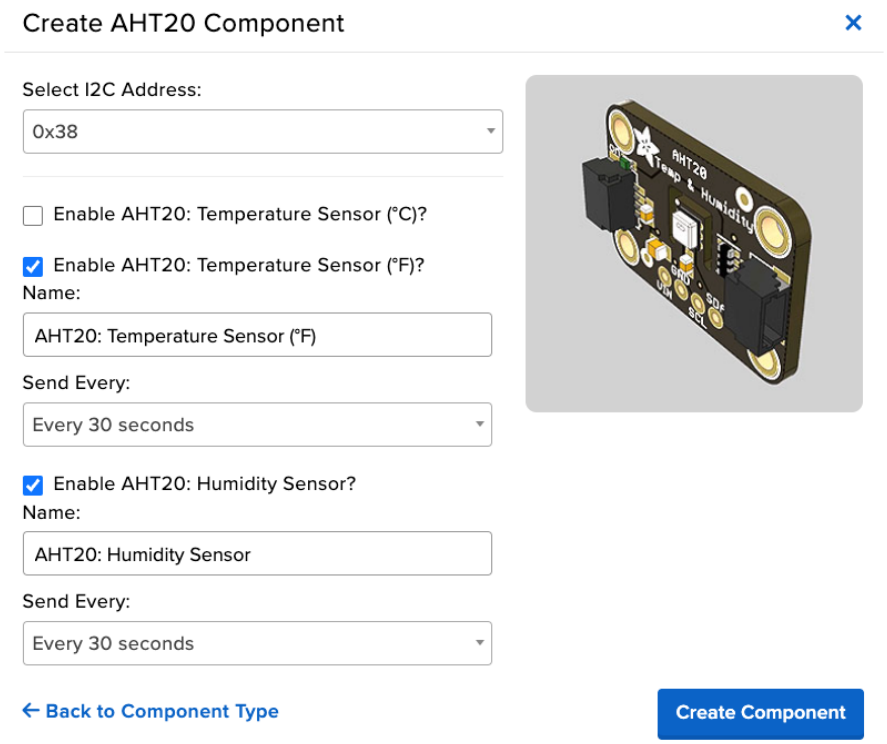
Under the **I2C Components** header, click **AHT20.**

On the component configuration page, the AHT20's I2C sensor address should be listed along with the sensor's settings.

The AHT20 sensor can measure ambient temperature and relative humidity. This page has individual options for reading the ambient temperature, in either degree Celsius or degree Fahrenheit, and the relative humidity. You may select the readings which are appropriate to your application and region.

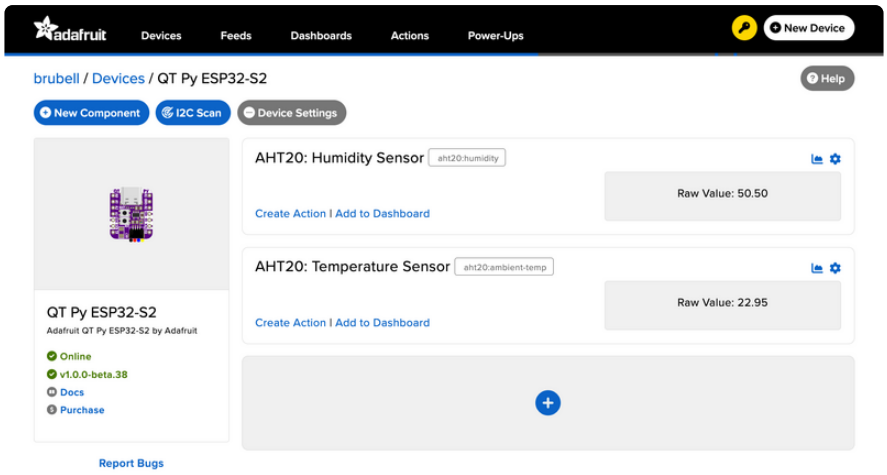
The **Send Every** option is specific to each sensor measurement. This option will tell the QT Py how often it should read from the AHT20 sensor and send the data to Adafruit IO. Measurements can range from every 30 seconds to every 24 hours.

For this example, set the **Send Every** interval for both seconds to **Every 30 seconds** and click **Create Component**.



The page should now show the AHT20 component you created.

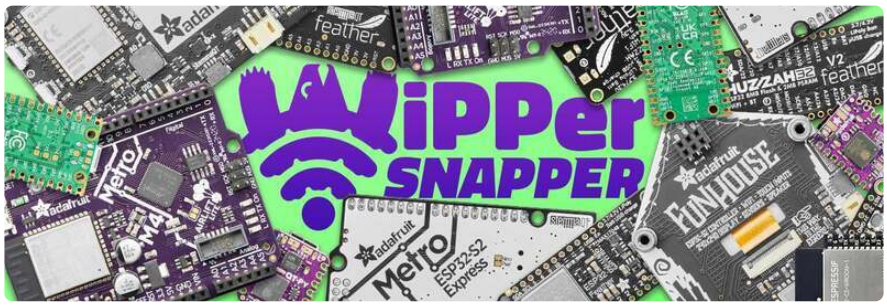
After the interval you configured elapses, WipperSnapper automatically reads values from the sensor and sends them to Adafruit IO.



Going Further

Want to learn more about Adafruit IO WipperSnapper? We have more complex projects on the Adafruit Learning System ().

WipperSnapper Essentials



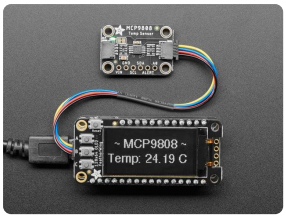
You've installed WipperSnapper firmware on your board and connected it to Adafruit IO. Next, let's learn how to use Adafruit IO!

The Adafruit IO supports a large number of components. Components are physical parts such as buttons, switches, sensors, servos, LEDs, RGB LEDs, and more.

The following pages will get you up and running with WipperSnapper as you interact with your board's LED, read the value of a push button, send the value of an I2C sensor to the internet, and wirelessly control colorful LEDs.

Parts

The following parts are **required** to complete the WipperSnapper essentials pages for this board:



Featured

Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/ precise we've ever seen, with a typical accuracy of ±0.25°C over the sensor's -40°C to...

https://www.adafruit.com/product/5027

Featured

STEMMA QT / Qwiic JST SH 4-pin Cable - 100mm Long

This 4-wire cable is a little over 100mm / 4" long and fitted with JST-SH female 4-pin connectors on both ends. Compared with the chunkier JST-PH these are 1mm pitch instead of...

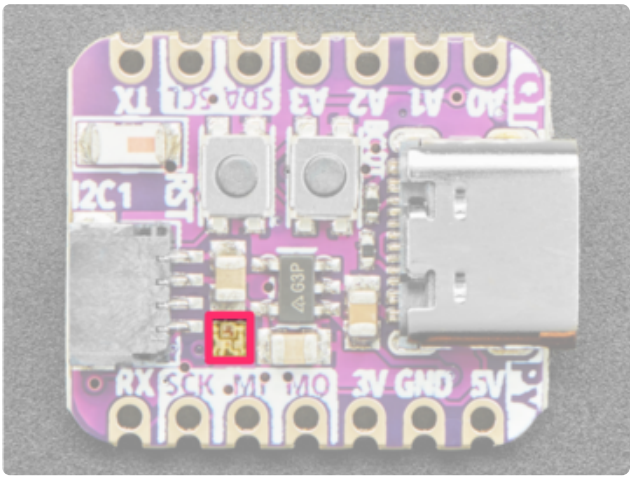
https://www.adafruit.com/product/4210

NeoPixel LED Blink

Your board has a WS281x RGB LED (NeoPixel, in Adafruit jargon) built in. Boards running the WipperSnapper firmware can be wirelessly controlled by Adafruit IO to interact with NeoPixels.

On this page, you'll learn how to change the color and brightness of the NeoPixel built into your board from Adafruit IO.

Where is the NeoPixel on my board?



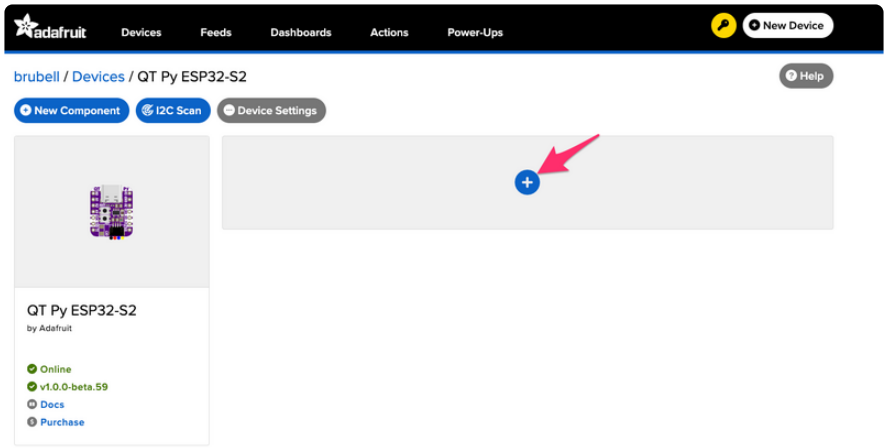
Above the SCK and MI on the silk, is the **RGB NeoPixel LED** (highlighted in pink)

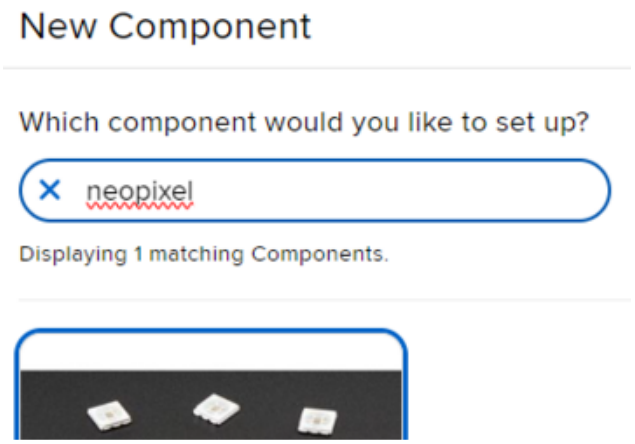
Create the NeoPixel Component

Text emphasized with a green exclamation:

If you used the Magic Config button to automatically configure the components on your board, then the NeoPixel component will already be configured. Skip ahead to the **Usage** section. Don't know about Magic Config? Click the Auto-Config button on the device page for further details!

On the device page, click the New Component (or "+") button to open the component picker.





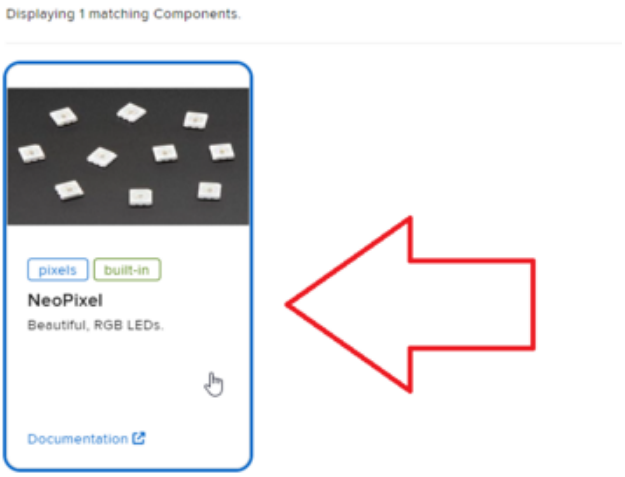
Search for the component name by entering `neopixel` into the text box on the component picker, the list of components should update as soon as you stop typing

Filtering and searching for components

Since WipperSnapper supports such a large number of components, you can use keyword filtering. Try searching for various keywords, like:

- component names: `aht20`, `servo`, `buzzer`, `button`, `neopixel`, etc
- sensor types: `light`, `temperature`, `pressure`, `humidity`, etc
- interface: `i2c`, `uart`, `ds18x20`, `pin`, etc (also I2C addresses e.g. `0x44`)
- vendor: `Adafruit`, `ASAIR`, `Infineon`, `Bosch`, `Honeywell`, `Sensirion`, etc

There is also added product and documentation links for every component, follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide

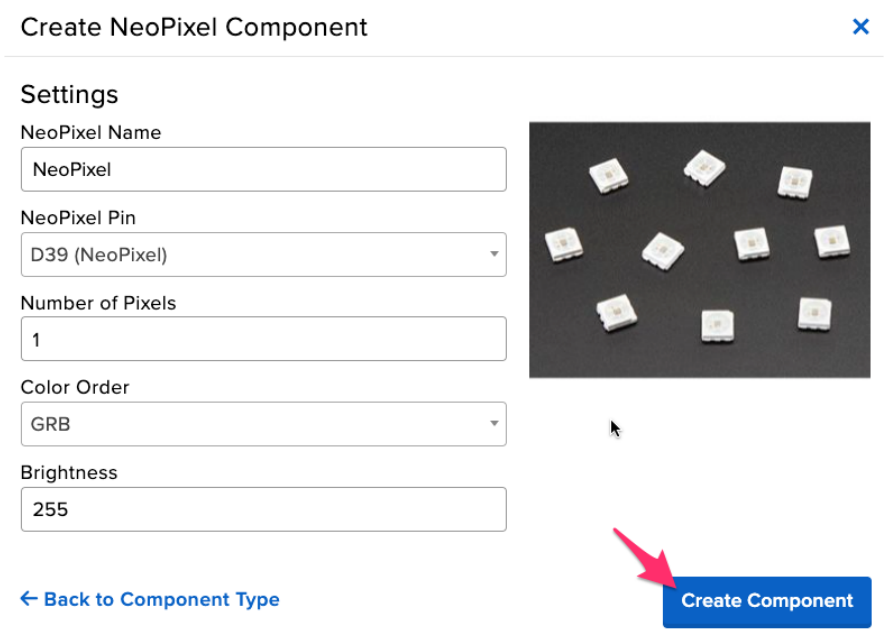


Select the **NeoPixel** from the list of results to go to the component configuration page.

There will be a back button if you select the wrong component, and you can use the Edit component icon (⚙) on the device page to update the component configuration in the future.

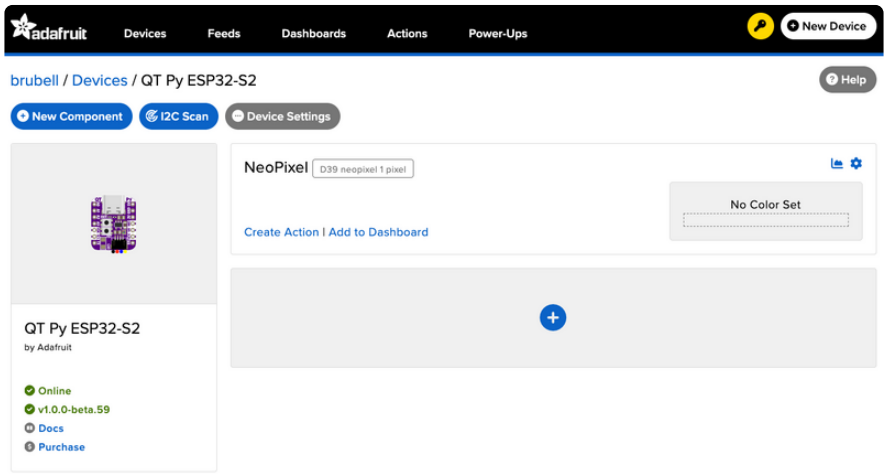
The board NeoPixel pin is automatically found and selected.

Click **Create Component**



Behind the scenes, Adafruit IO sends a command to your board running WipperSnapper firmware telling it to configure the pin as a NeoPixel component with the settings from the form.

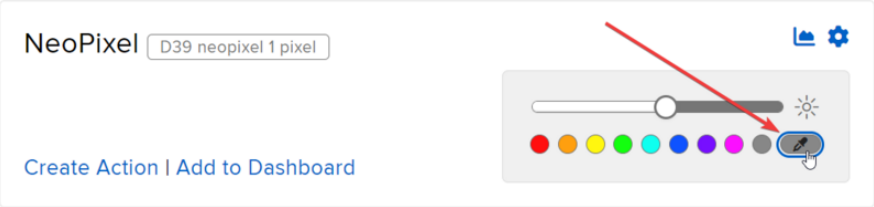
The Device page shows the NeoPixel component.



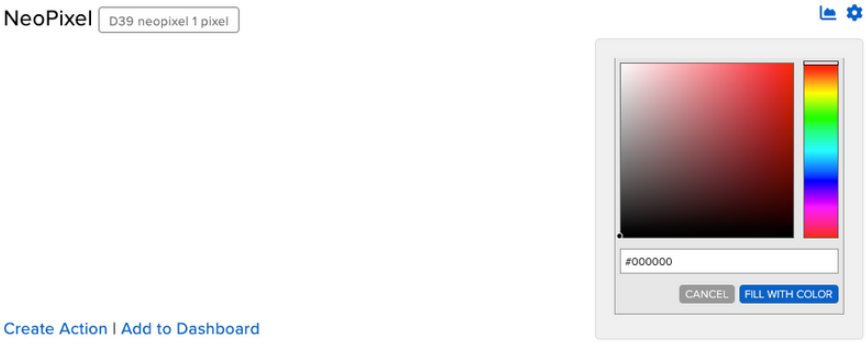
Set the NeoPixel's RGB Color

Since no colors have been set yet, the color picker's default value is `#000000` (black in hex color code) and appears "off". You can change that to make the NeoPixel shine brightly!

On the NeoPixel component, click the color dropper at the end of the color swatch list.



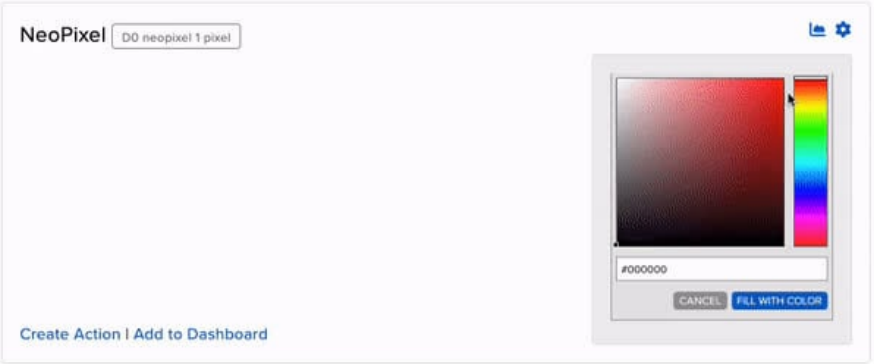
A color picker pops open! Next: learning how Adafruit IO uses hex color codes to represent the colors on your NeoPixel.



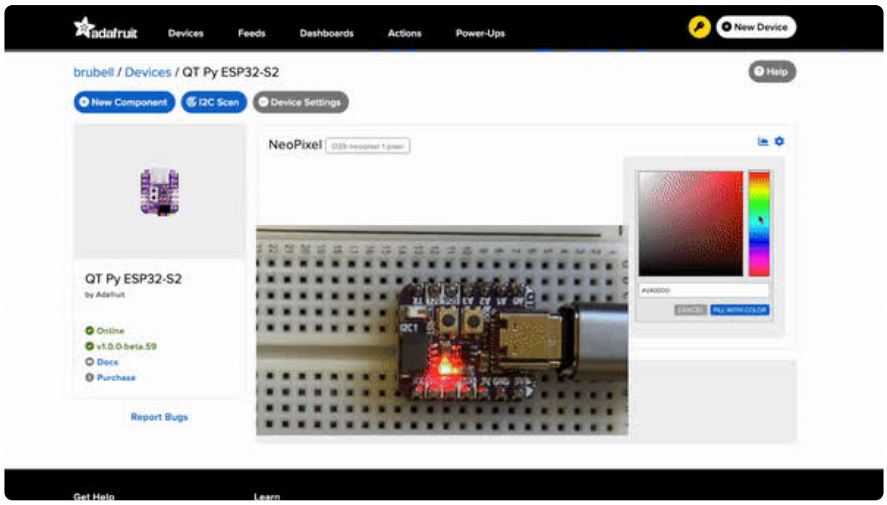
Hex Colors 101

The color picker on Adafruit IO uses hex color codes to represent Red, Green, and Blue values. For example, `#FF0000` is the hex color code for the color red. The colors (`#FF0000`) red component is `FF` (255 translated to decimal), the green component is `00` and the blue component is `00`. Translated to RGB format, the color is `RGB (255, 0, 0)`.

Using the color picker, or by manually entering a hex color code, select a color.

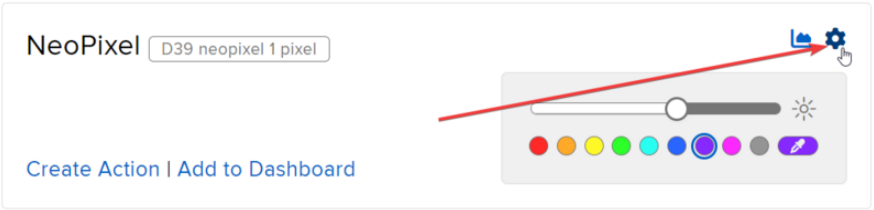


When you're ready to set the color of your device's NeoPixel, click **FILL WITH COLOR**. The NeoPixel will immediately glow!



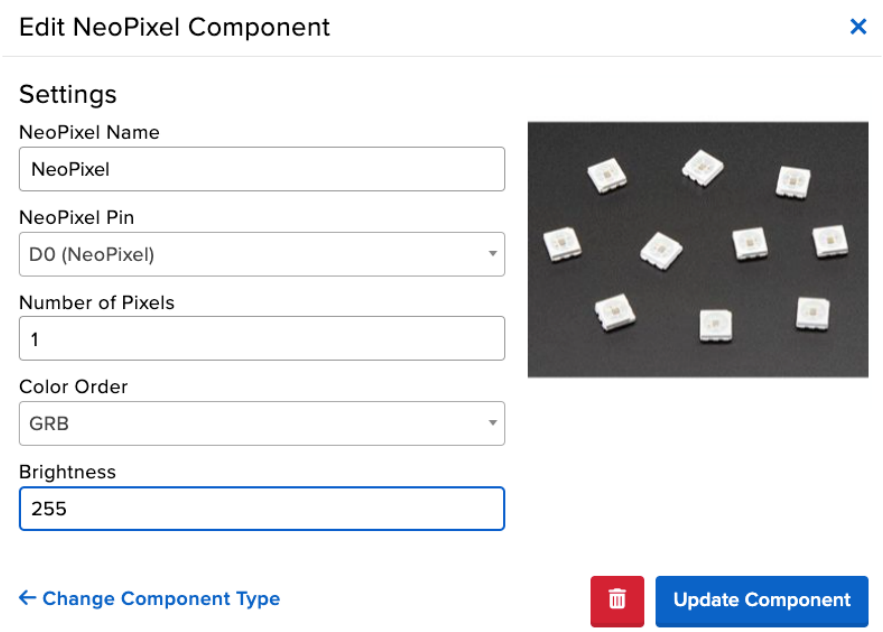
Set NeoPixel Brightness

If the NeoPixel is too bright (or too dim), you can change the overall brightness. Click the gear/cog icon on the NeoPixel component to open its settings.



On the NeoPixel component form, set Brightness to a value between 0 (fully off) and 255 (full brightness).

Click the **Update Component** button to send the updated configuration to your device.



Read a Push-button

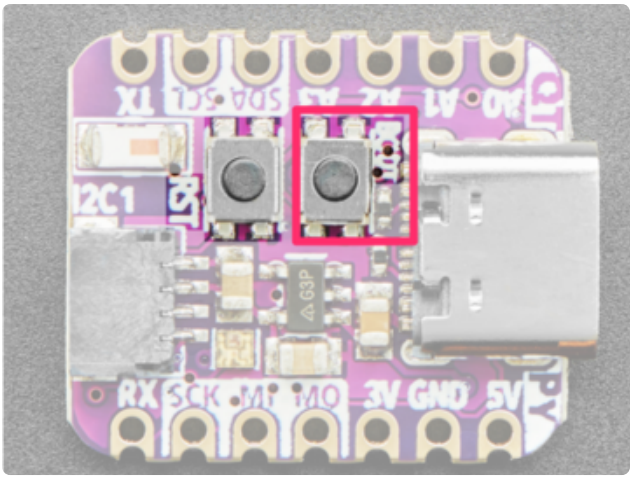
Text emphasized with a blue exclamation:	This demo shows reading the state of a push-button from WipperSnapper. But the same kind of control can be used for reading switches, break beam sensors, and other digital sensors.

You can configure a board running WipperSnapper to read data from standard input buttons, switches, or digital sensors, and send the value to Adafruit IO.

From Adafruit IO, you will configure one of the pushbuttons on your board as a push button component. Then, when the button is pressed (or released), a value will be published to Adafruit IO.

Button Location

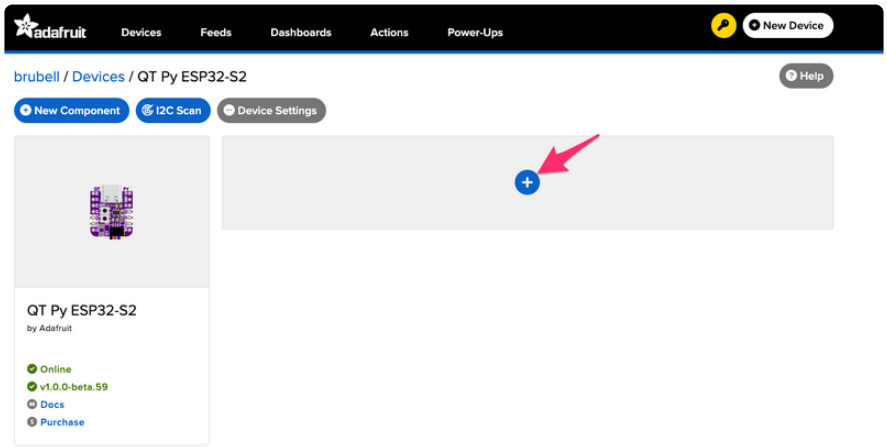
This example uses the board's built-in push-button and internal pull-up resistor instead of wiring a push-button up.

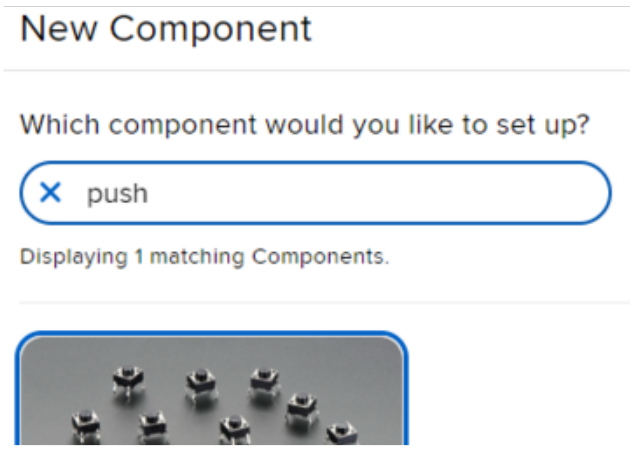


Your QT Py has two buttons: a reset button and a boot button. This page will use the boot button (highlighted in pink), located underneath the QT Py's USB-C cable.

Create a Push-button Component on Adafruit IO

On the device page, click the New Component (or "+") button to open the component picker.





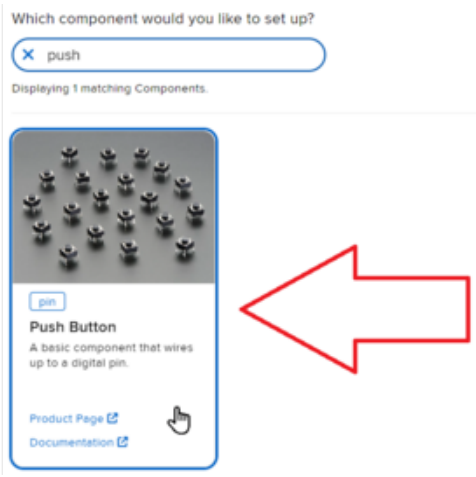
Search for the component name by entering `push` into the text box on the component picker, the list of components should update as soon as you stop typing.

Filtering and searching for components

Since WipperSnapper supports such a large number of components, you can use filtering. Try searching for various keywords, like:

- component names: `aht20`, `servo`, `buzzer`, `button`, `potentiometer`, etc
- sensor types: `light`, `temperature`, `pressure`, `humidity`, etc
- interface: `i2c`, `uart`, `ds18x20`, `pin`, etc (also I2C addresses e.g. `0x44`)
- vendor: `Adafruit`, `ASAIR`, `Infineon`, `Bosch`, `Honeywell`, `Sensirion`, etc

There are also added product and documentation links for every component. Follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide.

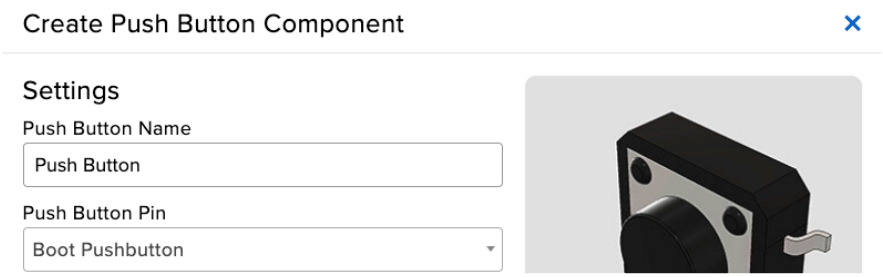


Select the **Push Button** from the list of results to go to the component configuration page.

There will be a back button if you select the wrong component, and you can use the Edit component icon (⚙) on the device page to update the component configuration in the future.

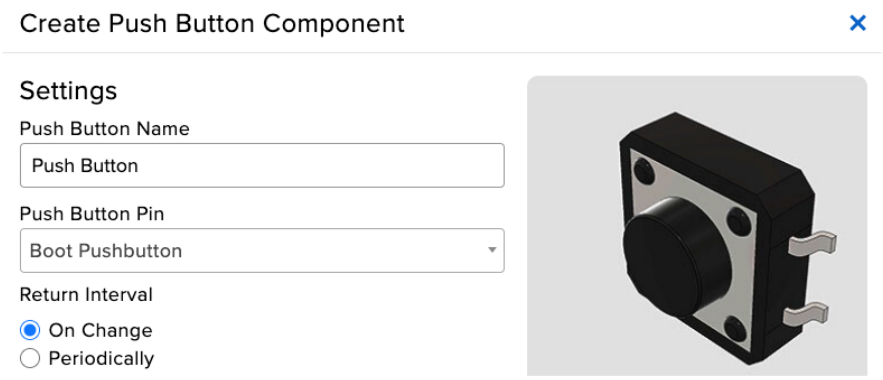
The "Create Push Button Component" form presents you with options for configuring the push button.

Start by selecting the board's pin connected to the push button.

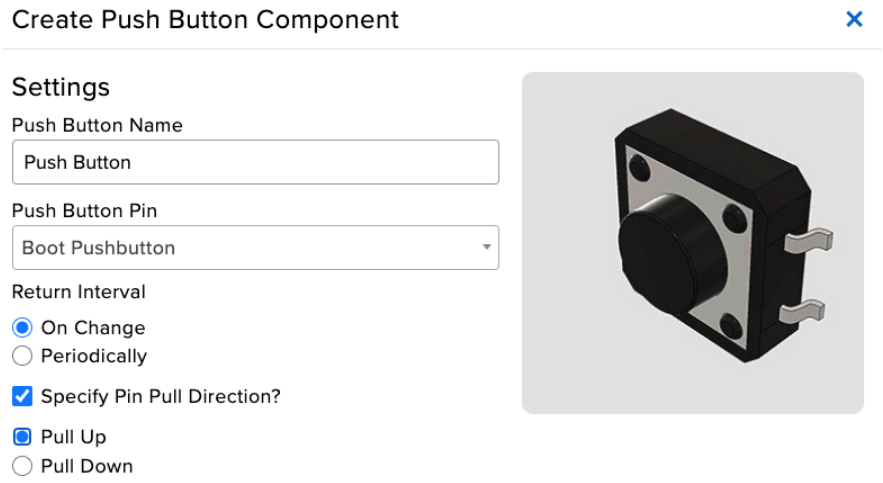


The Return Interval dictates how frequently the value of the push-button will be sent from the board to Adafruit IO.

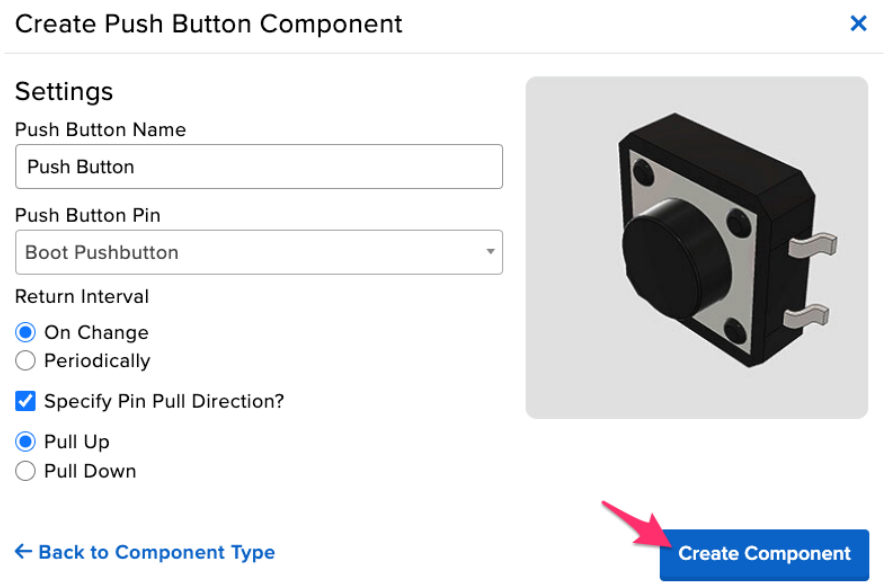
For this example, you will configure the push button's value to be only sent when the value changes (i.e. when it's either pressed or depressed).



Finally, check the **Specify Pin Pull Direction** checkbox and select the pull direction.

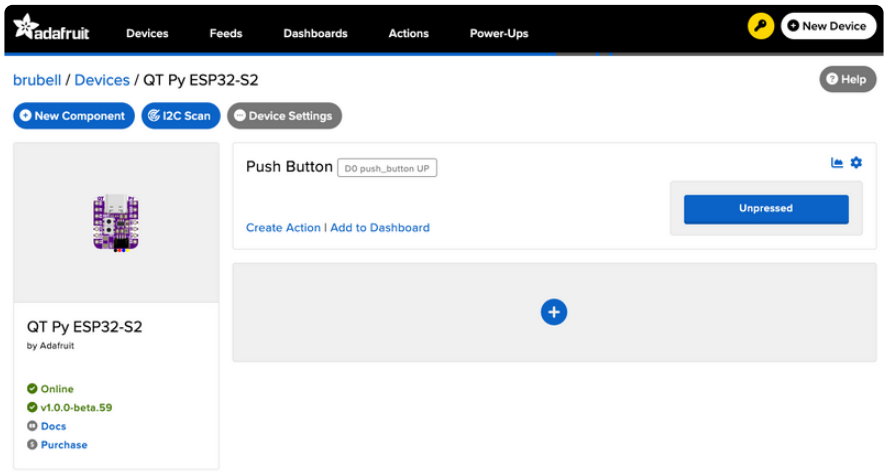


Make sure the form's settings look like the following screenshot. Then, click **Create Component**.

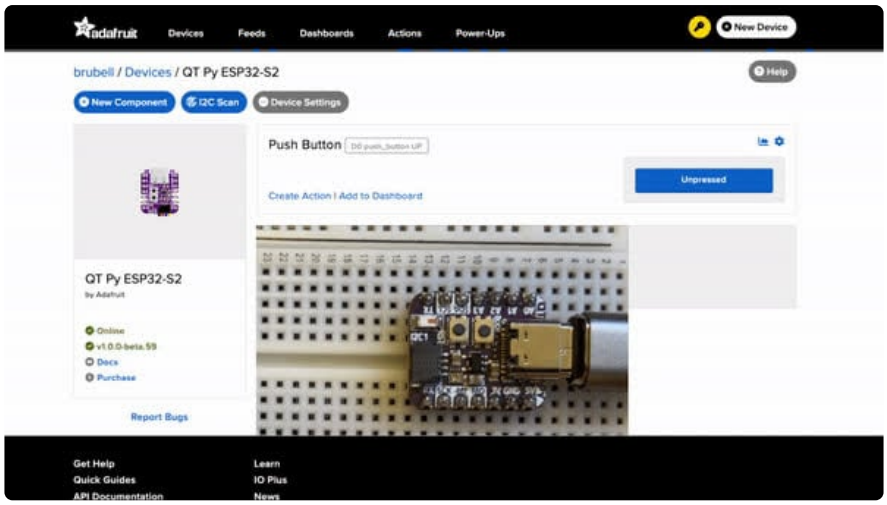


Adafruit IO sends a command to your WipperSnapper board, telling it to configure the GPIO pin you selected to behave as a digital input pin and to enable it to pull up the internal resistor.

Your board's page should also show the new push-button component.



Push the button on your board to change the value of the push-button component on Adafruit IO.



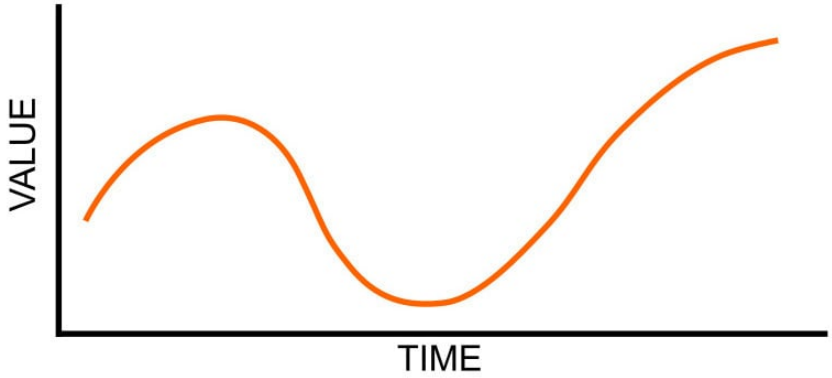
Analog Input

Your microcontroller board has both digital and analog signal capabilities. Some pins are analog, some are digital, and some are capable of both. Check the **Pinouts** page in this guide for details about your board.

Analog signals are different from digital signals in that they can be any voltage and can vary continuously and smoothly between voltages. An analog signal is like a dimmer switch on a light, whereas a digital signal is like a simple on/off switch.

Digital signals only can ever have two states, they are either are **on** (high logic level voltage like 3.3V) or **off** (low logic level voltage like 0V / ground).

By contrast, analog signals can be any voltage in-between on and off, such as 1.8V or 0.001V or 2.98V and so on.



Analog signals are continuous values which means they can be an infinite number of different voltages. Think of analog signals like a floating point or fractional number,

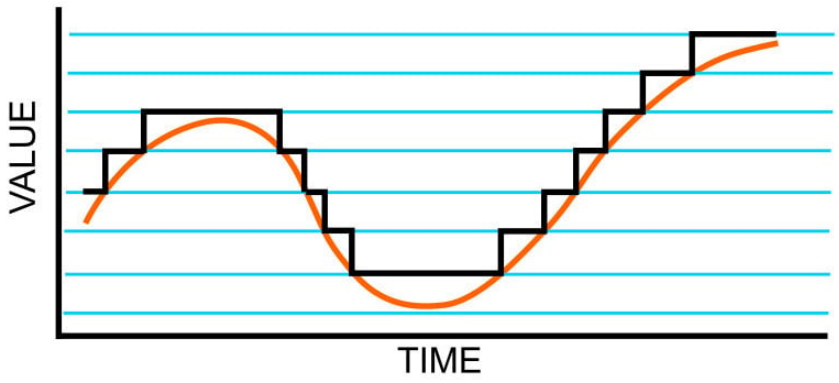
they can smoothly transiting to any in-between value like 1.8V, 1.81V, 1.801V, 1.8001V, 1.80001V and so forth to infinity.

Many devices use analog signals, in particular sensors typically output an analog signal or voltage that varies based on something being sensed like light, heat, humidity, etc.

Analog to Digital Converter (ADC)

An analog-to-digital-converter, or ADC, is the key to reading analog signals and voltages with a microcontroller. An ADC is a device that reads the voltage of an analog signal and converts it into a digital, or numeric, value. The microcontroller can't read analog signals directly, so the analog signal is first converted into a numeric value by the ADC.

The black line below shows a digital signal over time, and the red line shows the converted analog signal over the same amount of time.

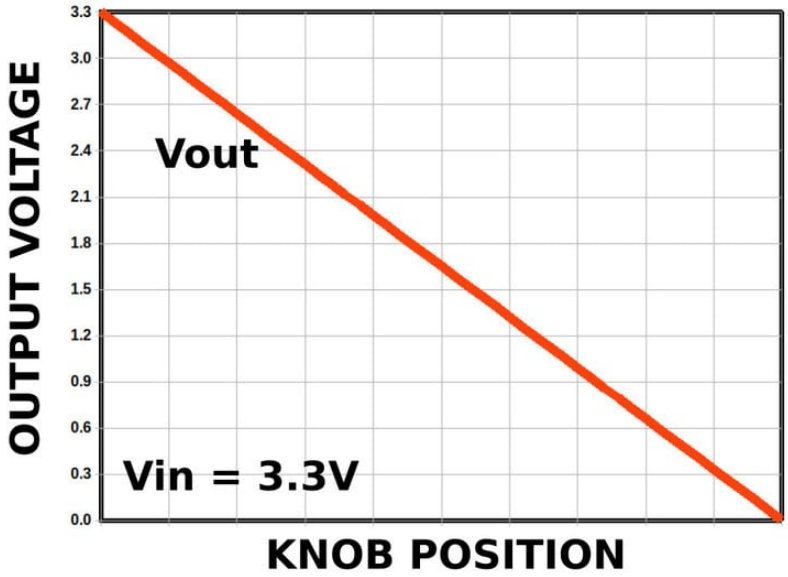


Once that analog signal has been converted by the ADC, the microcontroller can use those digital values any way you like!

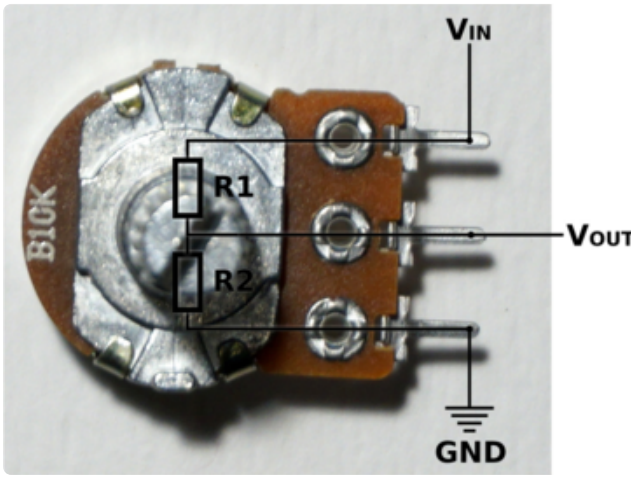
Potentiometers

A potentiometer is a small variable resistor that you can twist a knob or shaft to change its resistance. It has three pins. By twisting the knob on the potentiometer you can change the resistance of the middle pin (called the wiper) to be anywhere within the range of resistance of the potentiometer.

By wiring the potentiometer to your board in a special way (called a voltage divider) you can turn the change in resistance into a change in voltage that your board's analog to digital converter can read.



To wire up a potentiometer as a voltage divider:

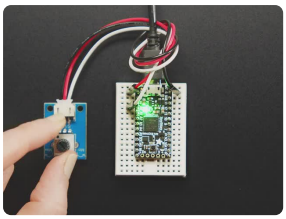


Connect **one outside pin** to **ground**
Connect **the other outside pin** to **voltage in (e.g. 3.3V)**
Connect **the middle pin** to **an analog pin (e.g. A0)**

Hardware

In addition to your microcontroller board, you will need the following hardware to follow along with this example.

Potentiometer



Featured

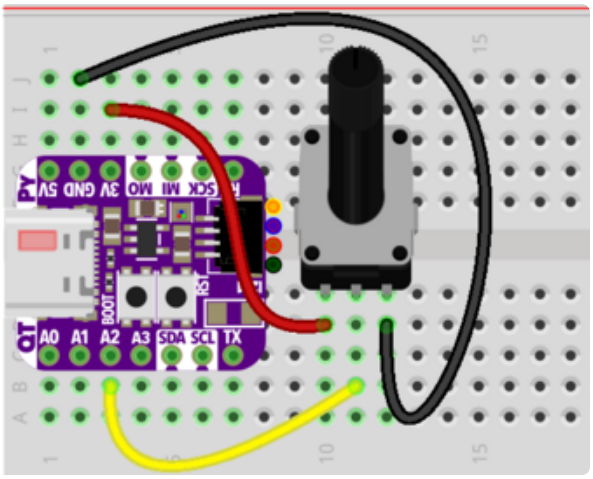
STEMMA Wired Potentiometer Breakout Board - 10K ohm Linear

For the easiest way possible to measure twists, turn to this STEMMA potentiometer breakout (ha!). This plug-n-play pot comes with a JST-PH 2mm connector and a matching

https://www.adafruit.com/product/4493

Wire Up the Potentiometer

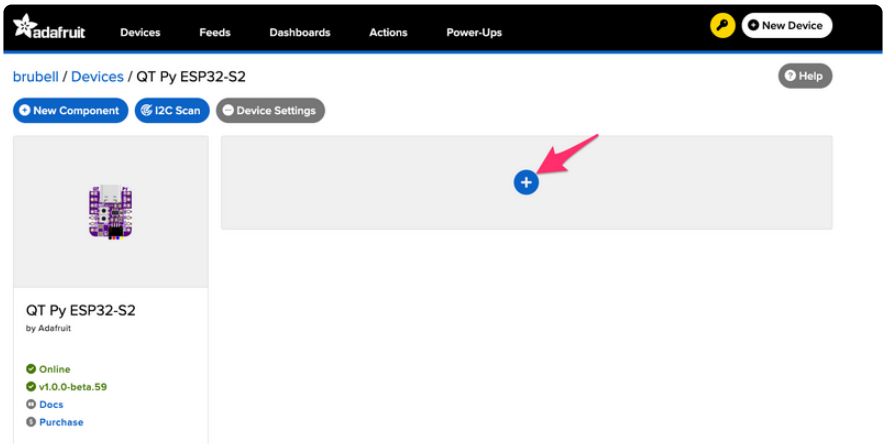
Connect the potentiometer to your board as follows.

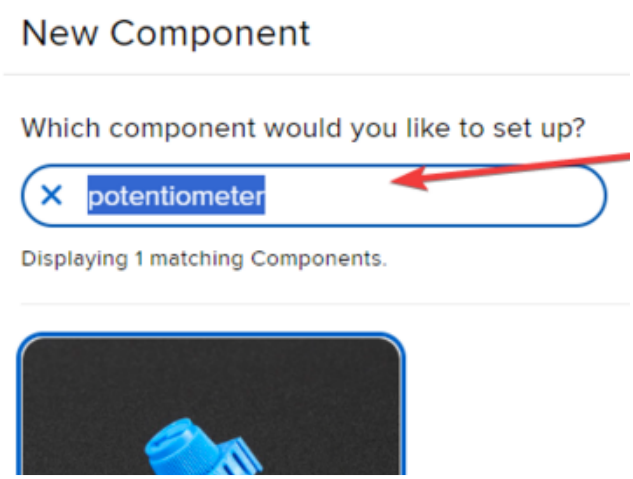


QT Py 3V to **potentiometer left pin**
QT Py A2 to **potentiometer middle pin**
QT Py GND to **potentiometer right pin**

Create a Potentiometer Component on Adafruit IO

On the device page, click the New Component (or "+") button to open the component picker.





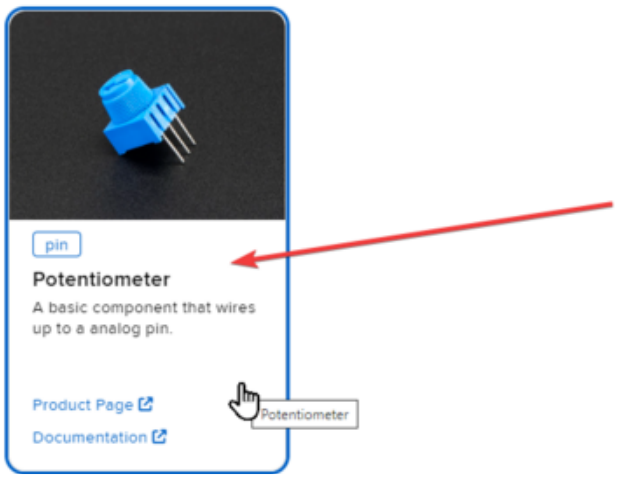
Search for the component name by entering `potentiometer` into the text box on the component picker, the list of components should update as soon as you stop typing.

Filtering and searching for components

Since WipperSnapper supports such a large number of components, there is keyword filtering. Try searching for various keywords, like:

- component names: `aht20`, `servo`, `buzzer`, `button`, `potentiometer`, etc
- sensor types: `light`, `temperature`, `pressure`, `humidity`, etc
- interface: `i2c`, `uart`, `ds18x20`, `pin`, etc (also I2C addresses e.g. `0x44`)
- vendor: `Adafruit`, `ASAIR`, `Infineon`, `Bosch`, `Honeywell`, `Sensirion`, etc

There are also added product and documentation links for every component, follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide.



Select the **Potentiometer** from the list of results to go to the component configuration page.

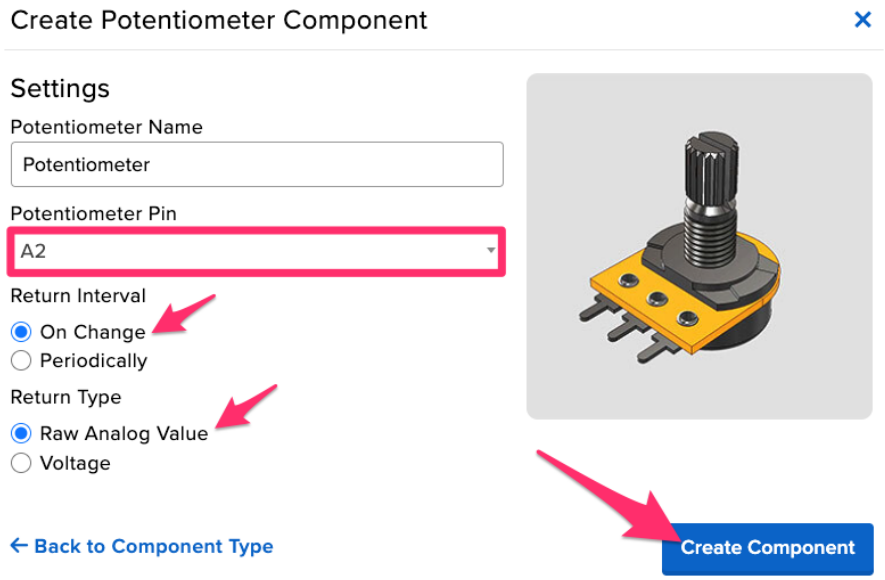
There will be a back button if you select the wrong component, and you can use the Edit component icon (⚙) on the device page to update the component configuration in the future.

On the Create Potentiometer Component form:

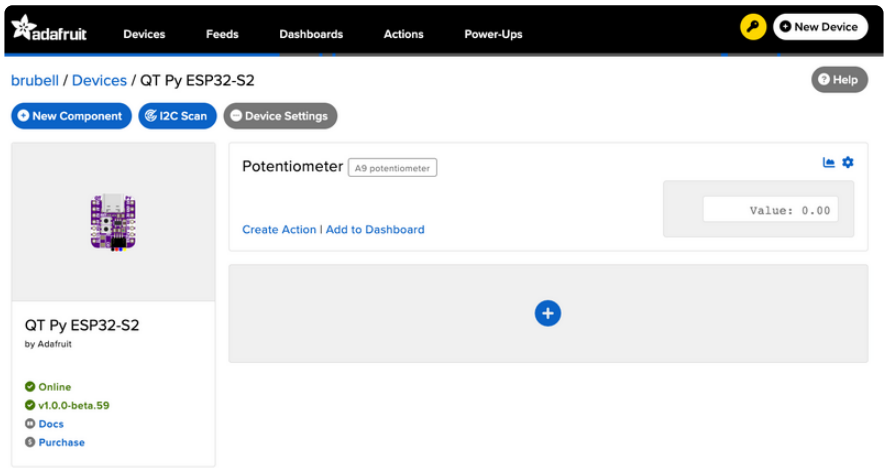
- Set **Potentiometer Pin** to **A2**

- Select **"On Change"** as the **Return Interval**
- Select **Raw Analog Value** as the **Return Type**

Then, click Create Component



The potentiometer component appears on your board page! Next, learning to read values from it.



Read Analog Pin Values

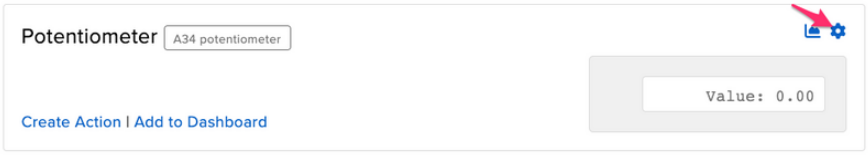
Rotate the potentiometer to see the value change.

What do these values mean?

WipperSnapper reports ADC "raw values" as 16-bit unsigned integer values. Your potentiometer will read between 0 (twisting the pot to the leftmost position) and 65535 (twisting the pot to the rightmost position).

Read Analog Pin Voltage Values

You can update the potentiometer component (or any analog pin component in WipperSnapper) to report values in Volts. To do this, on the right-hand side of the potentiometer component, click the cog button.



Under **Return Type**, click Voltage.

Click Update Component to send the updated component settings to your board running WipperSnapper.

Now, twist the potentiometer to see the value reported to Adafruit IO in Volts!

I2C Sensor

Text emphasized with a blue exclamation:	While this page uses the "MCP9808 High Accuracy I2C Temperature Sensor Breakout", the process for adding an I2C sensor to your board running WipperSnapper is similar for all I2C sensors.

Inter-Integrated Circuit, aka **I2C**, is a two-wire protocol for connecting sensors and "devices" to a microcontroller. A large number of sensors, including the ones sold by Adafruit, use I2C to communicate.

Typically, using I2C with a microcontroller involves programming. Adafruit IO and WipperSnapper let you configure a microcontroller to read data from an I2C sensor and publish that data to the internet without writing code.

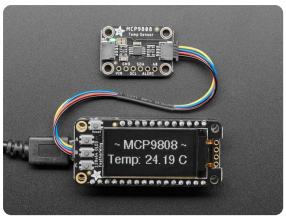
The WipperSnapper firmware supports a number of I2C sensors, viewable in list format here ().

- If you do not see the I2C sensor you're attempting to use with WipperSnapper, Adafruit has a guide on adding a component to Adafruit IO WipperSnapper here ().

On this page, you'll learn how to wire up an I2C sensor to your board. Then, you'll create a new component on Adafruit IO for your I2C sensor and send the sensor values to Adafruit IO. Finally, you'll learn how to locate, interpret, and download the data produced by your sensors.

Parts

You will need the following parts to complete this page:



Featured

Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/ precise we've ever seen, with a typical accuracy of ±0.25°C over the sensor's -40°C to...

https://www.adafruit.com/product/5027

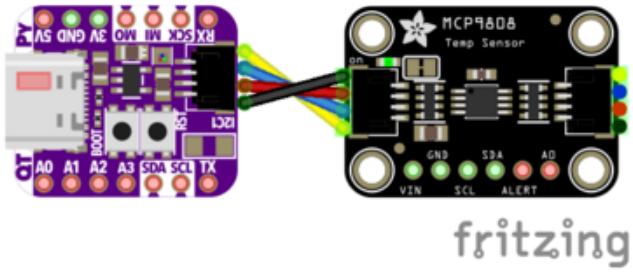
Featured

STEMMA QT / Qwiic JST SH 4-pin Cable - 100mm Long

This 4-wire cable is a little over 100mm / 4" long and fitted with JST-SH female 4-pin connectors on both ends. Compared with the chunkier JST-PH these are 1mm pitch instead of...

https://www.adafruit.com/product/4210

Wiring

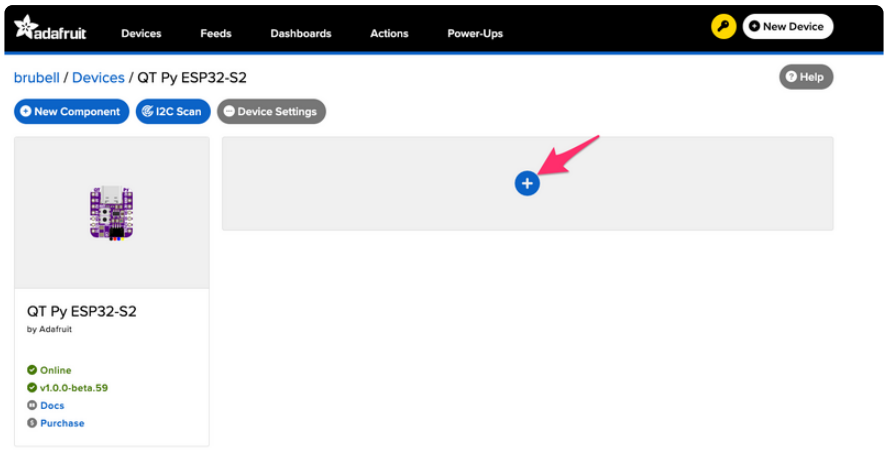


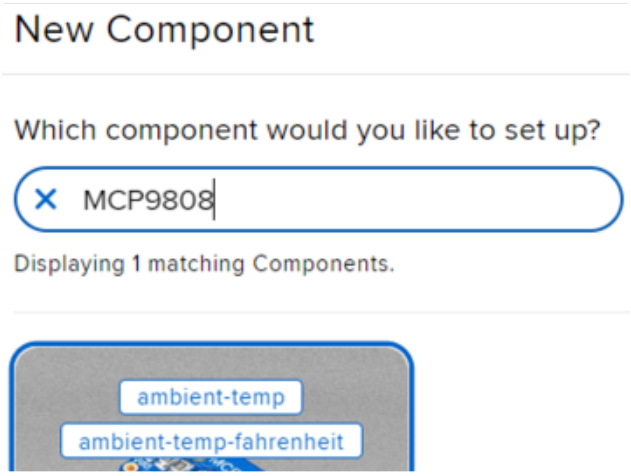
Connect the **QT Py's STEMMA QT Port** to **MCP9808's STEMMA QT Port**

NOTE: At the time of writing, **the QT Py's second I2C port** (mapped to the SCL/SDA pins on the board) **is not supported by WipperSnapper.** Only the STEMMA I2C port is supported.

Add an MCP9808 Component

On the device page, click the New Component (or "+") button to open the component picker.





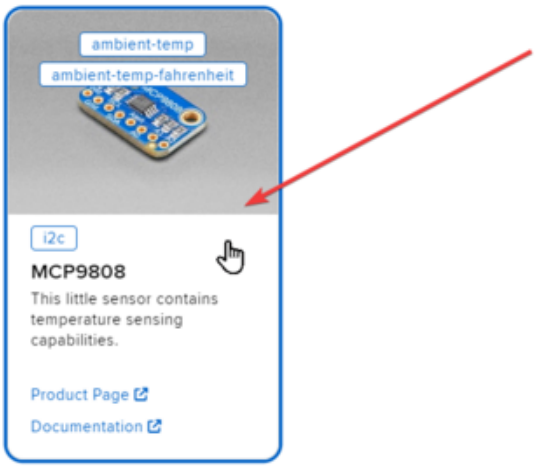
Search for the component name by entering `MCP9808` into the text box on the component picker, the list of components should update as soon as you stop typing.

Filtering and searching for components

Since WipperSnapper supports such a large number of components, there is keyword filtering. Try searching for various keywords, like:

- component names: `aht20`, `servo`, `buzzer`, `button`, `potentiometer`, etc
- sensor types: `light`, `temperature`, `pressure`, `humidity`, etc
- interface: `i2c`, `uart`, `ds18x20`, `pin`, etc (also I2C addresses e.g. `0x44`)
- vendor: `Adafruit`, `ASAIR`, `Infineon`, `Bosch`, `Honeywell`, `Sensirion`, etc

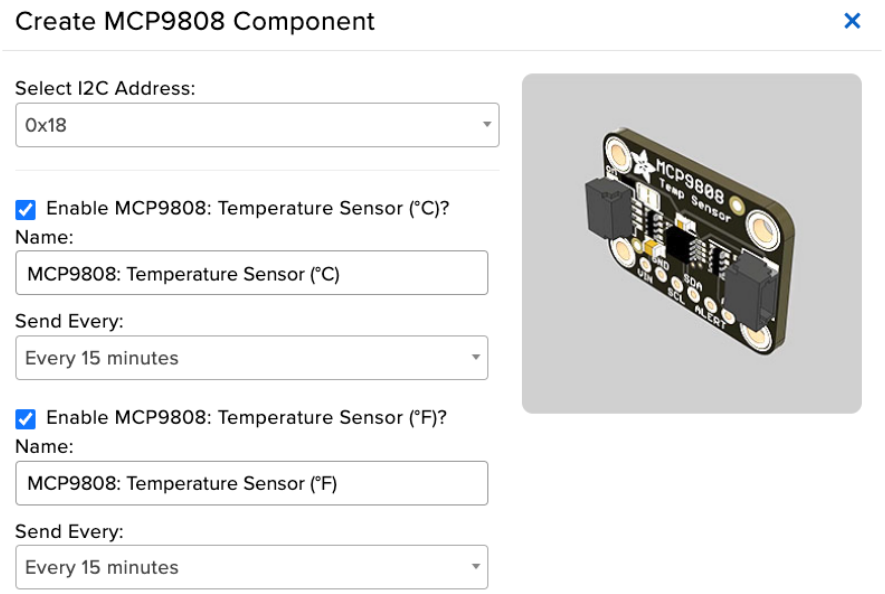
There are added product and documentation links for every component, follow the links beneath the component descriptions to be taken to the appropriate product page or Learn Guide.



Select the **MCP9808** from the list of results to go to the component configuration page.

There will be a back button if you select the wrong component, and you can use the Edit component icon (⚙) on the device page to update the component configuration in the future.

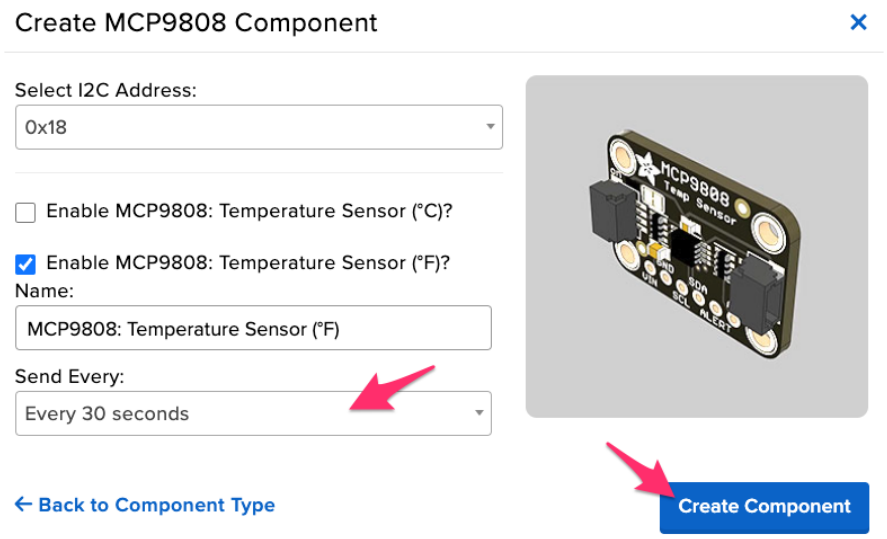
On the component configuration page, the MCP9808's I2C sensor address should be listed along with the sensor's settings.



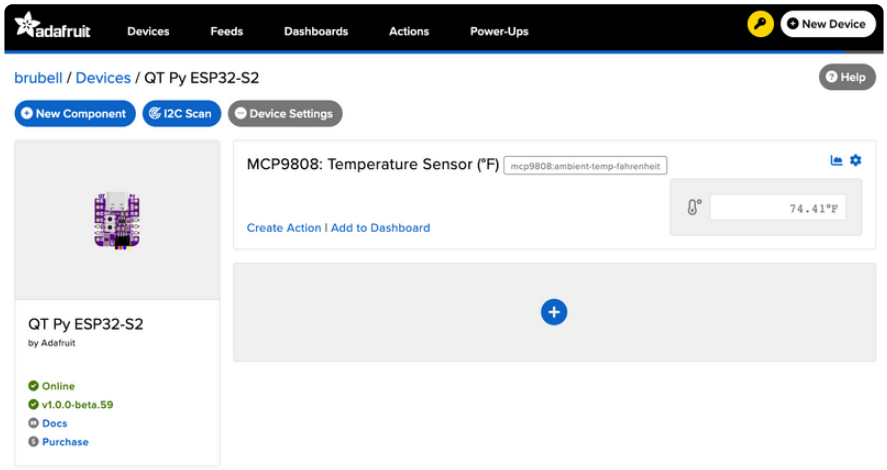
The MCP9808 sensor can measure ambient temperature. This page has individual options for reading the ambient temperature, in either Celsius or Fahrenheit. You may select the readings which are appropriate to your application and region.

The **Send Every** option is specific to each sensor measurement. This option will tell the board how often it should read from the sensor and send the data to Adafruit IO. Measurements can range from every 30 seconds to every 24 hours.

For this example, set the **Send Every** interval for both seconds to **Every 30 seconds.** Click **Create Component**.



The board page should now show the MCP9808 component you created. After the interval you configured elapses, the WipperSnapper firmware running on your board automatically reads values from the sensor and sends them to Adafruit IO.



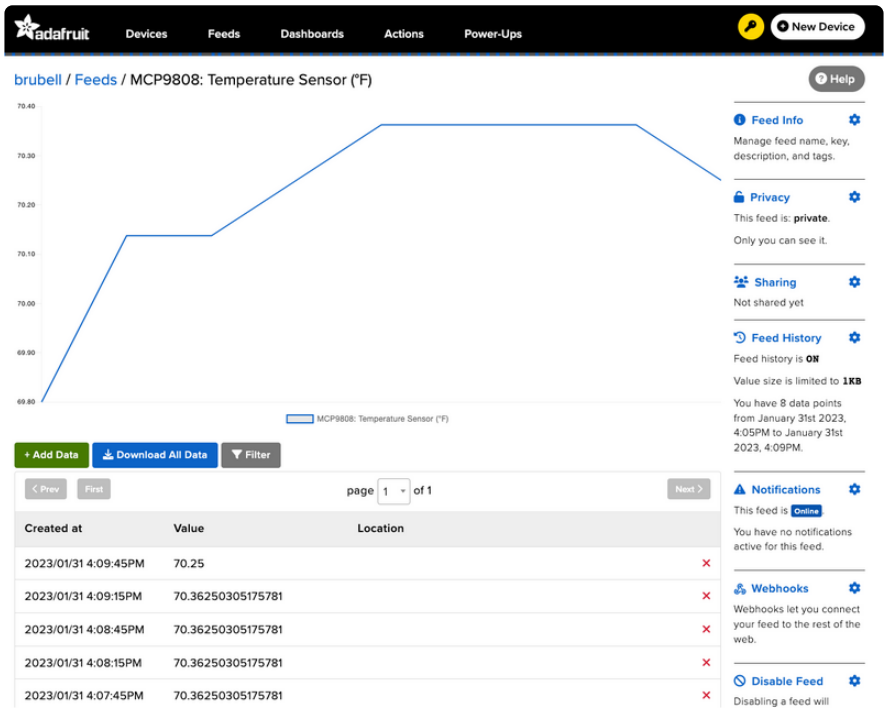
Read I2C Sensor Values

Now to look behind the scenes at a powerful element of using Adafruit IO and WipperSnapper. When a new component is created on Adafruit IO, an Adafruit IO Feed () is also created. This Feed holds your sensor component values for long-term storage (30 days of storage for Adafruit IO Free and 60 days for Adafruit IO Plus plans).

Aside from holding the **values** read by a sensor, the component's feed also holds **metadata** about the data pushed to Adafruit IO. This includes settings for whether the data is public or private, what license the stored sensor data falls under, and a general description of the data.

Next, to look at the sensor temperature feed. To navigate to a component's feed, click on the chart icon in the upper-right-hand corner of the component.

On the component's feed page, you'll each data point read by your sensor and when they were reported to Adafruit IO.



Doing more with your sensor's Adafruit IO Feed

This only scratches the surface of what Adafruit IO Feeds can accomplish for your IoT projects. For a complete overview of Adafruit IO Feeds, including tasks like downloading feed data, sharing a feed, removing erroneous data points from a feed, and more, head over to the "Adafruit IO Basics: Feed" learning guide ().

Factory Reset

The QT Py ships with a simple NeoPixel swirl demo.

It's lovely, but you probably had other plans for the board. As you start working with your board, you may want to return to the original code to begin again, or you may find your board gets into a bad state. Either way, this page has you covered.

You're probably used to seeing the **QTPYS2BOOT** drive when loading CircuitPython or Arduino. The **QTPYS2BOOT** drive is part of the UF2 bootloader, and allows you to drag and drop files, such as CircuitPython. However, on the ESP32-S2 the UF2 bootloader can become damaged.

If you want to check out the code for the factory reset/test its here! ()

Install the Factory Reset Firmware UF2

If you have a bootloader still installed - which means you can double-click to get the **QTPYS2BOOT** drive to appear, then you can simply drag this UF2 file over to the **BOOT** drive.

To enter UF2 bootloader mode, plug in the board into a USB cable with data/sync capability. Press the reset button once, wait till the RGB LED turns purple, then quickly press the reset button again. (You are doing a "slow double-click"). Then drag this file over:

Your board is now back to its factory-shipped state, running again with the original factory program. You can now begin again with your plans for your board.

Text emphasized with a blue exclamation:	Re-loading the factory reset UF2 isn't necessary to start over with the board, but it's a good way to show the board is working.

UF2 Bootloader Installation and Repair

What if you tried double-tapping the reset button, and you still can't get into the UF2 bootloader? Whether your board shipped without the UF2 bootloader, or something damaged it, this section has you covered.

Text emphasized with a yellow exclamation:

There is no protection for the UF2 bootloader. That means it is possible to erase or damage the UF2 bootloader, especially if you upload an Arduino sketch to an ESP32-S2/S3 board that doesn't "know" there's a bootloader it should not overwrite!

It turns out, however, the ESP32-S2/S3 comes with a second, built-in, bootloader, the ROM bootloader, which cannot be erased or damaged. You can always reload the UF2 bootloader using the ROM bootloader.

There are three ways to do a factory reset and bootloader repair. The first and easiest is to use the **OPEN INSTALLER** button () on the page for your board on circuitpython.org (). This method requires no manual downloads and guides you through the steps interactively. **We highly recommend this method as your first choice.**

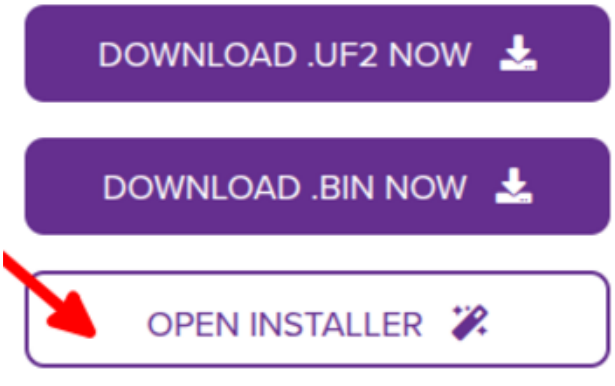
The second method is to use the browser-based **Adafruit WebSerial ESPTool**, and the third is to use `esptool.py` via the command line.

For both OPEN INSTALLER and the Adafruit WebSerial tool, you must use Firefox 151 or later, or Chrome 89 or later.

Text emphasized with a red exclamation:

Installing the UF2 bootloader will erase everything on your board, including CircuitPython, any Arduino program, and any files stored in flash! Be sure to back up your data first.

OPEN INSTALLER Method for UF2 Bootloader Installation



The OPEN INSTALLER button on circuitpython.org () is the easiest way to install or update the UF2 bootloader. See the Learn Guide Using the OPEN INSTALLER Button on circuitpython.org () for all the details.

Adafruit WebSerial ESPTool and `esptool.py` Methods for UF2 Bootloader Installation

The other methods for bootloader installation require you to download the bootloader yourself. The next section walks you through the prerequisite steps needed for both of these methods.

Step 1. Download the UF2 Bootloader **.bin** File for your board

Clik on the green button below to download the **.bin** file you need (there may be more than one listed), and save wherever is convenient for you. You will need to be able to access the file from the Adafruit WebSerial ESPTool or another upload method.

Text emphasized with a green exclamation:	Note that this file is approximately 3MB. This is not because the bootloader is that large, but because the bootloader needs to write data at both the beginning and end of flash. Most of the file is empty.

This TinyUF2 bootloader provides a single 2.8MB firmware partition, which is needed for CircuitPython 10.0.0-alpha.5 and later versions. For CircuitPython 9.1.x and 9.2.x,

either 4MB version below will work. See Update TinyUF2 Bootloader for CircuitPython 10 (4MB boards only) () for more details.

This TinyUF2 bootloader provides two 1.4MB firmware partitions, required for CircuitPython 9.0.x and earlier, to support `dualbank` functionality:

Step 2. Enter ROM bootloader mode

Entering the ROM bootloader is easy. Complete the following steps.

Before you start, make sure your ESP32-S2/S3 is plugged into USB port to your computer using a data/sync cable. Charge-only cables will not work!

To enter the bootloader:

1. **Press and hold the BOOT/DFU button down.** Don't let go of it yet!
2. **Press and release the Reset button.** You should still have the BOOT/DFU button pressed while you do this.
3. **Now you can release the BOOT/DFU button.**

No USB drive will appear when you've entered the ROM bootloader. This is normal!

Now that you've downloaded the **.bin** file and entered the ROM bootloader, you're ready to continue installing the UF2 bootloader. The next two sections walk you through using the Adafruit WebSerial ESPTool (Alternative A) or `esptool.py` (Alternative B).

(There is also an Alternative C below, using the Arduino IDE, but it is less desirable, because it does not allow you to choose the bootloader that will be used.)

Step 3: Alternative A. The Adafruit WebSerial ESPTool Method

Text emphasized with a green exclamation:

We highly recommend using the Adafruit WebSerial ESPTool method to perform a factory reset and bootloader repair. However, if you'd rather use **esptool.py** via the command line, you can skip this section and use Alternative B instead.

This method uses the Adafruit WebSerial ESPTool through a browser. Adafruit WebSerial ESPTool is a web-based option for programming ESP32-S2/S3 boards. It allows you to erase the the microcontroller flash and program up to four files at different offsets.

You have to use a browser that supports WebSerial for this to work. FIrefox versions 151 or later and and Chrome versions 89 or later work. Safari and is not supported because they don't have support for Web Serial functionality.

Follow the steps below to flash the UF2 bootloader.

Text emphasized with a yellow exclamation:

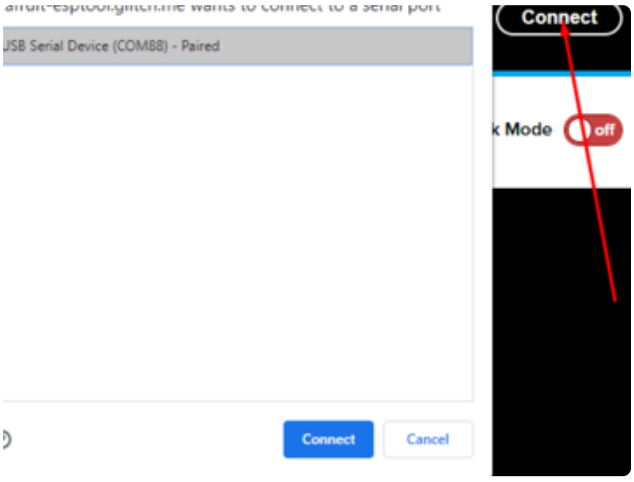
If you're using Chrome 88 or older, see the Older Versions of Chrome section at the end of this page for instructions on enabling Web Serial.

Connect

You should have plugged in **only the ESP32-S2/S3 that you intend to flash**. That way there's no confusion in picking the proper port when it's time!

In the **Chrome browser** visit [https://adafruit.github.io/Adafruit_WebSerial_ESPTool/ ()](). You should see something like the image shown.

Leave the **No reset for Passthrough updates** toggle **off**.



Press the **Connect** button in the top right of the web browser. You will get a pop up asking you to select the COM or Serial port.

Remember, you should remove all other USB devices so only the ESP32-S2/S3 board is attached, that way there's no confusion over multiple ports!

On some systems, such as MacOS, there may be additional system ports that appear in the list.

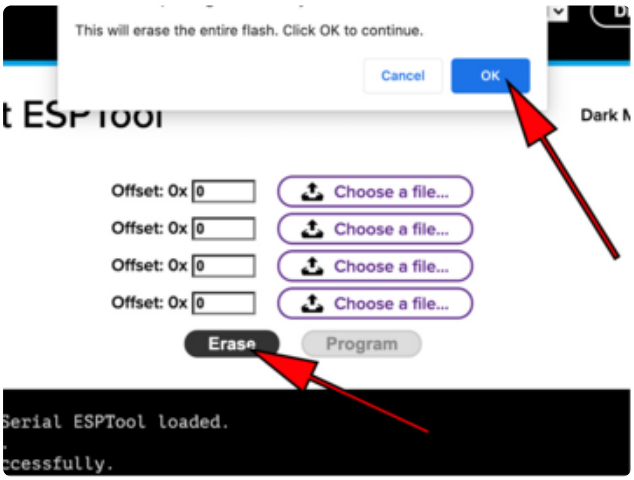
```
ESP Web Flasher loaded.
Connecting...
Connected successfully.
Try hard reset.
Chip type ESP32-S2
Connected to ESP32-S2
MAC Address: 7C:DF:A1:06:8D:D0
Uploading stub...
Running stub...
Stub is now running...
Detecting Flash Size
FlashId: 0x164020
Flash Manufacturer: 20
Flash Device: 4016
Auto-detected Flash size: 4MB
```

The JavaScript code will now try to connect to the ROM bootloader. It may timeout for a bit until it succeeds. On success, you will see that it is **Connected** and will print out a unique **MAC address** identifying the board along with other information that was detected.

Once you have successfully connected, the command toolbar will appear.

Erase the Contents of Flash

Text emphasized with a yellow exclamation:	This will erase everything on your board! If you have access, and wish to keep any code, now is the time to ensure you've backed up everything.



To erase the contents, click the Erase button. You will be prompted whether you want to continue. Click OK to continue or if you changed your mind, just click cancel.

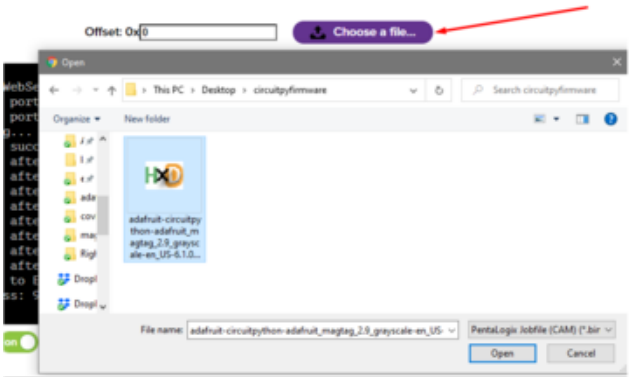
You'll see "Erasing flash memory. Please wait..." This will eventually be followed by "Finished." and the amount of time it took to erase.

Do not disconnect! Immediately continue on to programming the ESP32-S2/S3.

Text emphasized with a red exclamation:	Do not disconnect after erasing! Immediately continue on to the next step!

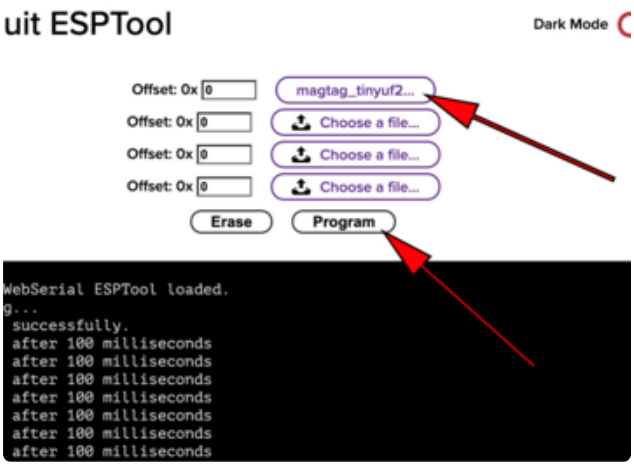
Flash the Bootloader **.bin** File

Programming the ESP32-S2/S3 can be done with up to four files at different locations, but with the board-specific bootloader **.bin** file, which you should have downloaded under **Step 1** on this page, you only need to use one file.

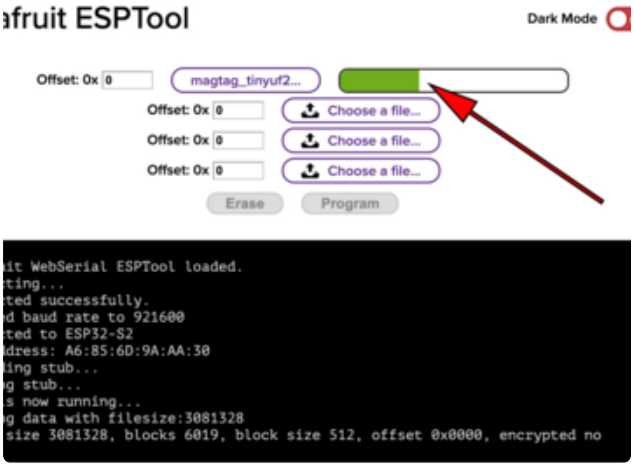


Click on the first **Choose a file...**. (The tool will only attempt to program buttons with a file and a unique location.) Then, select the bootloader **.bin** file you downloaded in Step 1 that matches your board.

Verify that the **Offset** box next to the file location you used is (0x) **0**.



Once you choose a file, the button text will change to match your filename. You can then select the **Program** button to begin flashing.



A progress bar will appear and after a minute or two, you will have written the firmware.

Once completed, you can skip down to the section titled Reset the Board.

Step 3: Alternative B. The `esptool.py` Method (for advanced users)

Text emphasized with a red exclamation:	If you used Adafruit WebSerial ESPTool, you do not need to complete the steps in this section!

Once you have entered ROM bootloader mode, you can then use Espressif's esptool program () to communicate with the chip! `esptool` is the 'official' programming tool and is the most common/complete way to program an ESP chip.

Install `esptool.py`

You will need to use the command line or Terminal to install and run `esptool`.

You will also need to have pip and Python installed (any version!).

Install the latest version using pip (you may be able to run `pip` without the `3` depending on your setup):

```
pip3 install --upgrade esptool
```

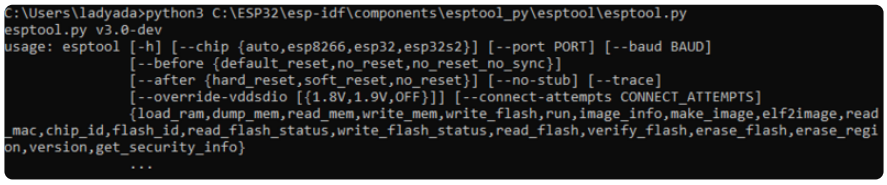
Then, you can run:

```
esptool.py
```

Text emphasized with a blue exclamation:	Make sure you are running esptool v3.0 or higher, which adds ESP32-S2/S3 support.

Test the Installation

Run `esptool.py` in a new terminal/command line and verify you get something like the below:



Find the Serial Port

First, you need to determine the name of the serial port you board has when it's plugged in and ROM bootloader mode.

- For Windows, it will be a COM port, such as `COM5`. Look in Device Manager -> Ports.
- For macOS, you can do `ls /dev/tty.*` in a Terminal to find the names of the serial ports.

- For Linux, you can do ls `/dev/tty*` in a terminal window. The name is often /dev/ttyACM0 or similar.

If you are not sure you've found the right port, unplug the board, and see if the port disappears.

For more information on determining serial ports, follow these links for help on Windows (), macOS (), or Linux ().

Connect

Run the following command, replacing the identifier after `--port` with the `COMxx`, `/dev/cu.usbmodemxx` or `/dev/ttySxx` you found above.

For example, if you're using Windows, and you think the board is on COM88, type this following:

```
esptool.py --port COM88 chip_id
```

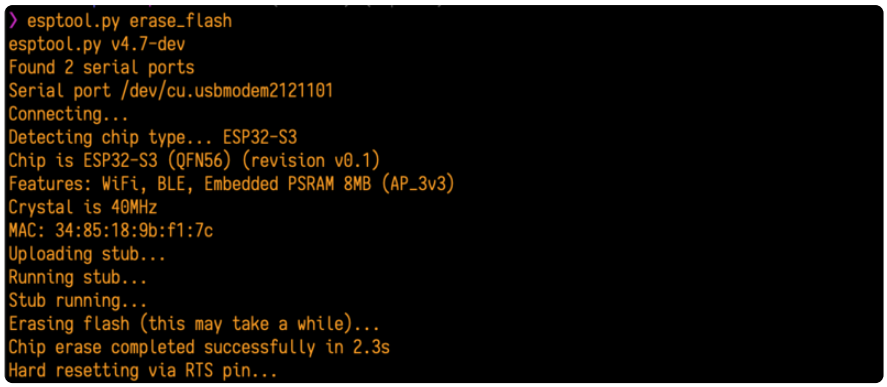
You should get a notice that it connected over that port and found an ESP32-S2/S3.

Erase the Flash

Before programming the board, it is a good idea to erase the flash. Run the following command.

```
esptool.py --port COM88 erase_flash
```

You must be connected (by running the command in the previous section) for this command to work as shown.



Flash the UF2 Bootloader

Run this command and replace the serial port name with your matching port and the file you just downloaded

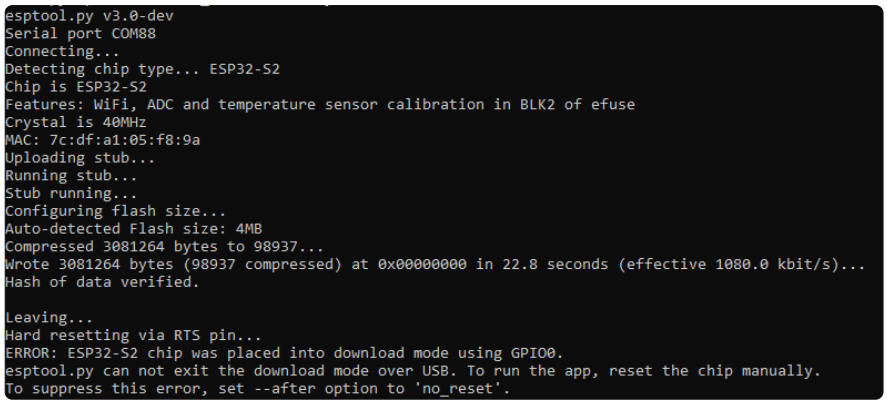
`esptool.py --port COM88 write_flash 0x0 tinyuf2-some-board-0.32.0-combined.bin`

Don't forget to change the `--port` name to match.

!	Text emphasized with a blue exclamation:	Adjust the bootloader filename accordingly if it differs from `tinyuf2-some-board-0.32.0-combined.bin`.

There might be a bit of a 'wait' when programming, where it doesn't seem like it's working. Give it a minute, it has to erase the old flash code which can cause it to seem like it's not running.

You'll finally get an output like this:



Once completed, you can continue to the next section.

Step 4. Reset the board

The board should restart automatically, and start up the UF2 bootloader. If not, press the reset button to start the bootloader.

The NeoPixel LED on the QT Py ESP32-S2 will light up in a rainbow swirl.

You've successfully returned installed the UF2 bootloader on your board. If you now want to return the board to a factory reset state, follow the instructions at the top of the page to **Install the Factory Reset Firmware UF2**.

Older Versions of Chrome

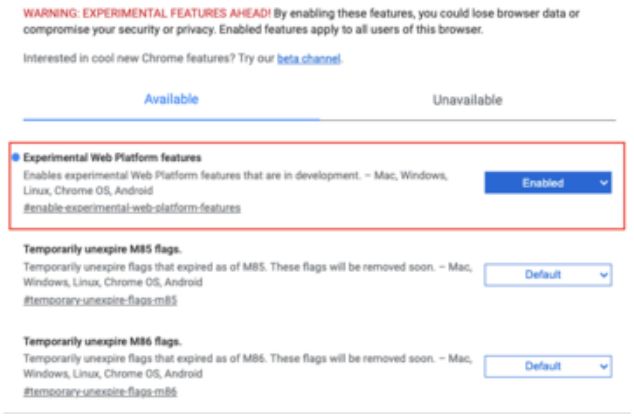
Text emphasized with a green exclamation:	As of chrome 89, Web Serial is already enabled, so this step is only necessary on older browsers.

We suggest updating to Chrome 89 or newer, as Web Serial is enabled by default.

If you must continue using an older version of Chrome, follow these steps to enable Web Serial.

If you receive an error like the one shown when you visit the Adafruit WebSerial ESPTool site, you're likely running an older version of Chrome.

You must be using Chrome 78 or later to use Web Serial.



To enable Web Serial in Chrome versions 78 through 88:

Visit **chrome://flags** from within Chrome.
Find and enable the **Experimental Web Platform features**
Restart Chrome

Step 3: Alternative C. The Flash an Arduino Sketch Method

This section outlines flashing an Arduino sketch onto your ESP32-S2/S3 board, which automatically installs the UF2 bootloader as well. However, it does not allow you to choose which UF2 bootloader to install, so Alternative A or B above are preferred.

Arduino IDE Setup

If you don't already have the Arduino IDE installed, the first thing you will need to do is to download the latest release of the Arduino IDE. ESP32-S2/S3 requires **version 1.8** or higher. Click the link to download the latest.

After you have downloaded and installed the latest version of Arduino IDE, you will need to start the IDE and navigate to the **Preferences** menu. You can access it from the **File** > **Preferences** menu in Windows or Linux, or the **Arduino** > **Preferences** menu on OS X.

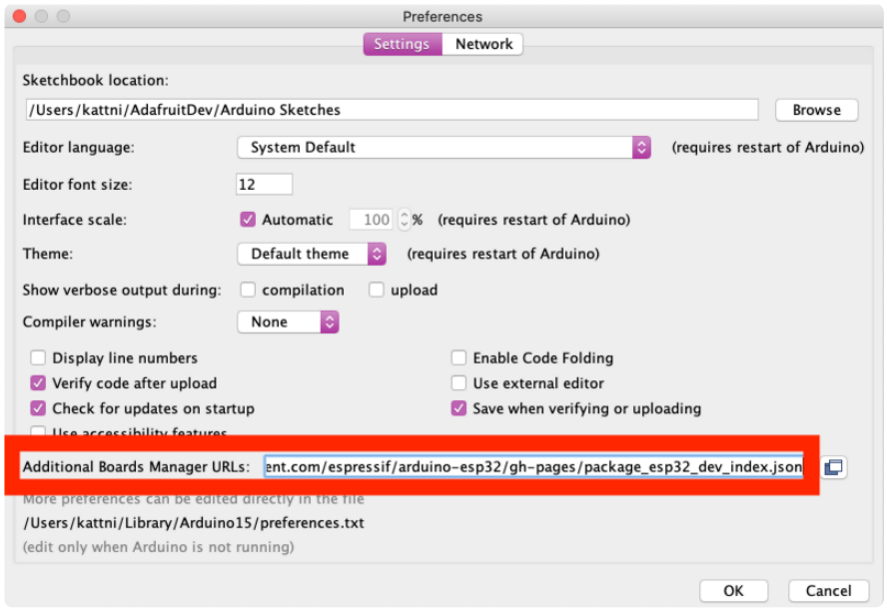
The **Preferences** window will open.

In the **Additional Boards Manager URLs** field, you'll want to add a new URL. The list of URLs is comma separated, and you will only have to add each URL once. The URLs point to index files that the Board Manager uses to build the list of available & installed boards.

Copy the following URL.

`https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_dev_index.json`

Add the URL to the the **Additional Boards Manager URLs** field (highlighted in red below).



Click **OK** to save and close **Preferences**.

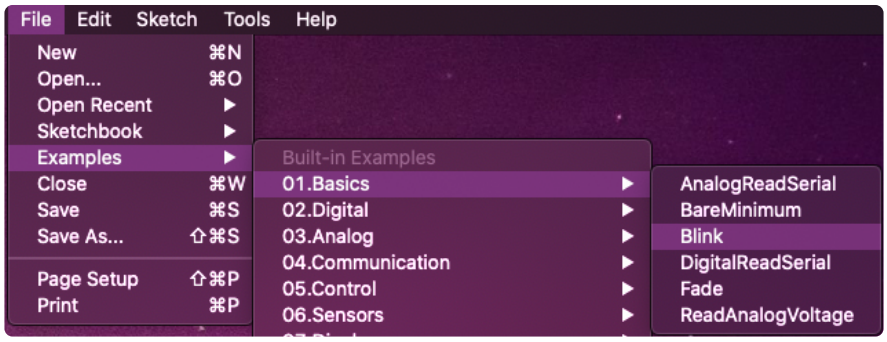
In the **Tools** > **Boards** menu you should see the **ESP32 Arduino** menu. In the expanded menu, it should contain the ESP32 boards along with all the latest ESP32-S2 boards.

Now that your IDE is setup, you can continue on to loading the sketch.

Load the Blink Sketch

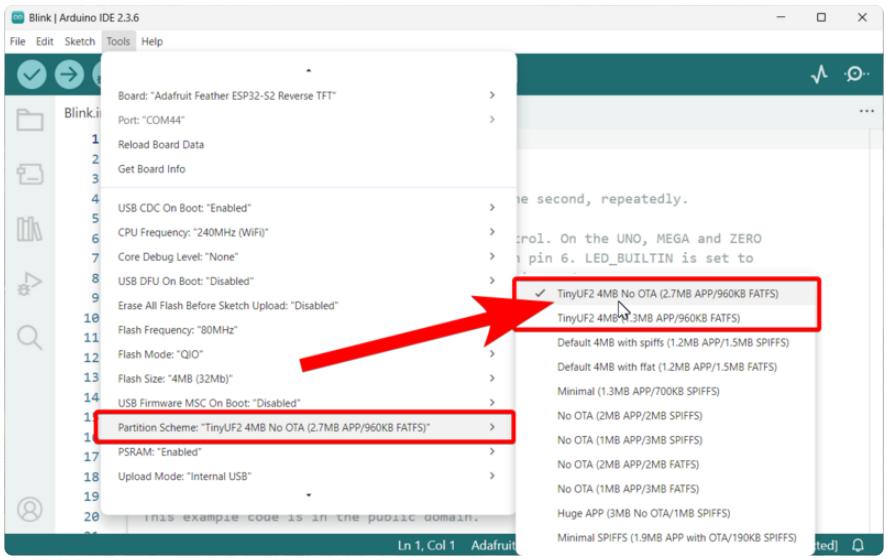
In the **Tools** > **Boards** menu you should see the **ESP32 Arduino** menu. In the expanded menu, look for the menu option for the **Adafruit QT Py ESP32-S2**.

Open the Blink sketch by clicking through **File** > **Examples** > **01.Basics** > **Blink**.

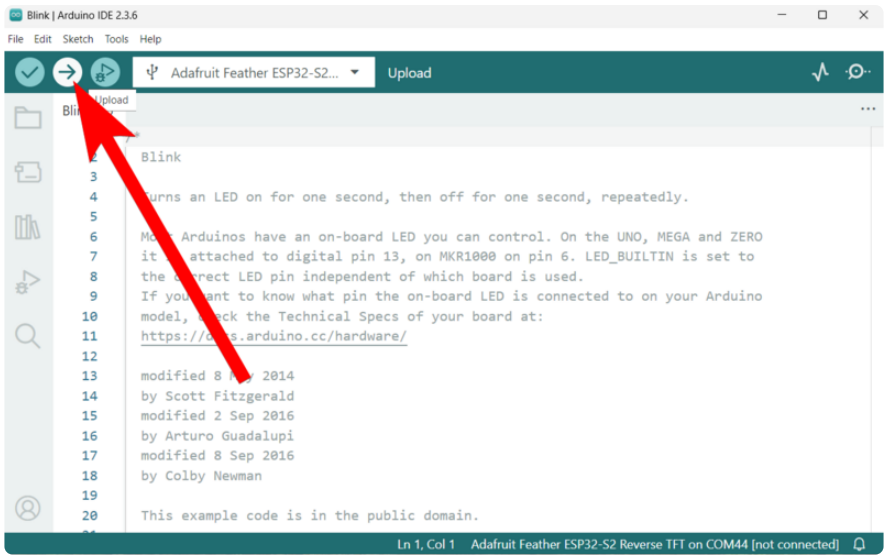


For the Arduino IDE to upload TinyUF2 you must select a TinyUF2 partition layout, under the **Tools** > **Partition Scheme** menu.

If you have a 4MB board, won't be doing Over-The-Air firmware updates (OTA), or are unsure, then choose the 'No OTA' TinyUF2 entry.



Finally, click Upload from the sketch window.



Once successfully uploaded, the little red LED will begin blinking once every second. At that point, you can now enter the bootloader.

Text emphasized with a red exclamation:	The QT Py ESP32-S2 does not have a little red LED, so the default Blink sketch will fail.

If you change `LED_BUILTIN` to `13`, the sketch will compile and upload. Be aware that, once the sketch is loaded, nothing will happen on the board. However, you will have a bootloader. The updated code would look like this:

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

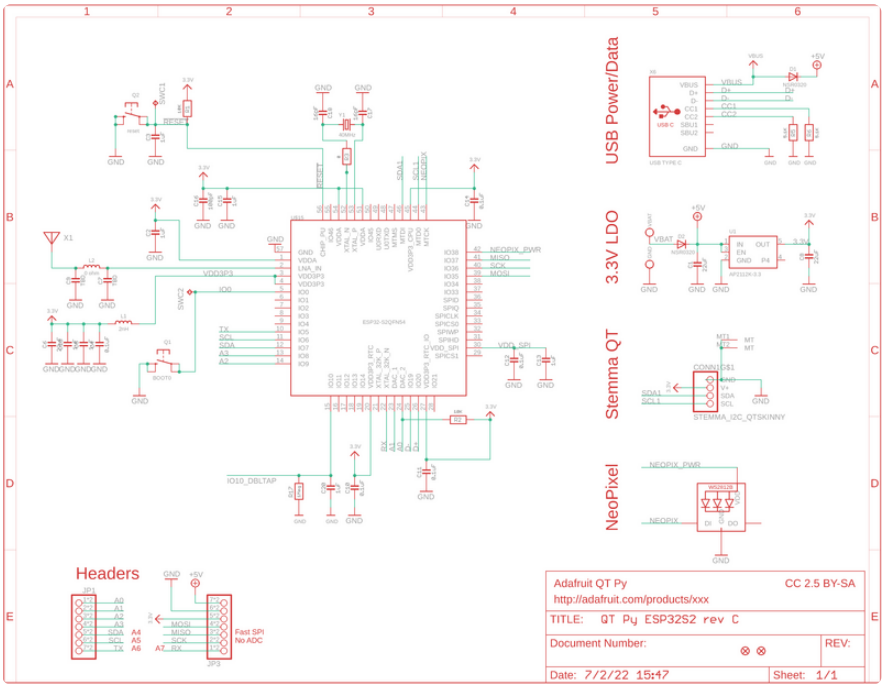
Alternatively, you could load a different sketch. It doesn't matter which sketch you use.

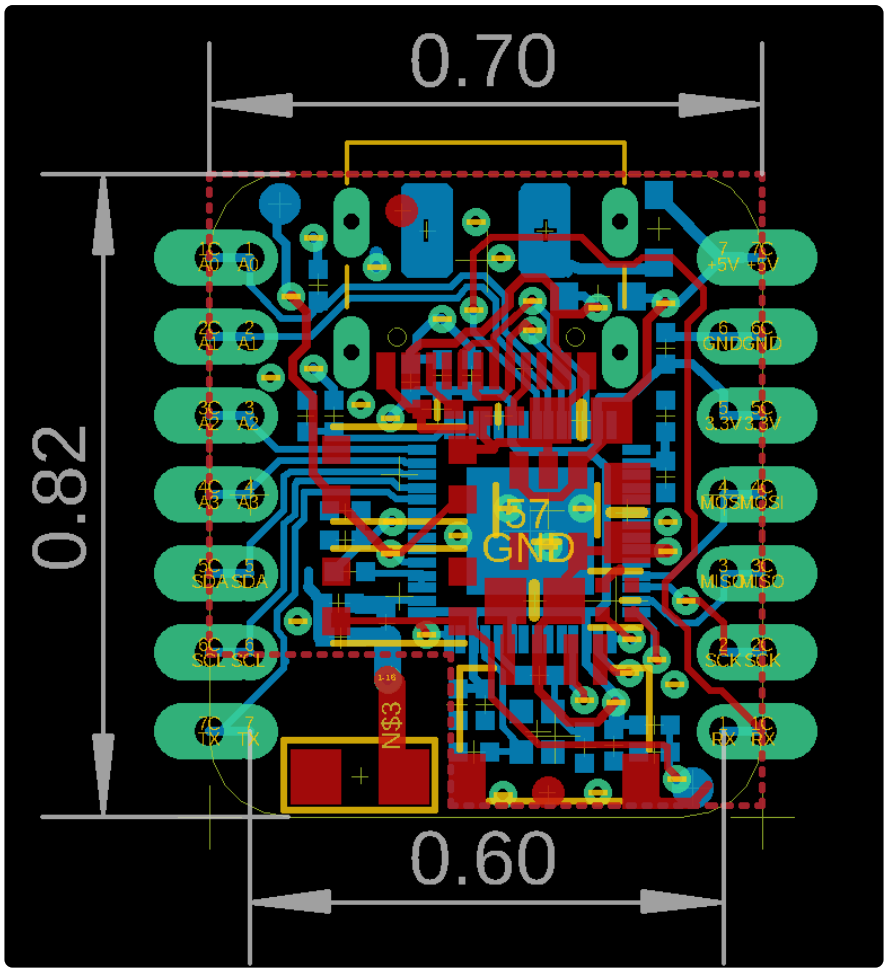
Downloads

Files:

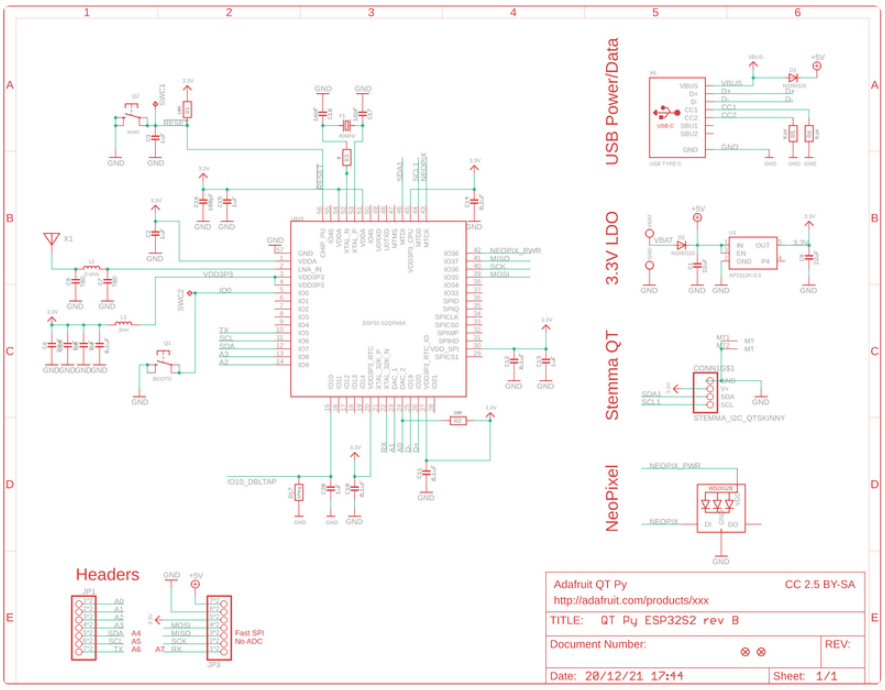
- ESP32-S2 product page with resources ()
- ESP32-S2 datasheet ()
- ESP32-S2 Technical Reference ()
- EagleCAD PCB files on GitHub ()
- 3D Models on GitHub ()
- Fritzing object in the Adafruit Fritzing Library ()
- PrettyPins PDF on GitHub ()
- PrettyPins SVG ()

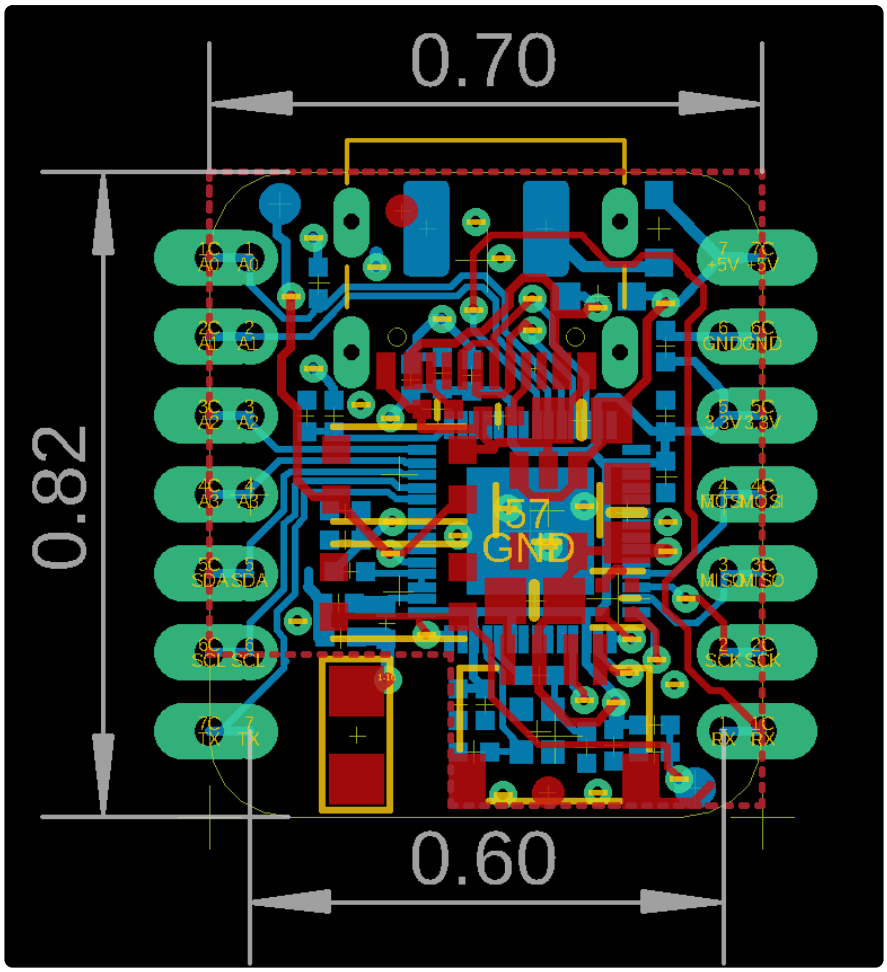
Schematic and Fab Print Rev C

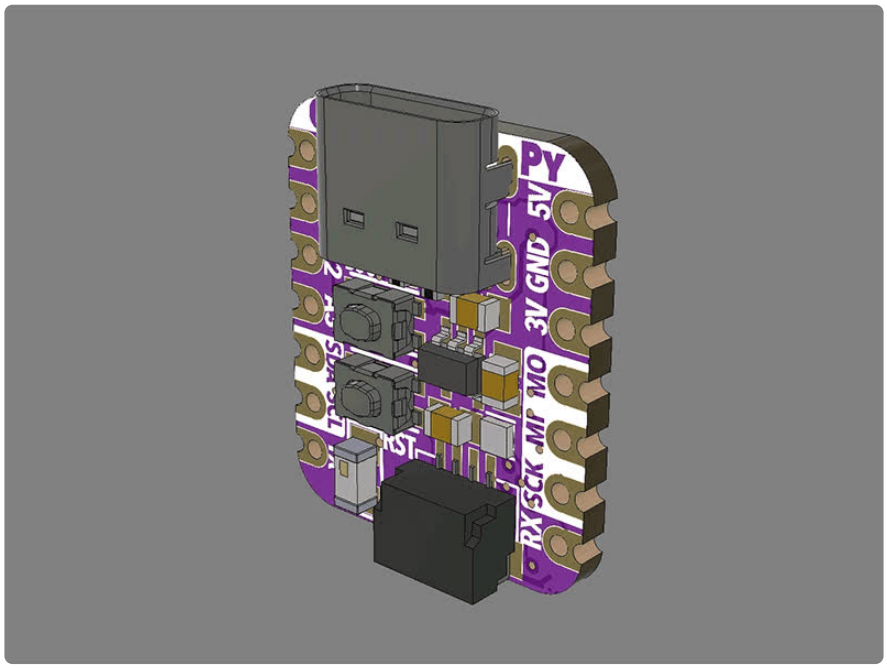




Schematic and Fab Print Rev B




0.70
0.82
0.60
A0
A1
A2
A3
A4
A5
SDA
SCL
TX
RX
+5V
GND
+3V
MOSI
MISO
SCK


5V
GND
3V
MO
SCK
RX